

UNIVERSITY GRANTS COMMISSION
NATIONAL ELIGIBILITY TEST

Computer Science & Applications

Complete Study Material for Paper I and Paper II

Notes, figures, worked examples and previous-year-pattern questions

TARGET: 250+ OF 300
about 126 of 150 questions correct

Contents

Exam Pattern & the 250+ Strategy	1
Exam Pattern at a Glance	1
What 250+ Really Means	1
About the PYQs	2
Golden Rules for 250+	2

I PAPER I

<i>TEACHING & RESEARCH APTITUDE</i>	4
1 Teaching Aptitude	5
1.1 What is Teaching?	5
1.2 Learner's Characteristics	8
1.3 Factors Affecting Teaching	9
1.4 Methods of Teaching	10
1.5 Teaching Support System	11
1.6 Evaluation Systems	13
1.7 Deeper Dive — Learning Theories, Micro-teaching & Teaching Models	14
1.8 Previous Year Questions (PYQ pattern)	16
2 Research Aptitude	19
2.1 Meaning & Characteristics	19
2.2 Types of Research	20
2.3 Variables & Hypothesis	21
2.4 Steps of Research	22
2.5 Sampling	22
2.6 Thesis & Article Writing	23
2.7 ICT in Research	24
2.8 Research Ethics	24
2.9 Deeper Dive — Research Designs, Reliability, Validity & Statistics	25
2.10 Previous Year Questions (PYQ pattern)	27
3 Comprehension	29
3.1 What Is Asked	29
3.2 Step-by-Step Strategy	29
3.3 Question Types & How to Crack Them	30
3.4 Tone Words Cheat Sheet	31
3.5 Common Traps	31
3.6 Solved Mini-Example	31
3.7 Deeper Dive — Two Full Practice Passages (solve in 8 minutes each)	31

3.8	Previous Year Questions (PYQ pattern — typical stems seen every cycle)	33
3.9	Practice Routine for 5/5	33
4	Communication	34
4.1	Meaning	35
4.2	Models of Communication	35
4.3	Types of Communication	36
4.4	Barriers to Communication	38
4.5	Inter-cultural & Group Communication	38
4.6	Mass Media & Society	39
4.7	Deeper Dive — Listening, Feedback, Networks & Classroom Talk	40
4.8	Previous Year Questions (PYQ pattern)	42
5	Mathematical Reasoning & Aptitude	44
5.1	Number Series — Pattern Checklist	44
5.2	Letter Series & Coding	45
5.3	Blood Relations	46
5.4	Classification (Odd one out)	46
5.5	Arithmetic Aptitude — Formula Bank	46
5.6	Worked Examples	49
5.7	Deeper Dive — More Question Types with Shortcuts	49
5.8	Previous Year Questions (PYQ pattern)	51
6	Logical Reasoning	53
6.1	Argument Basics	53
6.2	Categorical Propositions (A E I O)	54
6.3	Categorical Syllogism — Mood & Figure	55
6.4	Fallacies	57
6.5	Uses of Language; Connotation & Denotation	57
6.6	Analogies	58
6.7	Indian Logic (Nyaya) — Very High Weightage	58
6.8	Deeper Dive — Venn Diagram Practice, Conditionals & More Indian Logic	60
6.9	Previous Year Questions (PYQ pattern)	62
7	Data Interpretation	64
7.1	Data Basics	64
7.2	Graph Types — When to Use	65
7.3	Measures of Central Tendency & Dispersion	66
7.4	Speed-Calculation Tricks	66
7.5	Worked Data Set (typical NET format)	67
7.6	Data and Governance	67
7.7	Deeper Dive — Pie Chart & Line Graph Sets, Common Pitfalls	68
7.8	Previous Year Questions (PYQ pattern)	70
8	Information & Communication Technology (ICT)	71
8.1	Computer Basics	71
8.2	Abbreviations (must know)	72
8.3	Internet, Intranet, Extranet	73
8.4	E-mail	74
8.5	Audio & Video Conferencing	74
8.6	Digital Initiatives in Higher Education	75

8.7	ICT & Governance	75
8.8	Deeper Dive — Generations, Software, File Types & Emerging Tech	76
8.9	Previous Year Questions (PYQ pattern)	78
9	People, Development & Environment	80
9.1	MDGs → SDGs	81
9.2	Pollution — Sources & Health Effects	82
9.3	Waste Management	83
9.4	Natural & Energy Resources	83
9.5	Natural Hazards & Disasters	84
9.6	Environmental Laws & International Agreements	84
9.7	Deeper Dive — Climate Science, COP Timeline, Indian Programmes & Disasters	86
9.8	Previous Year Questions (PYQ pattern)	89
10	Higher Education System	91
10.1	Ancient Indian Centres of Learning	91
10.2	Evolution of Higher Education — Timeline	92
10.3	NEP 2020 — Higher Education Highlights (most asked)	92
10.4	Types of Institutions & Programmes	93
10.5	Regulatory & Professional Bodies	94
10.6	Skill-based & Professional Education	94
10.7	Value & Environmental Education	94
10.8	Governance & Administration	95
10.9	Deeper Dive — NEP 2020 Details, Quality Frameworks & Key Regulations	95
10.10	Previous Year Questions (PYQ pattern)	97

II PAPER II

COMPUTER SCIENCE & APPLICATIONS **99**

1	Discrete Structures & Optimization	100
1.1	Mathematical Logic	100
1.2	Sets & Relations	103
1.3	Counting & Probability	104
1.4	Algebraic Structures	105
1.5	Graph Theory	106
1.6	Boolean Algebra	108
1.7	Optimization	109
1.8	Deeper Dive — Worked Simplex, Transportation, Assignment, Groups & Recurrences	111
1.9	Previous Year Questions (PYQ pattern)	114
2	Computer System Architecture	117
2.1	Digital Logic	117
2.2	Data Representation	120
2.3	Register Transfer & Micro-operations	122
2.4	Basic Computer (Mano)	122
2.5	Control Unit	123
2.6	CPU Organisation	123
2.7	Pipelining	125
2.8	I/O Organisation	126
2.9	Memory Hierarchy	127

2.10	Multiprocessors	129
2.11	Deeper Dive — Booth Trace, Instruction Encoding, Cache Behaviour & Interrupts .	130
2.12	Previous Year Questions (PYQ pattern)	132
3	Programming Languages & Computer Graphics	135
3.1	Language Concepts	135
3.2	C Programming — Exam Traps	137
3.3	OOP Concepts	140
3.4	C++ Specifics	141
3.5	Web Programming	142
3.6	Computer Graphics	142
3.7	Deeper Dive — Scoping, More C Traps, Java Essentials & Graphics Traces	148
3.8	Previous Year Questions (PYQ pattern)	152
4	Database Management Systems	154
4.1	Architecture	154
4.2	ER Model	155
4.3	Keys & Integrity	156
4.4	Relational Algebra & Calculus	157
4.5	SQL	158
4.6	Functional Dependencies & Normalisation	158
4.7	Transactions	160
4.8	File Organisation & Indexing	163
4.9	Advanced & Enhanced Databases	164
4.10	Data Warehousing & Mining	164
4.11	Big Data & NoSQL	165
4.12	Deeper Dive — SQL on Real Tables, Minimal Cover, Full Normalisation & Schedules	167
4.13	Previous Year Questions (PYQ pattern)	170
5	System Software & Operating Systems	173
5.1	System Software	174
5.2	OS Basics	175
5.3	Process Management	176
5.4	Threads	178
5.5	CPU Scheduling	179
5.6	Deadlocks	180
5.7	Memory Management	182
5.8	Disk & Storage	185
5.9	File Systems & I/O	186
5.10	Protection & Security	187
5.11	Virtual Machines	187
5.12	Linux & Windows	188
5.13	Distributed Systems	188
5.14	Deeper Dive — More Scheduling, Semaphore Traces, Multilevel Paging & File Numericals	188
5.15	Previous Year Questions (PYQ pattern)	192
6	Software Engineering	195
6.1	Software Process Models	195
6.2	Requirements Engineering	199
6.3	Software Design	200

6.4	Software Quality	201
6.5	Estimation	203
6.6	Software Testing	204
6.7	Software Configuration Management	206
6.8	Maintenance & Re-engineering	207
6.9	Deeper Dive — Modelling Examples, Test Design, Estimation & Reliability Models	208
6.10	Previous Year Questions (PYQ pattern)	211
7	Data Structures & Algorithms	214
7.1	Asymptotic Analysis	214
7.2	Linear Data Structures	216
7.3	Trees	216
7.4	Hashing	219
7.5	Sorting	220
7.6	Searching	220
7.7	Algorithm Design Techniques	221
7.8	Graph Algorithms	222
7.9	Lower Bound Theory	224
7.10	Complexity Classes	224
7.11	Selected Topics	225
7.12	Advanced Algorithms	225
7.13	Deeper Dive — Traces You Must Be Able to Do by Hand	225
7.14	Previous Year Questions (PYQ pattern)	228
8	Theory of Computation & Compilers	231
8.1	Part A — Theory of Computation	231
8.2	Basics	231
8.3	Chomsky Hierarchy	232
8.4	Finite Automata	232
8.5	Regular Expressions	234
8.6	Context-Free Languages	235
8.7	Closure & Decision Properties (MOST ASKED TABLE)	236
8.8	Turing Machines	237
8.9	Undecidability	237
8.10	Part B — Compiler Design	238
8.11	Phases of a Compiler	238
8.12	Syntax Analysis (Parsing)	239
8.13	Syntax-Directed Translation	241
8.14	Run-Time Environment	242
8.15	Intermediate Code	243
8.16	Code Optimisation & Generation	244
8.17	Deeper Dive — Constructions & Parsing Traces	246
8.18	Previous Year Questions (PYQ pattern)	249
9	Data Communication & Computer Networks	252
9.1	Network Models	252
9.2	Signals & Data Rates	254
9.3	Digital & Analog Transmission	254
9.4	Multiplexing & Media	256
9.5	Switching	256

9.6	Data Link Layer	257
9.7	Medium Access Control	259
9.8	Network Layer	260
9.9	Transport Layer	262
9.10	Application Layer	264
9.11	Network Security	265
9.12	Mobile & Wireless	266
9.13	Cloud Computing & IoT	267
9.14	Deeper Dive — VLSM, Routing Tables, TCP Numbers, Hamming & Ciphers	268
9.15	Previous Year Questions (PYQ pattern)	271
10	Artificial Intelligence	274
10.1	Approaches to AI	274
10.2	State Space Search	275
10.3	Game Playing	278
10.4	Knowledge Representation	278
10.5	Planning	281
10.6	Natural Language Processing	281
10.7	Multi-Agent Systems	282
10.8	Fuzzy Sets & Logic (Zadeh, 1965)	282
10.9	Genetic Algorithms (John Holland, 1975)	284
10.10	Artificial Neural Networks	284
10.11	Deeper Dive — Worked Traces for Every Numerical Topic	287
10.12	Previous Year Questions (PYQ pattern)	290
III	REVISION	293
	Exam-Day Formula & Facts Sheet (UGC NET CS)	294
	Paper 1	294
	Discrete Mathematics	295
	Computer Architecture	295
	Programming & Graphics	295
	DBMS	296
	Operating Systems	296
	Software Engineering	296
	Algorithms	297
	TOC & Compilers	297
	Networks	297
	AI	298
	120-Day Study Plan for 250+ (Paper 1 + Paper 2)	299
	Overview	299
	Phase 1 — Concept Building (Day 1–60)	300
	Phase 2 — PYQs & Numerical Mastery (Day 61–90)	300
	Phase 3 — Mocks & Final Revision (Day 91–120)	300
	Score Tracker (fill in after every mock)	301
	Exam-Hall Strategy (3 hours, 150 questions, no negative marking)	301
	Recommended Standard Books (for depth where needed)	302
	Weekly Checklist	302

INTRODUCTION

Exam Pattern & the 250+ Strategy

...

Exam Pattern at a Glance

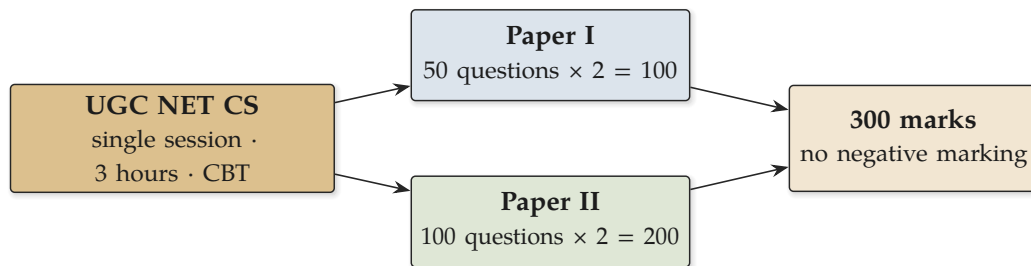


FIGURE 1. The examination at a glance: both papers are taken in one sitting, and a wrong answer costs nothing.

Item	Paper 1	Paper 2
Questions	50 (all compulsory)	100 (all compulsory)
Marks	100	200
Negative marking	None	None
Nature	Teaching/Research aptitude, reasoning	Core CS (10 units)

No negative marking → never leave a question blank.

What 250+ Really Means

$250 / 300 = 125$ correct out of 150 ($\approx 83\%$ accuracy).

Paper	Questions	Target correct	Marks	Allowed mistakes
Paper 1	50	42+	84+	8
Paper 2	100	84+	168+	16
Total	150	126	252	24

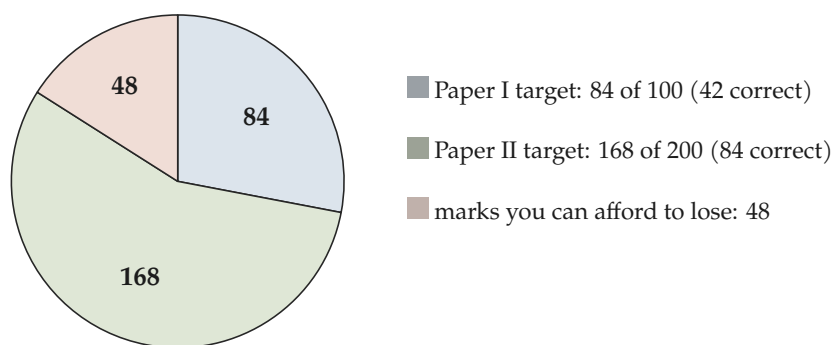


FIGURE 2. How 250+ out of 300 is built: about 83 % accuracy across both papers.

Where marks come from (approx. Paper 2 weightage — ~10 Qs per unit)

Unit	Expected Qs	Difficulty	Priority for 250+
Discrete Structures & Optimization	10–12	Medium	★★★
Computer System Architecture	8–10	Medium	★★★
Programming Languages & Graphics	8–10	Easy-Med	★★
DBMS	10–12	Easy-Med	★★★ (scoring)
System Software & OS	10–12	Medium	★★★ (scoring)
Software Engineering	8–10	Easy	★★★ (scoring, theory)
Data Structures & Algorithms	10–12	Medium	★★★
TOC & Compilers	10–12	Hard	★★★ (differentiator)
Networks	10–12	Medium	★★★
Artificial Intelligence	8–10	Medium	★★

Paper 1: roughly 5 questions per unit; **Data Interpretation, Reasoning, Comprehension** are “sure marks” (≈ 15 Qs) — aim for 100% there.

About the PYQs

The questions under “**Previous Year Questions (PYQ pattern)**” are reconstructed from the recurring question types and concepts asked in NTA UGC NET (2012–2025) and its predecessor CBSE NET papers. Numbers and wording may differ from the original papers, and year tags are deliberately left out where they could not be verified. Always pair these notes with the **official NTA question papers & answer keys** (ugcnet.nta.ac.in) for the last 5–6 cycles — solve them fully timed at least twice.

Golden Rules for 250+

1. **Paper 2 decides your score** — 2/3 of total marks. Spend ~70% of your time on it.
2. **Theory units are free marks** (SE, DBMS, OS, Networks, AI): learn definitions exactly.
3. **Numerical units are differentiators** (Discrete, COA, TOC, Algorithms): practise daily.

4. **PYQs repeat** — concepts recur cycle after cycle. Solve every PYQ in these files twice.
5. **Mocks**: at least 25 full-length mocks in the last 6 weeks; analyse every wrong answer.
6. **Attempt all 150** — no negative marking.

PART I

Paper I

Teaching & Research Aptitude

————— ... —————

Teaching Aptitude

Expected questions: ~5 (10 marks) · Target: 5/5

SYLLABUS COVERAGE

- ☐ Teaching: concept, objectives, levels (memory, understanding, reflective), characteristics & basic requirements
 - ☐ Learner's characteristics (adolescent & adult; academic, social, emotional, cognitive), individual differences
 - ☐ Factors affecting teaching (teacher, learner, support material, instructional facilities, environment, institution)
 - ☐ Methods of teaching: teacher-centred vs learner-centred; offline vs online (SWAYAM, SWAYAM-PRABHA, MOOCs)
 - ☐ Teaching support system: traditional, modern, ICT-based
 - ☐ Evaluation systems: elements & types; evaluation in CBCS; computer-based testing; innovations
-

1.1 What is Teaching?

Teaching is far more than "telling". A lecture delivered to an empty hall is not teaching, because nothing has changed in anyone's mind. We therefore judge teaching by its effect on the learner: has the student come to know, do or value something new? Seen this way, teaching is a *system* with inputs (the teacher's planning and personality, the content, the resources), a process (interaction in the classroom) and an output (learning). Examiners often test this systems view through the language of variables.

Teaching is a **purposeful, interactive process** that brings a desirable change in the learner's behaviour (knowledge, skills, attitudes).

Variable type	What it is	Example
Independent	Teacher (plans, organises, leads)	Teacher's planning
Dependent	Student (acts as per teacher)	Learner's performance
Intervening	Content, method, A/V aids, environment	Smart board

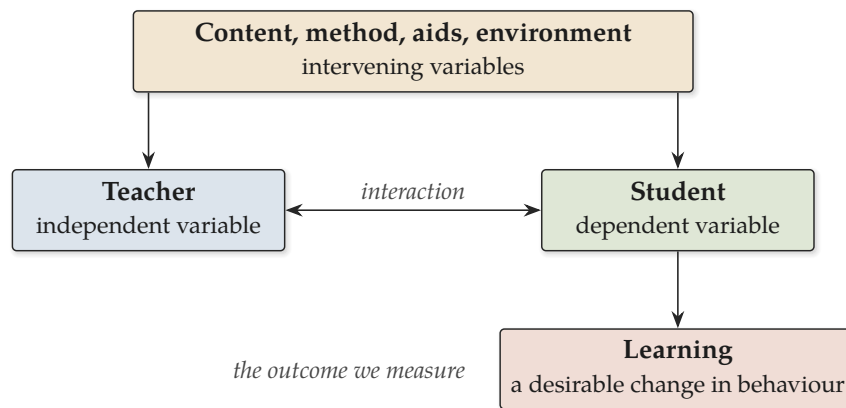


FIGURE 1.1. Teaching as a system of variables. The teacher acts, the learner responds, and everything that shapes the encounter — content, method, aids, environment — intervenes between the two.

1.1.1 Phases of Teaching (Philip W. Jackson)

Philip W. Jackson, observing real classrooms in *Life in Classrooms* (1968), noticed that a teacher's work does not begin when the bell rings or end when it rings again. Good teaching is planned beforehand, improvised sensitively during the lesson, and reviewed afterwards.

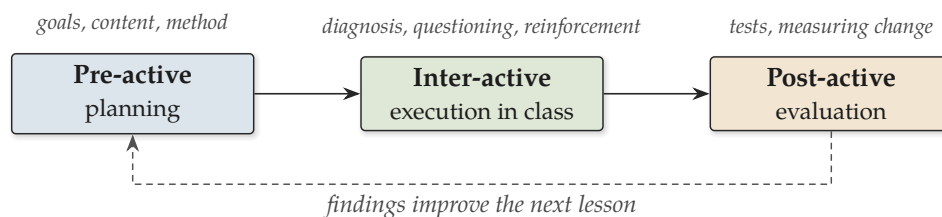


FIGURE 1.2. Jackson's three phases of teaching form a cycle: what is learnt in evaluation feeds the next plan.

- **Pre-active:** fixing goals, choosing content, deciding method.
- **Inter-active:** perception of class size, diagnosis, action (questioning, reinforcement).
- **Post-active:** test design, measuring changes, changing strategies.

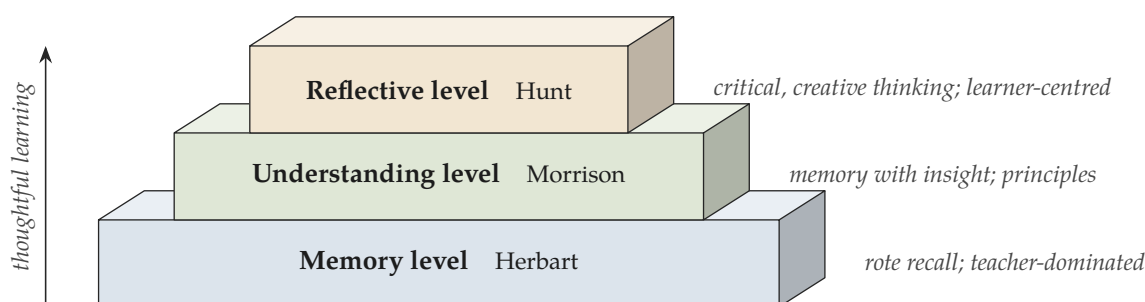


FIGURE 1.3. The three levels of teaching build on one another. Each level presupposes the one below it: there is no understanding without memory and no reflection without understanding.

1.1.2 Levels of Teaching (most asked!)

A useful way to remember the three levels is to ask *how much thinking the learner does*. At the memory level the learner reproduces; at the understanding level the learner sees relationships and principles; at the reflective level the learner questions, solves problems and creates. Each higher level contains the lower ones, which is why the levels are drawn as a stack rather than as alternatives.

Level	Proponent	Teacher role	Student role	Evaluation
Memory	Herbart	Dominant	Passive	Oral/written recall tests
Understanding	Morrison	Dominant	Semi-active	Objective + essay
Reflective	Hunt	Democratic/ secondary	Active	Essay, problem-solving

Herbartian 5 steps: **Preparation** → **Presentation** → **Comparison/Association** → **Generalisation** → **Application**. Morrison's steps: **Exploration** → **Presentation** → **Assimilation** → **Organisation** → **Recitation**.

1.1.3 Bloom's Taxonomy (Cognitive domain, revised 2001 by Anderson & Krathwohl)

Benjamin Bloom's committee (1956) classified educational objectives so that teachers and examiners could write questions at a deliberate level of difficulty. The 2001 revision made two changes that are examined repeatedly: the category names became **verbs** (what the learner *does*), and **Creating** replaced Synthesis at the very top, above Evaluating. When you meet a question that lists action verbs ("design", "justify", "classify"), map each verb to its level using the figure below.

Original (1956): Knowledge → Comprehension → Application → Analysis → **Synthesis** → **Evaluation**. Revised: Evaluation comes **before** Creating (Synthesis renamed Creating; nouns became verbs).

Three domains: **Cognitive** (Bloom), **Affective** (Krathwohl: Receiving → Responding → Valuing → Organisation → Characterisation), **Psychomotor** (Simpson / Dave: Imitation → Manipulation → Precision → Articulation → Naturalisation).

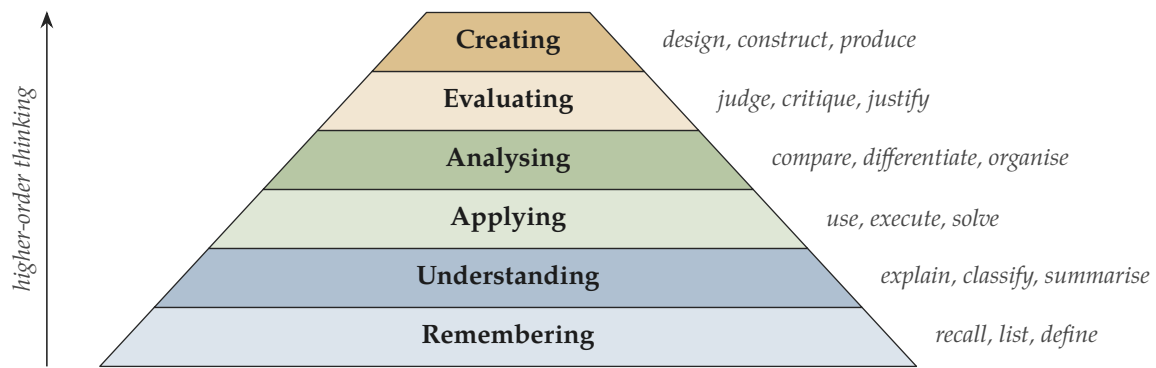


FIGURE 1.4. Revised Bloom's taxonomy (Anderson & Krathwohl, 2001). Each level uses verbs, not nouns, and *Creating* now sits above *Evaluating*.

1.1.4 Maxims of Teaching

Maxims are the time-tested rules of thumb that turn principles into classroom practice. Each one moves the learner from what is familiar and easy towards what is new and demanding — so the order of the pair is what examiners check (it is always *known* → *unknown*, never the reverse).

- Known → Unknown · Simple → Complex · Concrete → Abstract · Particular → General · Whole → Part
- Analysis → Synthesis · Empirical → Rational · Psychological → Logical · Induction → Deduction

1.2 Learner's Characteristics

No method works equally well for every learner. Questions in this area usually ask you to recognise which characteristic belongs to which group — for example, that adults are *self-directed* and use their *experience as a resource* (Malcolm Knowles's andragogy), or that adolescents are absorbed in forming an *identity* (Erik Erikson).

- **Pedagogy** = teaching children; **Andragogy** = teaching adults (Knowles); **Heutagogy** = self-determined learning.
- **Piaget stages**: Sensorimotor (0–2) → Pre-operational (2–7) → Concrete operational (7–11) → Formal operational (11+).
- **Vygotsky**: Zone of Proximal Development (ZPD), **scaffolding**, social constructivism.
- **Gardner** multiple intelligences: linguistic, logical-mathematical, spatial, musical, bodily-kinesthetic, interpersonal, intrapersonal, naturalistic (8).
- **VAK** learning styles: Visual, Auditory, Kinesthetic.

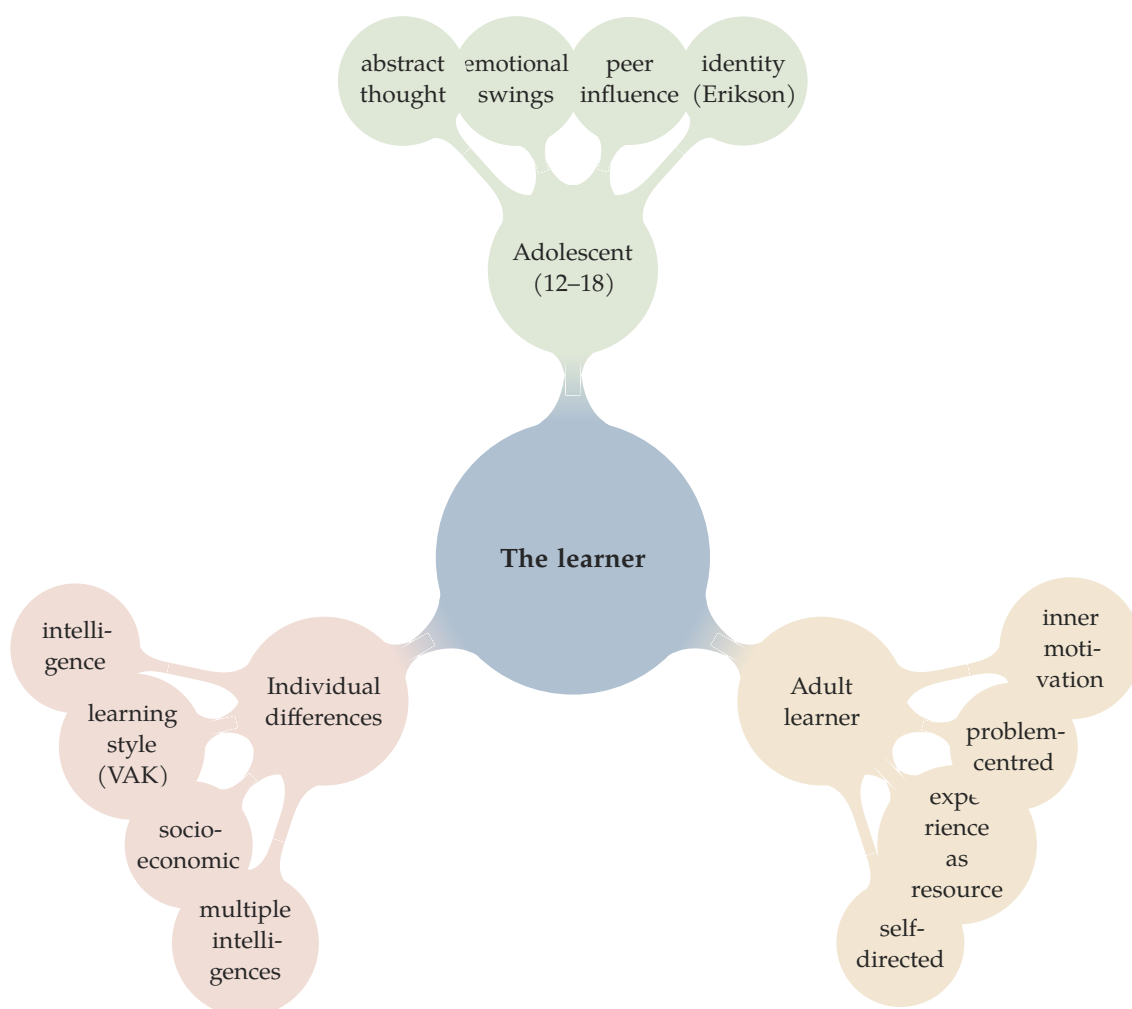


FIGURE 1.5. Learner characteristics. Adolescents are shaped by identity and peers; adults (andragogy) bring experience and self-direction; every class shows individual differences.

1.3 Factors Affecting Teaching

Teaching succeeds or fails through many interacting influences. It helps to group them as in the table: the people involved (teacher and learner), the materials and facilities they use, and the wider environment and institution in which they work. A question may describe a classroom problem and ask which factor is responsible.

Factor	Examples
Teacher	Subject knowledge, communication, personality, motivation
Learner	Aptitude, interest, motivation, readiness, prior knowledge
Support material	Textbooks, A/V aids, e-content
Instructional facilities	Labs, library, smart classroom
Learning environment	Class size, seating, light/ventilation, classroom climate
Institution	Policies, administration, culture

1.4 Methods of Teaching

The central distinction is *who is active*. In teacher-centred methods the teacher transmits and the learner receives; in learner-centred methods the learner investigates, discovers and constructs while the teacher guides. Modern policy (including NEP 2020) favours learner-centred and blended approaches, but every method has its place — a demonstration is still the quickest way to show a laboratory technique.

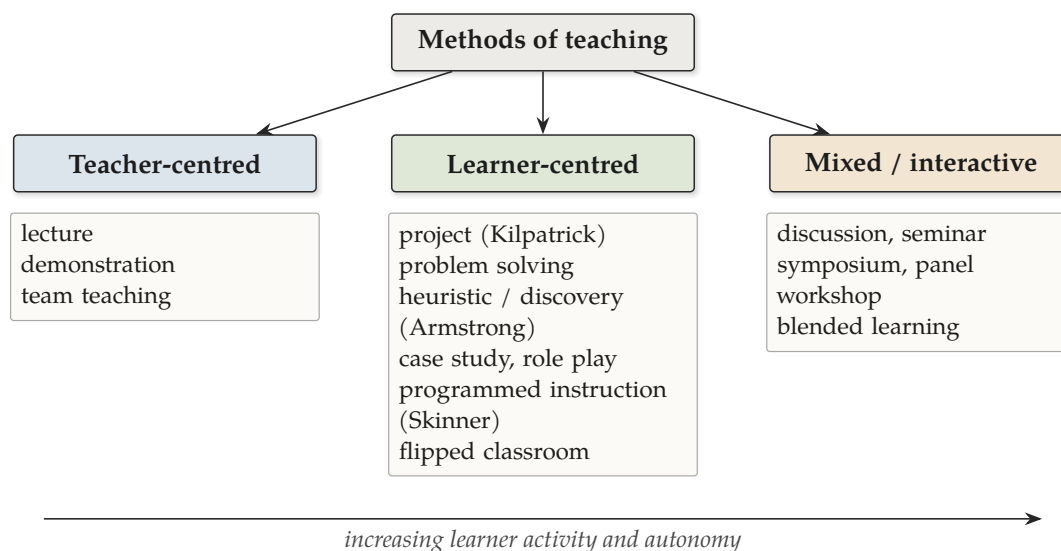


FIGURE 1.6. A map of teaching methods. Moving from left to right, responsibility for learning shifts from the teacher to the learner.

Method	Key idea	Proponent
Project method	Learning by doing, purposeful activity	W.H. Kilpatrick (based on Dewey)
Heuristic	Student as discoverer	H.E. Armstrong
Programmed instruction	Small steps, immediate feedback, linear	B.F. Skinner
Branching programming	Branches based on response	Norman Crowder
Dalton plan	Contract-based individual work	Helen Parkhurst
Montessori	Child-centred, didactic apparatus	Maria Montessori
Kindergarten	"Garden of children"	Froebel
Basic education (Nai Talim)	Craft-centred	Gandhi
Flipped classroom	Content at home, practice in class	Bergmann & Sams

Group discussion formats: Seminar (paper + discussion), Symposium (several speakers on aspects of one topic), Panel discussion (informal conversation among experts before audience), Workshop (hands-on).

1.4.1 Online Teaching — Government Initiatives (asked every cycle)

India has built a large public infrastructure for digital learning. Almost every NET paper contains at least one question on these initiatives, usually asking what a platform does or which body runs it. Learn the table as pairs: *platform* → *purpose* → *coordinating agency*.

Initiative	What it is
SWAYAM	Study Webs of Active-Learning for Young Aspiring Minds — MOOC platform (2017), 4 quadrants
SWAYAM 4 quadrants	e-Tutorial (video), e-Content (text), Self-assessment (quiz), Discussion forum
SWAYAM PRABHA	DTH educational channels (24×7), launched with 32, now 40+ channels, via GSAT-15
NPTEL	IITs + IISc engineering courses
NDL	National Digital Library (IIT Kharagpur)
e-PG Pathshala	PG e-content by UGC/INFLIBNET
e-Yantra	Robotics (IIT Bombay)
Shodhganga	Repository of Indian theses (INFLIBNET)
Shodhgangotri	Repository of synopses
DIKSHA	School teachers' platform (One Nation One Digital Platform)
Virtual Labs	MHRD remote labs
Spoken Tutorial	IIT Bombay software training
NMEICT	National Mission on Education through ICT
National coordinators of SWAYAM	AICTE, NPTEL, UGC, CEC, NCERT, NIOS, IGNOU, IIMB, NITTTR

MOOC: Massive Open Online Course. Term coined by Dave Cormier (2008). cMOOC (connectivist) vs xMOOC (extended, structured).

1.5 Teaching Support System

Support systems are the tools that carry a teacher's message. They range from the blackboard to virtual reality, and the choice among them should follow the learning objective, not fashion. Edgar Dale's cone, shown below, explains why concrete, hands-on experiences are remembered longer than words alone.

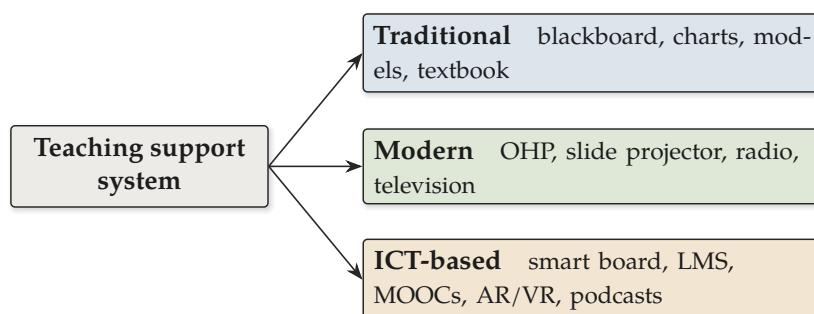


FIGURE 1.7. Three generations of teaching support, from chalk to cloud.

Edgar Dale's Cone of Experience (most concrete at base):

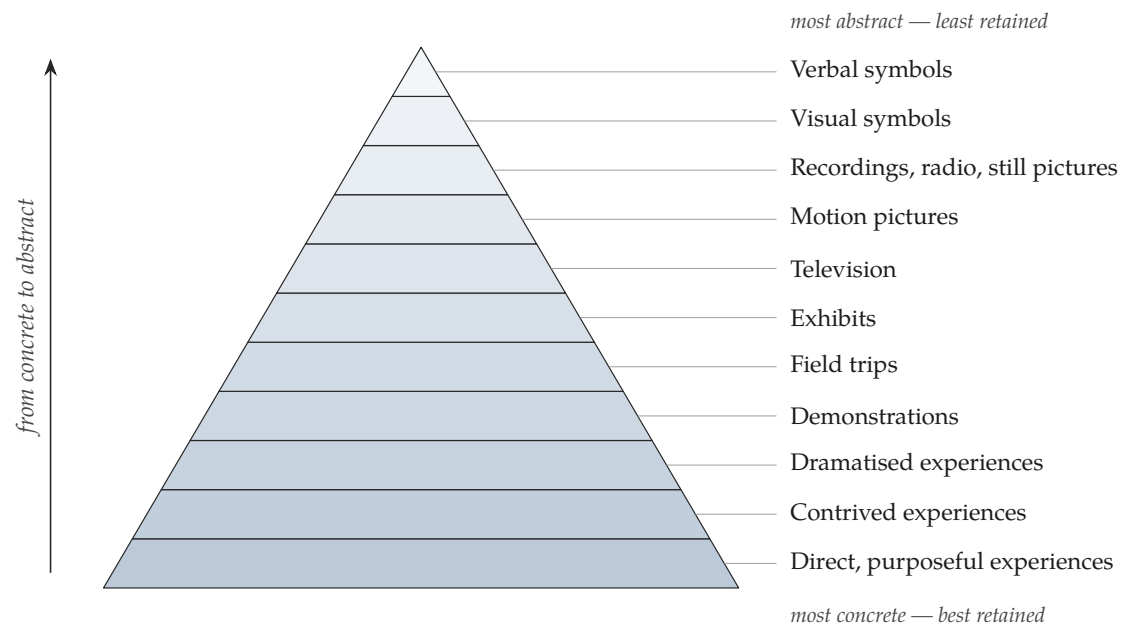


FIGURE 1.8. Edgar Dale's Cone of Experience. Learning is most durable at the broad base, where learners *do*; it becomes more abstract as we climb towards words and symbols.

People remember ~10% of what they read, ~90% of what they say and do.

Audio aids: radio, tape recorder, podcast. Visual: chart, map, blackboard. **Audio-visual:** TV, film, video. Projected aids: OHP, LCD; Non-projected: blackboard, charts, models.

1.6 Evaluation Systems

Evaluation answers two separate questions: *when* do we evaluate, and *against what* do we compare the result? Keeping these two axes apart removes most of the confusion in examination questions — a test can be both formative (timing) and criterion-referenced (interpretation).

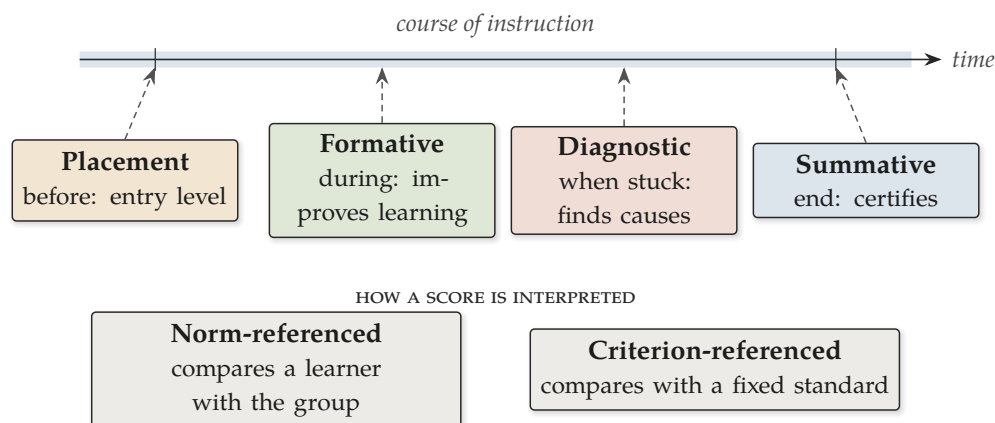


FIGURE 1.9. Types of evaluation arranged by *when* they occur (above) and by *how* results are interpreted (below).

Term	Meaning
Measurement	Quantitative description (numbers)
Assessment	Collecting information about learning
Evaluation	Measurement + value judgement (qualitative + quantitative)
CCE	Continuous & Comprehensive Evaluation — scholastic + co-scholastic
Reliability	Consistency of scores
Validity	Measures what it intends to measure (most important)
Objectivity	Free from examiner bias
Usability/Practicability	Easy to administer

Evaluation in CBCS: credit = 1 hr lecture/week per semester (or 2 hrs practical). Letter grades: O (10), A+ (9), A (8), B+ (7), B (6), C (5), P (4), F (0), Ab.

- $SGPA = \Sigma(C_i \times G_i) / \Sigma C_i$ (semester) • $CGPA = \Sigma(C_i \times S_i) / \Sigma C_i$ (across semesters)
- Percentage $\approx CGPA \times 9.5$ (UGC rule-of-thumb).
- CBCS course types: Core, Elective (Discipline-specific, Generic), Ability Enhancement (AECC, SEC).

Computer-based testing & innovations: CAT (Computer Adaptive Testing — difficulty adapts to answers), open-book exams, online proctoring, e-portfolios, rubrics, peer assessment, grading instead of marks, question banks.

1.7 Deeper Dive — Learning Theories, Micro-teaching & Teaching Models

1.7.1 Learning Theories (asked in almost every cycle)

A learning theory is an explanation of *how* learning happens. Behaviourists study only what can be observed and shaped by reinforcement; cognitivists look inside the mind at perception, memory and insight; constructivists hold that learners actively build their own understanding, often through social interaction; humanists emphasise the learner's needs, feelings and growth as a whole person.

Behaviourism <i>observable change</i>	Cognitivism <i>mental processing</i>	Constructivism <i>learner builds meaning</i>	Humanism <i>the whole person</i>
Pavlov — classical conditioning Thorndike — laws of learning Skinner — operant conditioning	Köhler — insight Bruner — discovery, spiral Ausubel — advance organisers	Piaget — cognitive stages Vygotsky — ZPD, scaffolding	Maslow — hierarchy of needs Rogers — learner-centred

FIGURE 1.10. Four families of learning theory and their principal thinkers.

Theorist	Key idea	Classroom implication
Pavlov	Classical conditioning: neutral stimulus + unconditioned stimulus → conditioned response (dog & bell)	Pleasant classroom associations reduce fear of a subject
Thorndike	Connectionism; laws of readiness, exercise, effect	Practice, rewards, readiness before teaching
Skinner	Operant conditioning; positive/negative reinforcement, punishment, shaping, schedules of reinforcement	Programmed instruction, immediate feedback
Kohler	Insight learning (chimpanzee Sultan) — sudden “aha” grasp of relationships	Present whole problems
Bruner	Discovery learning; modes: enactive → iconic → symbolic ; spiral curriculum	Revisit topics with increasing depth
Ausubel	Meaningful verbal learning; advance organisers	Start a lesson with an overview linking to prior knowledge
Bandura	Social learning / observational learning (Bobo doll); modelling; self-efficacy	Teacher as role model
Gagné	Conditions of learning; 8 types of learning; 9 events of instruction	Structured lesson design
Maslow	Physiological → safety → love/belonging → esteem → self-actualisation	Basic needs before academic growth

- **Negative reinforcement** = removing an unpleasant stimulus to *increase* behaviour (not punishment).
- Gagné's 9 events: gain attention → inform objectives → recall prior learning → present content → provide guidance → elicit performance → give feedback → assess performance → enhance retention and transfer.

1.7.2 Micro-teaching

Micro-teaching shrinks teaching down — fewer students, less time, a single skill — so that a trainee teacher can practise, receive precise feedback and try again immediately. It is the teacher-training counterpart of a musician practising one difficult passage rather than the whole piece.

Developed at **Stanford University (1961–63)** by **Dwight Allen** and colleagues. A scaled-down teaching encounter: one skill, 5–10 students, 5–10 minutes.

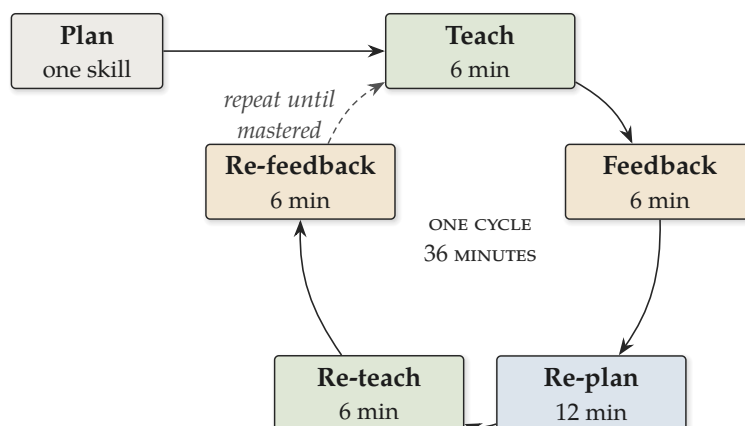


FIGURE 1.11. The micro-teaching cycle (Indian model): a single skill is taught to a small group, critiqued, re-planned and taught again.

- Indian model (NCERT/Passi): total cycle **36 minutes**.
- Skills: introducing a lesson, questioning (fluency, probing), explaining, illustrating with examples, **reinforcement**, stimulus variation, blackboard writing, closure, using A/V aids.
- **Simulated teaching** = role play of teaching in a training situation; **team teaching** = two or more teachers plan and teach together.

1.7.3 Models of Teaching

A model of teaching is a complete plan for a type of lesson: its goals, the sequence of activities, the roles of teacher and learner, and how success will be judged. Bruce Joyce and Marsha Weil grouped the many models into four families according to the kind of learning they aim at.

Model	Proponent	Key steps / idea
Basic Teaching Model	Robert Glaser (1962)	Instructional objectives → Entering behaviour → Instructional procedures → Performance assessment (with feedback loop)
Concept Attainment	Jerome Bruner	Examples & non-examples → identify concept attributes
Inductive Thinking	Hilda Taba	Concept formation → interpretation of data → application
Advance Organiser	David Ausubel	Organiser presented before content
Inquiry Training	Richard Suchman	Puzzling event → students ask yes/no questions → form theories
Jurisprudential	Oliver & Shaver	Debating public issues
Synectics	William Gordon	Creativity through metaphors & analogies

Families of teaching models (Joyce & Weil): Information processing, Social interaction, Personal, Behavioural systems.

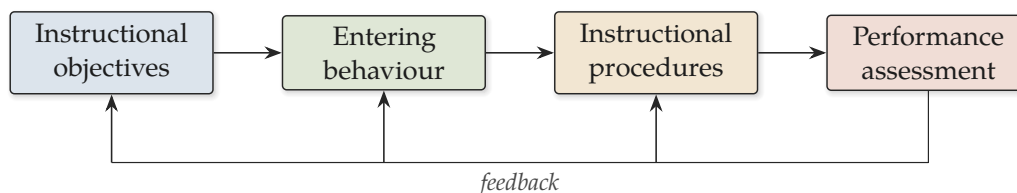


FIGURE 1.12. Glaser's Basic Teaching Model. Assessment feeds back into every earlier component.

1.7.4 Questioning & Classroom Management

Questions are the teacher's most flexible tool: they check understanding, provoke thinking and keep a class involved. Classroom management, in turn, creates the conditions in which questions can be asked and answered without disorder.

- **Convergent** questions (one correct answer, lower order) vs **divergent** (many answers, creativity).
- **Wait time** (Mary Budd Rowe): waiting 3–5 seconds after a question improves answer quality.
- **Kounin**: "withitness" (teacher aware of everything), overlapping, momentum, smoothness.
- Teacher leadership styles (Lewin): **autocratic**, **democratic**, **laissez-faire** — democratic is best for learning climate.
- **Hidden curriculum**: values and norms learnt implicitly from school culture.

1.8 Previous Year Questions (PYQ pattern)

Levels of teaching

1. The "reflective level" of teaching is associated with: (a) Herbart (b) Morrison (c) Hunt (d) Skinner — **Ans: (c)**
2. Which level of teaching is the most "thoughtless"? (a) Memory (b) Understanding (c) Reflective (d) Autonomous — **Ans: (a)**
3. Arrange the levels from lowest to highest: **Memory → Understanding → Reflective.**

Bloom's taxonomy

4. In revised Bloom's taxonomy the highest cognitive level is: (a) Evaluating (b) Creating (c) Analysing (d) Synthesis — **Ans: (b)**
5. "Characterisation by a value" belongs to which domain? — **Ans: Affective (Krathwohl)**
6. Match: Remembering–recall; Applying–use; Analysing–differentiate; Evaluating–judge. (Standard match question.)

Learner characteristics

7. Andragogy refers to: (a) teaching children (b) teaching adults (c) self-learning (d) group learning — **Ans: (b)**
8. ZPD is a concept given by: **Vygotsky**

9. Which is NOT a characteristic of adult learners? (a) self-directed (b) experience-based (c) externally driven (d) problem-centred — **Ans: (c)**

Methods

10. Programmed instruction is based on the theory of: (a) Classical conditioning (b) Operant conditioning (c) Insight (d) Constructivism — **Ans: (b) Skinner**
11. "Learning by doing" is the core of: **Project method (Kilpatrick)**
12. Which is a learner-centred method? (a) Lecture (b) Demonstration (c) Problem solving (d) Team teaching — **Ans: (c)**
13. In a flipped classroom, students first encounter new content: **at home (videos/readings)**

Online initiatives

14. SWAYAM PRABHA provides: (a) MOOCs (b) 24x7 DTH educational channels (c) Digital library (d) Thesis repository — **Ans: (b)**
15. How many quadrants does a SWAYAM course have? — **Ans: 4**
16. Shodhganga is maintained by: **INFLIBNET (Gandhinagar)**
17. The national coordinator for engineering courses on SWAYAM: **NPTEL**

Evaluation

18. Evaluation that takes place during the teaching-learning process is: **Formative**
19. A test that measures what it intends to measure is: **Valid**
20. Consistency of test scores refers to: **Reliability**
21. Norm-referenced test compares a student with: **performance of peer group**
22. Diagnostic evaluation is meant to: **identify learning difficulties**
23. In CBCS, the grade point for "O" (Outstanding) is: **10**

Teaching aids

24. In Edgar Dale's Cone, the most concrete experience is: **Direct purposeful experience**
25. Which is a non-projected aid? (a) LCD (b) OHP (c) Blackboard (d) Slide — **Ans: (c)**

Statement-based (common format)

26. Statement I: Effective teaching is learner-centred. Statement II: The teacher's role is only to transmit content. — **I true, II false**

More practice questions

27. Learning by insight was demonstrated by: **Wolfgang Kohler**
28. "Law of effect" is associated with: **E.L. Thorndike**
29. Advance organisers were proposed by: **David Ausubel**
30. Bruner's three modes of representation in order: **Enactive, Iconic, Symbolic**
31. Removing an unpleasant stimulus to increase a desired behaviour is: **negative reinforcement**
32. Micro-teaching was developed at: **Stanford University**
33. Duration of one micro-teaching cycle in the Indian model: **36 minutes**
34. The basic teaching model was given by: **Robert Glaser**

35. The first component of Glaser's model is: **instructional objectives**
36. Concept attainment model is associated with: **Jerome Bruner**
37. A question with several acceptable answers is: **divergent**
38. Observational (social) learning theory was given by: **Albert Bandura**
39. In Maslow's hierarchy, the highest need is: **self-actualisation**
40. Which leadership style creates the most positive classroom climate? **Democratic**
41. Statement I: Formative evaluation is diagnostic in nature. Statement II: Summative evaluation is used for certification. — **Both true**
42. Match: Kilpatrick–Project method; Skinner–Programmed learning; Armstrong–Heuristic; Parkhurst–Dalton plan.
43. Which is NOT a characteristic of effective teaching? (a) learner involvement (b) feedback (c) rote memorisation (d) use of examples — **Ans: (c)**

POINTS TO REMEMBER

- Memory–Herbart · Understanding–Morrison · Reflective–Hunt
- Formative = during; Summative = end; Diagnostic = difficulties; Placement = before
- Validity > Reliability (valid test is always reliable, not vice versa)
- SWAYAM (2017, MOOCs, 4 quadrants) · SWAYAM PRABHA (DTH channels, 32 at launch → 40+) · Shodhganga (theses, INFLIBNET)

Research Aptitude



Expected questions: ~5 (10 marks) · Target: 5/5

SYLLABUS COVERAGE

- ☐ Research: meaning, types, characteristics; positivism & post-positivistic approach
 - ☐ Methods: experimental, descriptive, historical, qualitative, quantitative
 - ☐ Steps of research
 - ☐ Thesis & article writing: format and styles of referencing
 - ☐ Application of ICT in research
 - ☐ Research ethics
-

2.1 Meaning & Characteristics

Research is curiosity disciplined by method. Everyday knowledge comes from habit, authority or intuition; research knowledge comes from evidence gathered in a way that another person could check and repeat. That is why the defining words are *systematic* (it follows a plan), *empirical* (it rests on observation) and *replicable* (others can verify it).

Research = **systematic, objective, controlled, empirical, critical** investigation to discover new facts or verify old ones. (*Re + search* = search again.) Characteristics: systematic, logical, empirical, replicable, reductive, controlled, generalisable.

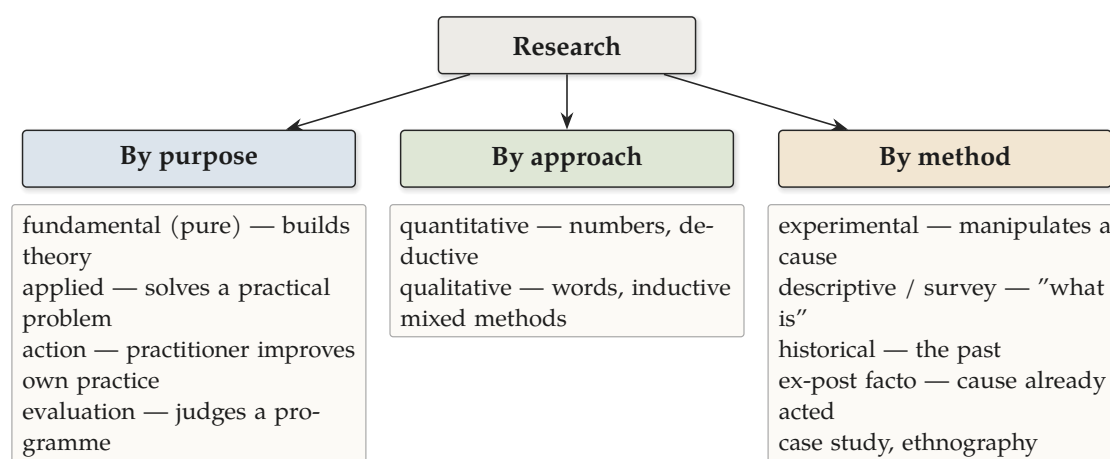


FIGURE 2.1. Three independent ways of classifying research. A single study can be, for example, applied in purpose, quantitative in approach and experimental in method.

2.2 Types of Research

Students often memorise the types as one long list and then confuse them. It is clearer to see that research is classified along three separate axes — its *purpose*, its broad *approach* and its specific *method* — and that each study takes one value on every axis.

Type	Key feature	Remember
Fundamental	Adds to knowledge/theory	Not immediately applicable
Applied	Solves immediate practical problem	Product-oriented
Action	Done by practitioner (teacher) to improve own practice; cyclic	Coined by Kurt Lewin; in education by Stephen Corey
Experimental	Manipulation of IV + control	Most scientific, cause-effect
Ex-post facto	IV already occurred, no manipulation	"After the fact"
Historical	Past events; primary & secondary sources	External criticism (authenticity) & internal criticism (credibility)
Descriptive	"What is" — survey, correlational	No manipulation
Ethnography	Culture studied by immersion	Participant observation
Grounded theory	Theory emerges from data	Glaser & Strauss
Phenomenology	Lived experiences	Husserl

2.2.1 Positivism vs Post-positivism

Behind every method lies a philosophy about what reality is and how we can know it. Positivism, founded by Auguste Comte, treats the social world like the physical world: objective, measurable and governed by laws. Interpretivist and post-positivist traditions reply that human behaviour carries meanings that must be *understood* from the inside, not merely counted.

Positivism (Auguste Comte)	Post-positivism / Interpretivism
Reality is objective, single	Reality is multiple, constructed
Quantitative, deductive	Qualitative (or critical realism)
Researcher detached	Researcher involved
Hypothesis testing, generalisation	Understanding meaning (<i>Verstehen</i> – Weber)

Paradigm (Thomas Kuhn) = ontology (nature of reality) + epistemology (nature of knowledge) + methodology.

2.3 Variables & Hypothesis

A variable is any characteristic that can take different values — marks, attitude, income. Research tries to explain how changes in one variable go with changes in another. A hypothesis states this expected relationship in a form that data can support or contradict, and the logic of testing it leads to the two kinds of error shown in the decision matrix below.

	H_0 IS TRUE	H_0 IS FALSE
RETAIN H_0	Correct decision confidence $1 - \alpha$	Type II error probability β
REJECT H_0	Type I error probability α	Correct decision power $1 - \beta$

FIGURE 2.2. The four outcomes of a hypothesis test. A Type I error convicts the innocent null hypothesis; a Type II error lets a false one go free.

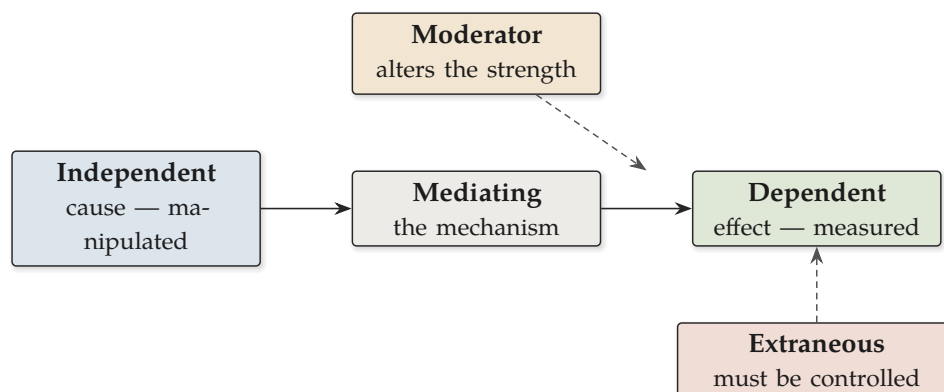


FIGURE 2.3. How variables relate in a study. Example: teaching method (independent) raises motivation (mediating), which raises achievement (dependent); the effect may be stronger for younger pupils (moderator) and can be distorted by prior knowledge (extraneous).

- **Hypothesis** = tentative, testable statement about relationship between variables.
- **Null hypothesis (H_0)**: no relationship/difference. **Alternative (H_1)**: there is.
- **Type I error (α)**: rejecting a TRUE null. **Type II error (β)**: accepting a FALSE null. Power = $1 - \beta$.
- Directional (one-tailed) vs non-directional (two-tailed).

2.4 Steps of Research

The steps below are the skeleton of every thesis and research proposal. Questions usually present the steps jumbled and ask for the correct order; remember that the problem comes first, the hypothesis comes only after the literature has been reviewed, and the report comes last.

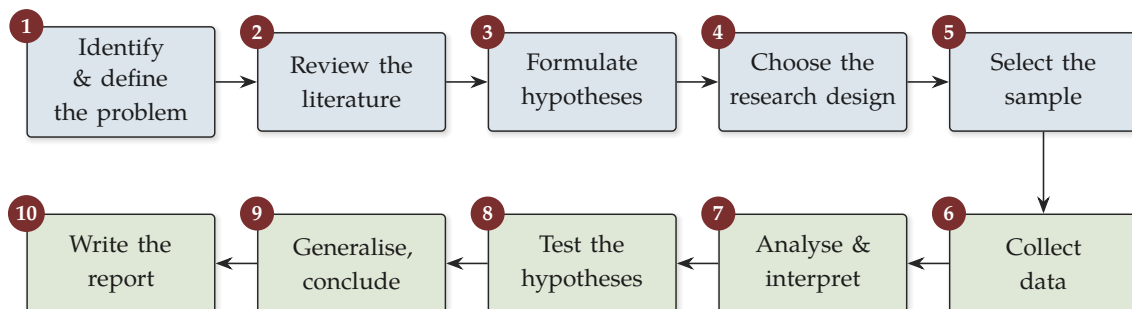


FIGURE 2.4. The research process as a sequence of ten steps. In practice the steps overlap and the researcher often loops back — for example, from the literature review to a sharper statement of the problem.

2.5 Sampling

Studying every member of a population is rarely possible, so researchers study a sample and generalise. The quality of that generalisation depends on how the sample was drawn. If each unit had a known chance of selection, statistics can tell us how far the sample might differ from the population; otherwise we rely on judgement.

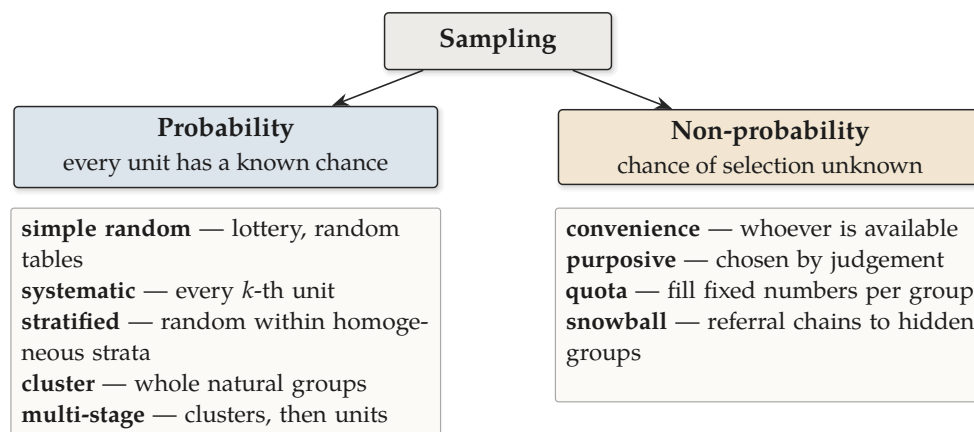


FIGURE 2.5. Sampling methods. Only probability sampling allows the sampling error to be estimated and the results to be generalised statistically.

Tools of data collection: questionnaire (closed/open), interview (structured, unstructured), observation (participant, non-participant), schedule (filled by enumerator), rating scales (**Likert** –

5-point agree/disagree; **Thurstone** – equal-appearing intervals; **Guttman** – cumulative; **Semantic differential** – Osgood; bipolar adjectives).

Scales of measurement (Stevens): Nominal (labels) → Ordinal (rank) → Interval (no true zero, e.g. °C) → Ratio (true zero, e.g. weight).

Parametric tests (normal data): t-test, z-test, ANOVA (F), Pearson r. **Non-parametric:** Chi-square, Mann-Whitney U, Wilcoxon, Kruskal-Wallis, Spearman rho, sign test.

2.6 Thesis & Article Writing

Writing is part of research, not an afterthought: findings that are not reported clearly cannot be checked, cited or used. Academic writing follows conventional structures so that readers know where to find the problem, the method, the evidence and the conclusions.

Thesis format: Preliminary pages (title, declaration, certificate, acknowledgement, contents, list of tables) → Main body (Introduction, Review of Literature, Methodology, Results, Discussion, Conclusion) → References → Appendices.

IMRAD structure of research article: Introduction, Methods, Results, And Discussion.

Style	Used in	Example in-text
APA (American Psychological Assoc.)	Social sciences, education	(Sharma, 2020) — author-date
MLA (Modern Language Assoc.)	Humanities, literature	(Sharma 45) — author-page
Chicago	History; notes-bibliography or author-date	Footnotes
Harvard	General author-date	(Sharma 2020)
IEEE	Engineering, CS	[1] numbered
Vancouver	Medicine	numbered

- *Ibid.* = same source as previous note; *Op. cit.* = work cited earlier (different page); *Loc. cit.* = same place cited earlier; *et al.* = and others.
- **Bibliography** = all sources consulted; **References** = only those cited.
- **Abstract:** ~150–300 words, written last, placed first. **Annotated bibliography** includes short summaries.

2.7 ICT in Research

Information technology now touches every stage of research — finding literature, managing references, collecting data online, analysing it statistically, checking originality and publishing. The exam expects you to match each task with a representative tool.

Need	Tool
Literature search	Google Scholar, Scopus, Web of Science, PubMed, Shodhganga, INFLIBNET N-LIST, e-ShodhSindhu
Reference management	Mendeley, Zotero, EndNote
Statistical analysis	SPSS, R, SAS, STATA, Excel
Qualitative analysis	NVivo, ATLAS.ti, MAXQDA
Plagiarism detection	Shodhshuddhi (DrillBit/Ouriginal-URKUND) , Turnitin, iThenticate
Writing	LaTeX, MS Word
Unique researcher ID	ORCID, Scopus Author ID, ResearcherID
Metrics	Impact Factor (Clarivate JCR), h-index (Hirsch), i10-index (Google Scholar), CiteScore (Scopus)

h-index: a researcher has index h if h papers have $\geq h$ citations each. **Impact factor (2024)** = citations in 2024 to items published in 2022–23 $\div$ citable items published in 2022–23.

2.8 Research Ethics

Ethics protects both the people who take part in research and the trust society places in its results. The core duties are honesty in reporting, respect for participants (informed consent, privacy, no harm) and fair credit to others' work.

- **Plagiarism:** UGC Regulations 2018 (Promotion of Academic Integrity and Prevention of Plagiarism):

Level	Similarity	Penalty (for students)
Level 0	up to 10%	Minor, no penalty
Level 1	10–40%	Resubmit within 6 months
Level 2	40–60%	Debarred from resubmission for 1 year
Level 3	> 60%	Registration cancelled

- Other ethics: informed consent, confidentiality, anonymity, no fabrication / falsification, honest authorship, avoid **predatory journals**; **UGC-CARE** list of quality journals (2019).
- **Salami slicing:** splitting one study into many papers. **Duplicate publication** is unethical.
- **COPE:** Committee on Publication Ethics.

2.9 Deeper Dive — Research Designs, Reliability, Validity & Statistics

2.9.1 Experimental Designs (Campbell & Stanley notation: *R* = random assignment, *O* = observation, *X* = treatment)

An experiment is persuasive only if alternative explanations have been ruled out. Designs differ in how well they do this: random assignment (*R*) and a control group are the two features that move a design from *pre-experimental* to *true experimental*.

Design	Notation	Notes
One-shot case study (pre-experimental)	X O	No comparison; weakest
One-group pre-test post-test	O X O	No control group
Static group comparison	X O / — O	Non-random groups
Pre-test post-test control group (true)	R O X O / R O — O	Classic true experiment
Post-test only control group	R X O / R — O	Avoids testing effect
Solomon four-group	Combines both above (4 groups)	Controls pre-test sensitisation; strongest
Quasi-experimental: non-equivalent control group	O X O / O — O	Intact groups (e.g. two existing classes)
Time-series	O O O X O O O	Repeated observations
Factorial design	2 × 2, 2 × 3 ...	Two or more IVs and their interaction

Threats to internal validity: history, maturation, testing, instrumentation, statistical regression, selection bias, experimental mortality (attrition). **External validity** = generalisability; threats: reactive effects (Hawthorne effect), interaction of selection and treatment.

2.9.2 Reliability & Validity

A measuring tool must give the same result when nothing has changed (reliability) and must measure the thing it claims to measure (validity). The target figure shows why the first is necessary but not sufficient for the second.

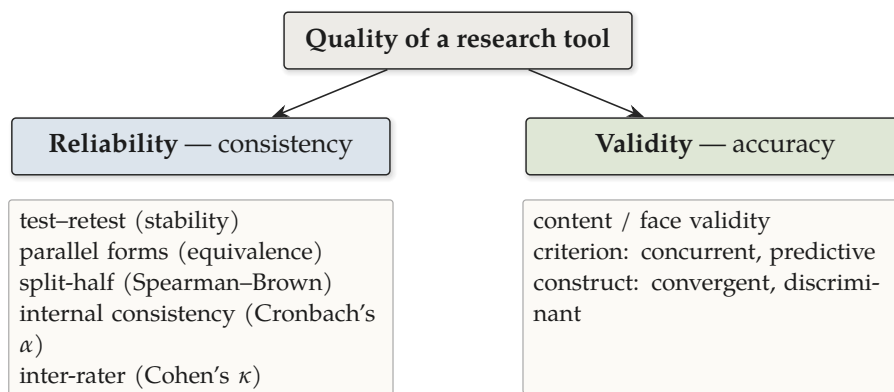


FIGURE 2.6. The two pillars of a good measuring instrument and the ways each is established.

- Reliability is necessary but **not sufficient** for validity. A valid test must be reliable.

- **Spearman–Brown:** full-test reliability $r_{\text{full}} = 2r_{\text{half}} / (1 + r_{\text{half}})$.
- Cronbach's $\alpha \geq 0.7$ is usually acceptable.

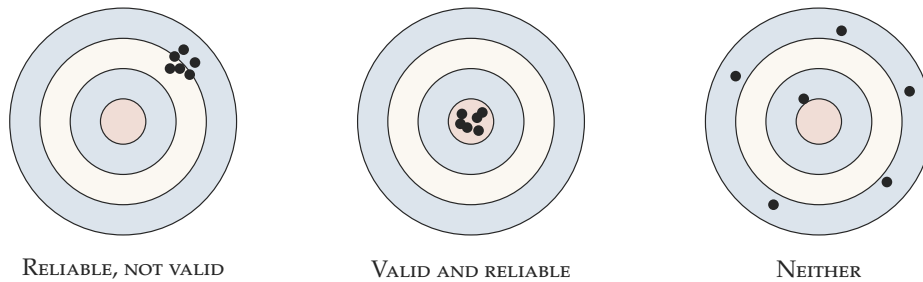


FIGURE 2.7. The target analogy. Reliability is a tight grouping of shots; validity is a grouping centred on the bull's-eye. A tool can be reliable without being valid, but a valid tool must be reliable.

2.9.3 Statistics for Researchers

Statistics summarise data and tell us how much confidence to place in a result. For the exam you need the meaning of the common measures and the shape of the normal distribution, on which most parametric tests rest.

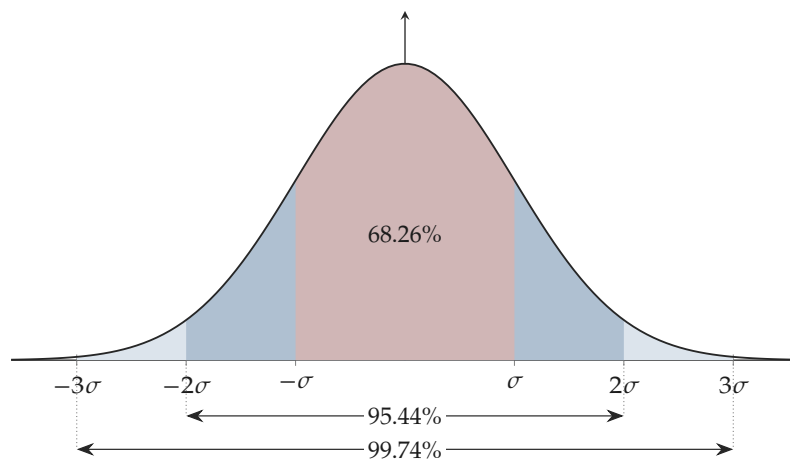


FIGURE 2.8. The normal curve: mean, median and mode coincide at the centre, and fixed proportions of cases lie within one, two and three standard deviations.

Concept	Key point
Correlation r (Pearson)	$-1 \leq r \leq +1$; $r = 0 \rightarrow$ no linear relation; correlation $\neq$ causation
Spearman rank ρ	$\rho = 1 - 6\sum d^2 / (n(n^2 - 1))$ for ordinal data
Coefficient of determination	r^2 = proportion of variance explained
Regression	Predicts Y from X : $Y = a + bX$
Level of significance	$\alpha = 0.05$ or 0.01 (probability of Type I error)
Degrees of freedom	t-test (one sample) $n - 1$; chi-square $(r - 1)(c - 1)$
Normal curve	Mean = median = mode; 68.26% within $\pm 1\sigma$, 95.44% within $\pm 2\sigma$, 99.74% within $\pm 3\sigma$
Skewness	Positive: mean $>$ median $>$ mode; Negative: mean $<$ median $<$ mode
Kurtosis	Leptokurtic (peaked), Mesokurtic (normal), Platykurtic (flat)

Worked (Spearman ρ): $n = 5, \Sigma d^2 = 4 \rightarrow \rho = 1 - (6 \times 4) / (5 \times 24) = 1 - 0.2 = 0.8$.

2.9.4 Research Proposal (Synopsis) Structure

A proposal is a promise: it tells a committee what will be studied, why it matters and how it will be done, before any data are collected. Its sections mirror the steps of research. Title → Introduction & background → Review of literature & research gap → Statement of the problem → Objectives → Hypotheses → Methodology (design, population, sample, tools, analysis) → Delimitations → Significance → Chapterisation → Time-frame → References.

- **Delimitations** = boundaries set by the researcher; **Limitations** = constraints beyond the researcher's control.
- **Operational definition** = defining a variable in terms of how it is measured.

2.10 Previous Year Questions (PYQ pattern)

Types of research

1. A teacher conducts research to solve a problem of absenteeism in her class. It is: **Action research**
2. Research aimed at developing theory is: **Fundamental**
3. Study of the effect of an event that has already happened without manipulation: **Ex-post facto**
4. "Who coined action research?" — **Kurt Lewin** (in education popularised by Stephen Corey)
5. In historical research, testing authenticity of a source is: **External criticism**

Positivism

6. Positivism was proposed by: **Auguste Comte**
7. Which is associated with qualitative research? (a) generalisation (b) hypothesis testing (c) thick description (d) large samples — **Ans: (c)**

Hypothesis & errors

8. Rejecting a true null hypothesis is: **Type I error**
9. Hypothesis that states "no difference" is: **Null hypothesis**
10. The variable manipulated by the researcher: **Independent variable**

Sampling

11. Studying drug users via referral chains: **Snowball sampling**
12. Population divided into homogeneous subgroups then randomly sampled: **Stratified random**
13. Which is a non-probability method? (a) cluster (b) systematic (c) quota (d) stratified — **Ans: (c)**

Scales & tools

14. Temperature in °C is measured on which scale? **Interval**
15. A 5-point agree–disagree scale is: **Likert scale**

16. Chi-square test is: **Non-parametric**

Writing & referencing

17. "Ibid." is used when: **citing the same source as the immediately preceding citation**

18. APA style follows: **author–date system**

19. The abstract of a thesis is written: **at the end, placed at the beginning**

20. Correct sequence of research steps: **Problem → Literature → Hypothesis → Design → Data → Analysis → Report**

ICT & Ethics

21. UGC's plagiarism software made available to universities: **Shodhshuddhi (URKUND/Original, later DrillBit)**

22. As per UGC 2018 regulations, similarity up to 10% is: **Level 0 (no penalty)**

23. h-index was proposed by: **J.E. Hirsch (2005)**

24. Which identifier uniquely identifies a researcher? **ORCID**

25. Reference management software: **Mendeley / Zotero**

More practice questions

26. The design that controls the effect of pre-testing using four groups: **Solomon four-group design**

27. Using two intact classes without random assignment is a: **quasi-experimental design**

28. Improvement in performance because subjects know they are being observed: **Hawthorne effect**

29. Consistency of scores over time is measured by: **test-retest reliability**

30. Cronbach's alpha measures: **internal consistency**

31. Split-half reliability is corrected by: **Spearman–Brown formula**

32. A test predicting future job performance has: **predictive validity**

33. In a normal distribution, the percentage of cases within $\pm 1\sigma$: **68.26%**

34. When mean > median > mode, distribution is: **positively skewed**

35. Degrees of freedom for a 3×4 contingency table chi-square: **6**

36. Boundaries set by the researcher on the scope of study: **delimitations**

37. $r = -0.9$ indicates: **strong negative correlation**

38. Which is NOT a threat to internal validity? (a) history (b) maturation (c) generalisability (d) attrition — **Ans: (c)**

39. Defining "academic achievement" as "marks obtained in the annual exam" is an: **operational definition**

POINTS TO REMEMBER

- Action research → practitioner, local, immediate · Ex-post facto → no manipulation
- Type I = reject true H_0 (α) · Type II = accept false H_0 (β)
- NOIR = Nominal, Ordinal, Interval, Ratio
- Plagiarism levels: 0 (≤ 10), 1 (10–40), 2 (40–60), 3 (> 60)

Comprehension

Expected questions: 5 (one passage) · Target: 5/5 — these are sure-shot marks

3.1 What Is Asked

Comprehension tests the oldest academic skill of all: reading closely and reasoning only from what the text actually says. It needs no subject knowledge, which is exactly why careless answers are common — an option can sound sensible and still be unsupported by the passage. A passage of ~400–600 words followed by **5 questions** on: central idea, title, author's tone, inference, vocabulary in context, factual detail, and "which statement is true/false".

3.2 Step-by-Step Strategy

Reading the questions first tells you what to look for, so the first reading of the passage becomes purposeful rather than passive. Treat each question type differently, as the figure shows: a factual question is answered by locating a line, while an inference must follow necessarily from several lines.

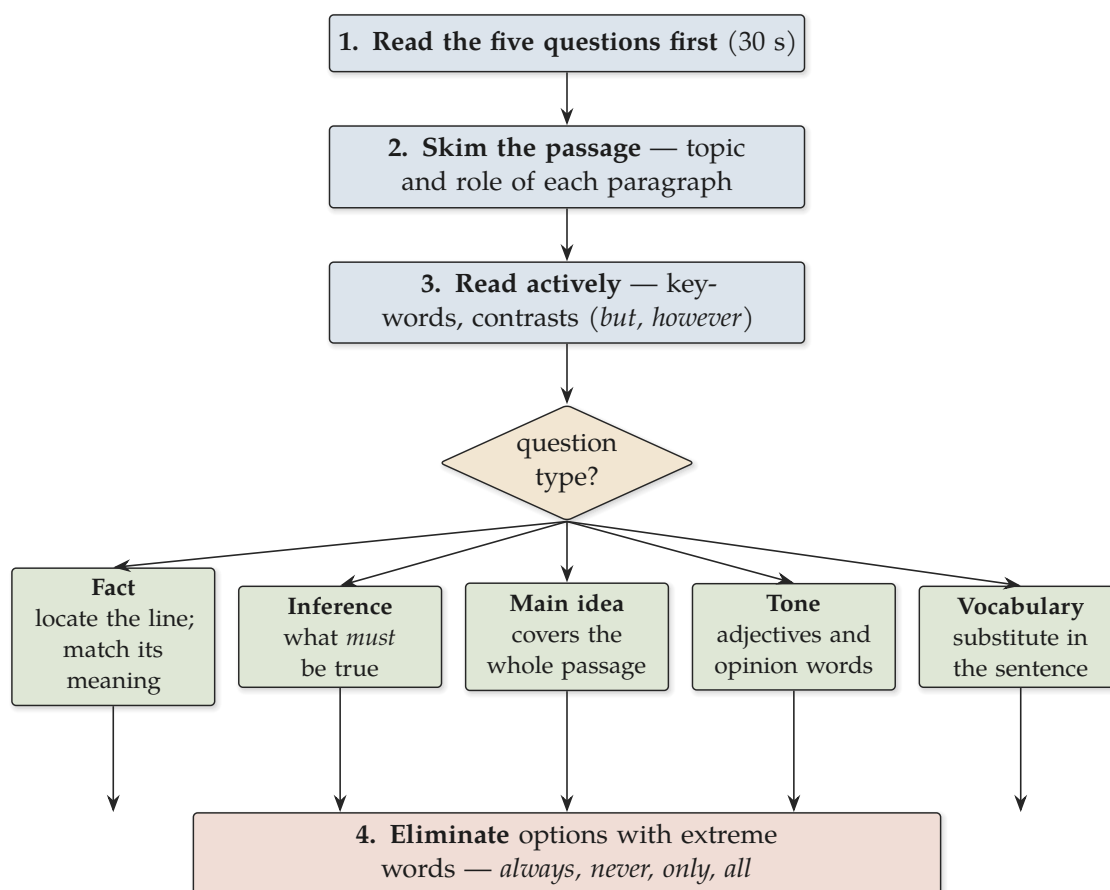


FIGURE 3.1. A reliable routine for reading-comprehension questions.

3.3 Question Types & How to Crack Them

Type	Signal words	Trick
Main idea	"central theme", "primarily concerned"	Answer must cover the whole passage; avoid too narrow/broad
Title	"suitable title"	Short, covers the main idea, often from first/last paragraph
Fact	"according to the passage"	Answer is <i>stated</i> ; paraphrase of a line
Inference	"it can be inferred", "implies"	Must logically follow; no outside knowledge
Tone/attitude	"author's tone"	Critical, analytical, sarcastic, optimistic, pessimistic, neutral, didactic
Vocabulary	"the word X means"	Meaning in <i>this context</i>
Assumption	"the author assumes"	Unstated premise the argument depends on
Not true / except	"which is NOT"	Verify each option in passage

3.4 Tone Words Cheat Sheet

Tone is the author's attitude towards the subject. Decide first whether the author approves, stays detached or disapproves; only then choose the precise word. Most passages in this examination are analytical or balanced, so strongly emotional options are usually wrong.

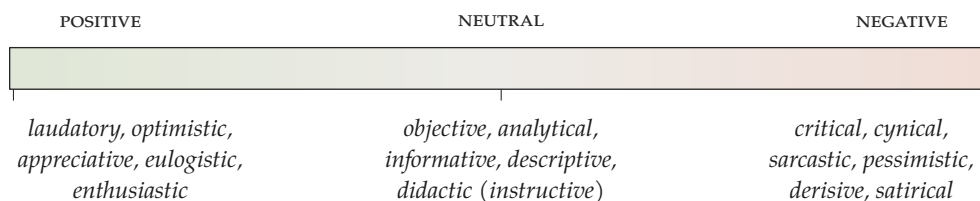


FIGURE 3.2. The spectrum of tone words. Locate the author first on this scale, then choose the most precise word.

3.5 Common Traps

1. **Option true in real life but not in passage** → wrong.
2. **Partially correct** option (one half right) → wrong.
3. **Extreme/absolute words** (always, never, all) → usually wrong.
4. **Out-of-scope** generalisation → wrong.
5. Repeated exact words from the passage used in a twisted sense → check carefully.

3.6 Solved Mini-Example

Passage (excerpt): "Digital technology has democratised access to knowledge. Yet access alone does not produce learning; without guidance, learners drown in information. Teachers therefore are not replaced by technology but repositioned — from transmitters of facts to curators and mentors."

1. Main idea? → **Technology changes, rather than eliminates, the teacher's role.**
2. The author's tone? → **Analytical / balanced.**
3. "Curators" means here? → **Selectors and organisers of relevant content.**
4. Which can be inferred? → (a) technology is harmful (b) access ensures learning (c) guidance is necessary for learning from digital resources (d) teachers are obsolete → (c)
5. Which is NOT stated? → "Teachers will become obsolete" — contradicted.

3.7 Deeper Dive — Two Full Practice Passages (solve in 8 minutes each)

3.7.1 Passage 1

India's traditional knowledge systems were never confined to scriptures alone. Farmers who read the sky to predict monsoon onset, healers who classified hundreds of plants by their effects, and artisans who worked metals without modern furnaces all practised forms of empirical inquiry. Yet because this knowledge travelled orally and through apprenticeship, it was rarely recorded in the formats that modern institutions recognise. Colonial administrators, trained to value written

evidence, frequently dismissed it as superstition.

The challenge today is not to romanticise this heritage but to examine it critically. Some practices survive rigorous testing; others do not. Treating every traditional claim as valid is as unscientific as rejecting all of them. What is needed is a dialogue in which communities are partners, not merely sources of data, and in which the benefits of any discovery return to those who preserved the knowledge.

1. The passage primarily argues that traditional knowledge should be: (a) accepted without question (b) rejected as superstition (c) critically examined in partnership with communities (d) recorded only in scriptures — **Ans: (c)**
2. According to the passage, traditional knowledge was dismissed during colonial times mainly because: **it was transmitted orally rather than in written form**
3. The author's tone is best described as: **balanced and analytical**
4. "Romanticise" in the passage means: **to present something as better or more ideal than it is**
5. Which statement is NOT supported by the passage? (a) Artisans practised empirical inquiry (b) All traditional practices survive testing (c) Communities should benefit from discoveries (d) Knowledge was passed through apprenticeship — **Ans: (b)**

3.7.2 Passage 2

Universities are often described as engines of economic growth, and policymakers increasingly measure them by patents, start-ups and graduate salaries. These indicators matter, but they capture only what is easily counted. A university also preserves languages, questions received wisdom and trains citizens to disagree without hostility. None of these appears in a ranking table.

When funding follows only measurable outputs, departments that produce long-term, uncertain or critical knowledge are the first to shrink. The irony is that many technologies now celebrated as commercial successes grew out of research that once looked useless. A wiser policy would balance accountability with patience, judging institutions over decades rather than budget cycles.

1. The central idea: **Universities have valuable functions beyond measurable economic outputs, and policy should recognise them.**
2. The "irony" mentioned refers to: **commercially successful technologies originating from research once considered useless**
3. The author would most likely support: **long-term evaluation of universities**
4. Which is an assumption of the author? **Funding decisions influence which departments survive.**
5. Suitable title: **"Beyond the Ranking Table: What Universities Are For"**

3.7.3 Inference vs Assumption vs Conclusion

Term	Meaning	Test
Inference	Logically follows from what is stated	"Must this be true if the passage is true?"
Assumption	Unstated premise the argument relies on	Negate it — does the argument collapse?
Conclusion	The main claim the author argues for	Usually signalled by "therefore", "thus", or the last line

3.8 Previous Year Questions (PYQ pattern — typical stems seen every cycle)

1. "The central idea of the passage is..." — pick the option that covers all paragraphs.
2. "Which of the following is the most appropriate title?"
3. "According to the passage, which of the following statements is/are correct? (A)... (B)... (C)...
Choose: A and B only / B and C only..." — verify each individually.
4. "The author's attitude towards X is..."
5. "What does the author mean by the phrase '...'?"
6. "Which of the following can be inferred from the passage?"
7. "Which one of the following is NOT supported by the passage?"

Past passages have covered: environment & sustainability, education policy, Indian philosophy/culture, technology & society, economics of development, ethics, colonial history, art & literature.

More practice: common distractor patterns

8. Option repeats a phrase from the passage but changes its meaning → **wrong (distortion)**
9. Option is true in general knowledge but not mentioned → **wrong (outside scope)**
10. Option covers only one paragraph when the question asks for the central idea → **too narrow**
11. Option makes a sweeping claim ("all", "never") → **usually extreme, wrong**

3.9 Practice Routine for 5/5

- Daily: 1 editorial (The Hindu / Indian Express) → write a 1-line main idea + tone.
- Weekly: 5 timed passages (8 minutes each).
- Maintain a vocabulary notebook of words met in context.

Communication



Expected questions: ~5 (10 marks) · Target: 5/5

SYLLABUS COVERAGE

- ☐ Communication: meaning, types and characteristics
 - ☐ Effective communication: verbal & non-verbal, inter-cultural & group communication, classroom communication
 - ☐ Barriers to effective communication
 - ☐ Mass-media and society
-

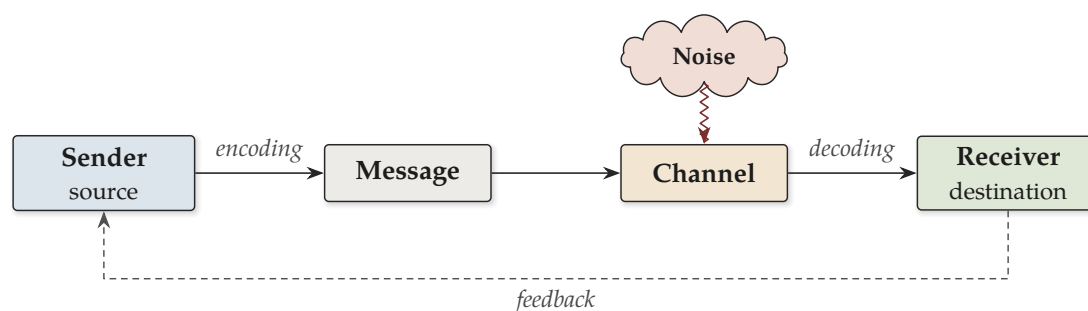


FIGURE 4.1. The general model of communication. Shannon and Weaver's contribution was the noise source, which can distort the message at any point on the channel.

4.1 Meaning

At its simplest, communication succeeds when the receiver understands what the sender meant. Every model in this unit is an attempt to explain how that shared meaning is achieved — and why it so often is not. Latin *communis* / *communicare* = "to make common / to share". Communication is the process of transmitting information, ideas and feelings to create **shared understanding**.

4.2 Models of Communication

Models simplify a complex process so that its parts can be studied. Early *linear* models (Aristotle, Lasswell, Shannon–Weaver) treat communication as a one-way transmission. *Interactive* models add feedback, and *transactional* models see both parties sending and receiving at once. The single most examined idea is **noise**, introduced by Shannon and Weaver: anything that distorts the message on its way.

Model	Type	Key point
Aristotle	Linear, oldest	Speaker → Speech → Audience (persuasion: ethos, pathos, logos)
Lasswell (1948)	Linear	<i>Who says What in which Channel to Whom with what Effect</i>
Shannon–Weaver (1949)	Linear, "mother of all models"	Introduced noise ; information source, transmitter, channel, receiver, destination
Berlo SMCR (1960)	Linear	Source, Message, Channel, Receiver
Osgood–Schramm	Circular	Both are encoder-interpreter-decoder
Schramm (1954)	Interactive	Field of experience must overlap
Westley–MacLean	Mass comm	Gatekeeper role
Dance (helical)	Transactional	Communication grows like a helix
Barnlund	Transactional	Simultaneous sending & receiving

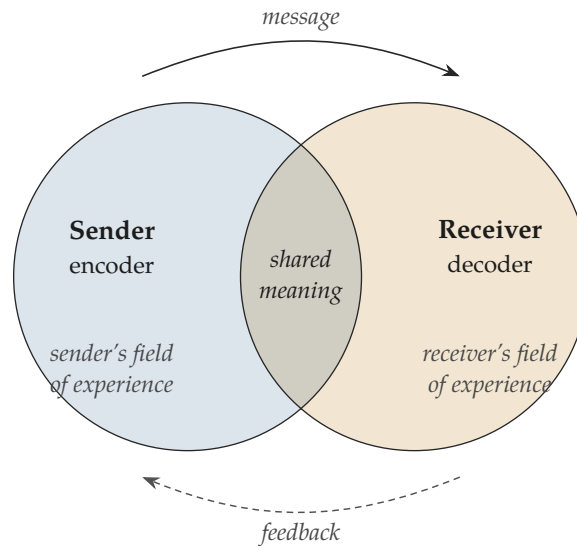


FIGURE 4.2. Schramm's model: a message can be understood only to the extent that the fields of experience of sender and receiver overlap.

4.3 Types of Communication

Communication can be classified by the channel used, the number of people involved, the direction of flow in an organisation, and its degree of formality. A single act — a principal's circular to teachers, for example — is written, downward, formal and group communication all at once.

By channel	By participants	By direction	By formality
verbal — oral, written non-verbal	intrapersonal (self) interpersonal (two) group, public mass (through media)	upward, downward horizontal (lateral) diagonal grapevine (informal)	formal informal

FIGURE 4.3. Four independent ways of classifying communication.

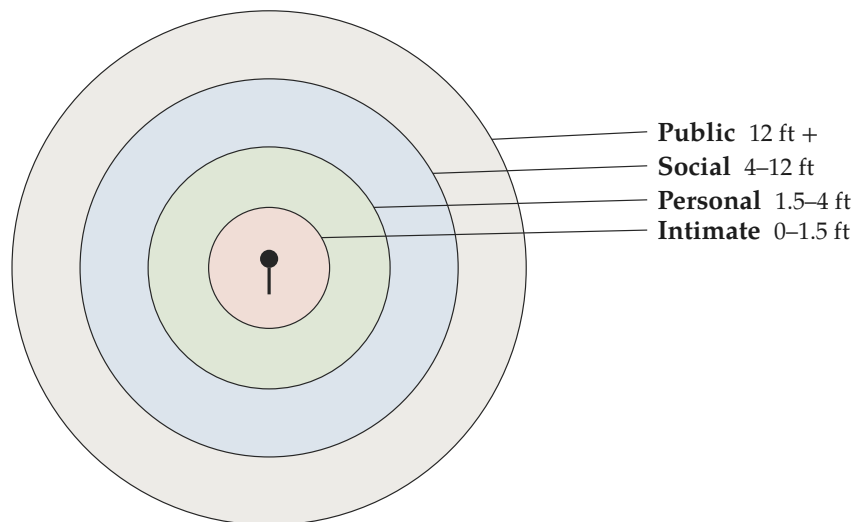


FIGURE 4.4. Edward T. Hall's proxemic zones: the distance people keep communicates the closeness of the relationship.

4.3.1 *Non-verbal communication (frequently asked)*

Much of what we communicate is carried not by words but by the body, the voice, space and time. The technical names (kinesics, proxemics, haptics, chronemics) are frequently asked; Edward T. Hall's zones of personal space are illustrated below.

Kind	Meaning
Kinesics	Body movements, gestures, facial expressions, posture
Proxemics	Use of space / distance (Edward T. Hall)
Haptics	Touch
Chronemics	Use of time
Paralanguage / Vocalics	Tone, pitch, volume, pauses — <i>how</i> it is said
Oculesics	Eye contact
Artifacts	Clothing, appearance, objects
Olfactics	Smell

Hall's proxemic zones: Intimate (0–1.5 ft), Personal (1.5–4 ft), Social (4–12 ft), Public (12 ft+).

Mehrabian's 7-38-55 rule (for feelings/attitudes): 7% words, 38% tone of voice, 55% body language.

4.3.2 *Classroom Communication*

- Mostly **interpersonal + group**; should be two-way, with feedback.
- **Flanders' Interaction Analysis Category System (FIACS)** — 10 categories (7 teacher talk, 2 student talk, 1 silence/confusion); teacher talk = indirect (1–4) + direct (5–7).
- Effective classroom communication: clarity, eye contact, questioning, active listening, empathy, using examples.

4.4 Barriers to Communication

A barrier is anything that prevents the intended meaning from reaching the receiver. Questions usually describe a situation — jargon in a lecture, a noisy hall, a student's prejudice — and ask you to name the barrier, so learn the categories with an example of each.

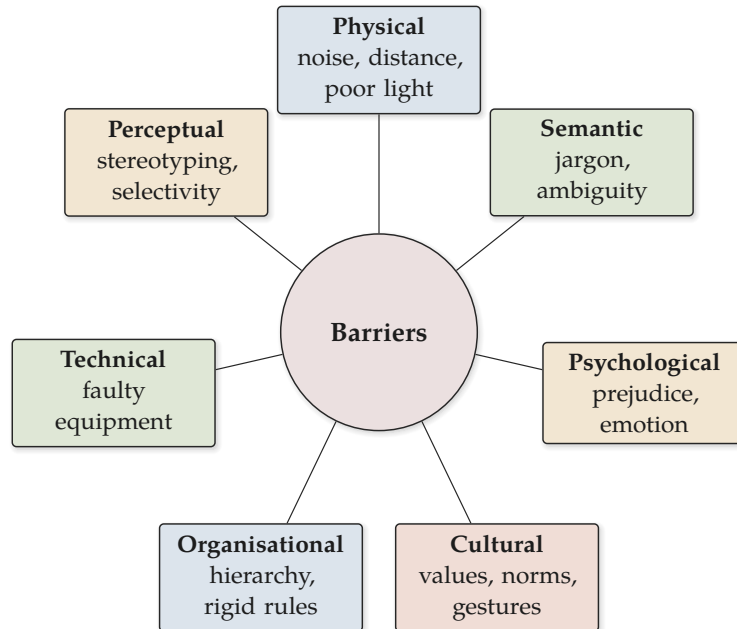


FIGURE 4.5. Seven families of barriers to effective communication.

7 Cs of effective communication: **Clear, Concise, Concrete, Correct, Coherent, Complete, Courteous.**

4.5 Inter-cultural & Group Communication

When sender and receiver come from different cultures, their fields of experience overlap less, and misunderstanding becomes more likely. Groups add their own dynamics: they pass through recognisable stages before they work well together.

- **Hofstede's cultural dimensions:** power distance, individualism–collectivism, masculinity–femininity, uncertainty avoidance, long-term orientation, indulgence.
- **High-context** cultures (India, Japan) rely on implicit cues; **low-context** (USA, Germany) are explicit (E.T. Hall).
- Ethnocentrism = judging other cultures by one's own — barrier.
- Tuckman's group stages: **Forming → Storming → Norming → Performing → Adjourning.**

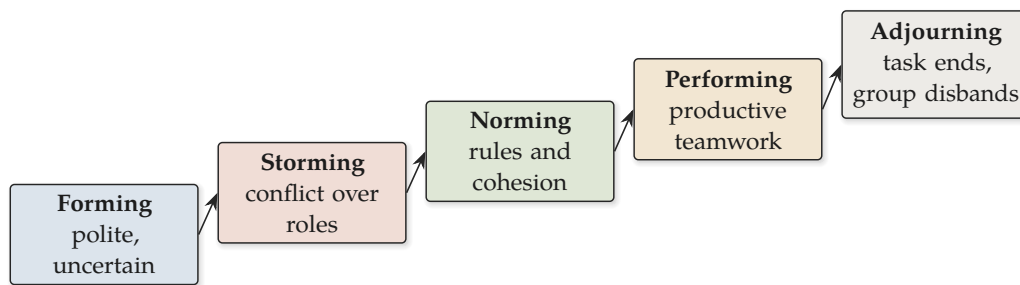


FIGURE 4.6. Tuckman's stages of group development.

4.6 Mass Media & Society

Theories of mass communication have swung between two views of the audience: as passive targets of powerful media (the "magic bullet") and as active users who select, interpret and resist. The two-step flow model introduced the influential middle layer of *opinion leaders*.

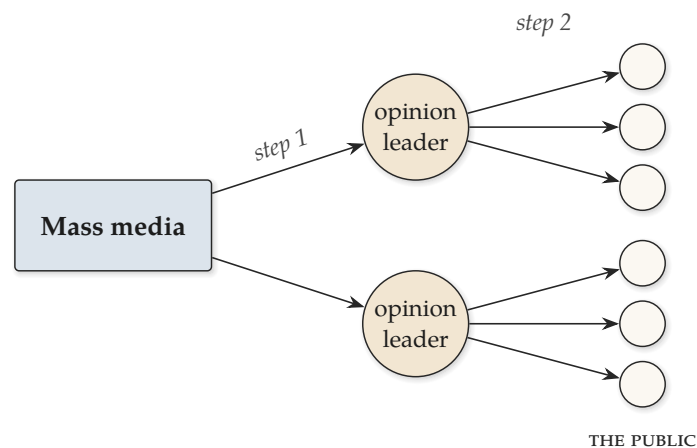


FIGURE 4.7. Lazarsfeld and Katz's two-step flow: media influence passes through opinion leaders to the wider public.

Theory	Idea
Hypodermic needle / Magic bullet	Media has direct, powerful effect on passive audience
Two-step flow (Lazarsfeld & Katz)	Media → opinion leaders → public
Agenda-setting (McCombs & Shaw)	Media tells us <i>what to think about</i>
Uses & Gratifications (Katz, Blumler)	Active audience uses media to satisfy needs
Cultivation (George Gerbner)	Long TV exposure shapes perception of reality ("mean world")
Spiral of silence (Noelle-Neumann)	Minority opinion holders stay silent
Medium is the message (Marshall McLuhan)	"Global village"; hot vs cool media
Gatekeeping (Kurt Lewin, David Manning White)	Editors filter news

Functions of mass media (Lasswell + Wright): surveillance, correlation, transmission of culture, entertainment. Indian landmarks: First newspaper — *Hicky's Bengal Gazette* (1780, James Augustus Hicky); Doordarshan (1959); All India Radio (1936, named; Akashvani 1957); Press Council of India (1966); Prasar Bharati (1997).

4.7 Deeper Dive — Listening, Feedback, Networks & Classroom Talk

4.7.1 Listening

Listening is the neglected half of communication. A message is complete only when it has been heard, understood and responded to, which is why the stages of listening end with *responding*.

Hearing is physiological; **listening** is psychological (attending, understanding, remembering, responding).

Type of listening	Purpose
Active / empathetic	Understand the speaker's feelings; paraphrase, nod, ask clarifying questions
Critical / evaluative	Judge the message, logic and evidence
Informational / comprehensive	Learn and retain content (lectures)
Appreciative	Enjoyment (music, poetry)
Selective	Hearing only what one wants — a barrier

Stages (HURIER model, Brownell): **Hearing → Understanding → Remembering → Interpreting → Evaluating → Responding.**

4.7.2 Feedback

Feedback turns a monologue into a dialogue. For the sender it confirms whether the message was understood; for the receiver it is information about how others see them — the idea captured by the Johari window.

- **Positive** (reinforces), **negative / corrective** (points out errors), **descriptive** (specific, non-judgemental), **evaluative** (judgement).
- Good feedback is specific, timely, focused on behaviour (not the person), and actionable.
- **Johari window** (Joseph Luft & Harrington Ingham): self-awareness in communication.

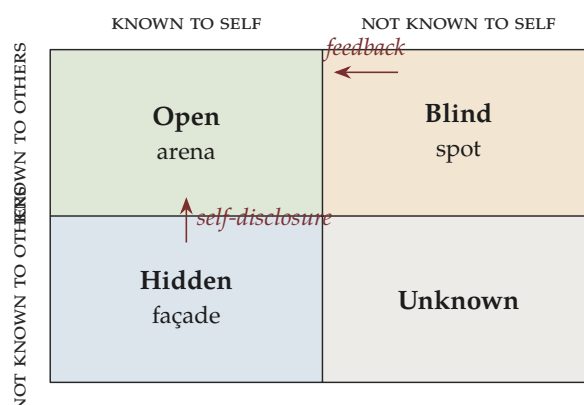


FIGURE 4.8. The Johari window. Feedback from others shrinks the blind area; self-disclosure shrinks the hidden area; both enlarge the open area in which communication is easiest.

4.7.3 Communication Networks (small groups)

Small-group experiments (Bavelas, Leavitt) showed that the *pattern* of who may talk to whom affects speed, accuracy, leadership and morale. Centralised networks are efficient for simple tasks; decentralised ones suit complex problems and keep members more satisfied.

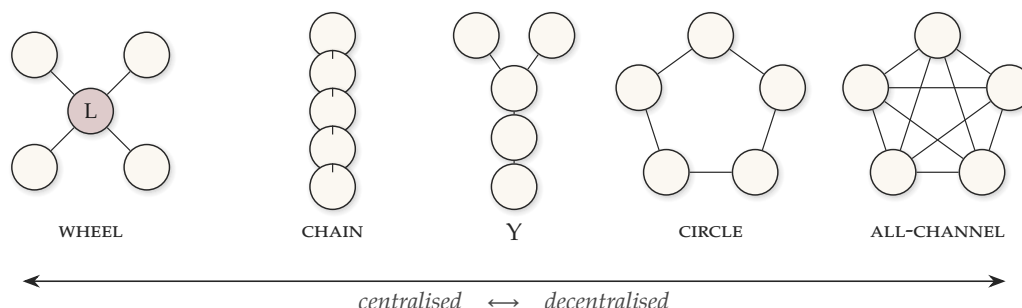


FIGURE 4.9. Small-group communication networks. Centralised patterns (left) are fast and produce a clear leader; decentralised ones (right) solve complex problems better and satisfy members more.

Network	Speed (simple tasks)	Accuracy	Member satisfaction	Leader emergence
Wheel	High	High	Low (except centre)	High
Chain	Moderate	High	Low	Moderate
Circle	Slow	Low	High	None
All-channel	Fast for complex tasks	Moderate	Highest	None

4.7.4 Effective Classroom Communication

- Use **simple language**, examples from learners' lives, and checks for understanding.
- Combine verbal with **non-verbal** cues (eye contact, gestures, movement).
- **Two-way**: questions, discussions, feedback.
- **Barriers** in class: noise, overcrowding, teacher's monotone, unclear objectives, learner's anxiety.
- **Teacher immediacy**: behaviours that reduce psychological distance (smiling, using names) → better learning.

4.7.5 Communication Ethics & Digital Communication

- Accuracy, honesty, respect for privacy, avoidance of plagiarism and misinformation.
- **Fake news / misinformation** (false but shared without intent to deceive) vs **disinformation** (deliberately false).
- **Netiquette**: rules of good online behaviour; ALL CAPS = shouting.
- **Digital divide**: unequal access to ICT.

4.8 Previous Year Questions (PYQ pattern)

1. "Who says what in which channel to whom with what effect" was given by: **Harold Lasswell**
2. Which model first introduced the concept of "noise"? **Shannon-Weaver**
3. The study of body movements as communication: **Kinesics**
4. Use of space in communication: **Proxemics**
5. The study of touch: **Haptics**
6. Tone and pitch of voice belong to: **Paralanguage**
7. Communication from subordinates to superiors: **Upward**
8. Informal communication network in an organisation: **Grapevine**
9. Jargon and ambiguous words cause which barrier? **Semantic**
10. Prejudice is a: **Psychological barrier**
11. "The medium is the message" — **Marshall McLuhan**
12. Media determines "what to think about" — **Agenda-setting theory**
13. Theory that the audience actively selects media to satisfy needs: **Uses and Gratifications**
14. Heavy TV viewers perceive the world as more dangerous — **Cultivation theory (Gerbner)**
15. "Global village" — **McLuhan**
16. Feedback in communication helps to: **verify that the message was understood as intended**
17. Classroom communication is basically: **Interactive / two-way, interpersonal + group**
18. FIACS was developed by: **Ned Flanders** (10 categories)
19. Arrange Tuckman's stages: **Forming, Storming, Norming, Performing, Adjourning**
20. The first newspaper in India: **Hicky's Bengal Gazette (1780)**
21. Communication with oneself: **Intrapersonal**
22. Statement: "Effective communication requires the sender and receiver to share a common field of experience." — **True (Schramm)**

More practice questions

23. The first stage of the listening process: **Hearing (receiving)**
24. In the Johari window, information known to others but not to self is the: **blind area**
25. Self-disclosure reduces the: **hidden area**
26. The most centralised small-group network: **wheel**
27. Network with highest member satisfaction: **all-channel**
28. Feedback should be: **specific, timely and descriptive**
29. Deliberately false information spread to deceive: **disinformation**
30. Typing in ALL CAPS in online communication is considered: **shouting (poor netiquette)**
31. Teacher behaviours that reduce psychological distance: **immediacy behaviours**
32. Which is NOT a characteristic of effective classroom communication? (a) two-way (b) jargon-heavy (c) clear (d) learner-centred — **Ans: (b)**

POINTS TO REMEMBER

- Lasswell = 5 Ws · Shannon-Weaver = noise · Schramm = field of experience · Berlo = SMCR
- Kinesics–body, Proxemics–space, Haptics–touch, Chronemics–time
- Agenda setting (what to think about) vs Cultivation (TV shapes reality) vs Two-step

(opinion leaders)

Mathematical Reasoning & Aptitude

...

Expected questions: ~5 (10 marks) · Target: 5/5 (pure practice unit)

SYLLABUS COVERAGE

- ☐ Types of reasoning
 - ☐ Number series, letter series, codes
 - ☐ Relationships (blood relations)
 - ☐ Classification
 - ☐ Mathematical aptitude: fraction, time & distance, ratio, proportion, percentage, profit & loss, interest & discounting, averages
-

5.1 Number Series — Pattern Checklist

A number series hides a rule that generates each term from the ones before it. The quickest way to find it is to test the simplest rules first — constant differences, then constant ratios — and only then look for squares, alternating patterns or mixed operations. Writing the differences beneath the series takes a few seconds and solves most questions.

Examples:

- 2, 6, 12, 20, 30, **42** → $n(n+1)$
- 3, 7, 15, 31, 63, **127** → $\times 2 + 1$
- 1, 8, 27, 64, **125** → n^3
- 2, 3, 5, 7, 11, **13** → primes
- 1, 1, 2, 3, 5, 8, **13** → Fibonacci
- 4, 9, 25, 49, 121, **169** → squares of primes
- 1, 2, 6, 24, 120, **720** → $\times 2, \times 3, \times 4 \dots$ (factorials)

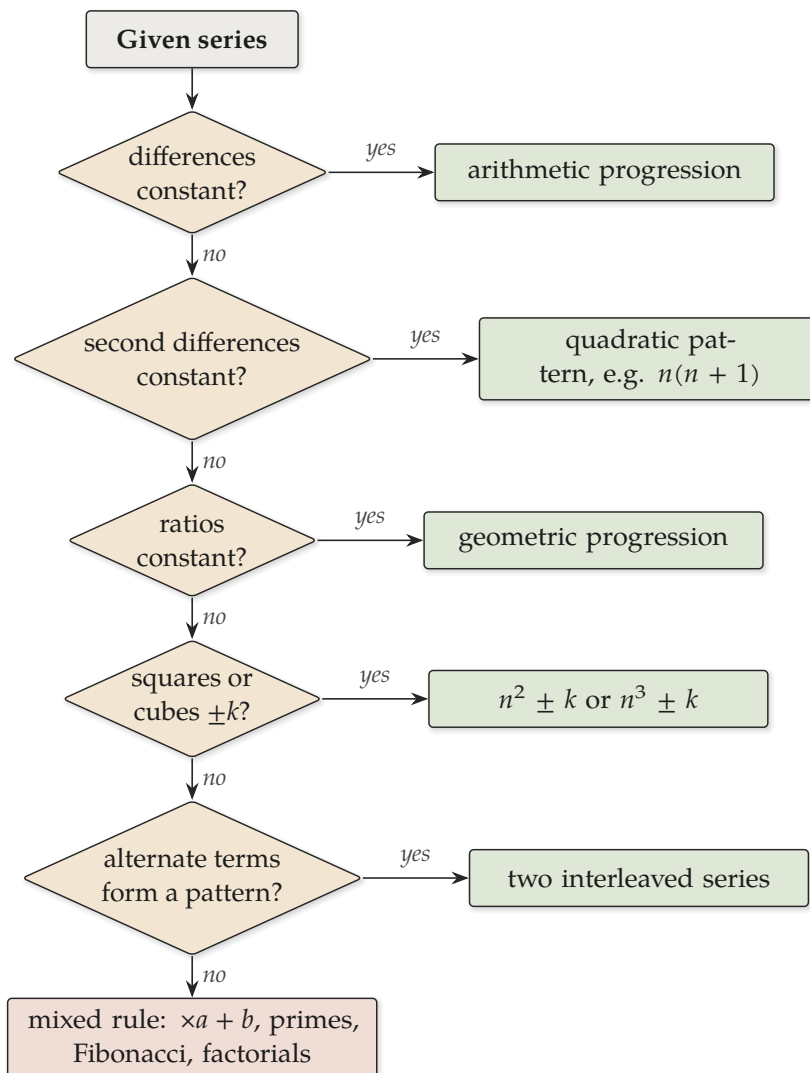


FIGURE 5.1. A checklist for number series: test the simple rules first and move down only when they fail.

5.2 Letter Series & Coding

Letter questions become number questions once each letter is replaced by its position in the alphabet. Memorise the positions and their reverses so that no time is lost counting; the reference table below is worth learning by heart.

Position table (memorise both directions):

Letter	A	B	C	D	E	F	G	H	I	J	K	L	M
Position	1	2	3	4	5	6	7	8	9	10	11	12	13
Reverse	26	25	24	23	22	21	20	19	18	17	16	15	14

Letter	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Position	14	15	16	17	18	19	20	21	22	23	24	25	26
Reverse	13	12	11	10	9	8	7	6	5	4	3	2	1

Trick: **EJOTY** = 5, 10, 15, 20, 25. Opposite pairs sum to 27 ($A \leftrightarrow Z$, $M \leftrightarrow N$).

– Coding: If CAT → DBU (+1 each), then DOG → **EPH**.

- If $CAT = 24$ ($3+1+20$), then $DOG = 4+15+7 = 26$.
- Reverse coding: $CAT \rightarrow XZG$ (opposite letters).

5.3 Blood Relations

Relationship puzzles are confusing in words and trivial on paper. Draw a small family tree as you read each clause, marking gender and generation, and the answer can simply be read off.

Use a **family tree diagram** with symbols:

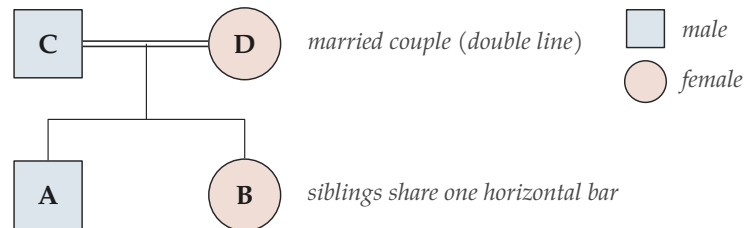


FIGURE 5.2. "A is the brother of B; B is the daughter of C; C is the husband of D." Drawn as a family tree, the answer is immediate: D is A's mother. Use squares for males and circles for females.

Coded relations: "P + Q means P is father of Q; $P \times Q$ means P is sister of Q" → draw step-by-step.

5.4 Classification (Odd one out)

Look for the property shared by all options but one: being prime, a perfect square, a multiple of a number, a vowel, a member of a category. When two different properties seem to work, prefer the more specific one. Find the common property in 3 of 4: primes, squares, multiples, vowels, same-category items. Example: 121, 144, 169, **190** → 190 is not a perfect square.

5.5 Arithmetic Aptitude — Formula Bank

Arithmetic questions in Paper I are not difficult, but they reward speed. Each formula below replaces several lines of working; understand where it comes from once, then use it directly in the examination.

5.5.1 Percentage

Percentages express every quantity as a fraction of 100, which makes comparison easy. Successive changes do *not* simply add: a 20% rise followed by a 20% fall leaves you 4% poorer, because the fall is taken on a larger base.

$x\%$ of y equals $y\%$ of x	$x\%$ of $y = y\%$ of x
Net effect of successive changes of $a\%$ and $b\%$ (negative for decreases)	$a + b + \frac{ab}{100}$
Cut in consumption needed to offset a price rise of $r\%$	$\frac{r}{100 + r} \times 100\%$
Single discount equivalent to $d_1\%$ followed by $d_2\%$	$d_1 + d_2 - \frac{d_1 d_2}{100}$

5.5.2 Profit & Loss

Profit and loss percentages are always calculated on the **cost price** unless the question says otherwise; discounts are always calculated on the **marked price**.

Profit percentage (on cost price)	$\text{Profit}\% = \frac{SP - CP}{CP} \times 100$
Selling price for a profit of $P\%$	$SP = CP \times \frac{100 + P}{100}$
Selling price after a discount of $d\%$ on marked price M	$SP = M \times \frac{100 - d}{100}$
Two articles at the same SP , one at $x\%$ gain and one at $x\%$ loss	always a loss of $\frac{x^2}{100}\%$
False weight: 900 g sold as 1 kg	$\text{gain} = \frac{100}{900} \times 100 = 11.11\%$

5.5.3 Simple & Compound Interest

Simple interest is earned only on the original principal; compound interest is also earned on interest already added. The difference between them grows with time, and the two-year difference formula is the most frequently used shortcut.

Simple interest	$SI = \frac{PRT}{100}$
Amount at compound interest	$A = P\left(1 + \frac{R}{100}\right)^T, \quad CI = A - P$
$CI - SI$ over two years	$P\left(\frac{R}{100}\right)^2$
$CI - SI$ over three years	$P\left(\frac{R}{100}\right)^2\left(3 + \frac{R}{100}\right)$
Sum doubles in T years at simple interest	$R = \frac{100}{T}$

5.5.4 Ratio, Proportion, Averages

A ratio compares two quantities; a proportion states that two ratios are equal. Averages smooth a set of values into one representative number — but note that the average *speed* over equal distances is a harmonic mean, not the simple average of the speeds.

Proportion	$a : b = c : d \iff ad = bc$
Mean proportional of a and b	$\sqrt{ab}$
Third proportional to a and b	$\frac{b^2}{a}$
Average	$\frac{\text{sum of values}}{\text{number of values}}$
One of n values replaced and the average changes by d	the value changed by nd
Average speed over two equal distances at x and y	$\frac{2xy}{x+y}$ (not $\frac{x+y}{2}$)

5.5.5 Time, Speed, Distance

Nearly every motion problem — trains, boats, races — reduces to the single relation distance = speed $\times$ time, applied carefully with consistent units and, where two bodies move, with their *relative* speed.

- Relative speed: same direction ($\mathbf{a} - \mathbf{b}$), opposite direction ($\mathbf{a} + \mathbf{b}$).
- Train crossing pole: time = L / S ; crossing platform: $(L + P) / S$.

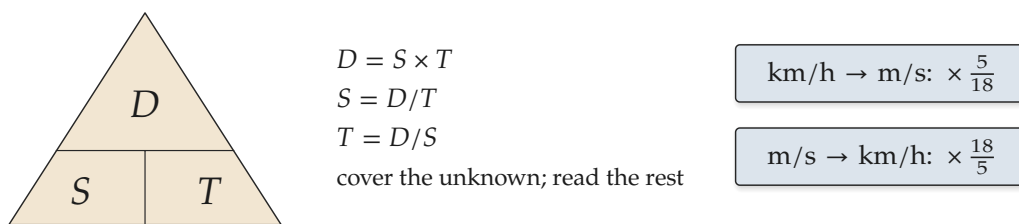


FIGURE 5.3. The distance–speed–time triangle and the two unit conversions that appear in almost every problem.

- Boats: downstream = $u + v$, upstream = $u - v$; still water speed = $(D + U)/2$, stream = $(D - U)/2$.

5.5.6 Time & Work

Think of work as a rate: if A finishes a job in a days, A does $1/a$ of it each day. Rates of people (or pipes) working together simply add.

A alone takes a days, B alone b days;
together $\frac{ab}{a+b}$ days

Pipes and cisterns add rates of filling pipes, subtract emptying ones

Men, days and hours per unit of work $\frac{M_1 D_1 H_1}{W_1} = \frac{M_2 D_2 H_2}{W_2}$

5.6 Worked Examples

1. A price rises 20% then falls 20%. Net? $\rightarrow 20 - 20 - 400/100 = -4\%$ (loss).
2. ₹5000 at 10% CI for 2 yrs: CI – SI = $5000 \times (0.1)^2 = \text{₹}50$.
3. A car goes at 40 km/h and returns at 60 km/h. Average speed = $2 \times 40 \times 60 / 100 = 48$ km/h.
4. A can finish in 10 days, B in 15. Together = $150/25 = 6$ days.
5. Two articles each sold at ₹990, one at 10% profit, another at 10% loss. $\rightarrow 1\%$ loss.

5.7 Deeper Dive — More Question Types with Shortcuts

5.7.1 Types of Reasoning

- **Deductive:** general $\rightarrow$ specific; conclusion certain. **Inductive:** specific $\rightarrow$ general; conclusion probable.
- **Abductive:** inference to the best explanation (a doctor diagnosing from symptoms).
- **Analogical:** reasoning from similarity of two cases.

5.7.2 Mixtures & Alligation

Alligation is a quick way of finding the ratio in which two ingredients at different prices (or concentrations) must be mixed to give a mixture of a desired mean value.

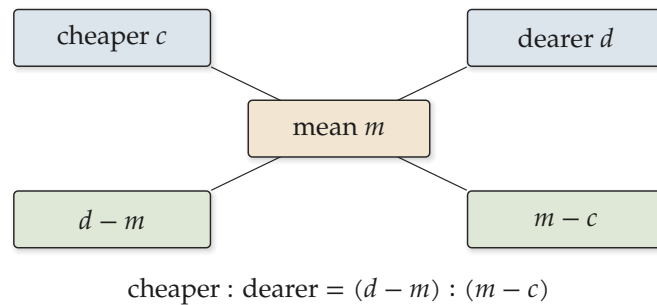


FIGURE 5.4. The rule of alligation: subtract crosswise to obtain the mixing ratio.

Example: Rice at ₹40/kg and ₹60/kg mixed to get ₹52/kg → ratio = $(60 - 52) : (52 - 40) = 8 : 12 = 2 : 3$.

Repeated dilution: from a vessel of x litres, y litres replaced by water n times → remaining pure = $x(1 - y/x)^n$.

5.7.3 Ages

Translate words into equations. "A is twice as old as B; 10 years ago A was three times as old as B": $A = 2B$, $A - 10 = 3(B - 10) \rightarrow 2B - 10 = 3B - 30 \rightarrow B = 20, A = 40$.

5.7.4 Partnership

Profit shared in ratio of **capital × time**. A invests ₹5000 for 12 months, B ₹6000 for 8 months → $60000 : 48000 = 5 : 4$.

5.7.5 Discounting (banker's & true discount)

True discount on amount A due in T years at $R\%$	$TD = \frac{ART}{100 + RT}$
Present worth	$PW = A - TD$
Banker's discount (simple interest on the amount)	$BD = \frac{ART}{100}$
Banker's gain	$BG = BD - TD = \text{SI on } TD$

Example: $A = ₹1100$ due in 1 year at 10%: $TD = 1100 \times 10/110 = ₹100$; $PW = ₹1000$; $BD = ₹110$; $BG = ₹10$.

5.7.6 Coding–Decoding Variants

Type	Example
Letter shift	COMPUTER → DPNQVUFS (+1)
Reverse alphabet	GOOD → TLLW (A↔Z)
Position sum	BAD = 2 + 1 + 4 = 7
Word reversal	READ → DAER
Symbol substitution	If + means ×, – means ÷: $8 + 2 - 4 = 8 \times 2 \div 4 = 4$

5.7.7 Fractions & Comparison

- To compare fractions quickly, cross-multiply: $5/7$ vs $7/10 \rightarrow 50$ vs $49 \rightarrow 5/7$ is larger.
- Recurring decimals: $0.333... = 1/3$; $0.2727... = 27/99 = 3/11$.

5.7.8 Worked Mixed Set

1. Successive discounts 20% and 10% = $20 + 10 - 2 = 28\%$.
2. A sum becomes ₹1331 in 3 years at 10% CI $\rightarrow$ principal = $1331/1.331 = ₹1000$.
3. 15 men finish work in 20 days; how many days for 25 men? $15 \times 20 / 25 = 12$ days.
4. Boat: 16 km downstream in 2 h, 8 km upstream in 2 h $\rightarrow$ speeds 8 and 4 $\rightarrow$ still water **6 km/h**, stream **2 km/h**.
5. Average of first 50 natural numbers = $(50 + 1)/2 = 25.5$.

5.8 Previous Year Questions (PYQ pattern)

1. Next term: 2, 5, 10, 17, 26, ? $\rightarrow 37$ ($n^2 + 1$)
2. Next term: 1, 4, 9, 16, 25, ? $\rightarrow 36$
3. Missing: 3, 12, 48, ?, 768 $\rightarrow 192$ ($\times 4$)
4. Next: 6, 11, 21, 36, 56, ? $\rightarrow 81$ (diffs 5,10,15,20,25)
5. Letter series: AZ, BY, CX, ? $\rightarrow$ DW
6. If TEACHER is coded as VGCEJGT (+2), then STUDENT $\rightarrow$ UVWFGPV
7. If PEN = 35 ($16+5+14$), INK = ? $\rightarrow 9+14+11 = 34$
8. Pointing to a photograph, a man says, "Her mother is the only daughter of my mother." The man is the girl's: **Maternal uncle**
9. A is B's sister, C is B's mother, D is C's father, E is D's mother. A is E's: **Great-granddaughter**
10. Odd one out: 27, 64, 125, 144 $\rightarrow 144$ (not a cube)
11. A shopkeeper marks 25% above CP and allows 10% discount. Gain% = $1.25 \times 0.9 = 1.125 \rightarrow 12.5\%$
12. A sum doubles in 8 years at SI. Rate = **12.5%**
13. A train 150 m long passes a pole in 15 s. Speed = $10 \text{ m/s} = 36 \text{ km/h}$
14. Average of 5 numbers is 20; one number removed, average becomes 18. Removed number = $100 - 72 = 28$

15. Ratio of A:B = 2:3, B:C = 4:5. A:B:C = **8:12:15**
16. If the population increases 10% annually, from 10000 after 2 years: **12100**
17. Salary increased by 25%; by what % must it be reduced to restore? $25/125 \times 100 = 20\%$
18. Two trains 120 m and 180 m at 50 & 40 km/h opposite directions. Time to cross = $300 / (90 \times 5/18) = 300/25 = 12 \text{ s}$

More practice questions

19. Milk at ₹30/L mixed with water (free) to sell at ₹30 with 20% profit. Ratio milk : water = **5 : 1**
(cost price of mixture 25 $\rightarrow (25 - 0) : (30 - 25)$)
20. Next term: 1, 3, 7, 15, 31, ? $\rightarrow$ **63**
21. Next term: 3, 5, 9, 17, 33, ? $\rightarrow$ **65** ($\times 2 - 1$)
22. Find the odd one: 3, 5, 11, 14, 17 $\rightarrow$ **14** (not prime)
23. A and B invest ₹20,000 and ₹30,000 for 6 and 4 months; profit ratio = $120000 : 120000 = 1 : 1$
24. Ten years ago, father was 4 times his son's age; now he is twice. Son's present age: $S - 10 = (2S - 10)/4 \rightarrow$ **15 years**
25. If 'SCHOOL' is coded as 'RBGNNK' (-1), then 'COLLEGE' is: **BNKKDFD**
26. Pointing to a lady, Rahul says, "She is the daughter of my grandfather's only son." The lady is Rahul's: **sister**
27. 40% of a number is 60. 75% of it = **112.5**
28. Ratio of present worth to true discount for ₹1210 due in 2 years at 10% SI: $PW = 1210/1.2 = 1008.33$; $TD = 201.67 \rightarrow 5 : 1$

POINTS TO REMEMBER

- Net % change = $a + b + ab/100$ · Same SP, $\pm x\% \Rightarrow$ loss $x^2/100 \%$
- CI-SI (2 yr) = $P(R/100)^2$ · Avg speed = $2xy/(x+y)$
- EJOY = 5,10,15,20,25 · Opposite letters sum to 27

Logical Reasoning

Expected questions: ~5 (10 marks) · Target: 5/5

SYLLABUS COVERAGE

- ☐ Structure of arguments: argument forms, categorical propositions, mood & figure, formal & informal fallacies, uses of language, connotations & denotations, classical square of opposition
 - ☐ Evaluating & distinguishing deductive and inductive reasoning
 - ☐ Analogies
 - ☐ Venn diagram: simple & multiple use for establishing validity of arguments
 - ☐ Indian Logic: means of knowledge (Pramanas), structure & kinds of Anumana (inference), Hetvabhasas (fallacies of inference)
-

6.1 Argument Basics

Logic studies the relation between evidence and conclusion. It does not ask whether premises happen to be true — that is the business of science or experience — but whether, *if* they are true, the conclusion must or probably follows. Keep these two questions apart: an argument can be perfectly valid and still have a false conclusion, because one of its premises is false.

An **argument** = premises + conclusion. Premise indicators: *since, because, for*. Conclusion indicators: *therefore, hence, thus, so*.

	Deductive	Inductive
Direction	General → particular (not strictly)	Particular → general
Conclusion	Necessarily follows	Probably follows
Evaluation	Valid / invalid ; sound = valid + true premises	Strong / weak ; cogent = strong + true premises
Example	All men are mortal; Socrates is a man; ∴ mortal	Every crow seen is black ∴ all crows are black

Validity is about **form**, not truth. An argument with false premises can be valid.

6.2 Categorical Propositions (A E I O)

Traditional (Aristotelian) logic analyses sentences that relate two classes: a subject class *S* and a predicate class *P*. There are only four standard forms, named by the vowels of the Latin *AffIrmo* ("I affirm") and *nEgO* ("I deny"). Whether a term is *distributed* — whether the proposition says something about every member of that class — governs nearly every rule of the syllogism.

Code	Form	Quantity	Quality	Distributes
A	All <i>S</i> are <i>P</i>	Universal	Affirmative	<i>S</i> only
E	No <i>S</i> is <i>P</i>	Universal	Negative	<i>S</i> and <i>P</i>
I	Some <i>S</i> are <i>P</i>	Particular	Affirmative	None
O	Some <i>S</i> are not <i>P</i>	Particular	Negative	<i>P</i> only

Memory: **AffIrmo** (A, I affirmative), **nEgO** (E, O negative). Distribution: "**ASEBINOP**" — A: Subject, E: Both, I: Nothing, O: Predicate.

6.2.1 Square of Opposition

The square arranges the four propositions so that their truth relations can be read at a glance. Questions give the truth or falsity of one proposition and ask about another; locate both corners on the square and apply the relation that joins them.

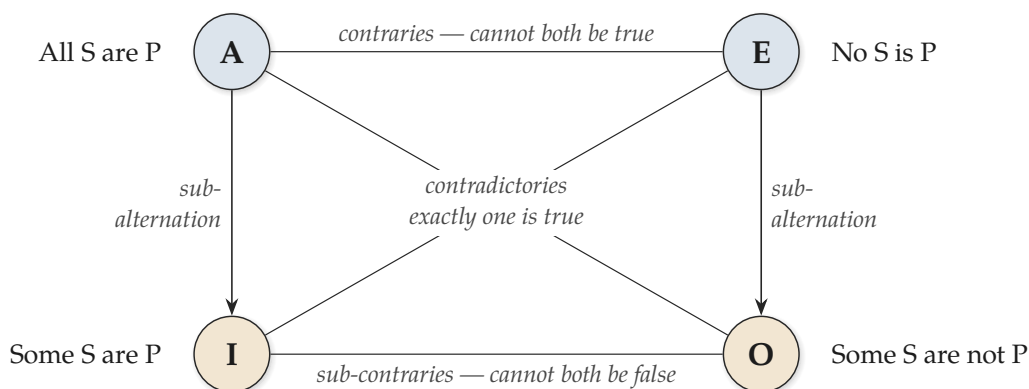


FIGURE 6.1. The traditional square of opposition. Universal propositions sit above, particular ones below; affirmatives on the left, negatives on the right.

Relation	Pair	Rule
Contradictory	A–O, E–I	Exactly one is true (opposite truth values always)
Contrary	A–E	Both cannot be true; both can be false
Sub-contrary	I–O	Both cannot be false; both can be true
Sub-alternation	A→I, E→O	If universal true ⇒ particular true; if particular false ⇒ universal false

Inference table (if given proposition is TRUE):

Given true	A	E	I	O
A	T	F	T	F
E	F	T	F	T
I	?	F	T	?
O	F	?	?	T

If given FALSE: A false $\rightarrow$ O true, E?, I?; E false $\rightarrow$ I true; I false $\rightarrow$ E true, A false, O true; O false $\rightarrow$ A true, E false, I true.

6.3 Categorical Syllogism — Mood & Figure

A categorical syllogism has two premises and a conclusion, and exactly three terms. The *mood* lists the types (A, E, I, O) of its three propositions; the *figure* records where the middle term stands in the premises.

Three terms: Major (P – predicate of conclusion), Minor (S – subject of conclusion), Middle (M – in both premises, not conclusion).

	Figure 1	Figure 2	Figure 3	Figure 4
Major premise	M – P	P – M	M – P	P – M
Minor premise	S – M	S – M	M – S	M – S
Conclusion	S – P	S – P	S – P	S – P
Middle term is	subject, predicate	predicate twice	subject twice	predicate, subject

Mood = the letters of the three propositions, e.g. AAA-1 (Barbara), EAE-1 (Celarent), AII-1 (Darii), EIO-1 (Ferio).

6.3.1 Rules of validity

1. Middle term must be distributed at least once (else **fallacy of undistributed middle**).
2. A term distributed in conclusion must be distributed in premise (else **illicit major/minor**).
3. Two negative premises $\rightarrow$ no conclusion (**exclusive premises**).
4. A negative premise $\rightarrow$ negative conclusion; and vice-versa.
5. Two particular premises $\rightarrow$ no conclusion.
6. Exactly three terms (else **four-term fallacy**).

6.3.2 Venn Diagram Method (for syllogism questions)

Venn diagrams turn syllogisms into pictures of overlapping classes. Draw the universal premise first, then the particular one, always choosing the arrangement that commits you to the *least*. If the conclusion is visible even in that weakest picture, it follows.

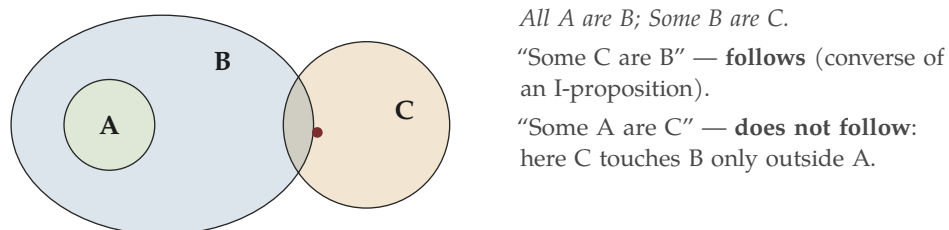


FIGURE 6.2. The Venn method. Draw the weakest picture the premises allow; a conclusion is valid only if it is true in every such picture.

Technique: draw the **minimum overlap** diagram; a conclusion follows only if it is true in **every** possible diagram.

6.4 Fallacies

A fallacy is an argument that looks persuasive but is defective. *Formal* fallacies break a rule of logical form and can be detected without knowing what the argument is about; *informal* fallacies depend on the content — an appeal to pity, a personal attack, an irrelevant distraction.

6.4.1 Formal fallacies

Undistributed middle, illicit major, illicit minor, affirming the consequent ($p \rightarrow q, q \therefore p$), denying the antecedent ($p \rightarrow q, \neg p \therefore \neg q$), four terms.

6.4.2 Informal fallacies

Fallacy	Meaning
Ad hominem	Attacking the person, not the argument
Ad populum (bandwagon)	Many people believe it $\Rightarrow$ true
Ad verecundiam	Appeal to inappropriate authority
Ad ignorantiam	Not proved false $\Rightarrow$ true
Ad misericordiam	Appeal to pity
Ad baculum	Appeal to force
Straw man	Distorting opponent's argument
Red herring	Diverting to an irrelevant issue
Begging the question (petitio principii)	Conclusion assumed in premise (circular)
Hasty generalisation	Too small a sample
False cause (post hoc)	After this, therefore because of this
Slippery slope	One step leads to disaster chain
False dilemma	Only two options presented
Equivocation	Word used in two senses
Composition / Division	Parts $\leftrightarrow$ whole wrongly transferred

6.5 Uses of Language; Connotation & Denotation

- Functions of language (Copi): **Informative, Expressive, Directive**, (also ceremonial, performative).
- **Denotation (extension)** = the set of objects a term refers to. **Connotation (intension)** = properties/attributes.
- As connotation increases, denotation decreases (inverse relation). e.g. "animal" $\rightarrow$ "mammal" $\rightarrow$ "dog".

6.6 Analogies

A : B :: C : ? — identify relation: part-whole, cause-effect, tool-worker, synonym, antonym, product-raw material, degree.

- Pen : Writer :: Scalpel : **Surgeon** · Cow : Calf :: Horse : **Foal** · Ornithology : Birds :: Entomology : **Insects**

6.7 Indian Logic (Nyaya) — Very High Weightage

The Indian tradition of logic (*nyāya*, *tarka*, *ānvīkṣikī*) asks how valid knowledge (*pramā*) is obtained. Its central topics — the means of knowledge, the structure of inference and the fallacies of inference — appear in almost every Paper I. Note that Indian inference always includes an *example*, which makes it more empirical than the Aristotelian syllogism.

6.7.1 *Pramanas (means of valid knowledge)*

	Pratyakṣa perception	Anumāna inference	Śabda testimony	Upamāna comparison	Arthāpatti postulation	Anupalabdhi non- perception	
Cārvāka	●	○	○	○	○	○	1
Buddhism, Vaiśeṣika	●	●	○	○	○	○	2
Sāṅkhya, Yoga, Jainism	●	●	●	○	○	○	3
Nyāya	●	●	●	●	○	○	4
Prābhākara Mīmāṃsā	●	●	●	●	●	○	5
Bhāṭṭa Mīmāṃsā, Advaita	●	●	●	●	●	●	6

FIGURE 6.3. How many means of valid knowledge (*pramāṇa*) each school accepts. Note the order in which they are added: Śabda (3) comes before Upamāna (4) because Sāṅkhya accepts testimony but not comparison.

School	Pramanas accepted	Count
Charvaka	Pratyaksha only	1
Buddhism, Vaisheshika	Pratyaksha, Anumana	2
Samkhya, Yoga, Jainism	+ Shabda	3
Nyaya	+ Upamana	4
Prabhakara Mimamsa	+ Arthapatti	5
Bhatta Mimamsa, Advaita Vedanta	+ Anupalabdhi	6

- **Arthapatti**: "Devadatta is fat but doesn't eat by day ⇒ he eats at night."
- **Anupalabdhi**: knowing the absence of a jar by not perceiving it.

6.7.2 Anumana — Five-membered syllogism (Panchavayava)

Inference for others (*parārthānumāna*) is set out in five members so that a listener can follow and check each step. The classic example concerns smoke and fire on a distant hill.

	Member	Meaning	Example
1	Pratijñā	proposition	The hill has fire.
2	Hetu	reason	Because it has smoke.
3	Udāharaṇa	example	Whatever has smoke has fire, as in a kitchen.
4	Upanaya	application	The hill has smoke pervaded by fire.
5	Nigamana	conclusion	Therefore the hill has fire.

Terms: **Paksha** (minor term: hill), **Sadhya** (major term: fire), **Hetu/Linga** (middle term: smoke). **Vyapti** = invariable concomitance (universal relation) between hetu and sadhya — the basis of inference.

Kinds of anumana:

- By purpose: **Svarthanumana** (for oneself) and **Pararthanumana** (for others, uses 5 members).
- By Nyaya (Gautama): **Purvavat** (cause → effect), **Sheshavat** (effect → cause), **Samanyatodrishta** (based on general observation).
- By vyapti: Kevalanvayi, Kevalavyatireki, Anvaya-vyatireki.

6.7.3 Hetvabhasa (fallacies of inference — 5)

A *hetvābhāsa* is a reason that only appears to be a reason. Each of the five arises when the middle term (*hetu*) fails one of the conditions that a sound reason must satisfy — for instance, being present wherever the probandum is, or not being contradicted by another means of knowledge.

Hetvabhasa	Meaning
Savyabhichara (Anaikantika)	Irregular / inconclusive middle — hetu found with and without sadhya
Viruddha	Contradictory — hetu proves the opposite
Satpratipaksha	Counter-balanced — another hetu proves the opposite
Asiddha (Sadhyasama)	Unproved — hetu itself not established
Badhita (Kalatita)	Contradicted by other pramana (e.g. "fire is cold because it is a substance")

6.8 Deeper Dive — Venn Diagram Practice, Conditionals & More Indian Logic

6.8.1 Six Standard Venn Situations

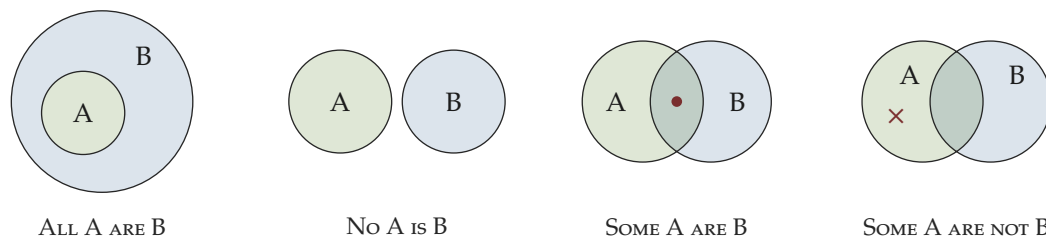


FIGURE 6.4. The four standard Venn situations. A dot or cross marks a region known to be non-empty.

Method for "Statements → Conclusions":

1. Draw the **minimum** case allowed by each statement.
2. A conclusion follows only if it holds in **every** possible diagram.
3. "Some A are not B" from "All A are B"? No. "Some B are A" from "All A are B"? Yes (conversion by limitation).

Worked: Statements: All pens are books. Some books are bags.

- Conclusion I: Some pens are bags — **does not follow** (bags may touch only the books outside pens).
- Conclusion II: Some bags are books — **follows** (conversion of "Some books are bags").
- "Either I or II" type options apply only when two conclusions form a complementary pair (e.g. Some A are B / No A is B).

6.8.2 Immediate Inference

Operation	From	To
Conversion	E: No S is P / I: Some S are P	No P is S / Some P are S (valid)
Conversion by limitation	A: All S are P	Some P are S
Obversion	All S are P	No S is non-P (change quality, negate predicate)
Contraposition	All S are P	All non-P are non-S

O-propositions cannot be converted.

6.8.3 Conditional (Hypothetical) Statements

Conditional arguments are tested constantly, often disguised in everyday language. Two forms are valid and two are tempting fallacies; learn them as a set of four.

Form	Premises	Conclusion	Valid?
Modus ponens	$P \rightarrow Q, P$	Q	yes
Modus tollens	$P \rightarrow Q, \neg Q$	$\neg P$	yes
Affirming the consequent	$P \rightarrow Q, Q$	P	no
Denying the antecedent	$P \rightarrow Q, \neg P$	$\neg Q$	no

"P only if Q" means $P \rightarrow Q$; "P unless Q" means $\neg Q \rightarrow P$.

6.8.4 Truth & Validity Combinations

Premises	Conclusion	Can the argument be valid?
True	True	Yes
True	False	No — impossible for a valid argument
False	True	Yes
False	False	Yes

6.8.5 More Indian Logic

- **Prama** = valid knowledge; **Aprama** = invalid knowledge (doubt — samshaya, error — viparyaya, hypothetical — tarka).
- **Pratyaksha** types (Nyaya): **Nirvikalpaka** (indeterminate) and **Savikalpaka** (determinate); also laukika (ordinary) and alaukika (extraordinary).
- **Upamana**: knowledge of a word–object relation through similarity (learning that a gavaya resembles a cow).
- **Shabda**: testimony of a reliable person (**apta vakya**); Vedic and secular.
- **Jain logic**: **Anekantavada** (many-sidedness), **Syadvada** — 7 modes of predication (**Saptabhangi naya**): syad asti, syad nasti, syad asti-nasti, syad avaktavya, ...
- **Buddhist logic**: Dignaga and Dharmakirti; accept only perception and inference.
- **Vyapti** is established through **anvaya** (agreement in presence: where smoke, there fire) and **vyatireka** (agreement in absence: where no fire, no smoke).
- Nyaya Sutra author: **Gautama (Akshapada)**; 16 categories (padarthas) beginning with pramana and prameya.

6.9 Previous Year Questions (PYQ pattern)

Square of opposition

1. If "All poets are dreamers" is true, which is false? → **"Some poets are not dreamers" (O)** and **"No poets are dreamers" (E)**
2. If "Some students are not intelligent" (O) is false, then "All students are intelligent" (A) is: **True**
3. Two propositions which cannot both be true but can both be false: **Contraries (A–E)**
4. Two propositions which cannot both be false but can both be true: **Sub-contraries (I–O)**
5. Which proposition distributes only the predicate? **O**

Syllogism

6. Premises: All cats are animals. Some animals are dogs. Conclusion "Some cats are dogs" → **Does not follow** (undistributed middle)
7. Statements: No A is B. All C are B. Conclusion: **No C is A** — follows.
8. Mood and figure of: "All M are P; All S are M; ∴ All S are P" → **AAA-1 (Barbara)**

Deductive/inductive

9. In a deductive argument, the conclusion: **follows necessarily**
10. An argument which is valid and has true premises is: **Sound**
11. Inductive arguments are evaluated as: **strong/weak**

Fallacies

12. "He is a criminal, so his argument about taxes is wrong" — **Ad hominem**
13. "No one has proved ghosts don't exist, so they exist" — **Ad ignorantiam**
14. "I wore a lucky shirt and won — so the shirt causes wins" — **Post hoc (false cause)**

Language

15. Denotation of a term refers to: **the objects to which it applies**
16. "Close the door" serves which function? **Directive**

Indian logic

17. Number of pramanas accepted by Nyaya: **4**
18. Charvakas accept only: **Perception**
19. Advaita Vedanta accepts how many pramanas? **6**
20. In "the hill has fire because it has smoke", the middle term (hetu) is: **smoke**; paksha is: **hill**
21. The invariable relation between hetu and sadhya is: **Vyapti**
22. Inference for others is: **Pararthanumana**
23. Arrange Panchavayava: **Pratijna, Hetu, Udaharana, Upanaya, Nigamana**
24. A hetu that proves the contrary of what is intended: **Viruddha**
25. Knowledge of non-existence is obtained through: **Anupalabdhi**
26. Arthapatti is accepted by: **Mimamsa (Prabhakara & Bhatta) and Advaita**

Analogy

27. Book : Author :: Statue : **Sculptor**

More practice questions

28. Obverse of "All S are P": **No S is non-P**
29. Which proposition cannot be converted? **O**
30. Converse of "All doctors are graduates" by limitation: **Some graduates are doctors**
31. A valid argument cannot have: **true premises and a false conclusion**
32. "If it rains, the match is cancelled. The match is cancelled. So it rained." — **affirming the consequent**
33. Statements: No cat is a dog. All dogs are animals. Conclusion "Some animals are not cats" — **follows**
34. Syadvada is associated with: **Jainism**
35. Number of modes in Saptabhangi: **7**
36. Determinate perception in Nyaya: **Savikalpaka**
37. Vyapti by agreement in absence: **Vyatireka**
38. Author of Nyaya Sutras: **Gautama (Akshapada)**
39. Knowledge from similarity (gavaya-cow): **Upamana**
40. Analogy: Doctor : Hospital :: Teacher : **School**

POINTS TO REMEMBER

- A(S) E(S,P) I(none) O(P) — distribution
- Contradictory A–O, E–I · Contrary A–E · Sub-contrary I–O
- Charvaka 1 · Buddhist 2 · Samkhya 3 · Nyaya 4 · Prabhakara 5 · Bhatta/Advaita 6
- 5 Hetvabhasa: Savyabhichara, Viruddha, Satpratipaksha, Asiddha, Badhita

Data Interpretation

...

Expected questions: 5 (one data set) · Target: 5/5 — pure calculation marks

SYLLABUS COVERAGE

- ☐ Sources, acquisition and classification of data
- ☐ Quantitative and qualitative data
- ☐ Graphical representation (bar chart, histograms, pie chart, table chart, line chart) and mapping of data
- ☐ Data interpretation
- ☐ Data and governance

7.1 Data Basics

Data are recorded observations. Before any calculation, ask what kind of data you have, because that decides what may legitimately be done with it — one can average marks, but not roll numbers or blood groups.

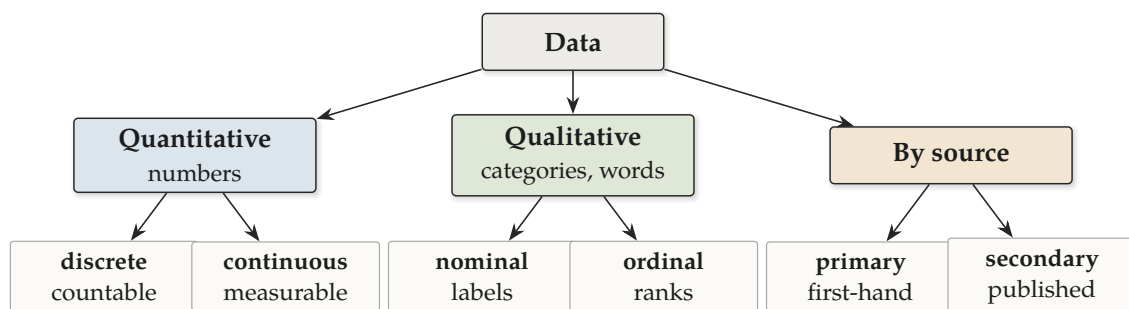


FIGURE 7.1. Ways of classifying data. The type of data decides which summary measures and charts are appropriate.

- Classification bases: **Geographical (spatial)**, **Chronological (temporal)**, **Qualitative**, **Quantitative**.
- Data mapping: choropleth maps (shades by region), dot maps, isopleths, cartograms.

7.2 Graph Types — When to Use

Every chart is designed for one kind of comparison. Examiners sometimes ask which chart suits a given purpose; the four panels below show the most common choices side by side.

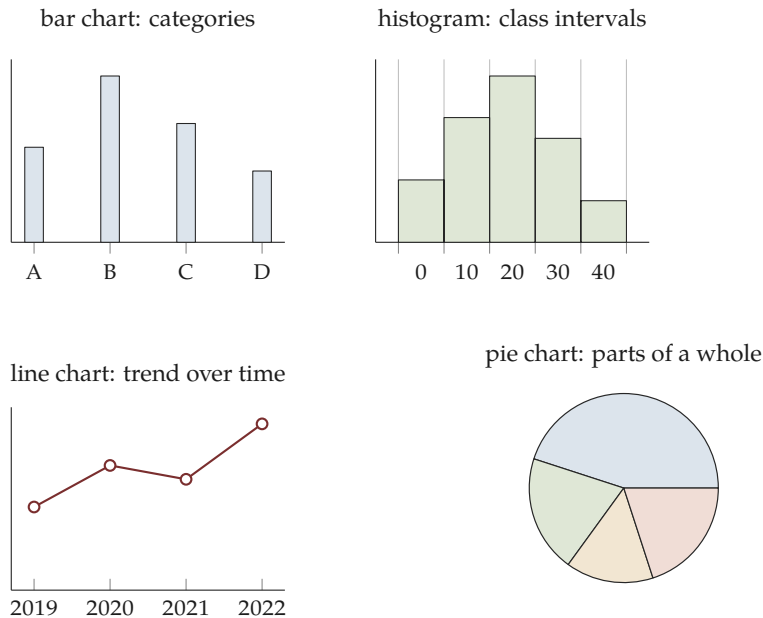


FIGURE 7.2. Choosing a chart. Bars compare separate categories; a histogram’s touching bars show a continuous distribution; a line shows change over time; a pie shows shares of one total.

Graph	Best for	Note
Bar chart	Comparing categories	Gaps between bars
Histogram	Frequency of continuous classes	No gaps ; area $\propto$ frequency
Frequency polygon	Joining midpoints of histogram	
Ogive	Cumulative frequency (less than / more than)	Intersection gives median
Pie chart	Part of a whole	Angle = (value/total) $\times$ 360°
Line chart	Trend over time	
Scatter plot	Correlation between two variables	
Table	Exact values	

One per cent of a pie chart

$$1\% = 3.6^\circ$$

Central angle of a sector

$$\theta = \frac{\text{value}}{\text{total}} \times 360^\circ$$

Percentage from an angle

$$\text{percentage} = \frac{\theta}{3.6}$$

7.3 Measures of Central Tendency & Dispersion

A measure of central tendency gives one representative value; a measure of dispersion tells how widely the values scatter around it. Two classes can share the same average and still be very different — which is why both kinds of measure are needed.

Measure	Formula	Note
Mean	$\Sigma x / n$	Affected by extreme values
Median	Middle value	Best for skewed data
Mode	Most frequent	Only one for nominal data
Empirical relation	Mode = 3 Median – 2 Mean	Moderately skewed
Range	Max – Min	
Variance	$\Sigma(x - \bar{x})^2 / n$	
SD	$\sqrt{\text{Variance}}$	
Coefficient of variation	SD / Mean $\times 100$	Compare consistency

7.4 Speed-Calculation Tricks

The data-interpretation set is the most calculation-heavy part of Paper I. Options are usually far apart, so approximate first and calculate exactly only when two options are close.

Percentage change	$\frac{\text{new} - \text{old}}{\text{old}} \times 100$
Percentage points	simple difference of two percentages
Comparing two ratios	$\frac{a}{b} > \frac{c}{d} \iff ad > bc \quad (b, d > 0)$
Compound annual growth rate over n years	$\left(\frac{\text{end}}{\text{start}}\right)^{1/n} - 1$

$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{7}$	$\frac{1}{8}$	$\frac{1}{9}$	$\frac{1}{11}$	$\frac{1}{12}$	$\frac{1}{16}$
50	33.33	25	20	16.67	14.28	12.5	11.11	9.09	8.33	6.25

Fractions and their percentage equivalents — learn these by heart.

Approximate first, then pick nearest option; options are usually far apart.

7.5 Worked Data Set (typical NET format)

The table below is typical of an examination set: four series over four years, followed by five questions on growth, averages, shares and angles. Work through each question before reading its solution.

Production of cars (thousands) by 4 companies:

Year	P	Q	R	S	Total
2019	40	30	25	15	110
2020	45	35	20	20	120
2021	50	30	30	25	135
2022	60	40	35	25	160

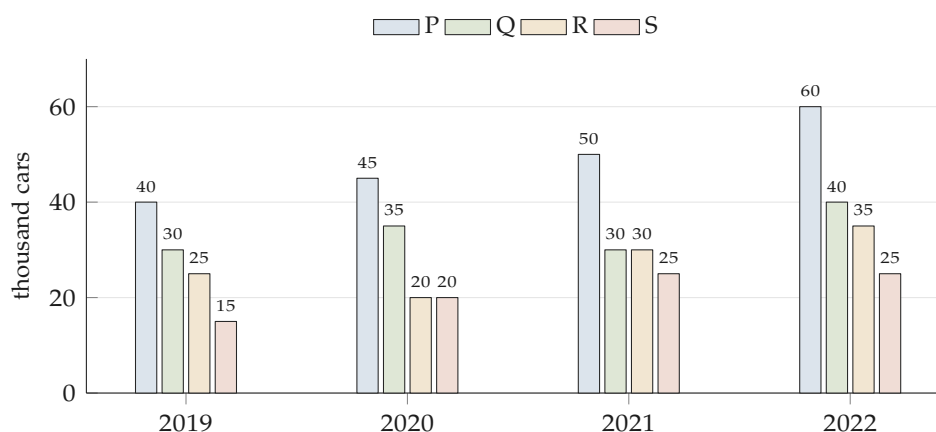


FIGURE 7.3. The worked data set as a grouped bar chart. Company P's steady growth, and R's dip in 2020, are visible at a glance — a reminder to sketch the data before calculating.

1. % increase in total from 2019 to 2022 = $50/110 \times 100 = 45.45\%$
2. Average production of Q = $135/4 = 33.75$
3. In which year did R have the highest share of total? 2019: 22.7%, 2020: 16.7%, 2021: 22.2%, 2022: 21.9% → **2019**
4. Ratio of P(2022) to S(2019+2020) = $60 : 35 = 12 : 7$
5. Pie-chart angle for S in 2021 = $25/135 \times 360 = 66.67^\circ$

7.6 Data and Governance

Governments are now among the largest producers and users of data. Questions in this area concern the institutions, laws and platforms through which public data are collected, protected and shared.

- **Open Government Data (OGD) Platform India** — data.gov.in (NDSAP 2012 — National Data Sharing & Accessibility Policy).
- **Digital Personal Data Protection Act, 2023 (DPDP)** — rights of Data Principal, duties of Data Fiduciary, Data Protection Board.

- **National Data Governance Framework Policy (2022)**, India Data Management Office.
- **Aadhaar (UIDAI, 2009; Aadhaar Act 2016)**, **e-governance**: Digital India (2015), DigiLocker, UMANG, GSTN, PFMS.
- Data governance principles: quality, privacy, security, accountability, transparency, interoperability.
- Census of India — every 10 years (Registrar General & Census Commissioner, MHA); NSO (merged NSSO + CSO, 2019) under MoSPI.

7.7 Deeper Dive — Pie Chart & Line Graph Sets, Common Pitfalls

7.7.1 Pie-Chart Set (*worked*)

A university's annual expenditure of ₹80 crore is distributed as follows:

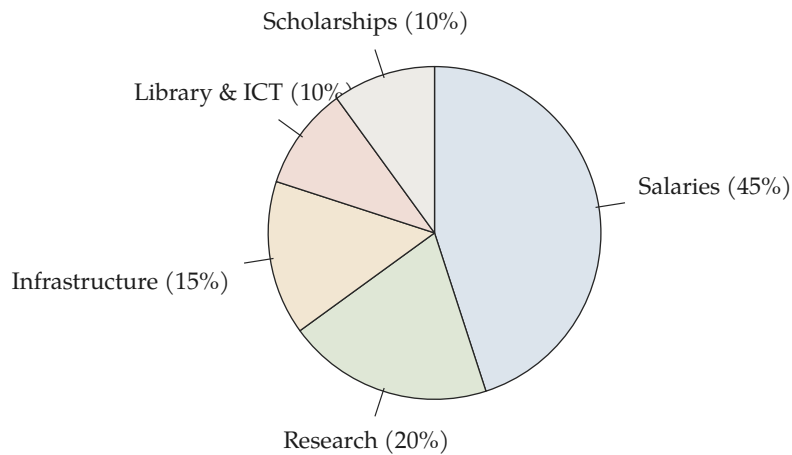


FIGURE 7.4. University expenditure of ₹80 crore by head. Each 1% of the circle is 3.6°.

Head	%	Amount (₹ crore)	Central angle
Salaries	45	36	162°
Research	20	16	72°
Infrastructure	15	12	54°
Library & ICT	10	8	36°
Scholarships	10	8	36°

1. Research exceeds Library & ICT by: $16 - 8 = ₹8 \text{ crore}$ (= 100% more).
2. If salaries rise by 10% and total stays the same, salaries' new share = $39.6/80 = 49.5\%$.
3. Ratio of infrastructure to scholarships = **3 : 2**.
4. Angle for research + scholarships = **108°**.
5. If research grants come only from a ₹12 crore grant plus internal funds, internal funds = $16 - 12 = ₹4 \text{ crore}$.

7.7.2 Two-Series Set (worked)

Number of PhD scholars admitted (two departments):

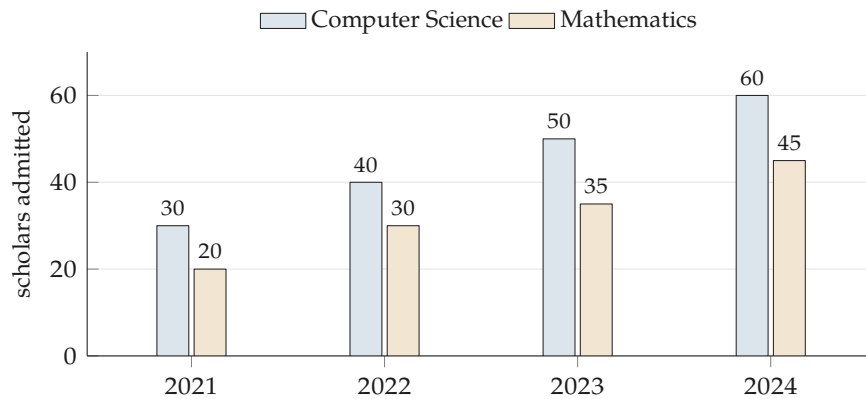


FIGURE 7.5. PhD admissions in two departments.

Year	CS	Maths	Total
2021	30	20	50
2022	40	30	70
2023	50	35	85
2024	60	45	105

1. Year with the highest % growth for Maths: 2022 (50%), 2023 (16.7%), 2024 (28.6%) → **2022**.
2. Average CS admissions = $180/4 = 45$.
3. Total growth 2021→2024 = $(105 - 50)/50 = 110\%$.
4. Ratio CS : Maths over four years = $180 : 130 = 18 : 13$.
5. CAGR of CS (2021→2024, 3 years) = $(60/30)^{(1/3)} - 1 \approx 26\%$.

7.7.3 Pitfalls Examiners Exploit

- **Percentage vs percentage points:** 20% → 25% is a rise of 5 percentage points but 25%.
- **Base year confusion:** “% increase from 2022 to 2023” uses 2022 as the base.
- **Average of percentages** ≠ overall percentage (weights differ).
- **Units:** lakhs vs crores (1 crore = 100 lakh); thousands in the table header.
- Read the **question first**, then compute only what is asked.

7.7.4 Data Mapping & Governance Extras

- **GIS** layers data on maps; **choropleth** uses shading per region; **isopleth** lines join equal values (isotherms, isobars).
- **Data lifecycle:** collection → storage → processing → analysis → sharing → archival/deletion.
- **Metadata** = data about data. **Open data** principles: accessible, machine-readable, licence-free.

7.8 Previous Year Questions (PYQ pattern)

DI questions always come as a set of 5 on one table/graph. Typical stems:

1. "What is the percentage increase in X from year A to year B?"
2. "In which year was the ratio of X to Y the maximum?"
3. "What is the average of X over the given years?"
4. "The central angle corresponding to X in the pie chart is..."
5. "What is the difference between total X and total Y?"
6. "For how many years was X above its average?"

Concept questions:

7. A histogram is used for: **continuous frequency distribution**
8. The median can be obtained graphically from: **Ogives (intersection of less-than and more-than)**
9. Data collected by the researcher for the first time: **Primary data**
10. Census data used by a researcher is: **Secondary data**
11. Pie chart: if a sector is 72° , it represents **20%**
12. Classification of data by time is called: **Chronological**
13. Which measure is best for ordinal data? **Median**
14. Which portal provides open government data in India? **data.gov.in**
15. Coefficient of variation is used to compare: **consistency/variability of two series**

More practice questions

16. A pie-chart sector of 54° represents what %? **15%**
17. If total = ₹500 crore and a sector is 18%, amount = **₹90 crore**
18. Rise from 40% to 50% is how many percentage points? **10**, and what % increase? **25%**
19. Average of 20, 30, 40 with frequencies 2, 3, 5 = $(40 + 90 + 200)/10 = 33$
20. Lines joining places of equal rainfall on a map: **isohyets** (a type of isopleth)
21. A map using colour shades by state for literacy rate: **choropleth map**

POINTS TO REMEMBER

- $1\% = 3.6^\circ$ · Mode = $3\text{Median} - 2\text{Mean}$ · $CV = SD/\text{Mean} \times 100$
- Histogram no gaps; bar chart gaps · Ogive → median
- Learn fraction ↔ percent table by heart

Information & Communication Technology (ICT)

...

Expected questions: ~5 (10 marks) · Target: 5/5 — easiest unit for a CS candidate

SYLLABUS COVERAGE

- ☐ ICT: general abbreviations and terminology
 - ☐ Basics of Internet, Intranet, E-mail, Audio and Video-conferencing
 - ☐ Digital initiatives in higher education
 - ☐ ICT and Governance
-

8.1 Computer Basics

For Paper I you need a clear picture of what the parts of a computer do rather than how they are built. Every computer, from a phone to a supercomputer, follows the same plan: data enter through input devices, are processed by the CPU using instructions held in memory, and leave through output devices.

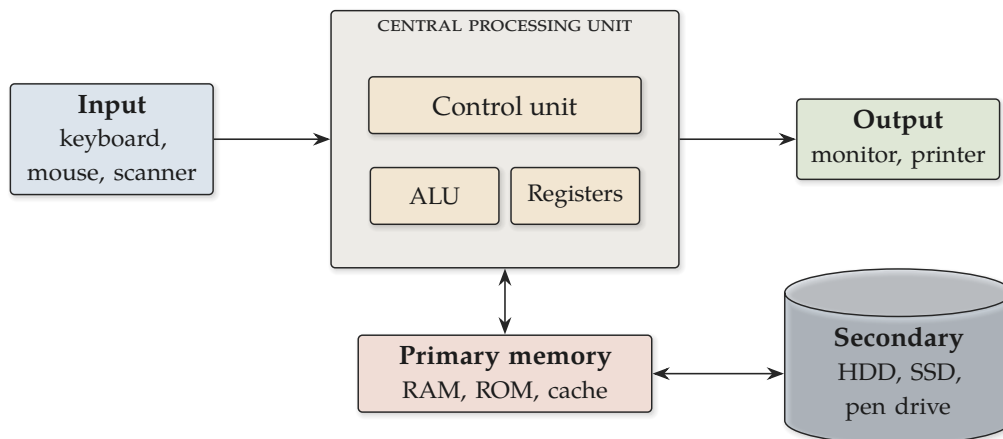


FIGURE 8.1. The basic organisation of a computer. The CPU fetches instructions and data from primary memory; secondary storage holds them permanently.

8.1.1 Memory hierarchy (fast/costly at top)

No single memory technology is both fast and cheap, so computers combine several in a hierarchy. Frequently used data are kept in the small, fast levels near the processor; everything else waits in the large, slow levels below.

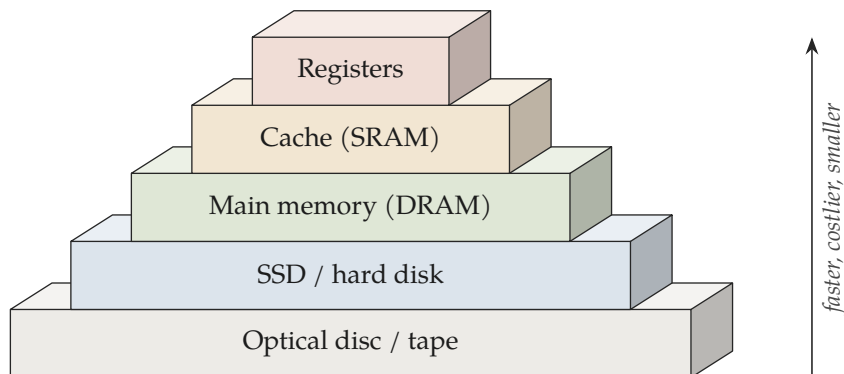


FIGURE 8.2. The memory hierarchy. Moving up, each level is faster and more expensive per byte but smaller.

8.1.2 Units

1 nibble	4 bits	1 KB	2^{10} B	1 PB	2^{50} B
1 byte	8 bits	1 MB	2^{20} B	1 EB	2^{60} B
		1 GB	2^{30} B	1 ZB	2^{70} B
		1 TB	2^{40} B	1 YB	2^{80} B

8.1.3 Number Systems (frequent question!)

Computers store everything in binary because electronic circuits have two reliable states. Octal and hexadecimal are compact ways of writing binary: each octal digit stands for three bits and each hexadecimal digit for four.

- Binary (2), Octal (8), Decimal (10), Hexadecimal (16).
- $(25)_{10} = (11001)_2 = (31)_8 = (19)_{16}$
- $(1010.101)_2 = 10.625$
- Group 3 bits $\rightarrow$ octal, 4 bits $\rightarrow$ hex.

8.2 Abbreviations (must know)

Abbr.	Full form
ALU	Arithmetic Logic Unit
BIOS	Basic Input Output System
CMOS	Complementary Metal Oxide Semiconductor
DNS	Domain Name System
DHCP	Dynamic Host Configuration Protocol

Abbr.	Full form
FTP	File Transfer Protocol
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IMAP	Internet Message Access Protocol
ISP	Internet Service Provider
LAN/MAN/WAN	Local/Metropolitan/Wide Area Network
MODEM	Modulator–Demodulator
OCR / OMR / MICR	Optical Character / Optical Mark / Magnetic Ink Character Recognition
POP3	Post Office Protocol v3
SMTP	Simple Mail Transfer Protocol
TCP/IP	Transmission Control Protocol / Internet Protocol
URL	Uniform Resource Locator
VoIP	Voice over Internet Protocol
VPN	Virtual Private Network
Wi-Fi	Wireless Fidelity
GIF, JPEG, PNG	Graphics Interchange Format, Joint Photographic Experts Group, Portable Network Graphics
PDF	Portable Document Format
CC / BCC	Carbon Copy / Blind Carbon Copy
IoT	Internet of Things
LMS	Learning Management System (Moodle)
NIC	National Informatics Centre
CERT-In	Indian Computer Emergency Response Team

8.3 Internet, Intranet, Extranet

The same Internet technologies — TCP/IP, web browsers, e-mail — can be used on a private network (intranet), shared with trusted partners (extranet) or opened to the world (Internet). What differs is who is allowed in.

- **Internet** originated from **ARPANET (1969)**, US DoD. **WWW** invented by **Tim Berners-Lee (1989, CERN)**.
- India's first: **ERNET (1986)**; public internet by **VSNL on 15 Aug 1995**.
- IPv4 = 32 bits; IPv6 = 128 bits. Domain types: .com, .org, .edu, .gov, .ac.in, .nic.in.
- Browser = client software (Chrome, Firefox); Search engine = website (Google, Bing).
- **Cookies** = small data files stored by websites in the browser.

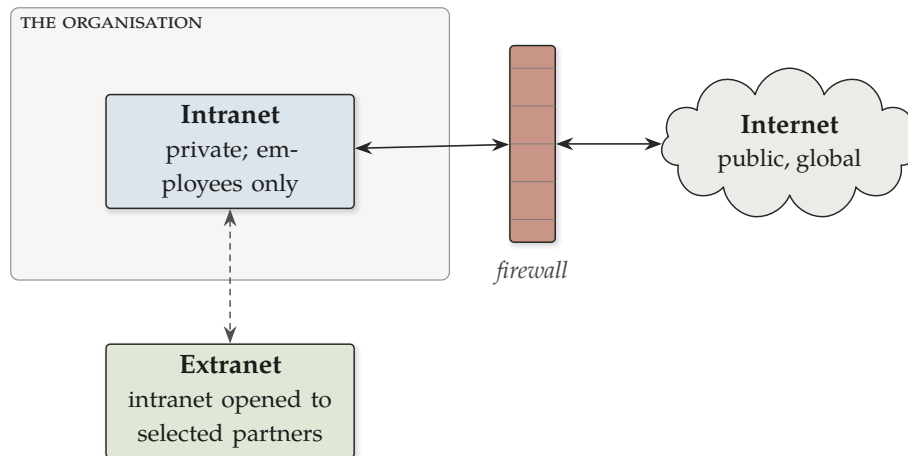


FIGURE 8.3. Intranet, extranet and Internet. A firewall filters traffic between the private network and the public Internet.

8.4 E-mail

E-mail is a store-and-forward system: messages wait on servers until the recipient collects them, which is why sender and recipient need not be online at the same time.

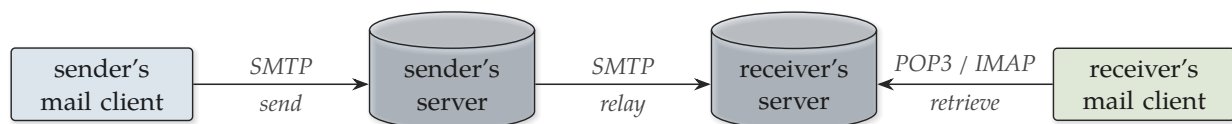


FIGURE 8.4. How e-mail travels. SMTP *pushes* mail towards the recipient's server; POP3 or IMAP lets the recipient *pull* it down.

- **SMTP** = sending (port 25/587); **POP3** = download & delete (110); **IMAP** = sync, mail stays on server (143).
- First email: **Ray Tomlinson (1971)**, introduced @.
- **BCC** recipients are hidden from others. **Spam** = unsolicited mail. **Phishing** = fraudulent mail to steal data.
- Attachments: MIME (Multipurpose Internet Mail Extensions).

8.5 Audio & Video Conferencing

- Real-time communication over IP: Zoom, Google Meet, Microsoft Teams, Webex; government: **JioMeet, Bharat VC (NIC's)**.
- Protocols: **H.323, SIP** (Session Initiation Protocol), RTP. Codecs compress audio/video.
- Synchronous (live) vs asynchronous (recorded, forums, email) learning.
- Webinar = web seminar; Podcast = audio episodes; Vodcast = video podcast.

8.6 Digital Initiatives in Higher Education

These initiatives, coordinated largely by the Ministry of Education, UGC, AICTE and INFLIBNET, together form India's digital infrastructure for higher education. Learn each name with its purpose.

Initiative	Purpose
SWAYAM	MOOCs (2017)
SWAYAM PRABHA	DTH educational TV channels
NDL India	National Digital Library (IIT Kharagpur)
e-PG Pathshala	PG e-content (UGC/INFLIBNET)
Shodhganga / Shodhgangotri	Theses / synopses repository
e-ShodhSindhu	E-journal consortium (merged UGC-INFONET, N-LIST, INDEST)
ONOS (One Nation One Subscription)	Nation-wide access to scholarly journals (2024/25)
NAD / DigiLocker	National Academic Depository — digital certificates
ABC	Academic Bank of Credits (NEP 2020)
NIRF	National Institutional Ranking Framework (2015)
AISHE	All India Survey on Higher Education
Virtual Labs, Spoken Tutorial, e-Yantra, FOSSEE	Lab/skill initiatives
SAMARTH	e-Governance suite for universities
VIDWAN	Database of experts (INFLIBNET)
PM e-VIDYA	Multi-mode access to digital education (2020)
NEAT	National Educational Alliance for Technology (AICTE, AI-based adaptive learning)
DIKSHA	Teachers' platform

8.7 ICT & Governance

E-governance uses ICT to deliver government services more quickly, transparently and cheaply. It is usually described by the parties it connects.

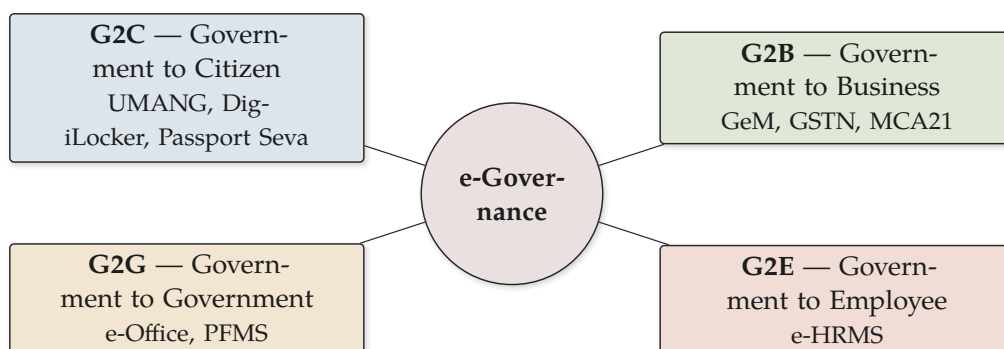


FIGURE 8.5. The four relationships served by e-governance, with representative Indian platforms.

- **Digital India** (1 July 2015) — 9 pillars incl. broadband highways, e-Kranti, Information for All.

- Key platforms: **UMANG** (unified app), **DigiLocker**, **GeM** (Government e-Marketplace), **BHIM-UPI** (NPCI), **CoWIN**, **MyGov**, **e-Hospital**, **Bhashini** (language AI).
- **IT Act 2000** (amended 2008) — legal recognition of e-records & digital signatures; Sec 66A struck down (Shreya Singhal, 2015). **DPDP Act 2023**.
- Cyber security: firewall, antivirus, encryption, 2FA. Malware: virus (needs host), worm (self-replicating), Trojan (disguised), ransomware, spyware.

8.8 Deeper Dive — Generations, Software, File Types & Emerging Tech

8.8.1 Generations of Computers

Generation	Period	Technology	Example
1st	1940–56	Vacuum tubes; machine language	ENIAC, UNIVAC
2nd	1956–63	Transistors; assembly, FORTRAN, COBOL	IBM 1401
3rd	1964–71	Integrated circuits; OS, multiprogramming	IBM 360
4th	1971–present	Microprocessors (VLSI); PCs, GUI	Intel 4004 (first microprocessor)
5th	Present & beyond	AI, ULSI, parallel processing, quantum	—

India: **PARAM 8000 (1991)** by C-DAC (first Indian supercomputer); **National Supercomputing Mission (2015)**; **AIRAWAT** (AI supercomputer).

8.8.2 Software

Hardware does nothing without instructions. Software is commonly divided into the programs that run the computer itself and the programs that do useful work for the user.

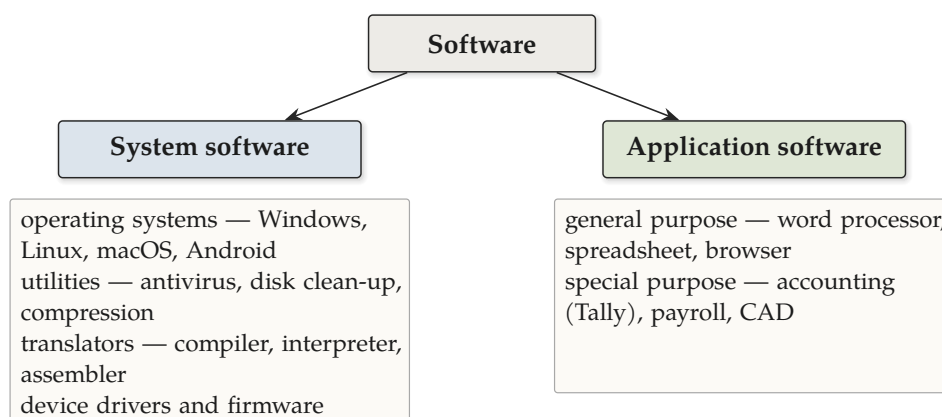


FIGURE 8.6. System software manages the machine; application software serves the user's tasks.

- **Open source** (source available, free to modify: Linux, LibreOffice, Moodle) vs **proprietary** (MS Office).
- **Freeware** (free, closed), **shareware** (trial), **firmware** (software in ROM/flash).

8.8.3 File Extensions & Office Shortcuts

Extension	Type	Extension	Type
.docx	Word document	.xlsx	Spreadsheet
.pptx	Presentation	.pdf	Portable document
.txt	Plain text	.csv	Comma-separated values
.jpg/.png/.gif	Images	.mp3/.wav	Audio
.mp4/.avi	Video	.zip/.rar	Compressed
.exe	Executable (Windows)	.html	Web page

Shortcut	Action	Shortcut	Action
Ctrl + C / X / V	Copy / cut / paste	Ctrl + Z / Y	Undo / redo
Ctrl + S	Save	Ctrl + P	Print
Ctrl + F	Find	Ctrl + H	Replace
Ctrl + A	Select all	F7	Spell check (Word)
F5	Slideshow / refresh	Alt + F4	Close window

Spreadsheet: cell address = column letter + row number (B3); **absolute reference** \$B\$3; functions SUM, AVERAGE, COUNT, IF, VLOOKUP.

8.8.4 Emerging Technologies (frequently asked one-liners)

Term	Meaning
Cloud computing	On-demand computing over the internet: SaaS, PaaS, IaaS
IoT	Physical devices with sensors connected to the internet
Artificial Intelligence	Machines performing tasks needing human intelligence; ChatGPT, Bhashini
Machine learning	Systems that learn patterns from data
Blockchain	Distributed, tamper-evident ledger (Bitcoin)
Big data	Volume, velocity, variety (+ veracity, value)
AR / VR	Augmented reality overlays digital on real; virtual reality is fully immersive
5G	Fifth-generation mobile; launched in India on 1 Oct 2022
Quantum computing	Qubits, superposition; India's National Quantum Mission (2023)
Open educational resources (OER)	Free, openly licensed learning materials (Creative Commons)

8.8.5 Internet Basics Extended

- Web 1.0 (read-only) → Web 2.0 (read-write, social media) → Web 3.0 (semantic, decentralised).
- **Search operators:** "exact phrase", site:ac.in, filetype:pdf, minus sign to exclude.
- **Bandwidth** measured in bps; **latency** in ms. Wired (Ethernet, fibre) vs wireless (Wi-Fi, 4G/5G, satellite).
- **IP address** identifies a device; **MAC address** identifies a network card (48 bits).

8.9 Previous Year Questions (PYQ pattern)

1. $(1011)_2 + (0111)_2 = ? \rightarrow (10010)_2$
2. $(237)_8$ in decimal = $2 \times 64 + 3 \times 8 + 7 = 159$
3. 1 TB equals: **1024 GB**
4. Which protocol is used to send e-mail? **SMTP**
5. Which protocol keeps mail on the server and syncs folders? **IMAP**
6. BCC stands for: **Blind Carbon Copy**
7. Which is NOT an input device? (a) scanner (b) light pen (c) plotter (d) OMR — **Ans: (c)**
8. WWW was invented by: **Tim Berners-Lee**
9. The internet originated from: **ARPANET**
10. Private network inside an organisation using internet technology: **Intranet**
11. URL stands for: **Uniform Resource Locator**
12. Which is a volatile memory? **RAM**
13. Which converts domain names to IP addresses? **DNS**
14. IPv6 address length: **128 bits**
15. "Academic Bank of Credits" is an initiative under: **NEP 2020**
16. The portal providing Indian theses: **Shodhganga**
17. The national ranking framework for HEIs: **NIRF**
18. Which is a G2B service? **GeM / GSTN**
19. Malware that replicates itself without a host program: **Worm**
20. Fraudulent attempt to obtain credentials via email: **Phishing**
21. Video conferencing protocol: **H.323 / SIP**
22. The Information Technology Act was passed in: **2000**
23. Arrange in increasing order of capacity: **KB < MB < GB < TB < PB**

More practice questions

24. The first Indian supercomputer: **PARAM 8000**
25. Integrated circuits were used in: **third-generation computers**
26. Which is open-source software? (a) MS Word (b) LibreOffice Writer (c) Photoshop (d) Windows — **Ans: (b)**
27. Absolute cell reference in a spreadsheet: **\$A\$1**
28. Shortcut to undo: **Ctrl + Z**
29. A .csv file stores: **tabular data as comma-separated text**
30. Web 2.0 is characterised by: **user-generated content and interactivity**
31. Software stored permanently in ROM: **firmware**

32. Which is NOT system software? (a) OS (b) compiler (c) spreadsheet (d) device driver — **Ans:** (c)
33. 5G services were launched in India in: **October 2022**
34. Search operator to restrict results to a website: **site:**

POINTS TO REMEMBER

- SMTP send · POP3 download · IMAP sync
- ARPANET 1969 · WWW 1989 (Berners-Lee) · India public internet 1995 (VSNL)
- IT Act 2000 · Digital India 2015 · DPDP Act 2023

People, Development & Environment

— . . . —

Expected questions: ~5 (10 marks) · Target: 4–5/5

SYLLABUS COVERAGE

- ☐ Development and environment: Millennium Development and Sustainable Development Goals
 - ☐ Human and environment interaction: anthropogenic activities and their impacts on environment
 - ☐ Environmental issues: local, regional and global; air, water, soil, noise pollution; waste (solid, liquid, biomedical, hazardous, electronic); climate change and its socio-economic and political dimensions
 - ☐ Impacts of pollutants on human health
 - ☐ Natural and energy resources: solar, wind, soil, hydro, geothermal, biomass, nuclear and forests
 - ☐ Natural hazards and disasters: mitigation strategies
 - ☐ Environmental Protection Act (1986), NAPCC, international agreements/efforts: Montreal Protocol, Rio Summit, CBD, Kyoto Protocol, Paris Agreement, ISA
-

9.1 MDGs → SDGs

Development was long measured by income alone. The global goals reflect a broader view: development must also reduce poverty, improve health and education, and do so without exhausting the environment on which future generations depend.



FIGURE 9.1. From the Millennium Development Goals to the Sustainable Development Goals.

8 MDGs: (1) eradicate extreme poverty & hunger (2) universal primary education (3) gender equality (4) reduce child mortality (5) improve maternal health (6) combat HIV/AIDS, malaria (7) environmental sustainability (8) global partnership.

17 SDGs (memorise numbers — directly asked):

#	Goal	#	Goal
1	No Poverty	10	Reduced Inequalities
2	Zero Hunger	11	Sustainable Cities
3	Good Health & Well-being	12	Responsible Consumption & Production
4	Quality Education	13	Climate Action
5	Gender Equality	14	Life Below Water
6	Clean Water & Sanitation	15	Life on Land
7	Affordable & Clean Energy	16	Peace, Justice & Strong Institutions
8	Decent Work & Economic Growth	17	Partnerships for the Goals
9	Industry, Innovation & Infrastructure		

5 Ps of SDGs: People, Planet, Prosperity, Peace, Partnership. In India, **NITI Aayog** monitors (SDG India Index, 2018).

Sustainable development (Brundtland Report "Our Common Future", 1987): *development that meets the needs of the present without compromising the ability of future generations to meet their own needs.*

Air	Water	Soil	Noise
PM _{2.5} , PM ₁₀ — lungs, heart SO ₂ , NO _x — acid rain CO — binds haemoglobin ground-level O ₃ lead — neurotoxic	fluoride — fluorosis arsenic — skin lesions mercury — Minamata cadmium — itai-itai nitrate — blue-baby	pesticides, fertilisers heavy metals salinisation	above 85 dB harmful hearing loss stress, sleep loss

FIGURE 9.2. Major forms of pollution and their best-known health effects.

9.2 Pollution — Sources & Health Effects

Pollution is the addition of substances or energy to the environment faster than it can disperse or neutralise them. Questions often pair a pollutant with the disease or disaster it caused, so learn the pairs in the figure and table together.

Disease/Event	Cause
Minamata disease	Mercury (Japan)
Itai-itai	Cadmium (Japan)
Blue baby syndrome (methaemoglobinaemia)	Nitrates in water
Black-foot disease	Arsenic
Fluorosis	Excess fluoride
Bhopal gas tragedy (1984)	Methyl isocyanate (MIC)
Chernobyl (1986)	Nuclear accident (Ukraine)
Silicosis	Silica dust
Asbestosis	Asbestos
London smog (1952)	SO ₂ + smoke (classical/sulphurous smog)
Photochemical smog	NO _x + VOC + sunlight → O ₃ , PAN

- **NAAQS** (CPCB): PM_{2.5} annual 40 µg/m³, 24-hr 60; PM₁₀ annual 60, 24-hr 100. **AQI** has 8 pollutants (PM₁₀, PM_{2.5}, NO₂, SO₂, CO, O₃, NH₃, Pb), 6 categories.
- **WHO 2021** guideline PM_{2.5} annual: 5 µg/m³.
- Greenhouse gases: CO₂, CH₄, N₂O, HFCs, PFCs, SF₆ (+ NF₃ under Doha amendment). GWP: CO₂=1, CH₄≈28, N₂O≈265, SF₆≈23,500.
- Ozone depletion: **CFCs** (chlorine radicals); ozone measured in **Dobson Units**; ozone layer in stratosphere.

9.3 Waste Management

India's 2016 set of waste rules shifted responsibility to the generators of waste and to producers (extended producer responsibility). The guiding principle is the waste hierarchy: the best waste is the waste never produced.

Waste	Rules in India
Solid waste	Solid Waste Management Rules 2016 — segregation at source (wet, dry, hazardous)
Plastic	Plastic Waste Management Rules 2016 (amended 2021/22) — single-use plastic ban from 1 July 2022
E-waste	E-Waste (Management) Rules 2016 → 2022 (EPR — Extended Producer Responsibility)
Bio-medical	Bio-Medical Waste Management Rules 2016 — colour-coded bins (yellow, red, white, blue)
Hazardous	Hazardous & Other Wastes Rules 2016
C&D waste	Construction & Demolition Waste Rules 2016

Waste hierarchy (most → least preferred): **Prevent → Reduce → Reuse → Recycle → Recover → Dispose.**

9.4 Natural & Energy Resources

India's energy transition — from coal towards solar, wind and other renewables — is a frequent source of questions, both for its targets and for its geography.

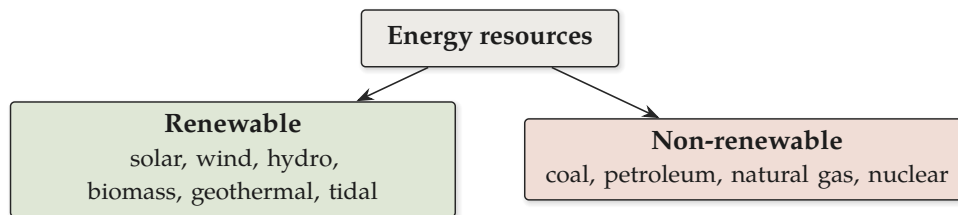


FIGURE 9.3. Renewable resources are replenished by nature within a human lifetime; non-renewable ones are not.

- India's targets: **500 GW non-fossil capacity by 2030; Net zero by 2070** (Panchamrit, COP26 Glasgow).
- Largest share of India's electricity: **coal (thermal)**. Largest renewable: **solar** (overtook wind).
- Top states: solar — Rajasthan, Gujarat; wind — Gujarat, Tamil Nadu.
- Nuclear: Tarapur (first, 1969). Geothermal sites: Puga Valley (Ladakh). Tidal potential: Gulf of Khambhat, Kutch.
- India's forest cover ~21.7% (ISFR 2021); tree + forest ~25%. National Forest Policy 1988 target: **33%**.

9.5 Natural Hazards & Disasters

A hazard becomes a disaster only when it strikes people who are vulnerable and unprepared. Disaster management therefore works continuously, not only after the event.

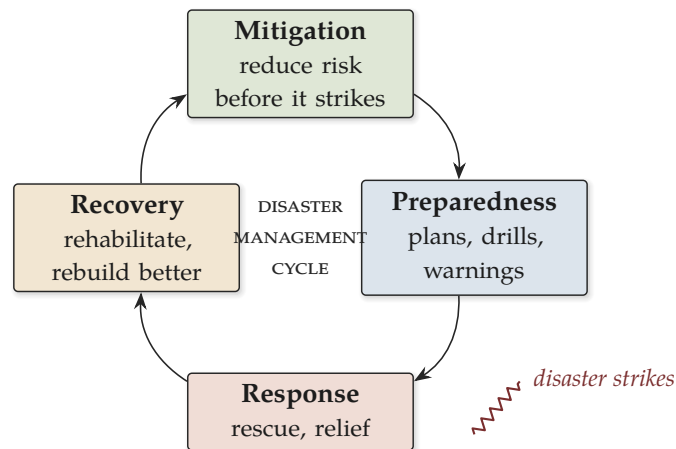


FIGURE 9.4. The disaster management cycle. Mitigation and preparedness come *before* the event; response and recovery follow it.

- **Disaster Management Act 2005** → **NDMA** (chaired by PM), **SDMA** (CM), **DDMA** (Collector). **NDRF** (response force).
- **Sendai Framework (2015–2030)** — 4 priorities, 7 targets (succeeded Hyogo Framework 2005–2015).
- Hazards: earthquake (Richter/moment magnitude), cyclone, flood, drought, landslide, tsunami (2004 Indian Ocean).
- Risk = Hazard × Vulnerability / Capacity.

9.6 Environmental Laws & International Agreements

Environmental law in India grew in response to specific crises — the Bhopal gas tragedy led directly to the Environment (Protection) Act, 1986. Internationally, cooperation advanced through a series of conferences and protocols whose years and subjects are a staple of the examination.

9.6.1 Indian Acts

Act	Year
Wildlife Protection Act	1972
Water (Prevention & Control of Pollution) Act	1974 (created CPCB)
Forest (Conservation) Act	1980
Air (Prevention & Control of Pollution) Act	1981
Environment (Protection) Act	1986 (post-Bhopal; umbrella act)
Biological Diversity Act	2002
National Green Tribunal Act	2010

NAPCC (2008) — 8 National Missions: Solar, Enhanced Energy Efficiency, Sustainable Habitat, Water, Sustaining Himalayan Ecosystem, Green India, Sustainable Agriculture, Strategic Knowledge on Climate Change.

9.6.2 International

- 1972 ● Stockholm Conference — UNEP created
- 1985 ● Vienna Convention for the ozone layer
- 1987 ● Montreal Protocol on ozone-depleting substances; Brundtland Report
- 1992 ● Rio Earth Summit — UNFCCC, CBD, Agenda 21
- 1997 ● Kyoto Protocol — binding targets for developed countries (in force 2005)
- 2002 ● Johannesburg Summit (Rio+10)
- 2012 ● Rio+20 — “The Future We Want”
- 2015 ● Paris Agreement (COP21): well below 2 °C, NDCs; International Solar Alliance
- 2016 ● Kigali Amendment — phase-down of HFCs
- 2021 ● COP26 Glasgow — India’s net-zero target of 2070

FIGURE 9.5. Milestones in global environmental cooperation.

Agreement	Key point
Ramsar (1971)	Wetlands
CITES (1973)	Trade in endangered species
Montreal (1987)	Ozone-depleting substances; most successful treaty
Basel (1989)	Transboundary hazardous waste
CBD (1992)	Biodiversity; Cartagena (biosafety, 2000), Nagoya (ABS, 2010), Kunming-Montreal GBF (2022, 30×30)
Kyoto (1997)	CDM, Joint Implementation, Emissions Trading; Annex-I countries' targets; 1st commitment 2008–12
Paris (2015)	NDCs every 5 years; global stocktake
Stockholm Convention (2001)	POPs (persistent organic pollutants)
Minamata Convention (2013)	Mercury
ISA	International Solar Alliance — launched by India & France at COP21 (2015); HQ Gurugram; “One Sun One World One Grid”

9.7 Deeper Dive — Climate Science, COP Timeline, Indian Programmes & Disasters

9.7.1 Greenhouse Effect & Climate Change

The greenhouse effect itself is natural and essential — without it the Earth would be about 33 °C colder. The problem is its *enhancement* by human emissions, which traps extra heat.

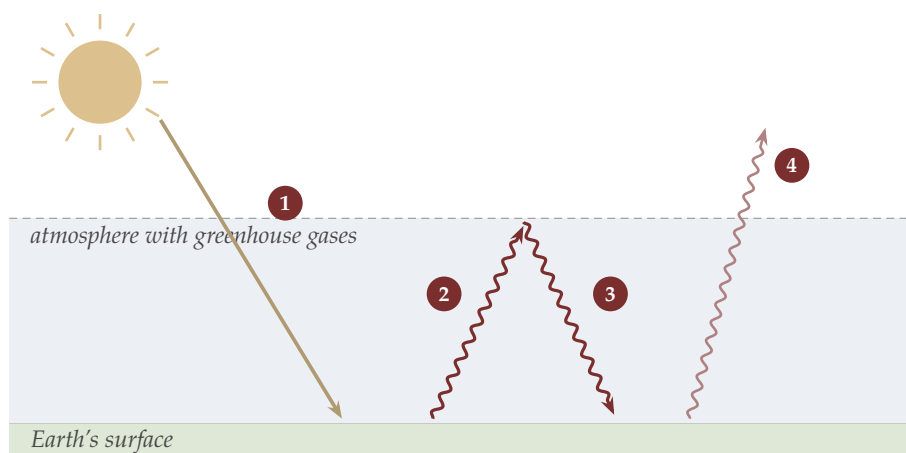


FIGURE 9.6. The greenhouse effect. (1) Short-wave sunlight passes through the atmosphere and warms the surface. (2) The warm surface radiates long-wave infrared, which greenhouse gases absorb. (3) The gases re-emit much of it back towards the surface. (4) The rest escapes to space. More greenhouse gas means more re-emission (3) and a warmer lower atmosphere.

- Pre-industrial CO₂ $\approx$ 280 ppm; now > 420 ppm. Largest anthropogenic GHG by volume: CO₂; methane is far more potent per molecule over 20 years.
- IPCC (1988, UNEP + WMO) assessment reports; AR6 (2021–23): warming about 1.1 °C above pre-industrial.
- Impacts: sea-level rise, glacier retreat, heatwaves, extreme rainfall, ocean acidification, coral bleaching, crop yield changes, vector-borne diseases.
- **Socio-economic & political dimensions:** climate justice; **CBDR-RC** (Common But Differentiated Responsibilities and Respective Capabilities) — principle from Rio 1992; loss and damage fund (COP27, Sharm el-Sheikh 2022); climate finance (USD 100 billion goal; NCQG at COP29 Baku 2024).
- Mitigation (reduce emissions) vs **adaptation** (adjust to impacts).
- **Carbon credit** = 1 tonne CO₂-equivalent reduced. India's Carbon Credit Trading Scheme (2023). **LiFE** (Lifestyle for Environment) mission launched by India at COP26.

9.7.2 Conference of Parties (COP) — Key Milestones

COP	Year & place	Outcome
COP1	1995 Berlin	Berlin Mandate
COP3	1997 Kyoto	Kyoto Protocol
COP8	2002 New Delhi	Delhi Declaration
COP13	2007 Bali	Bali Action Plan
COP15	2009 Copenhagen	Copenhagen Accord
COP16	2010 Cancun	Green Climate Fund
COP21	2015 Paris	Paris Agreement; ISA launched
COP26	2021 Glasgow	Glasgow Climate Pact; India Panchamrit, net zero 2070
COP27	2022 Sharm el-Sheikh	Loss & damage fund
COP28	2023 Dubai	First global stocktake; "transition away from fossil fuels"
COP29	2024 Baku	New climate finance goal (NCQG)

9.7.3 Indian Environmental Programmes

Programme	Year	Aim
Ganga Action Plan	1985	River cleaning → Namami Gange (2014)
National Clean Air Programme (NCAP)	2019	Cut PM by 20–30% (revised 40%) in non-attainment cities
Swachh Bharat Mission	2014	Sanitation, ODF
Pradhan Mantri Ujjwala Yojana	2016	LPG to poor households (reduces indoor air pollution)
UJALA	2015	LED bulbs — energy efficiency
National Green Hydrogen Mission	2023	Green hydrogen production
Nagar Van Yojana	2020	Urban forests
Green India Mission	NAPCC	Increase forest/tree cover
Bharat Stage VI emission norms	2020	Leapfrogged from BS-IV

Environmental movements: **Chipko** (1973, Uttarakhand — Sunderlal Bahuguna, Gaura Devi), **Appiko** (1983, Karnataka), **Silent Valley** (Kerala), **Narmada Bachao Andolan** (Medha Patkar), **Bishnoi** sacrifice (Khejarli, 1730, Amrita Devi).

9.7.4 Disasters — Scales & Facts

Hazard	Measure / fact
Earth-quake	Magnitude (Richter / moment magnitude — logarithmic); intensity (Modified Mercalli, I–XII); India has seismic zones II–V (Zone V highest: NE, J&K, Kutch)
Cyclone	IMD categories; named by WMO/ESCAP panel; Bay of Bengal more cyclone-prone than Arabian Sea
Tsunami	Caused by undersea earthquakes; INCOIS Hyderabad runs the warning centre
Flood	Most frequent disaster in India; Assam, Bihar most affected
Drought	Meteorological, hydrological, agricultural
Landslide	Himalayas and Western Ghats
Heat wave	Declared by IMD when max temp $\geq 40^{\circ}\text{C}$ (plains) and departure $\geq 4.5^{\circ}\text{C}$

Disaster management cycle: **Mitigation** → **Preparedness** → **Response** → **Recovery**.

9.7.5 Renewable Energy — India Snapshot

- Installed renewable capacity crossed **200 GW** (2024); solar is the largest renewable source.
- Largest solar park: **Bhadla (Rajasthan)**; large hybrid park: **Khavda (Gujarat)**.
- **PM-KUSUM** (solar pumps for farmers), **PM Surya Ghar Muft Bijli Yojana** (2024, rooftop solar).
- Hydro: Tehri dam (tallest in India); nuclear plants: Kudankulam (largest), Tarapur (oldest).

9.8 Previous Year Questions (PYQ pattern)

1. How many SDGs and targets? **17 goals, 169 targets**
2. SDG 4 is about: **Quality education**; SDG 13: **Climate action**
3. The Brundtland report is titled: **Our Common Future (1987)**
4. Minamata disease is caused by: **Mercury**
5. Blue baby syndrome is caused by excess: **Nitrates**
6. Bhopal gas tragedy gas: **Methyl isocyanate**
7. Which is NOT a greenhouse gas? (a) CO_2 (b) CH_4 (c) N_2 (d) N_2O — **Ans: (c)**
8. Montreal Protocol relates to: **ozone-depleting substances**
9. Kigali Amendment aims to phase down: **HFCs**
10. Kyoto Protocol came into force in: **2005**
11. Paris Agreement aims to limit warming to: **well below 2°C , pursue 1.5°C**
12. ISA headquarters: **Gurugram, India**
13. Environment (Protection) Act enacted in: **1986**
14. Number of missions in NAPCC: **8**
15. NDMA is chaired by: **Prime Minister**
16. Sendai Framework period: **2015–2030**
17. Arrange chronologically: Stockholm (1972), Montreal (1987), Rio (1992), Kyoto (1997), Paris (2015)

18. Which pollutant is NOT included in India's AQI? (a) PM_{2.5} (b) NH₃ (c) CO₂ (d) Pb — **Ans: (c)**
19. India's net-zero target year: **2070**
20. Ozone thickness is measured in: **Dobson units**
21. Which is a renewable resource? **Geothermal**
22. E-waste rules make producers responsible through: **Extended Producer Responsibility (EPR)**
23. CBD was opened for signature at: **Rio Earth Summit 1992**

More practice questions

24. The IPCC was established in: **1988 by UNEP and WMO**
25. The principle of CBDR originated at: **Rio Earth Summit 1992**
26. The loss and damage fund was agreed at: **COP27, Sharm el-Sheikh**
27. COP28 was held in: **Dubai (2023)**
28. National Clean Air Programme was launched in: **2019**
29. Chipko movement began in: **1973 (Uttarakhand)**
30. India's highest seismic zone: **Zone V**
31. Tsunami early-warning centre of India is at: **INCOIS, Hyderabad**
32. The most frequent natural disaster in India: **floods**
33. The LiFE mission was announced by India at: **COP26 Glasgow**
34. Bhadla solar park is in: **Rajasthan**
35. Arrange COPs chronologically: Kyoto (COP3) → Copenhagen (COP15) → Paris (COP21) → Glasgow (COP26)

POINTS TO REMEMBER

- MDG 8 (2000–15) · SDG 17/169 (2015–30)
- Hg–Minamata · Cd–Itai-itai · NO₃–blue baby · As–black foot · F–fluorosis
- EPA 1986 · NAPCC 2008 (8 missions) · DM Act 2005 · NGT 2010
- Montreal 87 · Rio 92 · Kyoto 97 · Paris 15 · Kigali 16 · ISA 2015 (Gurugram)

Higher Education System

Expected questions: ~5 (10 marks) · Target: 4–5/5

SYLLABUS COVERAGE

- ☐ Institutions of higher learning and education in ancient India
- ☐ Evolution of higher learning and research in post-independence India
- ☐ Oriental, conventional and non-conventional learning programmes in India
- ☐ Professional, technical and skill-based education
- ☐ Value education and environmental education
- ☐ Policies, governance and administration

10.1 Ancient Indian Centres of Learning

India had residential universities more than a thousand years before Europe. Questions usually ask who founded a centre, where it was, which foreign scholars studied there, or who destroyed it.

Centre	Location (present)	Key facts
Takshashila	Rawalpindi, Pakistan	Oldest (~6th c. BCE); Chanakya, Panini, Charaka, Jivaka; no formal degrees
Nalanda	Bihar	Founded by Kumaragupta I (Gupta, 5th c. CE); Xuanzang & I-tsing studied; library <i>Dharmaganja</i> (Ratnasagara, Ratnodadhi, Ratnaranjaka); destroyed by Bakhtiyar Khalji (~1193)
Vikramashila	Bhagalpur, Bihar	Founded by Dharmapala (Pala, 8th c.); Tantric Buddhism; Atisha Dipankara
Valabhi	Gujarat	Maitraka dynasty; Hinayana Buddhism
Odantapuri	Bihar	Gopala (Pala)
Somapura Mahavihara	Bangladesh	Dharmapala; UNESCO site
Kanchi (Kanchipuram)	Tamil Nadu	Pallavas; Ghatika
Pushpagiri	Odisha	
Jagaddala	Bengal	Ramapala

Vedic education: **Gurukul**; teaching via *Shravana* (listening) → *Manana* (reflection) → *Nididhyasana* (meditation). Upanayana ceremony marked entry. Buddhist education: *Pabbajja* (entry) and *Upasampada* (full ordination) ceremonies; viharas & monasteries.

10.2 Evolution of Higher Education — Timeline

Modern higher education in India has been shaped by a succession of commissions and policies. Their order and their key recommendations are among the most frequently asked facts in this unit.



1781	●	Calcutta Madrasa (Warren Hastings)
1791	●	Sanskrit College, Benares (Jonathan Duncan)
1813	●	Charter Act — ₹1 lakh a year for education
1835	●	Macaulay's Minute — English education
1854	●	Wood's Despatch — "Magna Carta of English education in India"
1857	●	Universities of Calcutta, Bombay and Madras
1882	●	Hunter Commission
1902	●	Raleigh Commission → Indian Universities Act, 1904
1917	●	Sadler (Calcutta University) Commission
1948	●	Radhakrishnan Commission — recommended the UGC
1953	●	UGC set up (inaugurated 28 December 1953)
1956	●	UGC Act — UGC becomes a statutory body
1964	●	Kothari Commission (1964–66) — 10+2+3 pattern
1968	●	First National Policy on Education
1986	●	National Policy on Education (revised 1992, Programme of Action)
2005	●	National Knowledge Commission (Sam Pitroda)
2009	●	Yash Pal Committee
2013	●	RUSA — funding for state institutions
2020	●	National Education Policy 2020 (K. Kasturirangan)

FIGURE 10.1. The evolution of modern higher education in India.

10.3 NEP 2020 — Higher Education Highlights (most asked)

The National Education Policy 2020 is the first comprehensive policy since 1986. Its higher-education reforms aim at flexibility for students, multidisciplinary institutions and lighter but tighter regulation.

- **HECI** (Higher Education Commission of India) — single umbrella regulator with 4 verticals: NHERC, NAC, HEGC, GEC (proposed; would subsume UGC/AICTE/NCTE except medical & legal).
- **ANRF** (Anusandhan National Research Foundation) Act 2023 — implements NRF idea.
- **Multidisciplinary Education & Research Universities (MERUs)**.

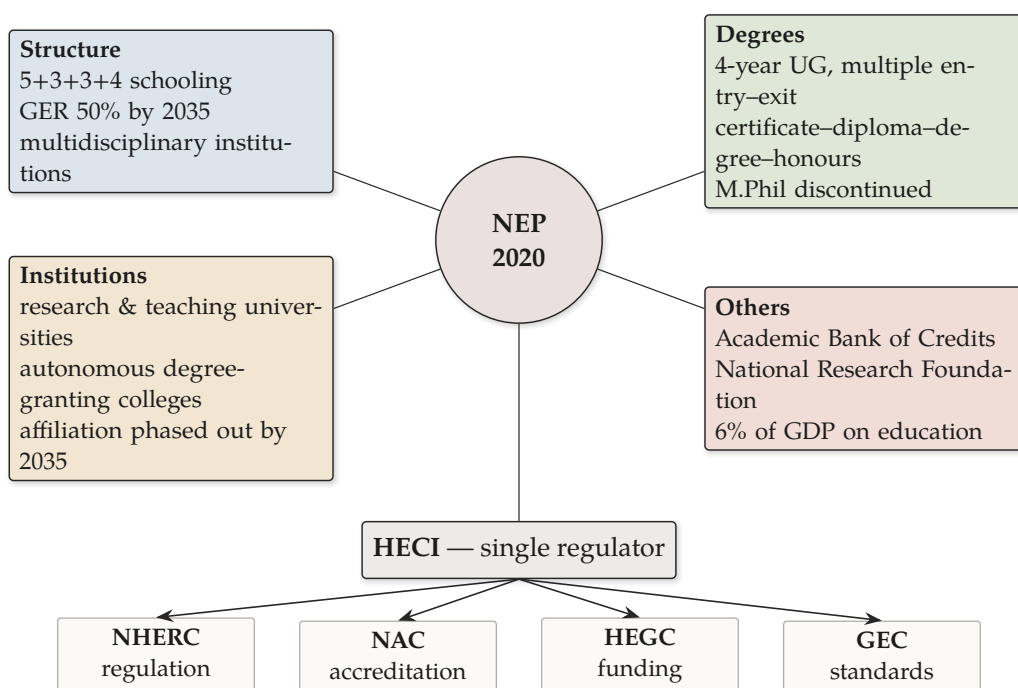


FIGURE 10.2. The main reforms of NEP 2020 for higher education, and the four verticals of the proposed Higher Education Commission of India.

10.4 Types of Institutions & Programmes

Type	Notes
Central university	Act of Parliament (e.g. JNU, DU, BHU, AMU)
State university	State Act
Deemed-to-be university	Sec 3 of UGC Act, declared by Central Govt on UGC advice
Private university	State Act; cannot affiliate colleges
Institutions of National Importance	Act of Parliament (IITs, NITs, AIIMS, IIMs)
Institution of Eminence	2017 scheme — 10 public + 10 private
Open universities	IGNOU (1985); first state open univ: Dr. B.R. Ambedkar Open Univ, Hyderabad (1982)

- **Oriental learning:** Sanskrit, Pali, Persian, Arabic institutions — e.g. Rashtriya Sanskrit Sansthan; Central Sanskrit University (2020).
- **Conventional:** classroom-based degrees. **Non-conventional:** distance, open, online, MOOCs, part-time.

10.5 Regulatory & Professional Bodies

India regulates higher education through a separate body for each sector. Matching each body with its field and its year of establishment is a standard question.

Body	Field	Est.
UGC	Universities (funding, standards)	1953 / Act 1956
AICTE	Technical education	1945 (statutory 1987)
NCTE	Teacher education	1973 (statutory 1993)
NMC	Medical (replaced MCI)	2020
BCI	Legal education	1961
ICAR	Agriculture	1929
PCI / INC / DCI	Pharmacy / Nursing / Dental	
COA	Architecture	1972
RCI	Rehabilitation	1992
NAAC	Accreditation of HEIs (Bengaluru)	1994
NBA	Accreditation of programmes (technical)	1994
NIRF	Ranking (MoE)	2015
NTA	Testing (UGC NET, CUET, NEET, JEE Main)	2017
CABE	Central Advisory Board of Education — oldest advisory body	1920/1935
NIEPA	Educational planning & admin	

NAAC grades: A++, A+, A, B++, B+, B, C, D on 4-point CGPA; criteria = 7 (Curricular aspects; Teaching-learning & evaluation; Research, innovations & extension; Infrastructure; Student support; Governance; Institutional values & best practices).

10.6 Skill-based & Professional Education

- **Skill India / PMKVY** (2015), **NSQF** (National Skills Qualification Framework, levels 1–10), **NCVET** (2018), B.Voc / D.Voc, Community colleges, **Apprenticeship embedded degree programmes**.
- **NHEQF** (National Higher Education Qualification Framework, levels 4.5–8) & **NCrF** (National Credit Framework).

10.7 Value & Environmental Education

- Value education: truth, non-violence, peace, love, right conduct (Sathya Sai model); constitutional values; NEP emphasises ethics, Indian Knowledge Systems (IKS).
- Environmental education made compulsory at all levels by **Supreme Court (M.C. Mehta case, 1991/2003)**; UGC mandated a 6-month core module course for UG.
- **Tbilisi Declaration (1977)** — first intergovernmental conference on environmental education (UNESCO-UNEP); Belgrade Charter (1975).

10.8 Governance & Administration

A university is governed by a hierarchy of officers and statutory bodies defined in its Act. The figure shows the usual arrangement; details vary between central and state universities.

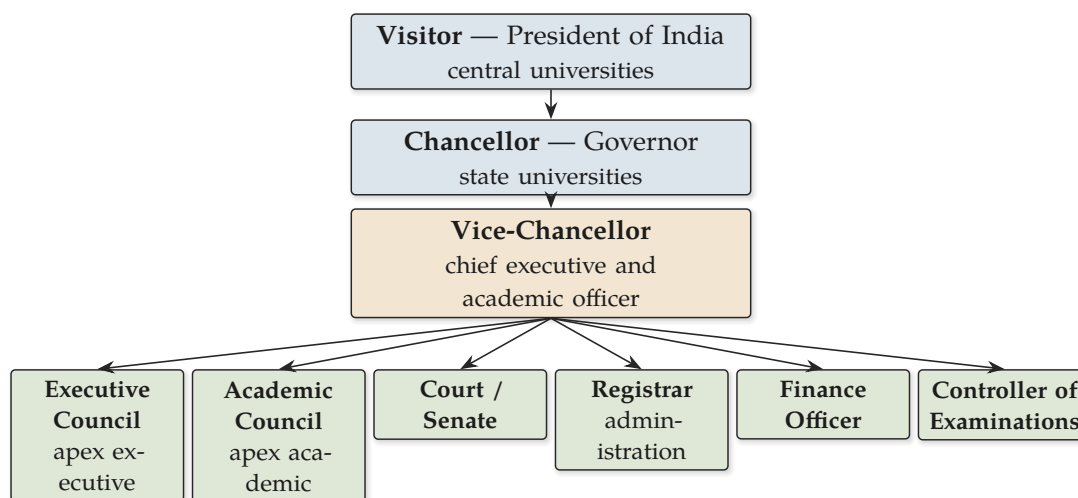


FIGURE 10.3. The governance structure of an Indian university.

- Education was moved from State List to **Concurrent List** by the **42nd Amendment (1976)**.
- **Article 21A** — right to education (6–14 yrs) via 86th Amendment 2002; RTE Act 2009.
- Article 30 — minorities' right to establish institutions; Article 45 — ECCE; Article 46 — education of SC/ST/weaker sections.
- Entry 66, Union List — coordination & determination of standards in higher education.
- **Ministry of Education** (renamed from MHRD in 2020). **RUSA (2013)** — funding to state HEIs.

10.9 Deeper Dive — NEP 2020 Details, Quality Frameworks & Key Regulations

10.9.1 NEP 2020 — Multiple Entry/Exit & Credit System

- **UGC Curriculum & Credit Framework for UG Programmes (CCFUP, 2022)**; credits stored in **Academic Bank of Credits (ABC)**, ID via DigiLocker / APAAR ID (One Nation One Student ID).
- Students may earn up to **40% of credits** through SWAYAM/online courses per semester (UGC Credit Framework for Online Learning Courses through SWAYAM, 2021).
- Course categories: Major (core), Minor, Multidisciplinary, Ability Enhancement (AEC), Skill Enhancement (SEC), Value-Added (VAC), internships, research project.
- **UGC (Minimum Standards and Procedures for Award of PhD) Regulations 2022**: 4-year UG (with 7.5 CGPA) eligible for PhD; M.Phil not required; 2-year coursework rule relaxed; publication requirement removed.
- **Professor of Practice** scheme (2022): industry experts can teach (up to 10% of sanctioned posts).
- Foreign university campuses permitted (UGC Regulations 2023); GIFT City.

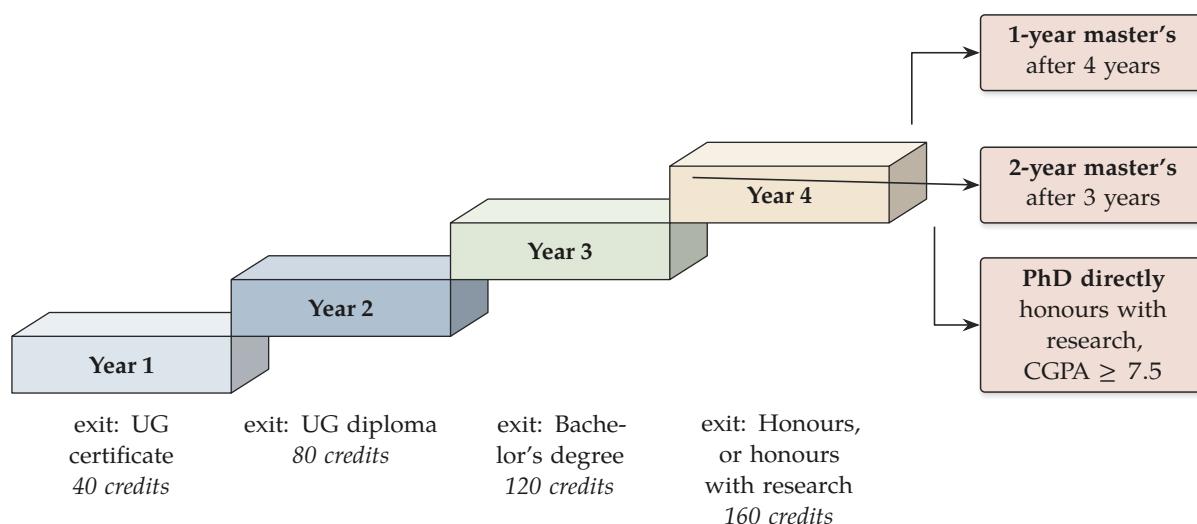


FIGURE 10.4. NEP 2020's four-year undergraduate programme, with an exit after every year and the routes onward.

- **Light but tight** regulation; graded autonomy; Board of Governors for institutions.

10.9.2 NIRF — Parameters (since 2016)

Parameter	Weight (overall category)
Teaching, Learning & Resources (TLR)	30%
Research & Professional Practice (RP)	30%
Graduation Outcomes (GO)	20%
Outreach & Inclusivity (OI)	10%
Perception (PR)	10%

10.9.3 NAAC Grading (cumulative GPA on a 4-point scale)

CGPA range	Grade	Status
3.51–4.00	A++	Accredited
3.26–3.50	A+	Accredited
3.01–3.25	A	Accredited
2.76–3.00	B++	Accredited
2.51–2.75	B+	Accredited
2.01–2.50	B	Accredited
1.51–2.00	C	Accredited
≤ 1.50	D	Not accredited

NAAC is moving to **binary accreditation** (accredited / not accredited) and **Maturity-Based Graded Levels (1–5)** following the Radhakrishnan Committee (2022–24) recommendations.

10.9.4 Constitutional & Legal Provisions on Education

Provision	Content
Article 21A	Free & compulsory education 6–14 years (86th Amendment, 2002)
Article 28	No religious instruction in fully state-funded institutions
Article 29	Protection of minorities' cultural & educational interests
Article 30	Minorities' right to establish & administer institutions
Article 45	Early childhood care & education below 6 years (DPSP)
Article 46	Promote education of SC/ST & weaker sections
Article 51A(k)	Parent's duty to provide education to child 6–14
Article 350A	Instruction in mother tongue at primary stage
Union List entry 63–66	Institutions of national importance; standards in higher education
Concurrent List entry 25	Education (since 42nd Amendment)

10.9.5 Value Education & Indian Knowledge Systems

- **IKS division** at AICTE (2020); Bharatiya Bhasha Samiti; courses on Indian philosophy, mathematics, astronomy, Ayurveda.
- Value frameworks: Delors Commission (UNESCO, 1996) "**Learning: The Treasure Within**" — **four pillars**: learning to know, to do, to live together, to be.
- **Mulya Pravah** (UGC, 2019, revised 2.0 in 2023): guidelines for human values & professional ethics in HEIs.
- **Deeksharambh**: student induction programme (UGC).
- **Paramarsh** (mentoring for NAAC accreditation), **STRIDE** (research), **LOCF** (learning outcomes-based curriculum framework).

10.10 Previous Year Questions (PYQ pattern)

1. Nalanda University was founded by: **Kumaragupta I**
2. Vikramashila was founded by: **Dharmapala**
3. Which Chinese traveller studied at Nalanda? **Xuanzang (Hiuen Tsang)** and I-tsing
4. Nalanda was destroyed by: **Bakhtiyar Khalji**
5. Takshashila is located in present-day: **Pakistan**
6. "Magna Carta of English education in India": **Wood's Despatch 1854**
7. The first three universities were established in: **1857**
8. University Education Commission (1948) was chaired by: **Dr. S. Radhakrishnan**
9. 10+2+3 pattern was recommended by: **Kothari Commission**
10. UGC became a statutory body in: **1956**

11. NEP 2020 GER target for higher education: **50% by 2035**
12. Which degree is discontinued by NEP 2020? **M.Phil**
13. NEP 2020 proposes a single regulator: **HECI**
14. NAAC is located in: **Bengaluru**
15. Programme-level accreditation for engineering: **NBA**
16. Education was placed in the Concurrent List by: **42nd Amendment, 1976**
17. Right to Education is under Article: **21A**
18. IGNOU was established in: **1985**
19. Apex academic body of a university: **Academic Council**
20. Who is the Visitor of central universities? **President of India**
21. Number of NAAC criteria: **7**
22. The Tbilisi conference (1977) dealt with: **Environmental education**
23. Arrange chronologically: Macaulay's Minute (1835), Wood's Despatch (1854), Hunter (1882), Sadler (1917), Radhakrishnan (1948), Kothari (1964–66)

More practice questions

24. Credits required for a UG certificate under NEP's multiple exit: **40**
25. Maximum credits that can be earned through SWAYAM per semester (UGC 2021): **40%**
26. "Learning: The Treasure Within" is the report of the: **Delors Commission (UNESCO, 1996)**
27. Four pillars of learning do NOT include: (a) to know (b) to do (c) to compete (d) to be —
Ans: (c)
28. Weight of Teaching, Learning & Resources in NIRF overall ranking: **30%**
29. NAAC grade for CGPA 3.30: **A+**
30. Article 30 deals with: **right of minorities to establish educational institutions**
31. Article 350A provides for: **instruction in mother tongue at the primary stage**
32. Mulya Pravah guidelines relate to: **human values and professional ethics in HEIs**
33. Student induction programme of UGC: **Deeksharambh**
34. UGC scheme for mentoring institutions towards accreditation: **Paramarsh**
35. Under UGC PhD Regulations 2022, a 4-year UG graduate needs a minimum CGPA of: **7.5**

POINTS TO REMEMBER

- Nalanda–Kumaragupta I · Vikramashila–Dharmapala · Destroyer–Khalji
- 1835 Macaulay · 1854 Wood · 1857 3 univs · 1948 Radhakrishnan · 1953/56 UGC · 1964-66 Kothari · 1968/1986 NPE · 2020 NEP
- NEP: GER 50% (2035), 4-yr UG, ABC, HECI (4 verticals), M.Phil out

PART II

Paper II

Computer Science & Applications

— . . . —

Discrete Structures & Optimization

...

Expected questions: 10–12 · Target: 9+ · Type: numerical + concept

SYLLABUS COVERAGE

- Mathematical logic: propositional & predicate logic, propositional equivalences, normal forms, predicates & quantifiers, nested quantifiers, rules of inference
 - Sets & relations: set operations, representation & properties of relations, equivalence relations, partially ordering
 - Counting, mathematical induction & discrete probability: basics of counting, pigeonhole principle, permutations & combinations, inclusion-exclusion, mathematical induction, probability, Bayes' theorem
 - Group theory: groups, subgroups, semigroups, product & quotients of algebraic structures, isomorphism, homomorphism, automorphism, rings, integral domains, fields, applications of group theory
 - Graph theory: simple graph, multigraph, weighted graph, paths & circuits, shortest paths, Eulerian & Hamiltonian, planar graph, graph colouring, bipartite graphs, trees & rooted trees, prefix codes, tree traversals, spanning trees & cut-sets
 - Boolean algebra: Boolean functions & representation, simplification
 - Optimization: linear programming (graphical, simplex, duality), integer programming, transportation & assignment, PERT-CPM
-

1.1 Mathematical Logic

Logic is the grammar of mathematical reasoning and of computing: every condition in a program and every gate in a circuit is a logical connective. In the examination, logic questions test three skills — reading a truth table, transforming formulas by known equivalences, and translating English into symbols.

1.1.1 Connectives & Truth Table

The only row that makes $p \rightarrow q$ false is the one where a true premise leads to a false conclusion. A conditional with a false antecedent is “vacuously” true, which surprises students but follows from treating $p \rightarrow q$ as a promise that is broken only when p holds and q fails.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
T	T	F	T	T	T	T	F
T	F	F	F	T	F	F	T
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	F

$p \rightarrow q$ is false **ONLY** when p is true and q is false.

Statement	Form	Equivalent to
conditional	$p \rightarrow q$	its contrapositive
converse	$q \rightarrow p$	the inverse
inverse	$\neg p \rightarrow \neg q$	the converse
contrapositive	$\neg q \rightarrow \neg p$	the conditional $p \rightarrow q$

1.1.2 Important Equivalences

Implication as disjunction	$p \rightarrow q \equiv \neg p \vee q$
Biconditional	$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
De Morgan's laws	$\neg(p \wedge q) \equiv \neg p \vee \neg q, \quad \neg(p \vee q) \equiv \neg p \wedge \neg q$
Absorption	$p \vee (p \wedge q) \equiv p, \quad p \wedge (p \vee q) \equiv p$
Exportation	$(p \wedge q) \rightarrow r \equiv p \rightarrow (q \rightarrow r)$
Common antecedent	$(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$
Common consequent	$(p \rightarrow r) \wedge (q \rightarrow r) \equiv (p \vee q) \rightarrow r$

- **Tautology**: always true ($p \vee \neg p$). **Contradiction**: always false. **Contingency**: neither.
- Functionally complete sets: $\{\wedge, \neg\}$, $\{\vee, \neg\}$, $\{\text{NAND}\}$, $\{\text{NOR}\}$, $\{\rightarrow, \neg\}$, $\{\rightarrow, \text{F}\}$.

1.1.3 Normal Forms

- **CNF** = AND of ORs (product of sums); **DNF** = OR of ANDs (sum of products).
- Number of Boolean functions on n variables = 2^{2^n} .

1.1.4 Rules of Inference

Rules of inference are valid argument patterns: whenever the premises are true, the conclusion must be. Proofs, resolution in artificial intelligence and many examination questions are chains of these few rules.

Rule	Form
Modus Ponens	$p, p \rightarrow q \vdash q$
Modus Tollens	$\neg q, p \rightarrow q \vdash \neg p$
Hypothetical syllogism	$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$
Disjunctive syllogism	$p \vee q, \neg p \vdash q$
Addition	$p \vdash p \vee q$
Simplification	$p \wedge q \vdash p$
Resolution	$p \vee q, \neg p \vee r \vdash q \vee r$
Universal instantiation	$\forall x P(x) \vdash P(c)$
Existential generalisation	$P(c) \vdash \exists x P(x)$

Fallacies: affirming the consequent ($q, p \rightarrow q \vdash p$ ✗), denying the antecedent ($\neg p, p \rightarrow q \vdash \neg q$ ✗).

1.1.5 Predicates & Quantifiers

Predicates make statements about objects, and quantifiers say how many objects satisfy them. The most common error is in translation: a universal statement uses implication, an existential one uses conjunction.

Negating quantifiers	$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x)$
$\forall$ distributes over $\wedge$; $\exists$ over $\vee$	$\forall x(P \wedge Q) \equiv \forall xP \wedge \forall xQ, \quad \exists x(P \vee Q) \equiv \exists xP \vee \exists xQ$
Order of mixed quantifiers	$\exists x \forall y P(x, y) \Rightarrow \forall y \exists x P(x, y) \quad (\text{not conversely})$
“All students are smart” — pair $\forall$ with $\rightarrow$	$\forall x (S(x) \rightarrow M(x))$
“Some students are smart” — pair $\exists$ with $\wedge$	$\exists x (S(x) \wedge M(x))$

1.2 Sets & Relations

A relation on a set A is simply a subset of $A \times A$, so with n elements there are 2^{n^2} of them. The counting formulas in the table follow by asking, for each ordered pair, how freely it may be included.

- $|A \cup B| = |A| + |B| - |A \cap B|$; $|P(A)| = 2^n$; $|A \times B| = mn$.
- Number of relations on A ($|A| = n$) = 2^{n^2} .

Relation type	Count on n elements
Reflexive	$2^{n^2 - n}$
Irreflexive	$2^{n^2 - n}$
Symmetric	$2^{n(n+1)/2}$
Antisymmetric	$2^n \cdot 3^{n(n-1)/2}$
Asymmetric	$3^{n(n-1)/2}$
Reflexive & symmetric	$2^{n(n-1)/2}$
Equivalence relations	Bell number B_n (1, 1, 2, 5, 15, 52, 203 ...)
Functions $A \rightarrow B$ ($ A = n, B = m$)	m^n
One-one ($m \leq n$)	$n!/(n-m)!$
Onto	$\sum (-1)^k C(n,k) (n-k)^m$
Bijections ($m = n$)	$n!$

1.2.1 Equivalence relation = Reflexive + Symmetric + Transitive $\rightarrow$ partitions the set into equivalence classes.

1.2.2 Partial order (POSET) = Reflexive + Antisymmetric + Transitive.

Hasse diagram of divisibility on $\{1, 2, 3, 6, 12\}$:

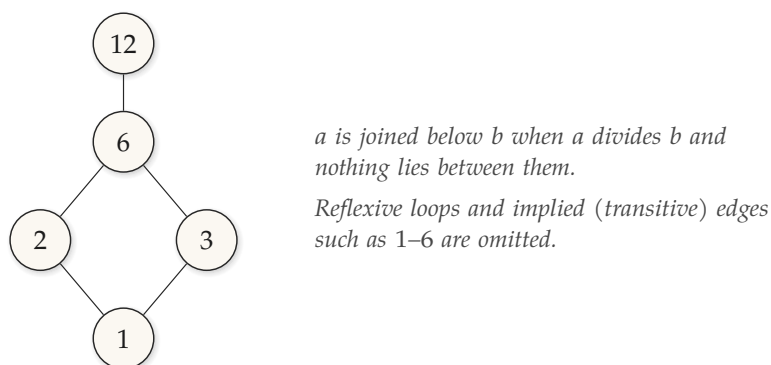


FIGURE 1.1. Hasse diagram of $\{1, 2, 3, 6, 12\}$ under divisibility.

- **Lattice:** every pair has LUB (join, $\vee$) and GLB (meet, $\wedge$).
- **Bounded:** has 0 (least) and 1 (greatest). **Complemented:** every element has a complement.
- **Distributive:** no sub-lattice isomorphic to M_3 (diamond) or N_5 (pentagon).
- **Boolean algebra** = complemented + distributive lattice. Divisor lattice D_n is Boolean iff n is square-free.

- Total order (chain): every pair comparable. Well-ordered: every nonempty subset has least element.

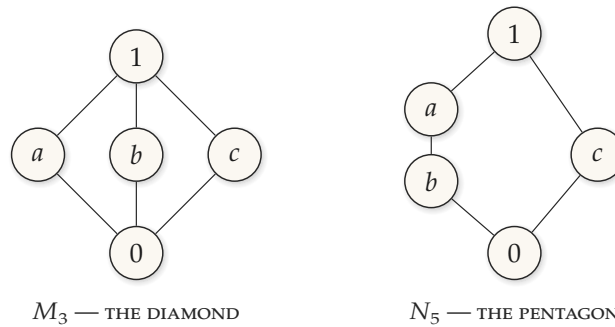


FIGURE 1.2. The two smallest non-distributive lattices. A lattice is distributive exactly when it contains neither as a sublattice.

1.3 Counting & Probability

Counting questions reduce to one of a few models: ordered or unordered selection, with or without repetition, in a line or a circle. Identify the model before reaching for a formula.

Permutations, combinations	${}^nP_r = \frac{n!}{(n-r)!}, \quad \binom{n}{r} = \frac{n!}{r!(n-r)!}$
Circular arrangements; necklace or garland	$(n-1)!$ and $\frac{(n-1)!}{2}$
Arrangements with repeated objects	$\frac{n!}{p!q!r!}$
Choosing r from n types with repetition (stars and bars)	$\binom{n+r-1}{r}$
Derangements	$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}; \quad D_1..D_5 = 0, 1, 2, 9, 44$
Catalan numbers (BSTs, bracketings)	$C_n = \frac{1}{n+1} \binom{2n}{n} = 1, 1, 2, 5, 14, 42, \dots$

- **Pigeonhole principle:** n items into m boxes $\Rightarrow$ some box has at least $\lceil n/m \rceil$ items. *e.g. Min. people to ensure 2 share a birth month = 13.*
- **Inclusion–exclusion:** $|A \cup B \cup C| = \Sigma A - \Sigma(A \cap B) + (A \cap B \cap C)$.
- **Mathematical induction:** base case + $(P(k) \Rightarrow P(k+1))$. Strong induction assumes all $P(1) \dots P(k)$.

1.3.1 Recurrence relations

- Linear homogeneous: $a_n = c_1 a_{n-1} + c_2 a_{n-2} \rightarrow$ characteristic equation $r^2 - c_1 r - c_2 = 0$. Distinct roots r_1, r_2 : $a_n = \alpha r_1^n + \beta r_2^n$; repeated root r : $a_n = (\alpha + \beta n) r^n$.
- Fibonacci: $F_n = F_{n-1} + F_{n-2}$. Tower of Hanoi: $T_n = 2T_{n-1} + 1 = 2^n - 1$.

1.3.2 Probability

Addition rule	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Conditional probability	$P(A B) = \frac{P(A \cap B)}{P(B)}$
Independence	$P(A \cap B) = P(A) P(B)$
Bayes' theorem	$P(A B) = \frac{P(B A) P(A)}{P(B)}$
Binomial distribution	$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}; \mu = np, \sigma^2 = npq$
Poisson distribution	$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}; \mu = \sigma^2 = \lambda$
Expectation and variance	$E[aX + b] = aE[X] + b, \text{Var } X = E[X^2] - (E[X])^2$

1.4 Algebraic Structures

Abstract algebra studies sets with operations by listing the laws they obey. The same names recur in computing: groups in cryptography, fields in error-correcting codes, Boolean algebras in circuit design.

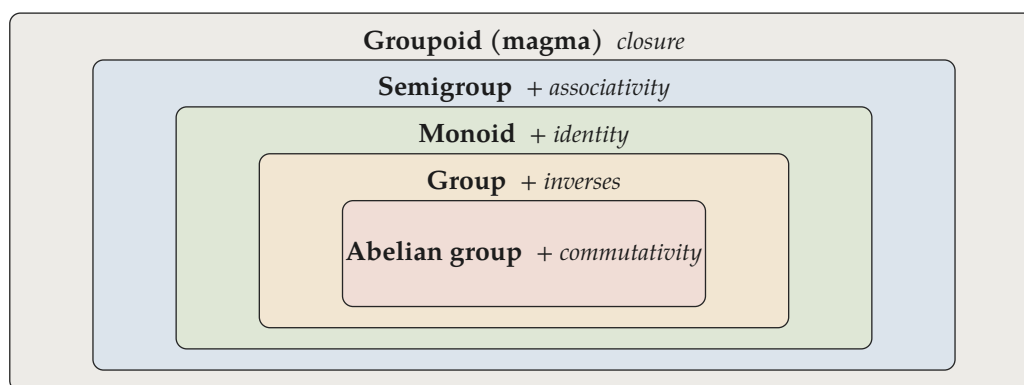


FIGURE 1.3. Algebraic structures with one operation, each adding one axiom to the one outside it. Every abelian group is a group, every group a monoid, and so on.

Key facts:

- **Lagrange's theorem**: order of a subgroup divides the order of the group (finite).
- Every group of **prime order** is cyclic and abelian. Every cyclic group is abelian.

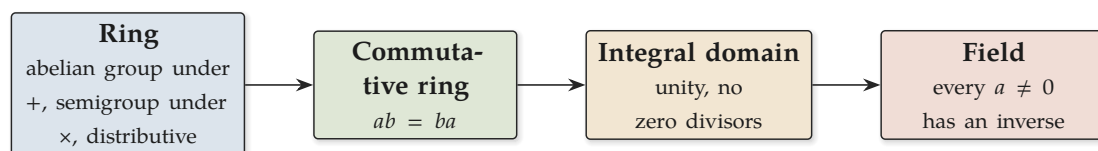


FIGURE 1.4. Structures with two operations. $\mathbb{Z}$ is an integral domain but not a field; $\mathbb{Z}_n$ is a field exactly when n is prime; every finite integral domain is a field.

- Number of generators of cyclic group of order $n = \varphi(n)$ (Euler totient).
- Groups of order ≤ 5 are abelian. Smallest non-abelian group: S_3 (order 6).
- $(\mathbb{Z}_n, +_n)$ is a group; $(\mathbb{Z}_n^*, \times_n)$ group of units has $\varphi(n)$ elements; $\mathbb{Z}_n$ is a **field** iff n is **prime**.
- Every finite integral domain is a field.
- **Homomorphism**: $f(a*b) = f(a) \circ f(b)$. **Isomorphism**: bijective homomorphism. **Automorphism**: isomorphism onto itself.
- **Normal subgroup** H : $gH = Hg \ \forall g$; quotient group G/H has order $|G|/|H|$.
- Order of an element $a =$ smallest k with $a^k = e$.

1.5 Graph Theory

A graph is a set of vertices joined by edges — an abstraction that models road networks, computer networks, dependencies and scheduling conflicts. Most examination questions turn on a handful of counting results and the characterisations of Euler graphs, planar graphs and bipartite graphs.

1.5.1 Basic results

Handshaking lemma	$\sum_v \deg v = 2 E \Rightarrow$ odd-degree vertices are even in number
Edges of K_n (maximum for a simple graph)	$\frac{n(n-1)}{2}$
Edges of $K_{m,n}$	mn
Tree on n vertices; forest of k trees	$n-1; \quad n-k$
Labelled trees on n vertices (Cayley)	n^{n-2} (= spanning trees of K_n)
Simple labelled graphs on n vertices	$2^{n(n-1)/2}$

1.5.2 Euler vs Hamilton

- K_n is Eulerian iff n is odd. K_n has $(n-1)!/2$ Hamiltonian cycles.

EULER: EVERY EDGE ONCE

HAMILTON: EVERY VERTEX ONCE

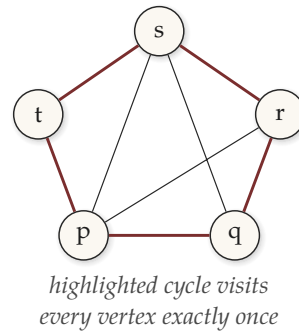
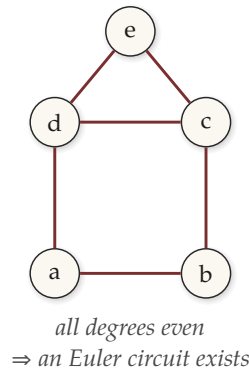


FIGURE 1.5. Euler and Hamilton problems. Euler's condition is a simple degree test; deciding whether a Hamiltonian cycle exists is NP-complete, though Dirac's and Ore's conditions guarantee one.

1.5.3 Planar Graphs

A graph is planar if it can be drawn in the plane without edges crossing. Euler's formula links the numbers of vertices, edges and faces of any such drawing, and the edge bounds derived from it give a quick test for non-planarity.

Euler's formula (connected planar graph)	$V - E + F = 2$	(k components: $V - E + F = k + 1$)
Simple planar graph, $V \geq 3$	$E \leq 3V - 6$	
Planar graph with no triangles	$E \leq 2V - 4$	
Kuratowski's theorem	G planar $\iff$ no subdivision of K_5 or $K_{3,3}$	

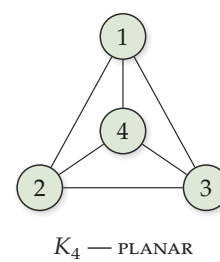
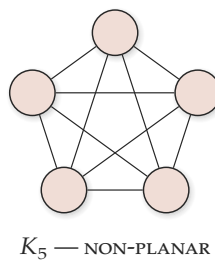
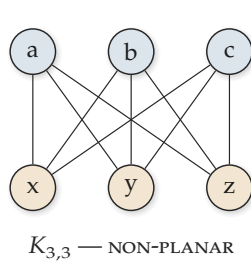


FIGURE 1.6. $K_{3,3}$ and K_5 cannot be drawn without crossings; K_4 can, with one vertex placed inside the triangle.

1.5.4 Colouring

Colouring assigns colours to vertices so that adjacent vertices differ — the model behind timetabling and register allocation. The chromatic number is easy to bound but hard to compute in general.

- **Chromatic number** $\chi(G)$: min colours so adjacent vertices differ.
- $\chi(K_n) = n$; $\chi(\text{bipartite with } \geq 1 \text{ edge}) = 2$; $\chi(\text{tree, } n \geq 2) = 2$; $\chi(\text{cycle } C_n) = 2$ if n even, 3 if odd; $\chi(\text{Wheel } W_n) = 3$ or 4.
- **Four colour theorem**: planar $\Rightarrow \chi \leq 4$.
- G is **bipartite** $\Leftrightarrow$ **no odd cycle** $\Leftrightarrow$ **2-colourable**.
- Chromatic polynomial of tree with n vertices: $k(k-1)^{n-1}$; of K_n : $k(k-1)\dots(k-n+1)$.

1.5.5 Other definitions

- **Independent set, clique, vertex cover**: $\alpha(G) + \beta(G) = n$ (independence number + vertex cover number).
- **Matching**; perfect matching. Edge chromatic number (Vizing): $\Delta \leq \chi' \leq \Delta+1$.
- **Cut vertex / articulation point, bridge, cut-set**. Vertex connectivity $\kappa \leq$ edge connectivity $\lambda \leq \delta$ (min degree) — Whitney.
- **Isomorphism** invariants: same #vertices, #edges, degree sequence, cycles.
- **Prefix codes**: Huffman coding gives optimal prefix-free code using a binary tree.

1.6 Boolean Algebra

Idempotent laws	$x + x = x, \quad x \cdot x = x$
Absorption	$x + xy = x, \quad x(x + y) = x$
Redundant literal	$x + \bar{x}y = x + y$
Distribution of $+$ over $\cdot$	$(x + y)(x + z) = x + yz$
Consensus theorem	$xy + \bar{x}z + yz = xy + \bar{x}z$
Duality	interchange $+$ $\leftrightarrow$ $\cdot$ and $0 \leftrightarrow 1$

K-maps and minimisation are covered in [Unit 2](#).

1.7 Optimization

Optimization chooses the best value of an objective subject to constraints. In linear programming both objective and constraints are linear, so the feasible region is a convex polygon (or polyhedron) and the optimum lies at a corner — the insight on which the simplex method is built.

1.7.1 Linear Programming (LPP)

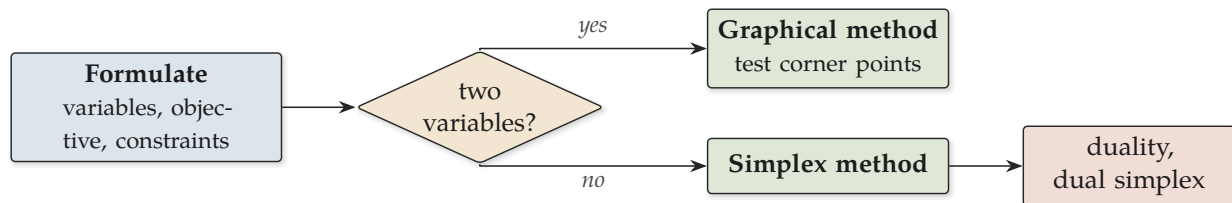


FIGURE 1.7. Choosing a method for a linear programme.

Graphical example: Max $Z = 3x + 5y$, s.t. $x + 2y \leq 8$, $3x + 2y \leq 12$, $x, y \geq 0$.

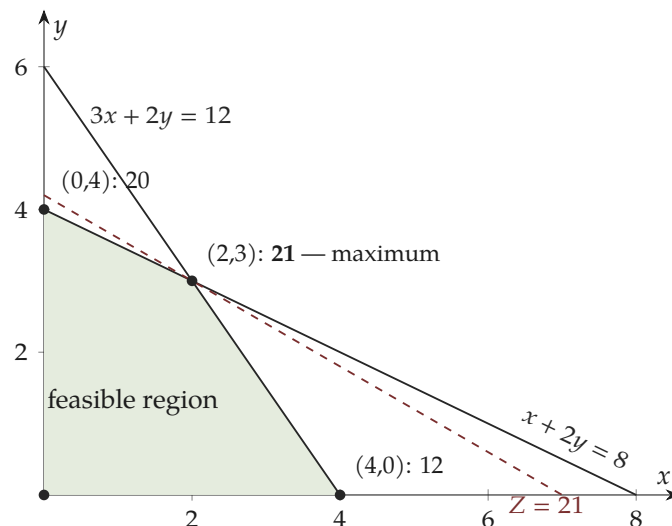


FIGURE 1.8. Graphical solution of: maximise $Z = 3x + 5y$ subject to $x + 2y \leq 8$, $3x + 2y \leq 12$, $x, y \geq 0$. The objective line, moved outward, last touches the region at the corner (2, 3).

- Optimal solution occurs at a **corner point** (extreme point) of the convex feasible region.
- Special cases: **unbounded**, **infeasible**, **alternate optima** (objective parallel to a binding constraint), **degeneracy** (basic variable = 0).
- Slack (for $\leq$), surplus (for $\geq$), artificial variables (Big-M / Two-phase).
- Simplex: entering variable = most negative $C_j - Z_j$ (max) ; leaving variable = **minimum ratio test**.
- **Duality**: primal max $\Leftrightarrow$ dual min; #constraints $\leftrightarrow$ #variables; dual of dual = primal. Strong duality: optimal values equal. Weak duality: any feasible dual value bounds the primal.
- **Dual simplex**: starts optimal but infeasible, keeps optimality till feasibility.

1.7.2 Transportation Problem

The transportation problem ships goods from sources to destinations at minimum cost. It is a linear programme with special structure, solved in two stages: find a starting allocation, then improve it until no cheaper route exists.

- m sources, n destinations; balanced if supply = demand (else add dummy).
- A basic feasible solution has $m + n - 1$ allocations (fewer $\rightarrow$ degenerate).
- Initial solution: **North-West Corner**, **Least Cost**, **Vogel's Approximation (VAM — best)**.
- Optimality: **MODI (u-v) method** or Stepping stone.

1.7.3 Assignment Problem

Assigning n jobs to n people one-to-one is a transportation problem with every supply and demand equal to one; the Hungarian method exploits this to solve it by simple row and column reductions.

- n jobs to n persons, one-to-one; solved by **Hungarian method** (row reduction, column reduction, cover zeros with min lines; if lines = $n \Rightarrow$ optimal).
- Special case of transportation (all supplies/demands = 1); highly degenerate.

1.7.4 Integer Programming

- Variables restricted to integers: **Branch and Bound**, **Gomory's cutting plane**.

1.7.5 PERT-CPM

Large projects are planned as networks of activities. The critical path is the longest path through the network: any delay on it delays the whole project, whereas activities off the path have slack (float).

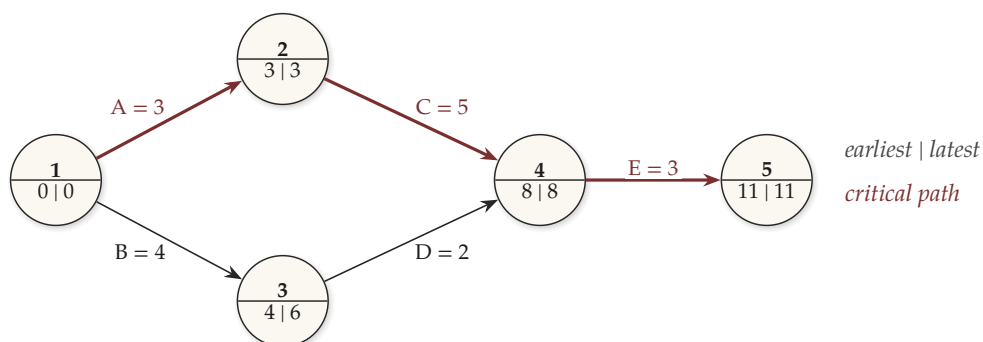


FIGURE 1.9. An activity-on-arrow network. Each event shows its earliest and latest times; where they are equal the event is critical. The critical path A-C-E (11 days) fixes the project duration, and activities B and D share 2 days of float.

Paths: A-C-E = $3+5+3 = 11$ (critical), B-D-E = $4+2+3 = 9$. Project duration = 11.

Concept	Formula
Earliest start (forward pass)	$ES = \max(\text{EF of predecessors})$
Latest finish (backward pass)	$LF = \min(\text{LS of successors})$
Total float	$LS - ES = LF - EF$
Critical activity	Total float = 0
PERT expected time	$t_e = (a + 4m + b)/6$
PERT variance	$\sigma^2 = ((b - a)/6)^2$
Probability	$Z = (T - T_e)/\sigma_{\text{path}}$

- **CPM**: deterministic time, cost-oriented (crashing). **PERT**: probabilistic (β -distribution), event-oriented.
- **Crashing**: reducing duration by adding resources; crash cost slope = (crash cost – normal cost)/(normal time – crash time).
- **Resource levelling**: smooth resource usage without changing project duration (use floats).

1.8 Deeper Dive — Worked Simplex, Transportation, Assignment, Groups & Recurrences

1.8.1 Simplex Method — Full Worked Example

The simplex method moves from corner to corner of the feasible region, always improving the objective, and stops when no adjacent corner is better. Each tableau below is one such corner.

Max $Z = 3x_1 + 5x_2$ subject to $x_1 \leq 4$, $2x_2 \leq 12$, $3x_1 + 2x_2 \leq 18$, $x_1, x_2 \geq 0$. Add slacks s_1, s_2, s_3 .

Tableau 0 (Z row written as $Z - 3x_1 - 5x_2 = 0$)

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
s_1	1	0	1	0	0	4	—
s_2	0	2	0	1	0	12	$12/2 = 6 \leftarrow$
s_3	3	2	0	0	1	18	$18/2 = 9$
Z	-3	-5 ↑	0	0	0	0	

x_2 enters (most negative), s_2 leaves (minimum ratio). Pivot = 2.

Tableau 1

Basic	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
s_1	1	0	1	0	0	4	4
x_2	0	1	0	$1/2$	0	6	—
s_3	3	0	0	-1	1	6	$6/3 = 2 \leftarrow$
Z	-3 ↑	0	0	$5/2$	0	30	

x_1 enters, s_3 leaves.

Tableau 2 (optimal — no negative entries in Z row)

Basic	x ₁	x ₂	s ₁	s ₂	s ₃	RHS
s ₁	0	0	1	1/3	-1/3	2
x ₂	0	1	0	1/2	0	6
x ₁	1	0	0	-1/3	1/3	2
Z	0	0	0	3/2	1	36

Optimal: $x_1 = 2$, $x_2 = 6$, $Z = 36$. Shadow prices (dual solution) are read from the Z row under the slacks: $y_1 = 0$, $y_2 = 3/2$, $y_3 = 1$ — and $4(0) + 12(3/2) + 18(1) = 36$ confirms strong duality.

1.8.2 Transportation Problem — NWC + MODI

	D1	D2	D3	Supply
S1	4	6	8	20
S2	5	8	7	30
S3	6	9	5	25
Demand	10	25	40	75 (balanced)

North-West Corner: $S1D1 = 10$, $S1D2 = 10$, $S2D2 = 15$, $S2D3 = 15$, $S3D3 = 25 \rightarrow 5$ allocations = $m + n - 1 \checkmark$. Cost = $40 + 60 + 120 + 105 + 125 = 450$.

MODI test (set $u_1 = 0$; for basic cells $u_i + v_j = c_{ij}$): $v_1 = 4$, $v_2 = 6$, $u_2 = 2$, $v_3 = 5$, $u_3 = 0$. Reduced cost of non-basic cell = $c_{ij} - u_i - v_j$: $S1D3 = 3$, **$S2D1 = 5 - 2 - 4 = -1$** (negative $\rightarrow$ improve), $S3D1 = 2$, $S3D2 = 3$.

Closed loop for $S2D1$: $S2D1(+)$ $\rightarrow$ $S1D1(-)$ $\rightarrow$ $S1D2(+)$ $\rightarrow$ $S2D2(-)$; $\theta = \min(10, 15) = 10$. New allocation: $S1D2 = 20$, $S2D1 = 10$, $S2D2 = 5$, $S2D3 = 15$, $S3D3 = 25 \rightarrow$ cost = $120 + 50 + 40 + 105 + 125 = 440$. Recomputing u, v gives all reduced costs $\geq 0 \rightarrow$ **optimal cost 440**.

1.8.3 Assignment Problem — Hungarian Method

	J1	J2	J3
A	9	2	7
B	6	4	3
C	5	8	1

$$\underbrace{\begin{pmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{pmatrix}}_{\text{cost}} \xrightarrow{\text{rows } -2, -3, -1} \begin{pmatrix} 7 & 0 & 5 \\ 3 & 1 & 0 \\ 4 & 7 & 0 \end{pmatrix} \xrightarrow{\text{columns } -3, 0, 0} \begin{pmatrix} 4 & \boxed{0} & 5 \\ \boxed{0} & 1 & 0 \\ 1 & 7 & \boxed{0} \end{pmatrix}$$

Three lines cover all zeros ($= n$), so an optimal assignment exists among the zeros: $A \rightarrow J2$, $B \rightarrow J1$, $C \rightarrow J3$, with minimum cost $2 + 6 + 1 = 9$.

1.8.4 Groups — Cayley Tables

$+_4$	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

$\cdot$	e	a	b	c
e	e	a	b	c
a	a	e	c	b
b	b	c	e	a
c	c	b	a	e

$(\mathbb{Z}_4, +_4)$: cyclic, generated by 1 or 3 Klein four-group V_4 : not cyclic; every element has order 2

- Both have order 4 and are abelian, but are **not isomorphic** ($\mathbb{Z}_4$ has an element of order 4; V_4 does not).
- Subgroups of $\mathbb{Z}_4$: $\{0\}$, $\{0, 2\}$, $\mathbb{Z}_4$. Cosets of $H = \{0, 2\}$: H and $\{1, 3\} \rightarrow \mathbb{Z}_4/H \cong \mathbb{Z}_2$.
- Every group of order 4 is isomorphic to $\mathbb{Z}_4$ or V_4 ; every group of order 6 is $\mathbb{Z}_6$ or S_3 .

1.8.5 Solving a Recurrence

$$a_n = 5a_{n-1} - 6a_{n-2}, a_0 = 1, a_1 = 0.$$

$$a_n = 5a_{n-1} - 6a_{n-2}, \quad a_0 = 1, a_1 = 0$$

$$r^2 - 5r + 6 = 0 \Rightarrow r = 2, 3$$

$$a_n = \alpha 2^n + \beta 3^n$$

$$\alpha + \beta = 1, \quad 2\alpha + 3\beta = 0 \Rightarrow \alpha = 3, \beta = -2$$

$$a_n = 3 \cdot 2^n - 2 \cdot 3^n \quad (\text{check: } a_2 = 12 - 18 = -6 = 5 \cdot 0 - 6 \cdot 1)$$

Non-homogeneous: $a_n = 2a_{n-1} + 1$ (Hanoi) $\rightarrow$ particular solution constant $c = 2c + 1 \Rightarrow c = -1$;
 $a_n = A \cdot 2^n - 1$; $a_1 = 1 \Rightarrow a_n = 2^n - 1$.

Generating functions: the sequence $1, 1, 1, \dots \leftrightarrow 1/(1-x)$; $a_n = C(n+k-1, k-1) \leftrightarrow 1/(1-x)^k$. Used to count integer solutions and to solve recurrences.

1.8.6 Graph Isomorphism & Colouring Checks

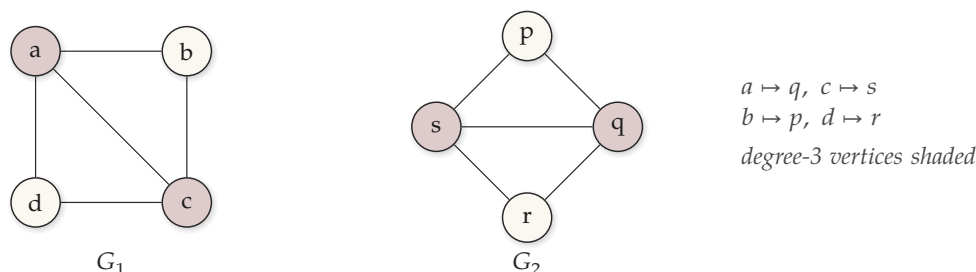


FIGURE 1.10. Two drawings of the same graph (K_4 minus one edge). Matching degree sequences suggest the isomorphism; exhibiting the mapping proves it.

- Both graphs have 4 vertices, 5 edges and degree sequence $(3, 3, 2, 2)$ — each is K_4 minus one edge.
- Mapping $a \mapsto q, c \mapsto s, b \mapsto p, d \mapsto r$ preserves every edge $\rightarrow$ **isomorphic**.

- $\chi(G_1) = 3$, because the triangle a–b–c needs three colours.

Quick invariant checklist for isomorphism: number of vertices and edges, degree sequence, number of cycles of each length, connectivity, bipartiteness. Matching invariants are necessary, not sufficient — finish by exhibiting a mapping.

1.9 Previous Year Questions (PYQ pattern)

Logic

1. Which is a tautology? (a) $p \rightarrow (p \vee q)$ (b) $(p \vee q) \rightarrow p$ (c) $p \wedge \neg p$ (d) $p \rightarrow \neg p$ — **Ans: (a)**
2. The contrapositive of "If it rains, the ground is wet" is: **"If the ground is not wet, it did not rain."**
3. $\neg \forall x (P(x) \rightarrow Q(x))$ is equivalent to: $\exists x (P(x) \wedge \neg Q(x))$
4. "Every student has a computer": with $S(x), C(x)$: $\forall x (S(x) \rightarrow C(x))$
5. Number of Boolean functions of 3 variables: $2^8 = 256$
6. Which is functionally complete? (a) $\{\wedge, \vee\}$ (b) $\{\uparrow\}$ NAND (c) $\{\vee, \leftrightarrow\}$ (d) $\{\wedge, \rightarrow\}$... — **NAND**
7. From $p \rightarrow q$ and $\neg q$ we infer $\neg p$ — this is: **Modus Tollens**

Sets, relations, functions

8. Number of reflexive relations on a set of 4 elements: $2^{12} = 4096$
9. Number of symmetric relations on 3 elements: $2^6 = 64$
10. Number of equivalence relations on $\{1, 2, 3, 4\}$: **15**
11. Number of onto functions from a 4-set to a 3-set: $3^4 - 3 \cdot 2^4 + 3 \cdot 1 = 81 - 48 + 3 = 36$
12. Relation " $\leq$ " on integers is: **partial order (in fact total)**
13. Which lattice is not distributive? **Pentagon N_5 / Diamond M_3**
14. D_{30} (divisors of 30) is a Boolean algebra because 30 is: **square-free** (D_{12} is not)

Counting

15. Minimum students so that at least 3 share birthday month: $2 \times 12 + 1 = 25$
16. Number of derangements of 4 letters: **9**
17. Number of ways to arrange letters of "MISSISSIPPI": $11! / (4!4!2!) = 34650$
18. Number of non-negative integer solutions of $x+y+z = 10$: $C(12, 2) = 66$
19. Solution of $a_n = 6a_{n-1} - 9a_{n-2}$ has form: **$(\alpha + \beta n)3^n$**
20. Number of distinct BSTs with 3 keys: **5** (Catalan)

Probability

21. Two dice thrown; probability sum = 7: **1/6**
22. A box: 3 defective of 10. Probability of picking 2 non-defective without replacement: $(7/10)(6/9) = 7/15$
23. Bayes problem: Machines A, B produce 60%, 40%; defect rates 2%, 3%. $P(A | \text{defective}) = 0.012/0.024 = 1/2$

Groups

24. Which is NOT a group? (a) $(\mathbb{Z}, +)$ (b) $(\mathbb{Q} - \{0\}, \times)$ (c) $(\mathbb{Z}, \times)$ (d) $(\mathbb{R}, +)$ — **Ans: (c)** (no inverses)
25. A group of order 7 is: **cyclic (prime order)**
26. Number of generators of cyclic group of order 12: $\phi(12) = 4$
27. Possible orders of subgroups of a group of order 15: **1, 3, 5, 15**
28. $\mathbb{Z}_n$ is a field iff: **n is prime**
29. Every finite integral domain is a: **field**

Graphs

30. Sum of degrees of a graph with 15 edges: **30**
31. A connected planar graph has 10 vertices and 15 edges. Number of faces: $15 - 10 + 2 = 7$
32. Max edges in simple planar graph with 8 vertices: $3 \times 8 - 6 = 18$
33. Which graph is non-planar? (a) K_4 (b) $K_{2,3}$ (c) $K_{3,3}$ (d) Q_3 — **(c)**
34. K_n has Euler circuit iff: **n is odd**
35. Chromatic number of a cycle with 7 vertices: **3**
36. Number of spanning trees of K_4 : $4^2 = 16$
37. Graph with degree sequence $(3, 3, 3, 1)$ — possible? Sum = 10 even $\rightarrow$ yes; $(3, 3, 3, 3, 1)$? Sum = 13 odd $\rightarrow$ **No**
38. Chromatic number of bipartite graph: **2**

Optimization

39. In a transportation problem with 4 sources and 5 destinations, a non-degenerate BFS has: **8 allocations**
40. The best initial basic feasible solution method: **Vogel's approximation**
41. Assignment problems are solved by: **Hungarian method**
42. The dual of a primal with 3 variables and 5 constraints has: **5 variables, 3 constraints**
43. In PERT, $a=2, m=5, b=14$: $t_e = (2+20+14)/6 = 6, \sigma^2 = (12/6)^2 = 4$
44. Activities on the critical path have total float: **zero**
45. Simplex leaving variable is chosen by: **minimum ratio test**
46. If the objective function is parallel to a binding constraint: **multiple optimal solutions**

More practice questions

47. In simplex, if all entries in the pivot column are ≤ 0 , the LPP has: **an unbounded solution**
48. If an artificial variable remains in the basis at a positive level in the optimal tableau: **the problem is infeasible**
49. Shadow price of a constraint equals the optimal value of the corresponding: **dual variable**
50. A transportation solution with fewer than $m + n - 1$ positive allocations is: **degenerate**
51. In MODI, a negative reduced cost $(c_{ij} - u_i - v_j)$ for an unoccupied cell means: **the solution can be improved**
52. Unbalanced assignment problems are balanced by adding: **dummy rows or columns with zero cost**
53. Number of subgroups of $\mathbb{Z}_4$: **3**
54. Klein four-group is: **abelian but not cyclic**
55. Solution of $a_n = 4a_{n-1} - 4a_{n-2}$ has the form: **$(\alpha + \beta n)2^n$**
56. Coefficient of x^5 in $1/(1-x)^3$: $C(7, 2) = 21$

57. The number of edges in a graph with degree sequence $(3, 3, 2, 2)$: **5**
58. Which statement is true? (a) every abelian group is cyclic (b) every cyclic group is abelian (c) every group of order 4 is cyclic (d) S_3 is abelian — **Ans: (b)**
59. Order of element 2 in $(\mathbb{Z}_6, +_6)$: **3**
60. Number of perfect matchings in K_4 : **3**

POINTS TO REMEMBER

- $p \rightarrow q \equiv \neg p \vee q \equiv \neg q \rightarrow \neg p$ · Relations on n : 2^{n^2}
- $V - E + F = 2$ · $E \leq 3V - 6$ · Cayley n^{n-2} · Odd-degree vertices even
- Lagrange: $|H|$ divides $|G|$ · $\mathbb{Z}_n$ field iff n prime · generators = $\phi(n)$
- Transport BFS = $m + n - 1$ · PERT $t_e = (a + 4m + b)/6$

Computer System Architecture

...

Expected questions: 8–10 · Target: 8+ · Type: numericals (cache, pipeline, number systems) + concepts (Morris Mano)

SYLLABUS COVERAGE

- ☐ Digital logic circuits & components: gates, Boolean algebra, K-maps, combinational circuits, flip-flops, sequential circuits, ICs, decoders, multiplexers, registers, counters, memory unit
 - ☐ Data representation: number systems, complements, fixed & floating point, error-detection codes, computer arithmetic
 - ☐ Register transfer & micro-operations
 - ☐ Basic computer organization & design: instruction codes, registers, timing & control, instruction cycle, interrupts
 - ☐ Programming the basic computer; microprogrammed control
 - ☐ CPU: general register & stack organisation, instruction formats, addressing modes, RISC vs CISC
 - ☐ Pipeline & vector processing
 - ☐ I/O organisation: interfaces, asynchronous transfer, modes of transfer, priority interrupt, DMA, serial communication
 - ☐ Memory hierarchy: main, auxiliary, associative, cache, virtual memory
 - ☐ Multiprocessors: interconnection structures, arbitration, synchronisation, cache coherence, multicore
-

2.1 Digital Logic

Every computer is ultimately built from logic gates that combine binary signals. Combinational circuits compute outputs purely from present inputs; sequential circuits add memory in the form of flip-flops, so their outputs also depend on the past.

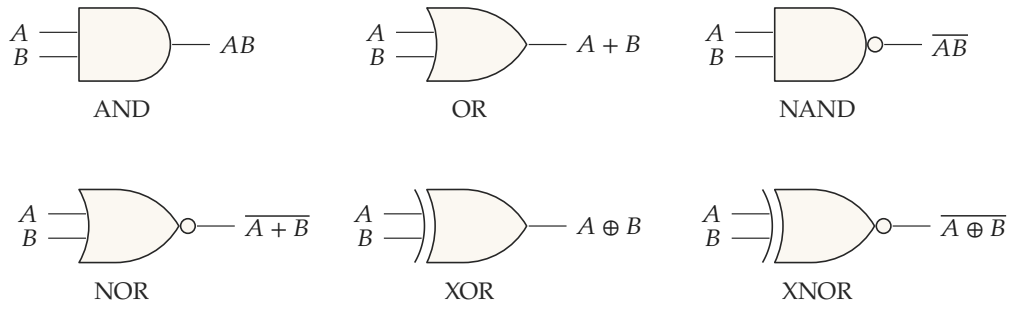


FIGURE 2.1. The two-input logic gates in the distinctive (US) shapes. NAND and NOR are *universal*: each alone can build every other gate.

2.1.1 Gates

Gate	Expression	Output 1 when
AND	AB	all inputs 1
OR	$A + B$	any input 1
NOT	A'	input 0
NAND	$(AB)'$	any input 0 — universal
NOR	$(A+B)'$	all inputs 0 — universal
XOR	$A \oplus B = A'B + AB'$	inputs differ (odd number of 1s)
XNOR	$(A \oplus B)'$	inputs equal

Gates needed using only NAND: NOT = 1, AND = 2, OR = 3, XOR = 4, XNOR = 5. Using only NOR: NOT = 1, OR = 2, AND = 3, XNOR = 4, XOR = 5.

2.1.2 K-Map Simplification

A Karnaugh map rearranges a truth table so that terms differing in a single variable sit side by side. Grouping adjacent 1s in blocks of 1, 2, 4 or 8 then eliminates the variables that change within the block, giving a minimal sum of products by inspection.

$AB \backslash CD$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

	00	01	11	10	
00	1	0	0	1	
01	0	1	1	0	$\bar{B}\bar{D}$ (four corners)
11	0	1	1	0	BD (centre block)
10	1	0	0	1	

CELL NUMBERING (GRAY CODE) $F = \sum m(0, 2, 5, 7, 8, 10, 13, 15) = BD + \bar{B}\bar{D}$

FIGURE 2.2. Karnaugh maps. Adjacent cells differ in one variable, and the map wraps around its edges, so the four corners form a single group.

Rules: group 1, 2, 4, 8, 16 adjacent cells (wrap-around allowed); larger groups = fewer literals; use don't-cares (X) when helpful.

- **Prime implicant:** largest possible group. **Essential PI:** covers a minterm no other PI covers.

2.1.3 Combinational Circuits

Adders, multiplexers, decoders and comparators are the standard building blocks of a processor's data path. The full adder below is worth being able to draw from memory.

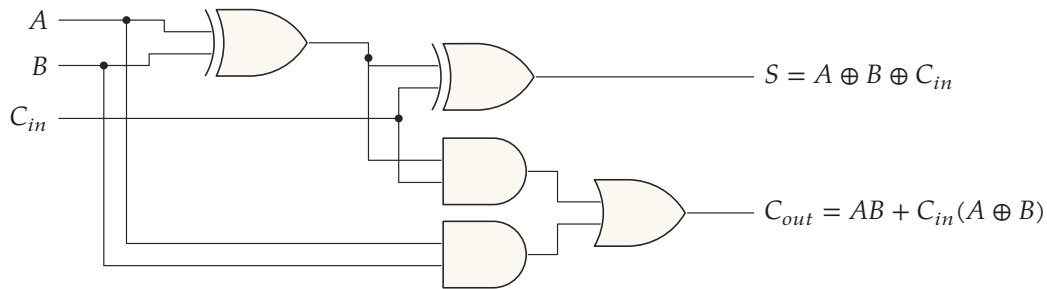


FIGURE 2.3. A full adder built from two half adders (the XOR–AND pairs) and an OR gate.

Circuit	Function
Multiplexer $2^n:1$	n select lines choose one of 2^n inputs; any n-var function with $2^n:1$ MUX (or n+1 var with $2^n:1$ + inverter)
Demultiplexer $1:2^n$	Routes one input to one of 2^n outputs
Decoder $n:2^n$	Activates one of 2^n outputs (minterm generator)
Encoder $2^n:n$	Reverse; priority encoder handles multiple inputs
Comparator	$A > B$, $A = B$, $A < B$
Ripple carry adder	n full adders; delay $\propto n$
Carry look-ahead adder	$G_i = A_i B_i$, $P_i = A_i \oplus B_i$; $C_{i+1} = G_i + P_i C_i$ — faster

2.1.4 Flip-Flops

A flip-flop stores one bit and changes state only on a clock edge. Its *characteristic* equation predicts the next state from the inputs; its *excitation* table answers the designer's reverse question — which inputs produce a desired transition.

FF	Characteristic equation	Notes
SR	$Q^+ = S + R'Q$ ($SR = 0$)	$S=R=1$ invalid
JK	$Q^+ = JQ' + K'Q$	$J=K=1$ toggles; race-around fixed by master-slave
D	$Q^+ = D$	Delay / data latch
T	$Q^+ = T \oplus Q$	Toggle; used in counters

Q	Q^+	S	R	J	K	D	T
0	0	0	×	0	×	0	0
0	1	1	0	1	×	1	1
1	0	0	1	×	1	0	1
1	1	×	0	×	0	1	0

Excitation table: the inputs required to move from Q to Q^+ (× = don't care).

2.1.5 Counters & Registers

- n flip-flops → **mod-2ⁿ** ripple counter. Ring counter with n FFs → mod- n . **Johnson (twisted ring)** with n FFs → **mod-2n**.
- Asynchronous (ripple) counter delay = $n \times t_{pd}$; synchronous counters clock all FFs together.
- Shift registers: SISO, SIPO, PISO, PIPO.
- Memory: $2^k \times n$ RAM needs k address lines and n data lines. e.g. $4K \times 8 \rightarrow 12$ address lines.
- Number of $1K \times 4$ chips to build $16K \times 8$: $(16 \times 8) / (1 \times 4) = 32$.

2.2 Data Representation

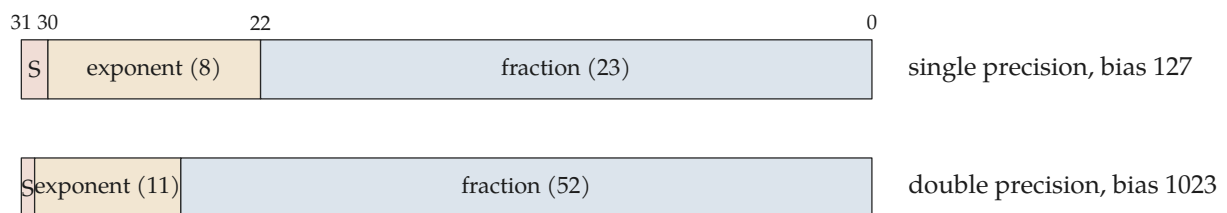
Computers represent integers in two's complement because it makes subtraction the same circuit as addition and gives a single zero. Real numbers use the IEEE 754 floating-point formats, which trade exactness for an enormous range.

2.2.1 Signed numbers (n bits)

Representation	Range	Zeros
Sign-magnitude	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	Two (+0, -0)
1's complement	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	Two
2's complement	$-2^{n-1} \dots +(2^{n-1}-1)$	One

8-bit 2's complement: -128 to $+127$. $-5 = 11111011$. **Overflow** in 2's-complement addition: carry into MSB $\neq$ carry out of MSB (or adding two same-sign numbers gives opposite sign). r 's complement of N (n digits) $= r^n - N$; $(r-1)$'s complement $= r^n - 1 - N$.

2.2.2 Floating Point — IEEE 754



$$\text{value} = (-1)^S \times 1.F \times 2^{E-\text{bias}}$$

FIGURE 2.4. IEEE 754 formats. The leading 1 of a normalised mantissa is implicit and not stored.

Example: $-6.25 = -110.01_2 = -1.1001 \times 2^2 \rightarrow S=1, E=129=10000001, M=1001000\dots \rightarrow \mathbf{C0C80000}$ (hex).

– $E=0, M=0 \rightarrow \pm 0$; $E=\text{all } 1\text{s}, M=0 \rightarrow \pm \infty$; $E=\text{all } 1\text{s}, M \neq 0 \rightarrow \text{NaN}$; $E=0, M \neq 0 \rightarrow \text{denormal}$.

2.2.3 Codes

- **BCD** (8421), **Excess-3** (BCD + 3, self-complementing), **Gray code** (one bit changes; $G = B \oplus (B \gg 1)$).
- **Parity** detects single-bit (odd number of) errors.
- **Hamming code**: number of parity bits r satisfies $2^r \geq m + r + 1$; parity bits at positions 1, 2, 4, 8 ...; detects 2, corrects 1 (with extra bit: SEC-DED).
- **CRC**: polynomial division; used in networks.

2.2.4 Arithmetic

- **Booth's algorithm** (signed multiplication): look at $Q_n Q_{n+1} \rightarrow 10$: subtract M ; 01 : add M ; $00/11$: shift only; arithmetic right shift each step.
- Division: restoring and non-restoring.

2.3 Register Transfer & Micro-operations

- RTL: $R2 \leftarrow R1, P: R2 \leftarrow R1$ (conditional on control P).
- Micro-operation types: register transfer, arithmetic (add, sub, inc, dec), logic, shift (logical, circular, arithmetic).
- Common bus with k registers of n bits: n multiplexers of $k \times 1$ each, $\log_2 k$ select lines.

2.4 Basic Computer (Mano)

Morris Mano's "basic computer" is the textbook model behind many questions: a 16-bit machine with eight registers and 4096 words of memory, all connected by one common bus.

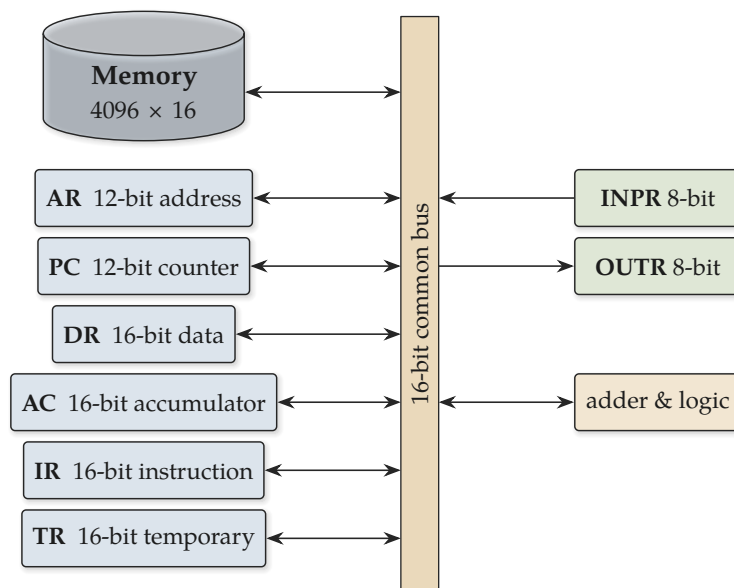


FIGURE 2.5. Mano's basic computer: memory and registers communicate through a single common bus; select lines decide which register drives the bus in each clock cycle.

Instruction format (16 bits): I (1) | Opcode (3) | Address (12) — I = 0 direct, 1 indirect.

- Instruction types: memory-reference (opcode 000–110), register-reference (0111...), I/O (1111...).

2.4.1 Instruction Cycle

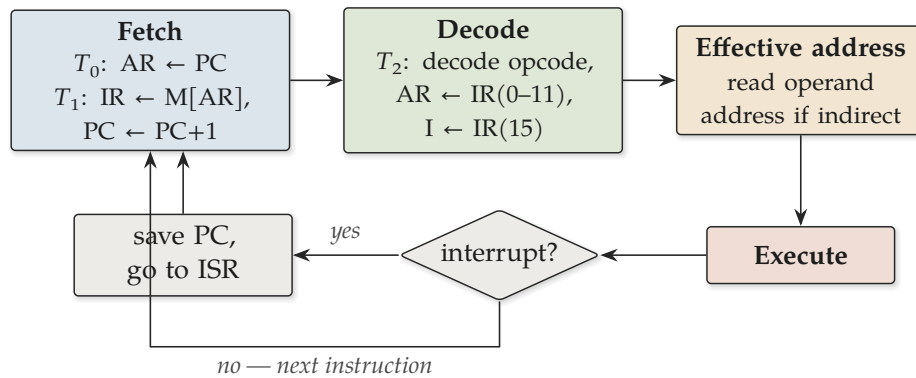


FIGURE 2.6. The instruction cycle of the basic computer, with the interrupt check after each instruction.

2.5 Control Unit

The control unit issues the sequence of control signals that makes each instruction happen. It can be built directly from gates (hardwired) or by reading control words from a small memory (microprogrammed).

Hardwired	Microprogrammed
Logic gates, decoders, sequence counter	Control memory stores microinstructions
Fast	Slower
Difficult to modify	Flexible, easy to modify
Used in RISC	Used in CISC

- Control word bits: **horizontal** microprogramming (one bit per control signal, wide, parallel, fast) vs **vertical** (encoded, narrow, needs decoder).
- Control address register (CAR), control data register (CDR), sequencer, mapping logic.
- Control memory size = (#microinstructions) × (width).

2.6 CPU Organisation

2.6.1 Instruction formats (by number of addresses)

Evaluate $X = (A + B) \times (C + D)$:

Type	Code
3-address	ADD R1, A, B · ADD R2, C, D · MUL X, R1, R2
2-address	MOV R1, A · ADD R1, B · MOV R2, C · ADD R2, D · MUL R1, R2 · MOV X, R1
1-address (accumulator)	LOAD A · ADD B · STORE T · LOAD C · ADD D · MUL T · STORE X
0-address (stack)	PUSH A · PUSH B · ADD · PUSH C · PUSH D · ADD · MUL · POP X

2.6.2 Addressing Modes (*very frequent*)

An addressing mode tells the processor how to find an operand. The choice trades instruction length against flexibility: immediate operands need no memory access at all, whereas indirect addressing needs an extra one but allows pointers.

Mode	Effective address	Use
Implied	Operand implicit	CMA, stack ops
Immediate	Operand in instruction	Constants
Register	Operand in register	Fast
Register indirect	$EA = (R)$	Pointers
Direct (absolute)	$EA = A$	Global variables
Indirect	$EA = M[A]$	Pointers
Auto-increment/ decrement	$EA = (R)$, then $R \pm 1$	Arrays, stacks
Relative	$EA = PC + A$	Branches, position-independent code
Indexed	$EA = XR + A$	Arrays
Base register	$EA = BR + A$	Relocation

Memory references to get operand: immediate 0, direct 1, indirect 2.

2.6.3 RISC vs CISC

RISC	CISC
Few, simple, fixed-length instructions	Many, complex, variable length
Few addressing modes	Many addressing modes
Load/store architecture	Memory operands in many instructions
Many registers, register windows	Fewer registers
Hardwired control	Microprogrammed
1 instruction per cycle, pipelining easy	Multi-cycle
ARM, MIPS, SPARC, RISC-V	x86, VAX, IBM 370

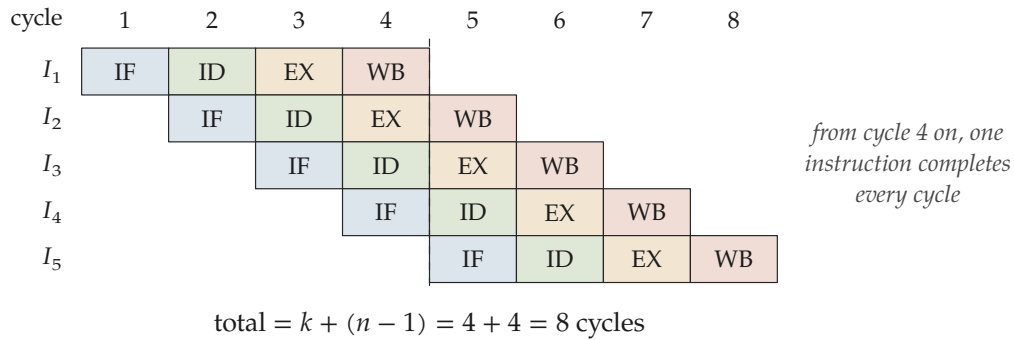


FIGURE 2.7. Space-time diagram of a four-stage pipeline executing five instructions.

2.7 Pipelining

Pipelining overlaps the execution of successive instructions, like an assembly line: while one instruction executes, the next is decoded and the one after is fetched. It does not make a single instruction faster, but it raises throughput towards one instruction per cycle.

Pipelined time for n instructions, k stages	$T_{\text{pipe}} = (k + n - 1) t_p, \quad t_p = \max(\text{stage}) + \text{latch}$
Non-pipelined time	$T_{\text{seq}} = n t_n, \quad t_n = \sum \text{stage delays}$
Speedup	$S = \frac{n t_n}{(k + n - 1) t_p} \rightarrow k \quad (n \rightarrow \infty, \text{equal stages})$
Efficiency and throughput	$\eta = \frac{S}{k}, \quad \text{throughput} = \frac{n}{T_{\text{pipe}}}$
With stalls	$\text{CPI}_{\text{pipe}} = 1 + \text{stalls per instruction}, \quad S = \frac{\text{CPI}_{\text{non-pipe}}}{\text{CPI}_{\text{pipe}}}$

2.7.1 Hazards

A hazard is any situation that prevents the next instruction from entering the pipeline in its expected cycle. Each type has its own family of remedies.

Structural hazard	Data hazard	Control hazard
<i>cause:</i> two instructions need the same resource <i>remedy:</i> duplicate resources; separate instruction and data caches	<i>cause:</i> an instruction needs a result not yet written (RAW; WAR and WAW when out of order) <i>remedy:</i> operand forwarding; stalls; register renaming	<i>cause:</i> branch outcome unknown when the next instruction is fetched <i>remedy:</i> branch prediction; delayed branch; flushing

FIGURE 2.8. The three kinds of pipeline hazard and how they are handled.

- Only **RAW** occurs in a simple in-order pipeline; WAR and WAW occur in out-of-order execution.

- **Flynn's taxonomy**: SISD (uniprocessor), **SIMD** (vector/array processors, GPU), MISD (rare; fault-tolerant), MIMD (multiprocessors).
- **Vector processing**: one instruction operates on arrays; **array processors** = SIMD with multiple PEs.
- **Amdahl's law**: $\text{Speedup} = 1 / ((1 - f) + f/s)$, f = fraction enhanced, s = speedup of that fraction.
- Superscalar (multiple issue), VLIW, superpipelining.

2.8 I/O Organisation

Input–output devices are slow and varied, so the processor talks to them through interfaces. The key design question is how much of the transfer the CPU itself must supervise.

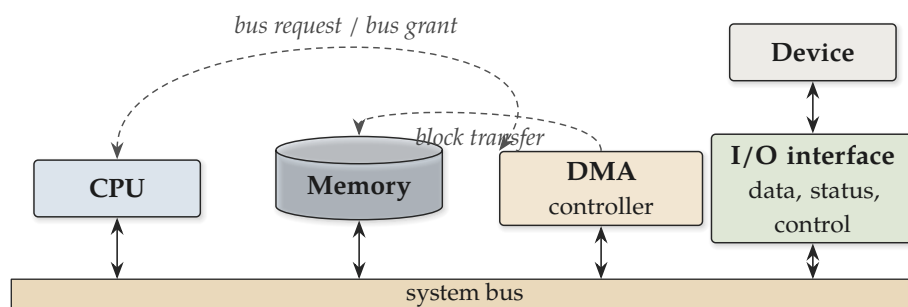


FIGURE 2.9. I/O organisation. A DMA controller borrows the bus (after a bus request and grant) to move whole blocks between a device and memory.

Mode	How	CPU involvement
Programmed I/O	CPU polls status flag	Very high (busy waiting)
Interrupt-driven	Device interrupts CPU	Per word
DMA	DMA controller transfers block directly to memory	Only at start/end
I/O processor (channel)	Dedicated processor	Minimal

- DMA modes: **burst** (block at once), **cycle stealing** (one word per cycle), **transparent** (when CPU not using bus).
- DMA signals: Bus Request (BR) and Bus Grant (BG).
- Asynchronous transfer: **strobe** (one control line) or **handshaking** (two lines — data valid & data accepted).
- **Priority interrupts**: polling (software), **daisy chaining** (serial hardware — nearest device highest priority), parallel priority (priority encoder).
- Vectored interrupt: device supplies ISR address. Non-maskable interrupt (NMI): cannot be disabled.
- Isolated I/O (separate address space, IN/OUT instructions) vs **memory-mapped I/O** (same address space).

- Serial communication: asynchronous (start bit, data, parity, stop bits) — UART; synchronous — USRT.

2.9 Memory Hierarchy

Programs tend to reuse recently used data (temporal locality) and data near it (spatial locality). The memory hierarchy exploits this by keeping copies of active data in small, fast memories close to the processor.

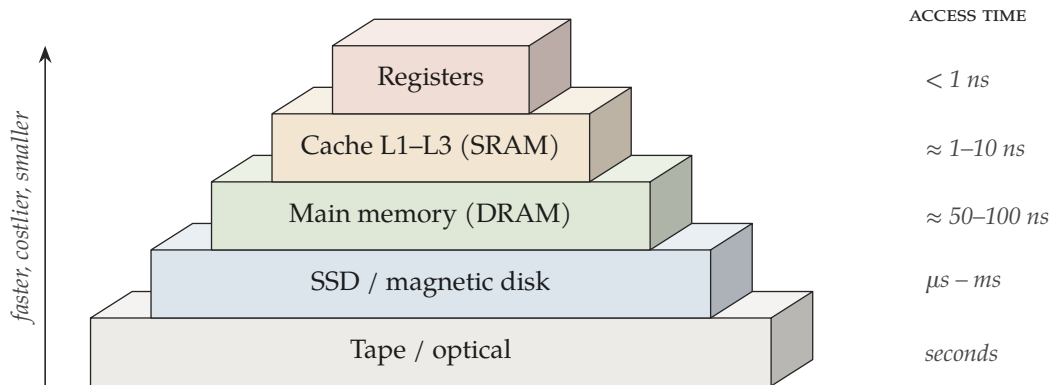


FIGURE 2.10. The memory hierarchy with typical access times. Locality of reference lets a small fast level satisfy most accesses.

- SRAM (flip-flops, no refresh, cache) vs DRAM (capacitors, refresh, main memory).
- ROM, PROM, EPROM (UV erase), EEPROM (electrical), Flash.
- **Associative memory (CAM)**: accessed by content; used in TLB.
- **Locality of reference**: temporal (same item soon) & spatial (nearby items soon).

2.9.1 Cache Mapping

A cache is far smaller than main memory, so each memory block must be assigned somewhere in it. The mapping scheme decides where a block may go — exactly one line, any line, or any line of one set — and so how the address is split into tag, index and offset.



32-bit address; 64 KB cache; 32-byte blocks (2048 lines)

FIGURE 2.11. How the same 32-bit address is divided under the three cache-mapping schemes.

Worked example: Main memory 4 GB (32-bit address), cache 64 KB, block 32 B.

$$\text{offset bits} = \log_2 32 = 5$$

$$\text{lines} = \frac{64 \text{ KB}}{32 \text{ B}} = 2048 = 2^{11} \Rightarrow 11 \text{ line bits}$$

$$\text{direct mapped: tag} = 32 - 11 - 5 = 16$$

$$4\text{-way: sets} = 2048/4 = 512 = 2^9, \quad \text{tag} = 32 - 9 - 5 = 18$$

$$\text{fully associative: tag} = 32 - 5 = 27$$

$$\text{tag directory (direct)} = 2048 \times 16 \text{ bits (+valid and dirty bits)}$$

Parallel (simultaneous) access	$T = h t_c + (1 - h) t_m$
Hierarchical (serial) access	$T = h t_c + (1 - h)(t_c + t_m)$
Average memory access time	AMAT = hit time + miss rate $\times$ miss penalty

- Replacement: LRU, FIFO, random (needed for associative/set-associative only).
- Write policies: **write-through** (update memory immediately; with write buffer) vs **write-back** (dirty bit, on eviction).
- Write miss: write-allocate (usually with write-back) vs no-write-allocate (with write-through).
- Misses: **Compulsory** (cold), **Capacity**, **Conflict** (3 Cs).

2.9.2 Virtual Memory

Covered in depth in **Unit 5 (OS)**: paging, TLB, page replacement.

- Effective access with TLB: $EAT = h(t_{\text{TLB}} + t_m) + (1 - h)(t_{\text{TLB}} + 2t_m)$ (single-level page table).

2.9.3 Magnetic Disk

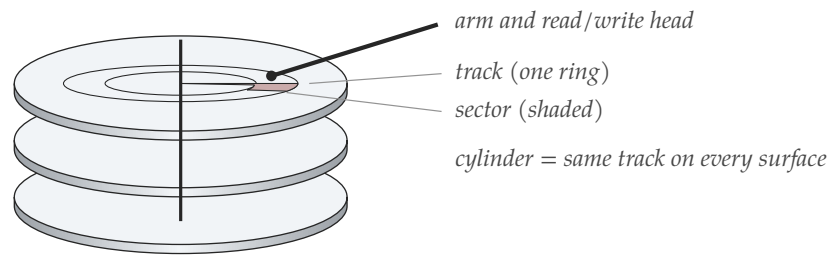


FIGURE 2.12. Geometry of a magnetic disk.

Access time	seek + rotational latency + transfer
Average rotational latency	$\frac{1}{2} \times \frac{60}{\text{RPM}}$ seconds
Capacity	surfaces $\times$ tracks $\times$ sectors per track $\times$ bytes per sector

e.g. 7200 RPM $\rightarrow$ one rotation 8.33 ms $\rightarrow$ avg latency **4.17 ms**.

2.10 Multiprocessors

Multiprocessors raise performance by running work in parallel. Their central difficulties are connecting processors to memory without bottlenecks and keeping their private caches consistent.

- **Tightly coupled** (shared memory, UMA/NUMA) vs **loosely coupled** (distributed memory, message passing).
- Interconnection structures: time-shared common bus, multiport memory, **crossbar switch** (n^2 switches, non-blocking), multistage networks (Omega: $(n/2) \log_2 n$ 2×2 switches), hypercube (n-cube: 2^n nodes, each with n links, diameter n).
- Inter-processor arbitration: serial (daisy chain), parallel, dynamic (time-slice, polling, LRU, FIFO, rotating daisy chain).
- Synchronisation: test-and-set, semaphores, mutual exclusion via hardware lock.
- **Cache coherence**: write-invalidate / write-update; snoopy protocols; **MESI** (Modified, Exclusive, Shared, Invalid); directory-based protocols.
- **Multicore**: multiple cores on one chip sharing L2/L3 cache; hyper-threading (SMT).

2.11 Deeper Dive — Booth Trace, Instruction Encoding, Cache Behaviour & Interrupts

2.11.1 Booth's Algorithm — Full Trace: $7 \times (-3)$, 4 bits

$M = 0111$ (7), $-M = 1001$, $Q = 1101$ (-3), $A = 0000$, $Q_{-1} = 0$.

Step	$Q_0 Q_{-1}$	Operation	A	Q	Q_{-1}
Init			0000	1101	0
1	1 0	$A \leftarrow A - M$	1001	1101	0
		Arithmetic shift right	1100	1110	1
2	0 1	$A \leftarrow A + M$	0011	1110	1
		Arithmetic shift right	0001	1111	0
3	1 0	$A \leftarrow A - M$	1010	1111	0
		Arithmetic shift right	1101	0111	1
4	1 1	Shift only	1110	1011	1

Result $A\ Q = 1110\ 1011 = -21$ in 8-bit 2's complement ✓. Booth reduces additions for runs of 1s; worst case is alternating bits (0101...).

2.11.2 Instruction Encoding Numericals

Fixed format: 32-bit instructions, 64 registers, 45 distinct opcodes, format opcode | R_d | R_s | immediate.

opcode (6)	R_d (6)	R_s (6)	immediate (14)	<i>immediate range $-8192 \dots +8191$</i>
------------	-----------	-----------	----------------	---

FIGURE 2.13. A 32-bit instruction with 45 opcodes ($\lceil \log_2 45 \rceil = 6$ bits) and 64 registers (6 bits each).

Expanding opcode: 16-bit instruction, 4-bit address fields.

Format	Opcode patterns available	Used / kept as escape
3-address	$2^4 = 16$	15 used, 1 escape
2-address	$1 \times 2^4 = 16$	14 used, 2 escapes
1-address	$2 \times 2^4 = 32$	31 used, 1 escape
0-address	$1 \times 2^4 = 16$	16 used

Rule: unused patterns at one level $\times 2^{\text{field width}} = \text{patterns available at the next level}$.

2.11.3 Mapping an Address into a Set-Associative Cache

2-way set associative, 128 lines, 16-byte blocks, 16-bit byte address **0x1A2B**.

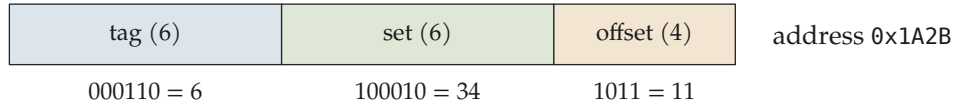


FIGURE 2.14. Address 0x1A2B in a 2-way set-associative cache of 128 lines with 16-byte blocks.

2.11.4 Hits & Misses for Three Organisations

Block reference string: **0, 4, 0, 4, 8, 0** with 4 cache lines.

Organisation	Trace	Misses
Direct mapped (line = block mod 4)	0, 4, 8 all map to line 0 → every access evicts the previous	6
2-way set assoc., LRU (set = block mod 2)	0 M, 4 M, 0 H, 4 H, 8 M (evict 0), 0 M (evict 4)	4
Fully associative, LRU	0 M, 4 M, 0 H, 4 H, 8 M, 0 H	3

Higher associativity removes **conflict misses**; the three first-time misses are **compulsory**.

2.11.5 Interrupt Cycle (Mano basic computer)

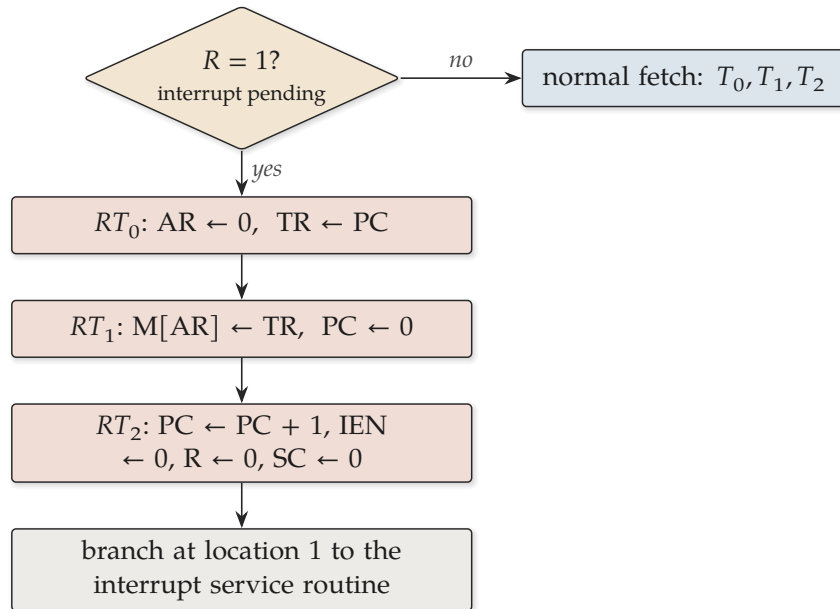


FIGURE 2.15. The interrupt cycle: the return address is saved in location 0 and further interrupts are disabled.

Return address is saved in memory location 0; the ISR ends with an indirect branch through location 0 (BUN 0 I), re-enabling interrupts with ION.

2.11.6 Microprogrammed Control Numericals

Mano's microinstruction: F1 (3) | F2 (3) | F3 (3) | CD (2) | BR (2) | AD (7) = **20 bits**;
control memory 128×20 .

- Horizontal: n control signals $\rightarrow n$ bits. Vertical: encode k mutually exclusive signals in $\lceil \log_2(k + 1) \rceil$ bits.
- **Example:** 48 control signals in 4 mutually-exclusive groups of 12 $\rightarrow$ vertical needs $4 \times \lceil \log_2 13 \rceil = 4 \times 4 = 16$ bits (vs 48 horizontal).

2.11.7 DMA & I/O Numericals

- Disk transfers 4 MB/s in 32-bit words using cycle stealing on a memory that supports 100 million cycles/s: words/s = 4 MB / 4 B = 1 M $\rightarrow$ fraction of memory cycles stolen = 1 M / 100 M = **1%**.
- Programmed I/O polling overhead: polling 100 times/s $\times$ 500 cycles each = 50,000 cycles/s $\rightarrow$ **0.05%** of a 100 MHz CPU.

2.11.8 Memory Interleaving

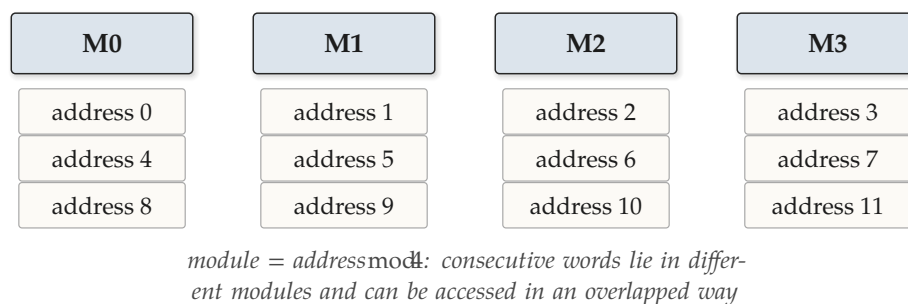


FIGURE 2.16. Low-order interleaving across four memory modules.

2.12 Previous Year Questions (PYQ pattern)

Number systems & codes

1. 2's complement of 01101000: **10011000**
2. Range of 8-bit 2's complement: **−128 to +127**
3. Gray code of binary 1011: 1, $1 \oplus 0$, $0 \oplus 1$, $1 \oplus 1 \rightarrow$ **1110**
4. Excess-3 of decimal 5: **1000**
5. Hamming code for 4 data bits needs **3** parity bits ($2^3 \geq 4+3+1$)
6. IEEE 754 single precision bias: **127**; exponent bits **8**; mantissa **23**
7. (-0.75) in IEEE 754 single: $S=1$, -1.1×2^{-1} , $E=126 \rightarrow$ **BF400000**

Digital logic

8. Minimum number of NAND gates for XOR: **4**
9. Simplify $F = \sum m(0,1,2,3)$: **A'** (for 3 variables A,B,C) — $F = A'$

10. Number of select lines for 32:1 MUX: **5**
11. Johnson counter with 4 FFs has **8** states
12. Characteristic equation of JK FF: $Q^+ = JQ' + K'Q$
13. Race-around condition occurs in: **JK flip-flop when $J=K=1$ and clock pulse wide** — solved by master-slave
14. A 3-to-8 decoder with an OR gate can implement: **any 3-variable Boolean function**
15. $16K \times 8$ memory needs **14** address lines

CPU & addressing

16. Addressing mode used for position-independent code: **Relative (PC-relative)**
17. Addressing mode in which operand is part of instruction: **Immediate**
18. Which addressing mode needs two memory accesses to get operand (after fetch)? **Indirect**
19. Zero-address instructions are used in: **stack-organised computers**
20. Which is a feature of RISC? **Fixed-length instructions / load-store / hardwired control**
21. Horizontal microprogramming: **one bit per control signal, longer control words**

Pipelining

22. A 5-stage pipeline, stage delay 20 ns, executes 100 instructions. Time = $(5 + 99) \times 20 = 2080$ ns
23. Ideal speedup of a k-stage pipeline: **k**
24. Stages 150, 120, 160, 140 ns, latch 5 ns. Pipeline cycle = **165 ns**; non-pipelined instruction = 570 ns
25. RAW hazard is also called: **true data dependency**; fixed by **operand forwarding**
26. Amdahl: 40% of program sped up 10 $\times$. Speedup = $1/(0.6 + 0.04) = 1.5625$
27. Vector processors belong to which Flynn category? **SIMD**

Memory & cache

28. Cache 2 ns, memory 20 ns, hit ratio 0.9, hierarchical: $0.9 \times 2 + 0.1 \times 22 = 4$ ns
29. Direct-mapped cache with 128 lines, block 16 words; 64K-word memory $\rightarrow$ address 16 bits: TAG 5, LINE 7, WORD 4
30. Which miss cannot be reduced by increasing cache size? **Compulsory miss**
31. Write-back cache uses a: **dirty bit**
32. Associative memory is accessed by: **content**
33. Disk 6000 RPM: average rotational latency = **5 ms**

I/O

34. Which mode transfers data without CPU intervention? **DMA**
35. In daisy chaining, priority is determined by: **position of device in the chain**
36. DMA cycle stealing means: **DMA takes the bus for one cycle at a time**
37. Handshaking uses: **two control lines**

Multiprocessors

38. Number of crosspoints in a crossbar connecting 8 processors to 8 memories: **64**
39. MESI protocol is used for: **cache coherence**

40. In a 4-dimensional hypercube, number of nodes: **16**, links per node: **4**

More practice questions

- 41. Booth's algorithm when $Q_0Q_{-1} = 01$: **add M then shift**
- 42. A machine has 32-bit instructions and 128 registers; a 3-register format leaves how many opcode bits? $32 - 21 = 11$
- 43. 16-bit instructions with 6-bit address fields: 2-address instructions use 4-bit opcodes; with 14 two-address instructions used, one-address instructions possible: $2 \times 64 = 128$
- 44. 4-way set associative cache, 256 lines, 32-byte blocks, 32-bit addresses: set bits **6**, offset **5**, tag **21**
- 45. Which miss is eliminated by full associativity? **conflict miss**
- 46. In Mano's interrupt cycle, the return address is stored in: **memory location 0**
- 47. Vertical microprogramming requires a: **decoder** for the encoded control fields
- 48. Low-order interleaving places consecutive addresses in: **different memory modules**
- 49. A DMA transfer of 1 M words/s on 50 M memory cycles/s steals: **2%** of cycles
- 50. Booth multiplication of $2 \times (-4)$ in 4 bits gives $AQ = 1111\ 1000$ (-8)

POINTS TO REMEMBER

- NAND: XOR=4 · NOR: XNOR=4 · Johnson n FF $\rightarrow 2n$ states
- IEEE single 1/8/23 bias 127 · double 1/11/52 bias 1023
- Pipeline time $(k + n - 1)t$ · speedup $\rightarrow k$ · Amdahl $1/((1-f)+f/s)$
- Hierarchical cache $T = h \cdot tc + (1-h)(tc + tm)$
- Direct: TAG|LINE|OFFSET · Set-assoc: TAG|SET|OFFSET · Fully: TAG|OFFSET

Programming Languages & Computer Graphics

...

Expected questions: 8–10 · Target: 8+ · Type: C/C++ output prediction, OOP concepts, graphics algorithms & transforms

SYLLABUS COVERAGE

- ☐ Language design & translation: paradigms, binding times, virtual computers, syntax, stages of translation
 - ☐ Elementary data types: scalar & composite
 - ☐ Programming in C: tokens, data types, control, arrays, structures, unions, strings, pointers, functions, files, command-line args, preprocessor
 - ☐ OOP: class, object, inheritance, encapsulation, abstract class, polymorphism
 - ☐ C++: constructors/destructors, overloading, virtual functions, templates, exceptions, streams & files
 - ☐ Web programming: HTML, DHTML, XML, scripting, Java, servlets, applets
 - ☐ Graphics: display devices, raster/random scan, line (DDA, Bresenham), circle & ellipse (mid-point), polygon fill, boundary/flood fill
 - ☐ 2-D transforms & viewing: homogeneous coordinates, composite transforms, window-to-viewport, clipping (Cohen-Sutherland, Liang-Barsky, Sutherland-Hodgman)
 - ☐ 3-D: polygon/quadric surfaces, splines, Bezier & B-spline, illumination, shading, projections
-

3.1 Language Concepts

Every programming language makes design decisions about how programs are structured, when names are bound to meanings, how data are typed and how arguments reach procedures. Questions in this part ask you to recognise these decisions and their consequences.

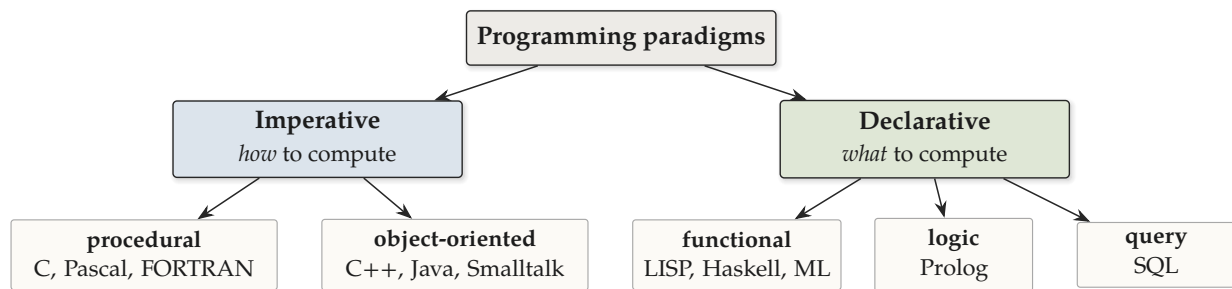


FIGURE 3.1. The main programming paradigms. Event-driven, scripting and concurrent styles cut across both families.

3.1.1 Paradigms

Language	Year/creator	Paradigm
FORTRAN	1957, John Backus	First high-level, scientific
LISP	1958, John McCarthy	Functional, AI
COBOL	1959, Grace Hopper (CODASYL)	Business
ALGOL	1958/60	Block structure, BNF first used
Simula 67	Dahl & Nygaard	First OO language (classes)
Smalltalk	Alan Kay	Pure OO
C	1972, Dennis Ritchie	Procedural
Prolog	1972, Colmerauer	Logic
C++	1979–83, Bjarne Stroustrup	OO + procedural
Ada	1980, US DoD	Concurrency (tasks, rendezvous)
Java	1995, James Gosling (Sun)	OO, platform independent
Python	1991, Guido van Rossum	Multi-paradigm

3.1.2 Binding Time

A binding associates a name or entity with an attribute — a type, an address, a value. The earlier the binding, the more efficient but less flexible the program; the figure orders the possible binding times. Binding = association of an attribute with an entity.

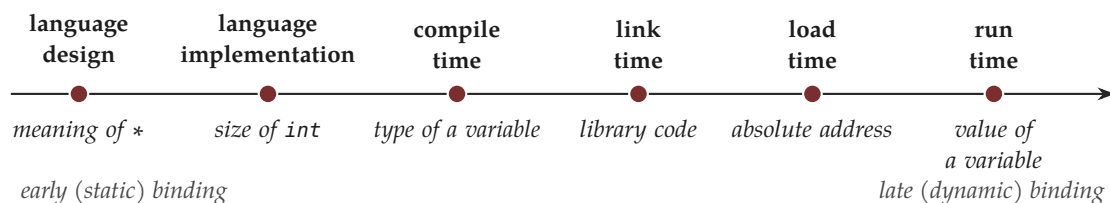


FIGURE 3.2. Binding times, from the earliest to the latest, with a typical attribute bound at each.

- **Static binding** (early) before run time; **dynamic binding** (late) at run time.
- **Static scoping (lexical)**: binding by program text (C, Java). **Dynamic scoping**: by calling sequence (early LISP, Perl local).
- Storage: static (globals), stack-dynamic (locals), explicit heap-dynamic (new/malloc), implicit heap-dynamic.

3.1.3 Parameter Passing

How arguments are passed determines whether a procedure can change the caller's variables and when the argument expressions are evaluated. Questions typically give a short program and ask what it prints under each mechanism.

Method	Behaviour	Language
Call by value	Copy of value	C (default)
Call by reference	Address passed; changes visible	C++ &, Pascal var
Call by value-result (copy-in/ copy-out)	Copy in, copy back at return	Ada in out
Call by name	Textual substitution, evaluated each use (Jensen's device)	ALGOL 60
Call by need	Lazy evaluation, memoised	Haskell

Classic question:

```
void swap(int x, int y) { int t = x; x = y; y = t; } // by value: caller unchanged
```

With $a = 1$, $i = 1$, $A[1] = 5$, call $f(i, A[i])$ where f does $x = x + 1$; $y = y + 1$; by name $\rightarrow A[2]$ incremented (since i changed first); by reference $\rightarrow A[1]$ incremented.

3.1.4 Data Types

- **Scalar**: integer, real, character, Boolean, enumeration, pointer.
- **Composite**: array, record/struct, union, string, set, list, file.
- Strong typing (Java, Ada), weak typing (C). Type equivalence: **name** vs **structural**.
- Type checking static (compile) vs dynamic (run time). Coercion = implicit type conversion.

3.2 C Programming — Exam Traps

Output-prediction questions on C test a small number of rules: operator precedence and associativity, integer arithmetic, pointer arithmetic, the behaviour of macros, and the few constructs whose behaviour is undefined. Work each example by hand before reading its comment.

3.2.1 Operator precedence (high → low)

Level	Operators	Associativity
1	() [] -> . postfix ++ --	left to right
2	unary ! ~ ++ -- + - * & (type) sizeof	right to left
3	* / %	left to right
4	+ -	left to right
5	<< >>	left to right
6	< <= > >=	left to right
7	== !=	left to right
8	&, then ^, then	left to right
9	&&, then	left to right
10	?:	right to left
11	= += -= ...	right to left
12	,	left to right

3.2.2 Output-prediction examples

```
int i = 5;
printf("%d %d", i++, ++i);    // UNDEFINED behaviour (unsequenced modification)

int a = 10, b = 3;
printf("%d %d", a / b, a % b); // 3 1
printf("%d", -7 % 3);          // -1 (sign follows dividend in C99)

int x = 0;
if (x = 5) printf("yes");      // prints yes (assignment, value 5)

char s[] = "NET";
printf("%zu %zu", sizeof(s), strlen(s)); // 4 3

int arr[] = {10, 20, 30};
int *p = arr;
printf("%d", *(p + 2));        // 30 ; arr[i] == *(arr + i) == i[arr]

int k = 3;
printf("%d", k << 2);          // 12 (multiply by 4)

static int counter() { static int c = 0; return ++c; } // retains value: 1, 2, 3 ...

#define SQ(x) x*x
printf("%d", SQ(2+3));          // 2+3*2+3 = 11 (macro pitfall)
```

3.2.3 Pointers

Declaration	Meaning
<code>int *p;</code>	pointer to int
<code>int **pp;</code>	pointer to a pointer to int
<code>int *a[10];</code>	array of 10 pointers to int
<code>int (*a)[10];</code>	pointer to an array of 10 ints
<code>int *f();</code>	function returning a pointer to int
<code>int (*f)();</code>	pointer to a function returning int

Read declarations from the name outwards: `()` and `[]` bind more tightly than `*`.

- Pointer arithmetic scales by `sizeof(type)`. `p + 1` on `int*` (4 bytes) adds 4.
- `void*` generic pointer; dangling pointer (freed memory); NULL pointer; wild pointer (uninitialised).
- Memory: `malloc` (uninitialised), `calloc` (zeroed, $n \times \text{size}$), `realloc`, `free`.

3.2.4 Structures vs Unions

```
struct S { int i; char c; double d; }; // size = sum + padding (typically 16)
union U { int i; char c; double d; }; // size = largest member (8) – members share memory
```

3.2.5 Storage Classes

Class	Scope	Lifetime	Default	Storage
auto	Block	Block	Garbage	Stack
register	Block	Block	Garbage	CPU register (no &)
static	Block/file	Whole program	0	Data segment
extern	Global	Whole program	0	Data segment

3.2.6 Files, Command line, Preprocessor

- `FILE *fp = fopen("a.txt", "r");` modes `r`, `w`, `a`, `r+`, `w+`, `a+`, `rb`. `fgetc`, `fputc`, `fgets`, `fprintf`, `fscanf`, `fread`, `fwrite`, `fseek`, `ftell`, `rewind`.
- `int main(int argc, char *argv[])` — `argv[0]` is the program name; `argc ≥ 1`.
- Preprocessor: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if`, `#pragma`, `#undef`; runs **before compilation**.

3.3 OOP Concepts

Object-oriented programming organises software around objects that combine state and behaviour. Its four principles, shown below as the pillars of a building, are among the most frequently examined definitions in the paper.

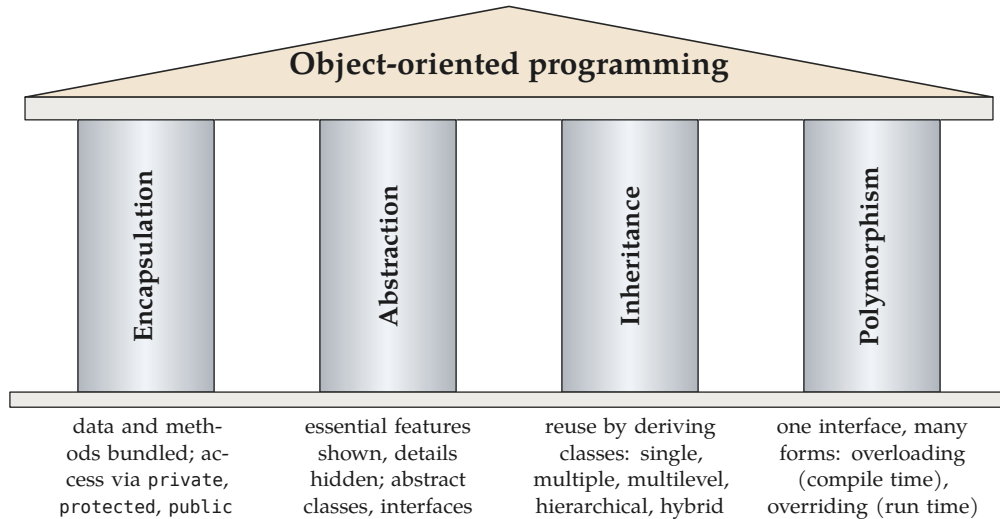


FIGURE 3.3. The four pillars of object-oriented programming.

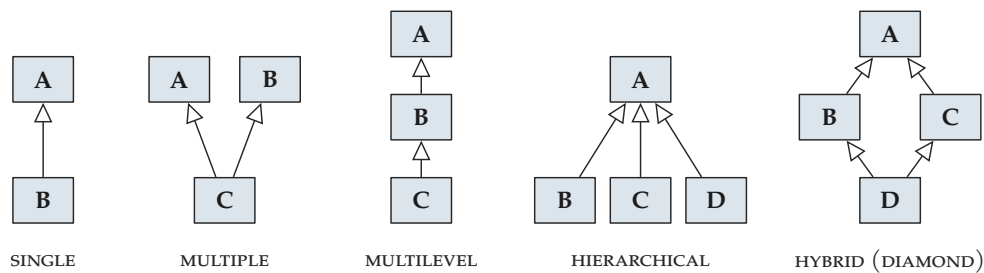


FIGURE 3.4. Kinds of inheritance, drawn with the UML generalisation arrow (pointing to the base class). The diamond causes duplicate copies of A in D unless A is declared a *virtual* base class in C++.

- Diamond problem in C++ solved by **virtual base class**. Java avoids multiple class inheritance (uses interfaces).

3.4 C++ Specifics

Topic	Key point
Constructor	Same name as class, no return type; default, parameterised, copy constructor <code>X(const X&)</code>
Destructor	<code>~X()</code> , no args, cannot be overloaded; called in reverse order of construction
Order of construction	Base → member objects → derived; destruction reverse
this pointer	Hidden pointer to invoking object; not available in static functions
Friend function	Not a member but accesses private data; not inherited
Static member	Shared by all objects; static function accesses only static members
Inline function	Expanded at call site
Function overloading	Same name, different parameter lists (return type alone not enough)
Operator overloading	Cannot overload <code>::</code> , <code>.</code> , <code>.*</code> , <code>?:</code> , <code>sizeof</code>
Virtual function	Run-time polymorphism via vtable/vptr ; call through base pointer/reference
Pure virtual	<code>virtual void f() = 0;</code> → class becomes abstract (cannot be instantiated)
Virtual destructor	Needed when deleting derived object via base pointer
Templates	Generic programming: function & class templates (compile-time polymorphism)
Exceptions	<code>try</code> , <code>throw</code> , <code>catch(...)</code> catches all
Streams	<code>cin</code> (istream), <code>cout</code> (ostream), <code>cerr</code> (unbuffered), <code>clog</code> ; <code>ifstream</code> , <code>ofstream</code> , <code>fstream</code>
Access in inheritance	public base: public→public, protected→protected; protected base: both→protected; private base: both→private; private members never accessible

```
class Base { public: virtual void show() { cout << "Base"; } };
class Der : public Base { public: void show() { cout << "Derived"; } };
Base *b = new Der();
b->show();    // "Derived" (virtual) - without virtual: "Base"
```

3.5 Web Programming

Technology	Key facts
HTML	Markup; tags <code><a href></code> , <code></code> , <code><table><tr><td></code> , <code><form action method></code> ; HTML5 adds <code><canvas></code> , <code><video></code> , <code><audio></code> , <code><section></code> , <code><article></code>
DHTML	HTML + CSS + JavaScript + DOM → dynamic pages
CSS	Inline > internal > external (priority); selectors <code>.class</code> , <code>#id</code>
XML	User-defined tags, data description, must be well-formed (single root, closed tags, case-sensitive); validated by DTD/XSD ; XSLT for transformation, XPath for navigation
JavaScript	Client-side scripting; <code>document.getElementById</code>
Server-side	PHP, JSP, ASP.NET, Servlets
Java Applet	Runs in browser; life cycle: <code>init()</code> → <code>start()</code> → <code>paint()</code> → <code>stop()</code> → <code>destroy()</code> ; no <code>main()</code> (now obsolete)
Servlet	Runs on server; life cycle: <code>init()</code> → <code>service()</code> (<code>doGet/doPost</code>) → <code>destroy()</code> ; container e.g. Tomcat
Java	Bytecode on JVM ("write once run anywhere"); garbage collection; no pointers; <code>final</code> , <code>abstract</code> , <code>interface</code>
HTTP methods	GET (in URL, idempotent), POST (in body), PUT, DELETE

3.6 Computer Graphics

Computer graphics turns mathematical descriptions of shapes into pixels. The syllabus covers the hardware, the scan-conversion algorithms that draw lines and circles, the transformations that move objects, and the clipping and projection steps that fit a scene onto a screen.

3.6.1 Display Devices

Raster scan	Random (vector) scan
Beam sweeps every row (top to bottom)	Beam draws only the lines of the picture
Uses frame buffer (refresh buffer)	Uses display file (display list)
Realistic images, shading	Line drawings only, smooth lines
Aliasing (jaggies)	No aliasing
TV, monitors	Pen plotters, early CAD

Frame-buffer size	$\frac{\text{width} \times \text{height} \times \text{bits per pixel}}{8}$ bytes
Example: 1024 × 768, 24-bit colour	$1024 \times 768 \times 3 = 2.25 \text{ MB}$
Number of displayable colours	$2^{\text{bits per pixel}}$
Aspect ratio	width : height

- CRT parts: electron gun, focusing & deflection system, phosphor coated screen. **Persistence** = time phosphor glows. Shadow-mask (RGB) and beam-penetration (limited colours) colour CRTs.
- LCD, LED, plasma (flat panel); DVST (Direct View Storage Tube). Colour lookup table saves memory.

3.6.2 Line Drawing

A raster display can light only whole pixels, so drawing a line means choosing, column by column, the pixel closest to the true line. DDA does this with floating-point increments; Bresenham's algorithm reaches the same choices with integer additions and a sign test.

DDA (Digital Differential Analyzer)

Differences and number of steps	$\Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1, \quad \text{steps} = \max(\Delta x , \Delta y)$
Increments	$x_{inc} = \frac{\Delta x}{\text{steps}}, \quad y_{inc} = \frac{\Delta y}{\text{steps}}$
Each step	$x \leftarrow x + x_{inc}, \quad y \leftarrow y + y_{inc}, \quad \text{plot}(\text{round}(x), \text{round}(y))$

Bresenham ($|m| < 1$) — integer only:

Initial decision parameter ($ m < 1$)	$p_0 = 2\Delta y - \Delta x$
If $p_k < 0$: next pixel $(x_k + 1, y_k)$	$p_{k+1} = p_k + 2\Delta y$
Otherwise: next pixel $(x_k + 1, y_k + 1)$	$p_{k+1} = p_k + 2\Delta y - 2\Delta x$

Example $(20,10) \rightarrow (30,18)$: $dx=10, dy=8, p_0 = 6, 2dy = 16, 2dy-2dx = -4$.

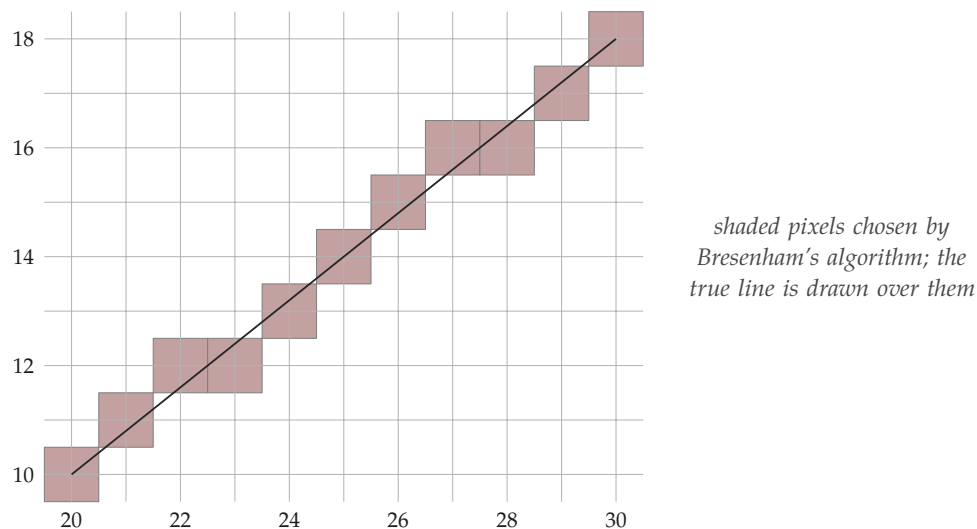


FIGURE 3.5. Bresenham's algorithm for the line from $(20, 10)$ to $(30, 18)$: at each column it picks the pixel nearer the true line using only integer arithmetic.

k	p_k	next pixel
0	6	(21, 11)
1	2	(22, 12)
2	-2	(23, 12)
3	14	(24, 13)
4	10	(25, 14)
5	6	(26, 15)

3.6.3 Mid-point Circle

The mid-point algorithm decides between two candidate pixels by testing whether the midpoint between them lies inside or outside the circle. Symmetry means only one-eighth of the circle need be computed.

- 8-way symmetry: compute one octant (x from 0 to $x = y$), reflect to 8.

Initial decision parameter	$p_0 = 1 - r$ (or $\frac{5}{4} - r$)
If $p_k < 0$: next pixel $(x_k + 1, y_k)$	$p_{k+1} = p_k + 2x_{k+1} + 1$
Otherwise: next pixel $(x_k + 1, y_k - 1)$	$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$

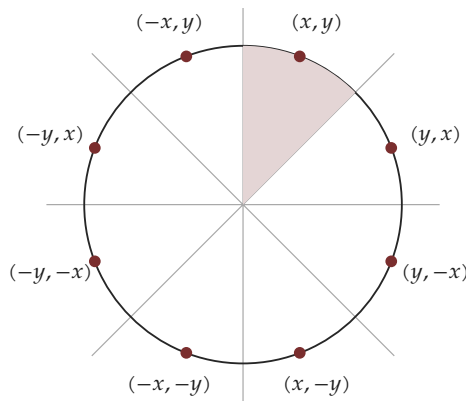


FIGURE 3.6. Eight-way symmetry: the algorithm computes only the shaded octant (x from 0 up to $x = y$) and reflects each point into the other seven.

Ellipse: **4-way symmetry**, two regions (slope -1 boundary).

3.6.4 Filling

- **Scan-line polygon fill**: intersect each scan line with edges, sort x , fill pairs; uses edge table & active edge table.
- **Boundary fill**: fills until boundary colour found. **Flood fill**: replaces a specific interior colour.
- 4-connected vs 8-connected (8-connected may leak through diagonal gaps).

- Inside test: **odd-even (even-odd) rule, non-zero winding number rule.**

3.6.5 2-D Transformations (Homogeneous coordinates)

Writing a point (x, y) as $(x, y, 1)$ lets translation, which is not linear, be expressed as matrix multiplication like rotation and scaling. Any sequence of transformations then collapses into a single matrix product — applied right to left.

$$T(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \quad S(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Reflection about the x -axis, the y -axis, the origin

$$\text{diag}(1, -1, 1), \quad \text{diag}(-1, 1, 1), \quad \text{diag}(-1, -1, 1)$$

Reflection about the line $y = x$

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Shear along x ; along y

$$x' = x + sh_x y; \quad y' = y + sh_y x$$

- Homogeneous coordinates allow **translation as matrix multiplication** → composite transforms.
- Rotation about arbitrary point (x_r, y_r) : $T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$ (applied right-to-left).
- Successive translations add; successive rotations add angles; successive scalings multiply.
- Matrix multiplication is generally **not commutative** (except e.g. two rotations, two scalings, rotation & uniform scaling).

3.6.6 Viewing & Window-to-Viewport

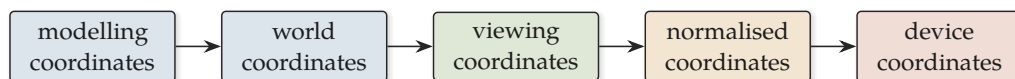


FIGURE 3.7. The two-dimensional viewing pipeline.

Horizontal mapping

$$x_v = x_{v \min} + (x_w - x_{w \min}) s_x, \quad s_x = \frac{x_{v \max} - x_{v \min}}{x_{w \max} - x_{w \min}}$$

Vertical mapping

$$y_v = y_{v \min} + (y_w - y_{w \min}) s_y, \quad s_y = \frac{y_{v \max} - y_{v \min}}{y_{w \max} - y_{w \min}}$$

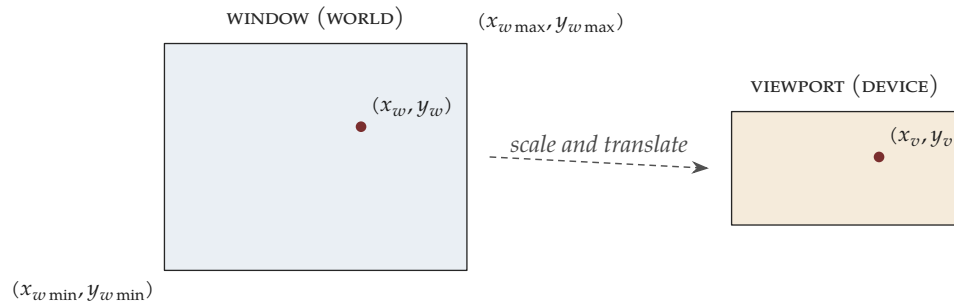


FIGURE 3.8. Window-to-viewport mapping preserves the relative position of a point.

3.6.7 Clipping

Clipping removes the parts of a picture that fall outside the window. Fast algorithms first reject or accept whole lines with cheap tests and compute intersections only when they must.

Cohen–Sutherland line clipping — 4-bit region code **TBRL** (Top, Bottom, Right, Left):

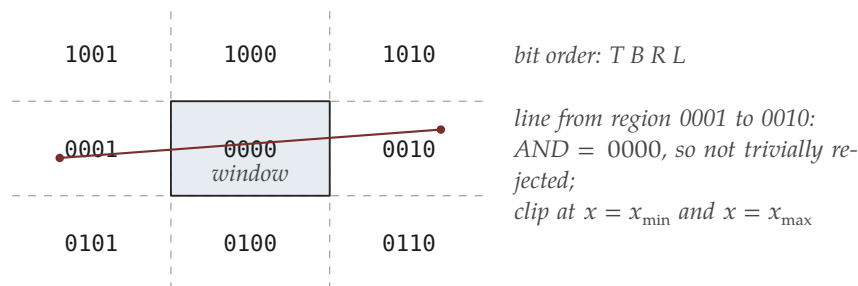


FIGURE 3.9. Cohen–Sutherland region codes. Both endpoint codes 0000: accept; codes with a common 1 bit: reject; otherwise clip against one edge and repeat.

- Both codes 0000 → trivially accept. (code1 AND code2) ≠ 0 → trivially reject. Else clip against an edge and repeat.

Algorithm	For
Cohen–Sutherland	Lines (region codes)
Liang–Barsky	Lines (parametric, faster)
Cyrus–Beck	Lines vs convex polygon window
Nicholl–Lee–Nicholl	Lines (fewest comparisons)
Mid-point subdivision	Lines (binary search)
Sutherland–Hodgman	Polygon clipping (convex window; edge by edge)
Weiler–Atherton	Polygon clipping (concave too)

3.6.8 3-D Graphics

- **Polygon surfaces:** vertex, edge, polygon tables; plane equation $Ax + By + Cz + D = 0$.
- **Quadric surfaces:** sphere, ellipsoid, torus.
- **Splines:** interpolation (passes through control points) vs approximation (near them).

Bezier curve	B-spline curve
Degree = $n - 1$ for n control points	Degree independent of number of points
Passes through first & last control points	Generally doesn't pass through endpoints (uniform)
Global control — moving one point changes whole curve	Local control
Lies within convex hull of control points	Lies within convex hull
Bernstein basis polynomials	B-spline basis functions

Cubic Bezier: $P(t) = (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3, 0 \leq t \leq 1$.

3.6.9 Illumination & Shading

An illumination model computes how bright a surface point appears; a shading method decides at which points to apply the model and how to fill in between.

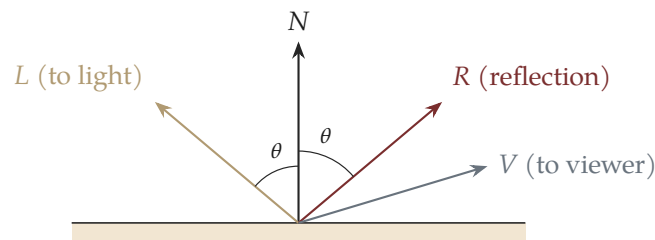


FIGURE 3.10. Vectors in the Phong illumination model.

$$I = \underbrace{I_a k_a}_{\text{ambient}} + \underbrace{I_l k_d (N \cdot L)}_{\text{diffuse (Lambert)}} + \underbrace{I_l k_s (R \cdot V)^n}_{\text{specular (Phong; } n=\text{shininess)}}$$

Shading	Method	Quality
Flat (constant)	One intensity per polygon	Faceted
Gouraud	Interpolate intensities at vertices	Smooth, may miss specular highlights, Mach bands
Phong	Interpolate normals	Best quality, costlier

3.6.10 Projections

A projection maps a three-dimensional scene onto a two-dimensional view plane. Parallel projections keep parallel lines parallel and are used in engineering drawings; perspective projections look natural because distant objects appear smaller.

- Perspective: realistic, foreshortening, parallel lines meet at **vanishing points**; does not preserve parallelism/size.
- Cavalier (receding lines full length, 45°) vs Cabinet (half length, 63.4°).
- Hidden surface removal: **Z-buffer (depth buffer)**, painter's (depth sort), scan-line, BSP tree, back-face detection ($N \cdot V > 0 \rightarrow$ back face), Warnock (area subdivision), ray casting.

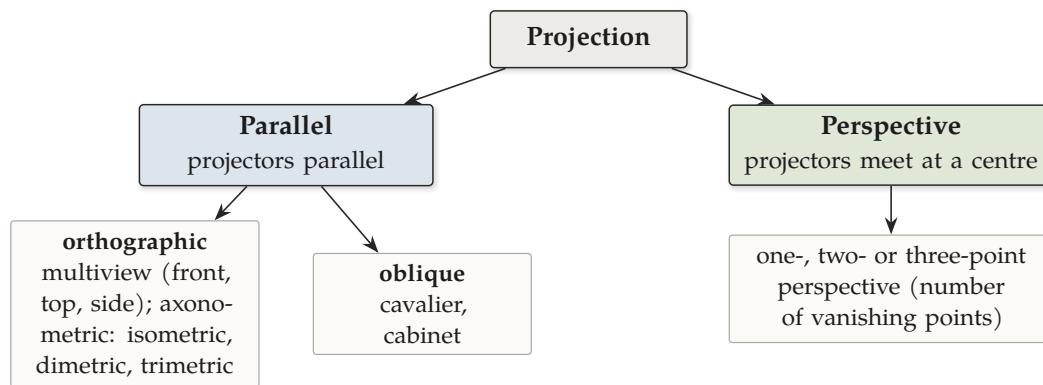


FIGURE 3.11. The family of planar geometric projections.

3.7 Deeper Dive — Scoping, More C Traps, Java Essentials & Graphics Traces

3.7.1 Static vs Dynamic Scoping — Classic Question

```

int x = 1;
void f() { printf("%d", x); }
void g() { int x = 2; f(); }
int main() { g(); }
  
```

- **Static (lexical) scoping:** f's free variable x refers to the global x → prints **1** (C, Java, Pascal).
- **Dynamic scoping:** x is looked up in the most recent active frame (g's) → prints **2**.

3.7.2 Recursion Trace

```

int fun(int n) { if (n <= 1) return 1; return n * fun(n - 2); }
fun(7) = 7 × fun(5) = 7 × 5 × fun(3) = 7 × 5 × 3 × fun(1) = 105
  
```

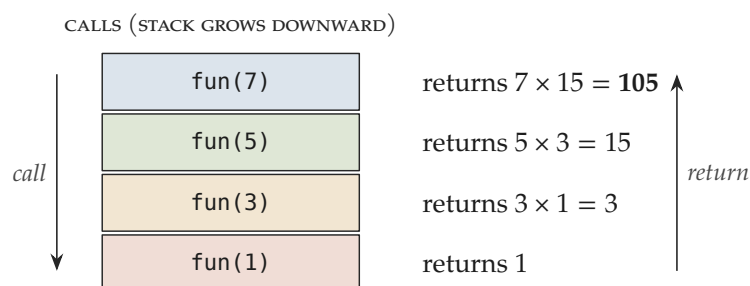


FIGURE 3.12. Activation records for fun(7): frames are pushed as calls are made and popped, with their results, as the recursion unwinds.

3.7.3 More C Output Questions (with reasons)

```
// 1. switch fall-through
int k = 2;
switch (k) { case 1: printf("A"); case 2: printf("B"); case 3: printf("C"); break; default: printf("D"); }
// Output: BC    (no break after case 2)

// 2. bitwise
printf("%d %d %d", 5 & 3, 5 | 3, 5 ^ 3);    // 1 7 6
printf("%d", ~5);                          // -6  (2's complement: ~x = -x - 1)

// 3. pointer to string
char *s = "UGCNET";
printf("%s", s + 3);                        // NET
printf("%c", *s + 1);                       // V   ('U' + 1)

// 4. post-increment in loop condition
int i = 0; while (i++ < 3); printf("%d", i); // 4

// 5. integer division & casting
printf("%.2f", (float)7 / 2);                // 3.50
printf("%d", 7 / 2 * 2);                    // 6

// 6. comma operator
int a = (1, 2, 3); printf("%d", a);          // 3

// 7. short-circuit
int p = 0, q = 5;
if (p && (q = 10)) {} printf("%d", q);       // 5 (second operand never evaluated)

// 8. 2-D array pointer arithmetic
int m[2][3] = {{1,2,3},{4,5,6}};
printf("%d", *(m + 1) + 2);                 // 6  (m[1][2])
```

3.7.4 Java Essentials

Modifier	Same class	Same package	Subclass (other pkg)	World
private	✓	✗	✗	✗
default (none)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Abstract class	Interface
Can have constructors, state (fields), concrete methods	Constants + abstract methods (default/static methods since Java 8)
Single inheritance (extends)	A class can implements many interfaces
Use for "is-a" with shared code	Use for capability contracts

- final variable = constant; final method = cannot be overridden; final class = cannot be inherited (String).
- static members belong to the class. super refers to the parent; this to the current object.

- JVM components: **class loader** → **bytecode verifier** → **interpreter / JIT compiler**; runtime areas: heap, method area, stack, PC registers.
- Exceptions: **checked** (IOException — must be handled or declared) vs **unchecked** (RuntimeException, e.g. NullPointerException, ArithmeticException).
- String is immutable; StringBuffer (synchronised) and StringBuilder (faster) are mutable.

3.7.5 Graphics Traces

DDA: $(2, 3) \rightarrow (8, 6)$; $dx = 6$, $dy = 3$, steps = 6, $x\text{-inc} = 1$, $y\text{-inc} = 0.5$.

x	2	3	4	5	6	7	8
y (exact)	3	3.5	4	4.5	5	5.5	6
plotted pixel	(2,3)	(3,4)	(4,4)	(5,5)	(6,5)	(7,6)	(8,6)

Mid-point circle, $r = 10$ (first octant, start $(0, 10)$, $p_0 = 1 - r = -9$):

k	p_k	Next pixel	p_{k+1}
0	-9	(1, 10)	-6
1	-6	(2, 10)	-1
2	-1	(3, 10)	6
3	6	(4, 9)	-3
4	-3	(5, 9)	8
5	8	(6, 8)	5
6	5	(7, 7)	stop ($x \geq y$)

Composite transformation: rotate $P(4, 2)$ by 90° anticlockwise about pivot $(2, 2)$.

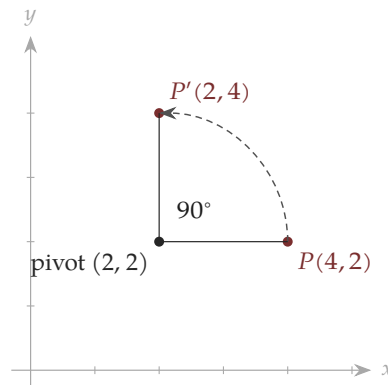


FIGURE 3.13. Rotation about a pivot: translate the pivot to the origin, rotate, translate back — $T(2, 2) R(90^\circ) T(-2, -2)$.

translate by $(-2, -2)$: $(4, 2) \mapsto (2, 0)$

rotate by 90° : $(x \cos 90^\circ - y \sin 90^\circ, x \sin 90^\circ + y \cos 90^\circ) = (0, 2)$

translate by $(+2, +2)$: $(0, 2) \mapsto (2, 4) = P'$

Scaling about fixed point (xf, yf): $T(xf, yf) \cdot S(sx, sy) \cdot T(-xf, -yf) \rightarrow x' = xf + (x - xf) \cdot sx$.

Cohen–Sutherland: window (0, 0)–(10, 10); line (–5, 5) $\rightarrow$ (15, 5). Codes 0001 and 0010; AND = 0000 $\rightarrow$ not trivially rejected; clip at $x = 0 \rightarrow (0, 5)$ and at $x = 10 \rightarrow (10, 5)$.

Liang–Barsky: same window; line (–5, 3) $\rightarrow$ (15, 9); $\Delta x = 20, \Delta y = 6$.

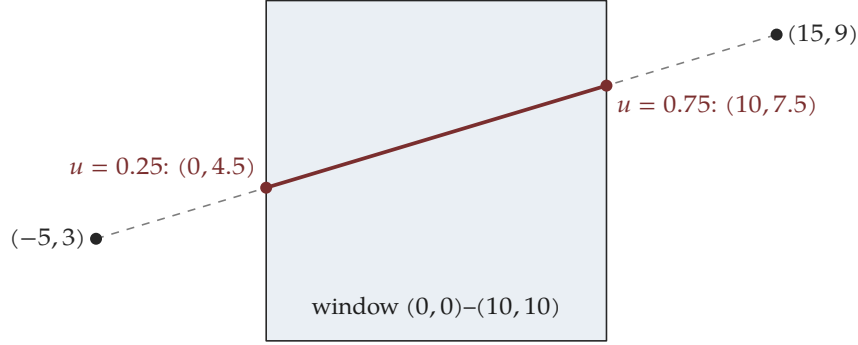


FIGURE 3.14. Liang–Barsky clipping of the line from (–5, 3) to (15, 9).

Left edge ($p_1 = -\Delta x$, entering)	$p_1 = -20, q_1 = x_1 - x_{\min} = -5 \Rightarrow r_1 = 0.25$
Right edge ($p_2 = \Delta x$, leaving)	$p_2 = 20, q_2 = x_{\max} - x_1 = 15 \Rightarrow r_2 = 0.75$
Bottom edge (entering)	$p_3 = -6, q_3 = 3 \Rightarrow r_3 = -0.5$
Top edge (leaving)	$p_4 = 6, q_4 = 7 \Rightarrow r_4 \approx 1.17$
Parameter range of the visible part	$u_1 = \max(0, 0.25, -0.5) = 0.25, \quad u_2 = \min(1, 0.75, 1.17) = 0.75$

3.7.6 3-D Transformation Matrices (homogeneous 4×4)

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation about the x -axis $y' = y \cos \theta - z \sin \theta, \quad z' = y \sin \theta + z \cos \theta$

Rotation about the y -axis $z' = z \cos \theta - x \sin \theta, \quad x' = z \sin \theta + x \cos \theta$

Perspective (centre at origin, view plane $z = d$) $x' = \frac{x d}{z}, \quad y' = \frac{y d}{z}$

3.8 Previous Year Questions (PYQ pattern)

Language concepts

1. First object-oriented language: **Simula 67**
2. Which uses call by name? **ALGOL 60**
3. Binding of a variable to its type in C occurs at: **compile time**
4. Dynamic scoping resolves a non-local name using: **the calling sequence (most recent active binding)**
5. Prolog is a: **logic programming language**
6. LISP is primarily used for: **list processing / AI (functional)**

C

7. `int a = 5; a = a++ + ++a;` → **undefined behaviour** (option often given as 12 or 13; correct per standard: UB)
8. Output of `printf("%d", sizeof('A'))` in C: **4** (char constant is int in C; 1 in C++)
9. `#define MUL(a,b) a*b; MUL(2+3, 4)` → **2+3*4 = 14**
10. Size of union `{int a; char b[10];}` (int=4): **12** with alignment (largest member 10, padded to multiple of 4)
11. Which storage class retains value between function calls? **static**
12. `char *p = "hello"; printf("%c", *(p+1));` → **e**
13. `int (*p)[5];` declares: **pointer to an array of 5 integers**
14. Which function allocates zero-initialised memory? **calloc**

OOP/C++

15. Run-time polymorphism is achieved via: **virtual functions**
16. A class with at least one pure virtual function is: **abstract class**
17. Which operator cannot be overloaded? **:: (scope resolution)** (also `., ?:, sizeof`)
18. The copy constructor takes argument by: **reference** (by value would cause infinite recursion)
19. Diamond problem is resolved by: **virtual base class**
20. Templates support: **generic programming (compile-time polymorphism)**
21. Order of destructor calls: **reverse of constructor calls**
22. Java does not support multiple inheritance of classes; achieved via: **interfaces**

Web

23. Applet life cycle method called first: **init()**
24. Servlet method handling each request: **service()**
25. XML document conforming to DTD is called: **valid**; following syntax rules: **well-formed**
26. DHTML combines: **HTML, CSS, JavaScript, DOM**

Graphics

27. Frame buffer for 640×480 with 8 bits per pixel: **300 KB** (307200 bytes)
28. Bresenham's line algorithm uses: **only integer arithmetic**
29. Initial decision parameter for mid-point circle with $r = 10$: **$1 - 10 = -9$**

30. Circle uses __-way symmetry, ellipse __-way: **8, 4**
31. Composite transformation for rotation about point P: $T(P) \cdot R(\theta) \cdot T(-P)$
32. Region code of a point above-left of window (TBRL): **1001**
33. Line endpoints codes 0101 and 0110: $AND = 0100 \neq 0 \rightarrow$ **trivially rejected**
34. Polygon clipping algorithm: **Sutherland–Hodgman**
35. Shading that interpolates normal vectors: **Phong**
36. Z-buffer algorithm is a: **image-space hidden surface method**
37. Bezier curve with 4 control points has degree: **3**
38. Which curve has local control? **B-spline**
39. Projection in which parallel lines converge: **Perspective**
40. Isometric projection is a type of: **axonometric orthographic projection**
41. Scaling matrix with $s_x = s_y = -1$ is equivalent to: **reflection about origin (rotation by 180°)**
42. Translating (2,3) by (4,-1): **(6,2)**

More practice questions

43. Under dynamic scoping, the code in §7.1 prints: **2**
44. Output of `printf("%d", ~0);` in C: **-1**
45. Output of `printf("%d", 10 >> 1 << 2);`: **(10 » 1) « 2 = 20**
46. In Java, a member visible only within its package has: **default (package-private) access**
47. A Java class that cannot be subclassed is declared: **final**
48. NullPointerException is a: **unchecked (runtime) exception**
49. Number of steps in DDA for line (1, 1) to (9, 4): **8**
50. Rotating point (1, 0) by 90° anticlockwise about origin gives: **(0, 1)**
51. Reflection of (3, -2) about the line $y = x$: **(-2, 3)**
52. In Liang–Barsky, $p_k < 0$ means the line is: **entering** the clip boundary
53. Shear in x with $sh_x = 2$ applied to (1, 3): **(7, 3)**
54. Homogeneous coordinates for 3-D points use: **4-element vectors / 4×4 matrices**

POINTS TO REMEMBER

- Simula 67 first OO · ALGOL call-by-name · C static scope
- `::`, `.`, `?:`, `sizeof` can't be overloaded · Pure virtual $\rightarrow$ abstract
- Bresenham $p_0 = 2dy - dx$ · Mid-point circle $p_0 = 1 - r$
- TBRL codes; $AND \neq 0$ reject; both 0 accept
- Gouraud = intensity interpolation · Phong = normal interpolation
- Bezier global control, passes endpoints · B-spline local control

Database Management Systems

...

*Expected questions: 10–12 · Target: 10+ · Type: normalisation, keys, SQL output, transactions, concepts
— very scoring*

SYLLABUS COVERAGE

- ☐ Database system concepts & architecture: data models, schemas, instances, three-schema architecture, data independence, database languages & interfaces, centralised & client/server architecture
 - ☐ Data modelling: ER diagram, relational model (constraints, languages, design, programming), relational algebra & calculus, codd rules
 - ☐ SQL: data definition, types, constraints, queries, insert, delete, update, views, stored procedures, functions, triggers, SQL injection
 - ☐ Normalisation: functional dependencies, 1NF, 2NF, 3NF, BCNF, multivalued dependencies & 4NF, join dependencies & 5NF, inclusion dependencies
 - ☐ Transaction processing, concurrency control & recovery
 - ☐ Physical database design: file organisation, indexing (single-level, multilevel, B-tree, B+-tree), query processing & optimisation
 - ☐ Object & object-relational databases; database security
 - ☐ Enhanced data models: temporal, multimedia, deductive, XML & internet databases, mobile, GIS, genome, distributed databases
 - ☐ Data warehousing & data mining: OLAP, data cube, association rules, classification, clustering
 - ☐ Big data systems: characteristics, types, architecture, MapReduce, Hadoop, NoSQL (CAP theorem, document, key-value, column, graph)
-

4.1 Architecture

A database management system stands between users and stored data. Its central achievement is *data independence*: applications describe what data they want, and the system decides how it is stored and found, so that storage can change without rewriting programs.

4.1.1 Three-Schema (ANSI/SPARC) Architecture

- **Logical data independence**: change conceptual schema without changing external views (harder to achieve).
- **Physical data independence**: change internal schema without changing conceptual schema.
- **Schema** = structure (intension, changes rarely); **Instance** = data at a moment (extension).
- Languages: **DDL** (CREATE, ALTER, DROP, TRUNCATE), **DML** (SELECT, INSERT, UPDATE, DELETE), **DCL** (GRANT, REVOKE), **TCL** (COMMIT, ROLLBACK, SAVEPOINT).

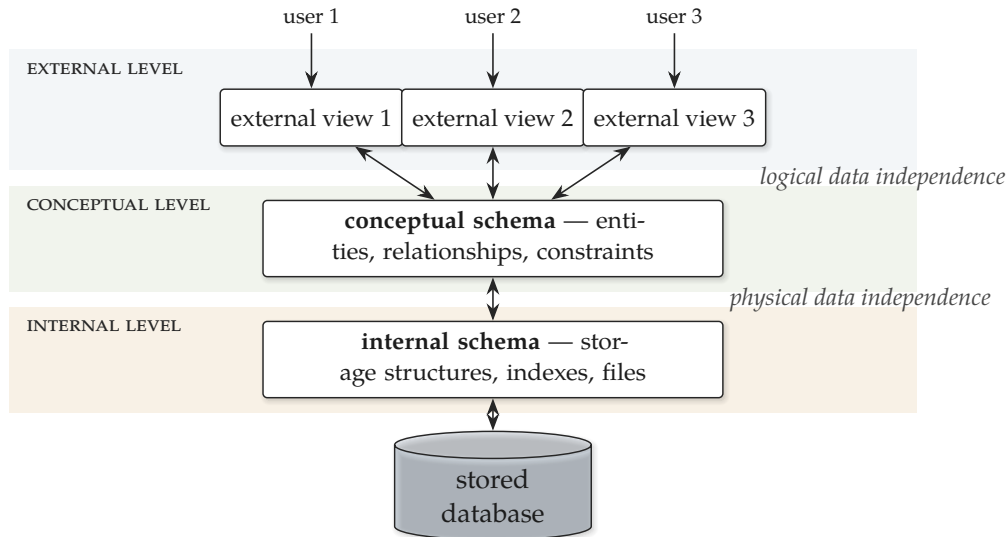


FIGURE 4.1. The ANSI/SPARC three-schema architecture. Mappings between adjacent levels insulate each level from changes in the one below it.

- Data models: hierarchical (IMS, tree), network (CODASYL, graph), relational (Codd 1970), object-oriented, object-relational.
- Architectures: centralised, **2-tier** client/server, **3-tier** (client – application server – DB server).

4.1.2 DBMS Components

Query processor (DDL interpreter, DML compiler, query evaluation engine), storage manager (authorisation, transaction manager, file manager, buffer manager), data dictionary (metadata).

4.2 ER Model

The entity–relationship model is a picture of the real world before any tables exist: things (entities), their properties (attributes) and the associations between them (relationships). Chen's notation, shown below, is the one used in the examination.

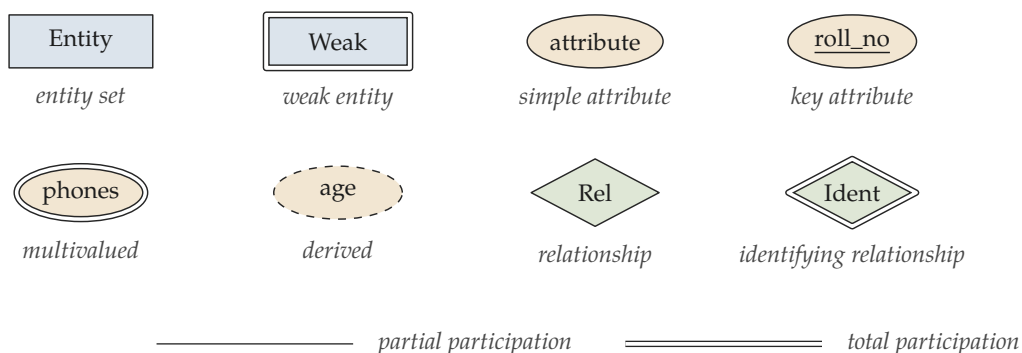


FIGURE 4.2. Chen's ER notation.

- **Cardinality:** 1:1, 1:N, M:N. **Participation:** total (every entity participates — double line) or partial.
- **Weak entity:** no key of its own; identified via owner entity + partial key (discriminator).

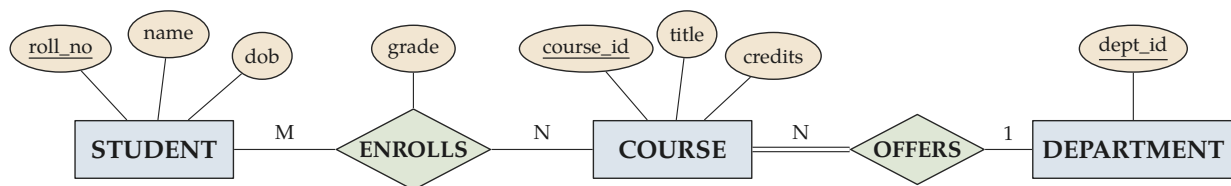


FIGURE 4.3. An ER diagram for a university. ENROLLS is many-to-many and carries the attribute *grade*; every course must be offered by a department (total participation, double line).

- **Specialisation** (top-down), **generalisation** (bottom-up), **aggregation** (relationship as entity). Disjoint/overlapping, total/partial constraints.

4.2.1 ER → Relational mapping (minimum tables)

Case	Tables
Strong entity	1 table each
1:1 relationship	Merge into either side (total participation side preferred)
1:N relationship	FK on N-side; no separate table
M:N relationship	Separate table with both PKs
Multivalued attribute	Separate table
Weak entity	Table with owner's PK + partial key

Classic: $E1 \text{ ---} M:N \text{ ---} E2 \text{ ---} 1:N \text{ ---} E3 \rightarrow \text{minimum tables} = 3 \text{ entities} + 1 \text{ (for } M:N) = 4$.

4.3 Keys & Integrity

Keys are how a relational database identifies rows and links tables. The three key concepts are nested, as the figure shows; integrity constraints then forbid NULL primary keys and dangling foreign keys.

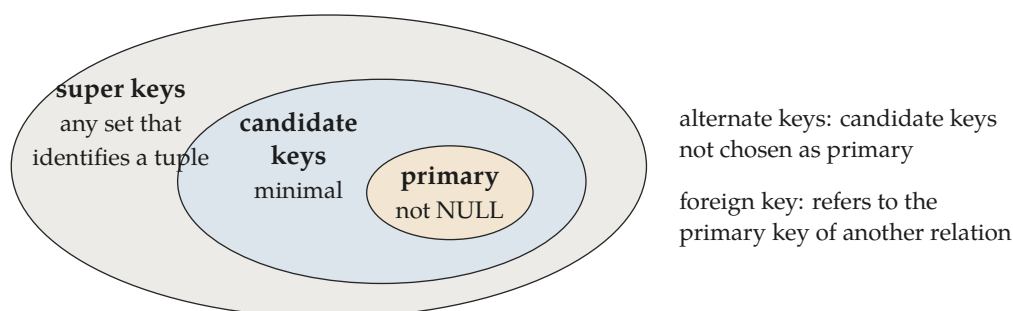


FIGURE 4.4. Every primary key is a candidate key, and every candidate key is a (minimal) super key.

- Number of super keys of $R(A_1 \dots A_n)$ with single candidate key A_1 : 2^{n-1} .
- With candidate keys A_1 and A_2 : $2^{n-1} + 2^{n-1} - 2^{n-2} = 3 \cdot 2^{n-2}$.
- **Entity integrity**: PK cannot be NULL. **Referential integrity**: FK value must match an existing PK or be NULL.

– ON DELETE CASCADE / SET NULL / RESTRICT.

Codd's 12 rules (13 with Rule 0): Rule 0 foundation; 1 information; 2 guaranteed access; 3 systematic NULLs; 4 active online catalog; 5 comprehensive data sublanguage; 6 view updating; 7 high-level insert/update/delete; 8 physical data independence; 9 logical data independence; 10 integrity independence; 11 distribution independence; 12 non-subversion.

4.4 Relational Algebra & Calculus

Relational algebra is a small set of operations on tables that produce tables. Every SQL query is translated into an algebra expression before it is optimised, which is why the algebra is examined so often.

Operation	Symbol	Notes
Selection	$\sigma_{\text{cond}}(R)$	Rows; commutative
Projection	$\pi_{\text{attrs}}(R)$	Columns; removes duplicates
Union	$R \cup S$	Union-compatible
Difference	$R - S$	
Cartesian product	$R \times S$	
Rename	ρ	
Intersection	$R \cap S = R - (R - S)$	Derived
Natural join	$R \bowtie S$	Equal common attrs
Theta join	$R \bowtie_{\theta} S$	$\sigma_{\theta}(R \times S)$
Division	$R \div S$	"for all" queries
Outer joins	$\bowtie \ltimes \bowtie$	Keep unmatched with NULLs

Fundamental: $\sigma, \pi, \cup, -, \times, \rho$. Relational algebra is **procedural**; relational calculus (TRC/DRC) is **non-procedural**. Safe calculus = relational algebra in power (relational completeness).

Operation ($ R = m, S = n$)	Number of tuples
$R \times S$	$m n$
$R \bowtie S$	$0 \dots m n$
$R \cup S$	$\max(m, n) \dots m + n$
$R \cap S$	$0 \dots \min(m, n)$
$R - S$	$0 \dots m$
R left outer join S	at least m

Division example: "students who took **all** courses": $\pi_{\{\text{sid}, \text{cid}\}}(\text{Enrolled}) \div \pi_{\{\text{cid}\}}(\text{Course})$.

TRC: $\{ t \mid t \in \text{Student} \wedge t.\text{age} > 20 \} \cdot \text{DRC: } \{ \langle n \rangle \mid \exists a (\langle n, a \rangle \in \text{Student} \wedge a > 20) \}$.

4.5 SQL

```
SELECT dept, COUNT(*) AS n, AVG(salary)
FROM   employee
WHERE  salary > 20000          -- filters rows (before grouping)
GROUP BY dept
HAVING COUNT(*) > 2          -- filters groups
ORDER BY n DESC;
```

Logical order of execution: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **DISTINCT** → **ORDER BY** → **LIMIT**.

- **COUNT(*)** counts all rows; **COUNT(col)** ignores NULLs; aggregates (SUM/AVG/MAX/MIN) ignore NULLs.
- NULL comparisons give UNKNOWN; use **IS NULL**. **NULL = NULL** → UNKNOWN.
- **DELETE** (DML, row-wise, rollback possible, WHERE allowed) vs **TRUNCATE** (DDL, all rows, faster) vs **DROP** (removes table structure).
- **IN**, **EXISTS**, **ANY/SOME**, **ALL**: $x > \text{ALL (subquery)}$ = greater than max; $x > \text{ANY}$ = greater than min.
- **UNION** removes duplicates; **UNION ALL** keeps them.
- Correlated subquery: inner query refers to outer — evaluated per outer row.
- Constraints: **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE** (allows NULL), **NOT NULL**, **CHECK**, **DEFAULT**.
- **View**: virtual table (stored query); updatable if based on single table without aggregates/DISTINCT/GROUP BY. **Materialised view** stores results.
- **Trigger**: ECA (Event–Condition–Action) rule; BEFORE/AFTER, row-level/statement-level.
- **Stored procedure** (CALL, may not return value) vs **function** (returns value, usable in SELECT).
- **SQL injection**: attacker inserts SQL via input, e.g. ' OR '1'='1. Prevent with **prepared statements/parameterised queries**, input validation, least privilege.

Nth highest salary:

```
SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp); -- 2nd highest
```

4.6 Functional Dependencies & Normalisation

A functional dependency $X \rightarrow Y$ says that the value of X determines the value of Y . Redundancy arises when a table stores such facts about only part of its key, or about non-key attributes; normalisation removes it by splitting the table so that every fact is stored once.

4.6.1 Armstrong's Axioms

- **Reflexivity**: $Y \subseteq X \Rightarrow X \rightarrow Y$ · **Augmentation**: $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$ · **Transitivity**: $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$
- Derived: Union, Decomposition, Pseudo-transitivity ($X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$).
- Sound and complete.

4.6.2 Attribute Closure — Finding Candidate Keys

$R(A, B, C, D, E), F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow E\}$

$$A^+ = \{A, B, C\}$$
$$(AD)^+ = \{A, D, B, C, E\} = \text{all attributes} \Rightarrow AD \text{ is a key}$$

Since A and D never appear on a right-hand side, every key must contain both; AD is therefore the only candidate key.

Trick: attributes not on any RHS must be in every key; attributes only on RHS are never in a key.

Minimal (canonical) cover: single attribute on RHS $\rightarrow$ remove extraneous LHS attributes $\rightarrow$ remove redundant FDs.

4.6.3 Normal Forms

Each normal form forbids one more kind of undesirable dependency. The nested diagram is a reminder that higher normal forms imply all the lower ones.

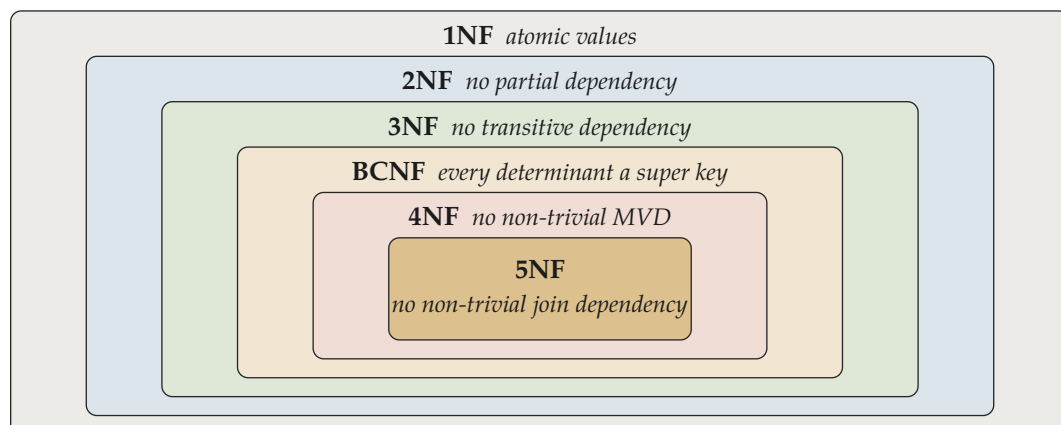


FIGURE 4.5. Each normal form is stricter than the one enclosing it: a relation in BCNF is also in 3NF, 2NF and 1NF.

NF	Condition (for every non-trivial FD $X \rightarrow A$)
1NF	All attributes atomic (no repeating groups / multivalued)
2NF	1NF + no partial dependency (non-prime attribute depends on part of a candidate key)
3NF	X is a super key OR A is a prime attribute
BCNF	X is a super key
4NF	For every non-trivial MVD $X \twoheadrightarrow Y$, X is a super key
5NF	Every join dependency is implied by candidate keys

- Relation with **only two attributes** is **always in BCNF**.
- If all candidate keys are single attributes $\Rightarrow$ automatically 2NF.
- If all attributes are prime $\Rightarrow$ automatically 3NF.

4.6.4 Decomposition Properties

Property	Test
Lossless join (must)	R into R1, R2 is lossless iff $(R1 \cap R2) \rightarrow R1$ or $(R1 \cap R2) \rightarrow R2$
Dependency preserving (desirable)	$(F1 \cup F2)^+ = F^+$

- 3NF decomposition: always lossless **and** dependency-preserving (synthesis algorithm).
- BCNF decomposition: always lossless but **not always** dependency-preserving.

Worked: $R(A,B,C)$, $F = \{AB \rightarrow C, C \rightarrow B\}$. Keys: AB, AC. $C \rightarrow B$: C not super key but B is prime $\rightarrow$ 3NF, **not** BCNF. BCNF decomposition (C,B) , (A,C) loses $AB \rightarrow C$.

4.7 Transactions

A transaction is a unit of work — such as a fund transfer — that must happen completely or not at all, even when many transactions run at once and the system may crash. The ACID properties state these guarantees.

4.7.1 ACID

Property	Ensured by
Atomicity (all or nothing)	Recovery manager (undo log)
Consistency	Programmer + integrity constraints
Isolation	Concurrency control manager
Durability	Recovery manager (redo log)

4.7.2 Transaction States

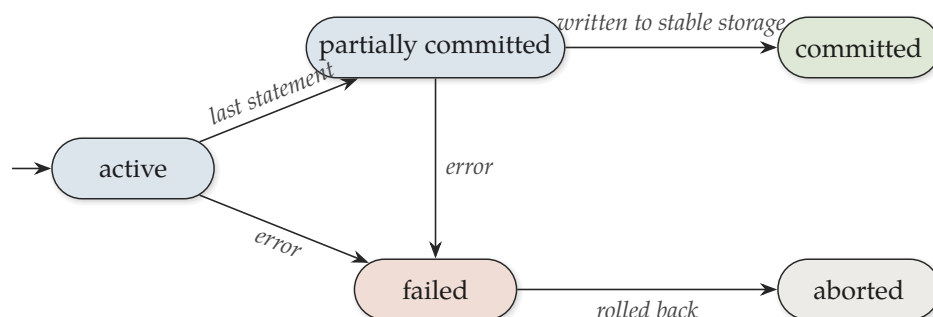


FIGURE 4.6. The states of a transaction.

4.7.3 Schedules & Serializability

When transactions interleave, the result is correct if it is equivalent to some serial order. Conflict serializability is tested by drawing a precedence graph and looking for a cycle.

- **Conflicting operations:** same data item, different transactions, at least one write (RW, WR, WW).
- **Conflict serializable** $\Leftrightarrow$ **precedence graph is acyclic** (edge $T_i \rightarrow T_j$ if T_i 's op conflicts with and precedes T_j 's).
- **View serializable:** same initial reads, same reads-from, same final writes. Every conflict-serializable schedule is view serializable; a view-serializable schedule that is not conflict-serializable has **blind writes**.
- Number of serial schedules of n transactions: $n!$.

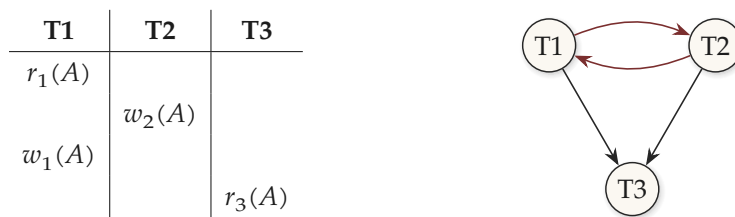


FIGURE 4.7. A schedule and its precedence graph. The cycle $T1 \leftrightarrow T2$ (from $r_1(A)$ before $w_2(A)$ and $w_2(A)$ before $w_1(A)$) means the schedule is not conflict serializable.

4.7.4 Recoverability hierarchy

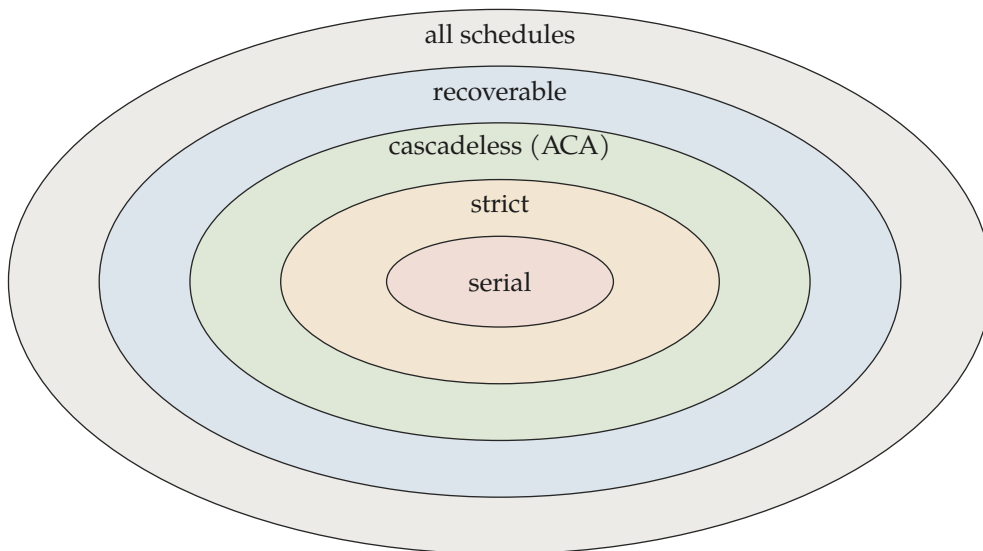


FIGURE 4.8. Classes of schedules: $\text{serial} \subset \text{strict} \subset \text{cascadeless} \subset \text{recoverable}$.

- **Recoverable:** if T_j reads from T_i , T_i commits before T_j commits.
- **Cascadeless:** read only committed data.
- **Strict:** no read/write of item until the last writer commits/aborts.

4.7.5 Concurrency problems

Lost update (WW), dirty read (WR — uncommitted dependency), unrepeatable read (RW), phantom read (insertions). SQL isolation levels: READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ < SERIALIZABLE.

4.7.6 Concurrency Control Protocols

Protocol	Key points
Lock-based (S/X)	S compatible with S only
Basic 2PL	Growing phase then shrinking phase; ensures conflict serializability; deadlock possible , cascading rollback possible
Strict 2PL	Hold X locks till commit → cascadeless, strict
Rigorous 2PL	Hold all locks till commit; serial order = commit order
Conservative 2PL	Acquire all locks before starting → deadlock-free
Timestamp ordering	Older first; Read: if $TS(T) < W-TS(X)$ → rollback; Write: if $TS(T) < R-TS(X)$ or $< W-TS(X)$ → rollback. Deadlock-free, may starve
Thomas' write rule	Ignore obsolete write instead of rollback → allows some view-serializable schedules
Validation (optimistic)	Read → validate → write phases
Multiversion (MVCC)	Readers don't block writers
Multiple granularity	Intention locks IS, IX, SIX

Deadlock prevention with timestamps:

Scheme	Older asks for younger's lock	Younger asks for older's lock
Wait-die (non-pre-emptive)	older <i>waits</i>	younger <i>dies</i> (rolled back)
Wound-wait (pre-emptive)	older <i>wounds</i> (aborts) younger	younger <i>waits</i>

4.7.7 Recovery

- **Log-based**: $\langle T, \text{start} \rangle, \langle T, X, \text{old}, \text{new} \rangle, \langle T, \text{commit} \rangle$.
- **Deferred update** (NO-UNDO/REDO), **Immediate update** (UNDO/REDO).
- **Checkpoint** reduces log scanning. After crash: transactions committed after checkpoint → REDO; uncommitted → UNDO.
- **ARIES** (WAL, LSN, repeating history): Analysis → Redo → Undo.
- **WAL (Write-Ahead Logging)**: log record written to stable storage before data page.
- Shadow paging: no log needed; copy-on-write page table.

4.8 File Organisation & Indexing

An index is to a table what the index of a book is to its pages: a small, ordered structure that leads straight to the wanted rows. B⁺-trees dominate because they stay balanced and short even for very large files.

Index	Description
Primary	On ordering key field; sparse (one entry per block)
Clustering	On ordering non-key field
Secondary	On non-ordering field; dense
Dense	Entry for every record
Sparse	Entry for some records (needs sorted file)
Multilevel	Index on index — $\log_{fo}(\text{blocks})$ levels

- A file can have **at most one primary or clustering index** but many secondary indexes.

4.8.1 B-Tree vs B+ Tree

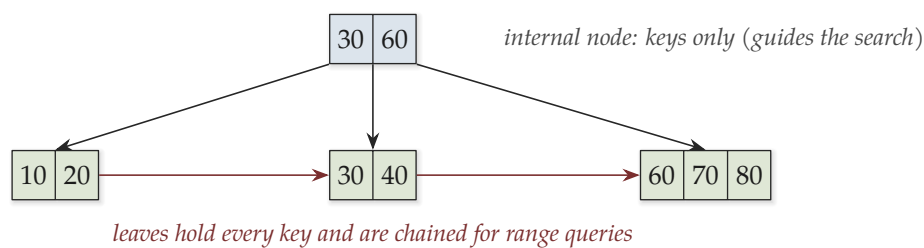


FIGURE 4.9. A B⁺-tree of order 3. Unlike a B-tree, internal keys are repeated in the leaves, and the leaf chain supports sequential access.

B-Tree	B+ Tree
Data pointers in all nodes	Data pointers only at leaves
No duplicate keys	Internal keys repeated in leaves
Leaves not linked	Leaves linked → efficient range/sequential access
Lower fan-out	Higher fan-out (internal nodes smaller) → fewer levels

Order p (max children): every node $\leq p$ children; non-root internal $\geq \lceil p/2 \rceil$ children; root ≥ 2 .

Order calculation: block 1024 B, key 9 B, block pointer 6 B, record pointer 7 B.

- B+ internal: $p \cdot 6 + (p-1) \cdot 9 \leq 1024 \rightarrow 15p \leq 1033 \rightarrow p = 68$.
- B+ leaf: $q(9 + 7) + 6 \leq 1024 \rightarrow q = 63$.
- B-tree node: $p \cdot 6 + (p-1)(9 + 7) \leq 1024 \rightarrow 22p \leq 1040 \rightarrow p = 47$.

Hashing: static (overflow chains), dynamic — extendible hashing (directory doubles, global/local depth), linear hashing.

4.8.2 Query Optimisation

- Heuristics: perform **selection early**, projection early, most restrictive selection first, avoid Cartesian products, replace $\times + \sigma$ with joins.
- Cost-based: estimate block transfers & seeks. Join algorithms: nested-loop ($nr \cdot bs + br$), block nested-loop ($br \cdot bs + br$), indexed, sort-merge, hash join.
- Query tree; equivalence rules (σ commutes, cascade of π , joins associative/commutative).

4.9 Advanced & Enhanced Databases

Model	Notes
OODBMS	Objects, OID, inheritance; ODMG standard, OQL
ORDBMS	Relational + UDTs, inheritance (PostgreSQL, Oracle)
Temporal	Valid time & transaction time
Multimedia	Images, audio, video; content-based retrieval
Deductive	Facts + rules; Datalog (Prolog-like)
XML DB	XPath, XQuery; native XML DBs
Mobile	Intermittent connectivity, caching
GIS	Spatial data: raster/vector; R-trees, quad-trees
Genome	Biological sequence data
Distributed DB	Fragmentation (horizontal, vertical, mixed), replication, transparency; 2PC (two-phase commit: prepare $\rightarrow$ commit) — blocking; 3PC non-blocking

Security: discretionary (GRANT/REVOKE, WITH GRANT OPTION), mandatory (Bell-LaPadula: no read up, no write down), role-based access control, statistical DB inference control, encryption.

4.10 Data Warehousing & Mining

Operational databases record today's transactions; a data warehouse collects years of history from many sources for analysis. Data mining then searches that history for patterns.

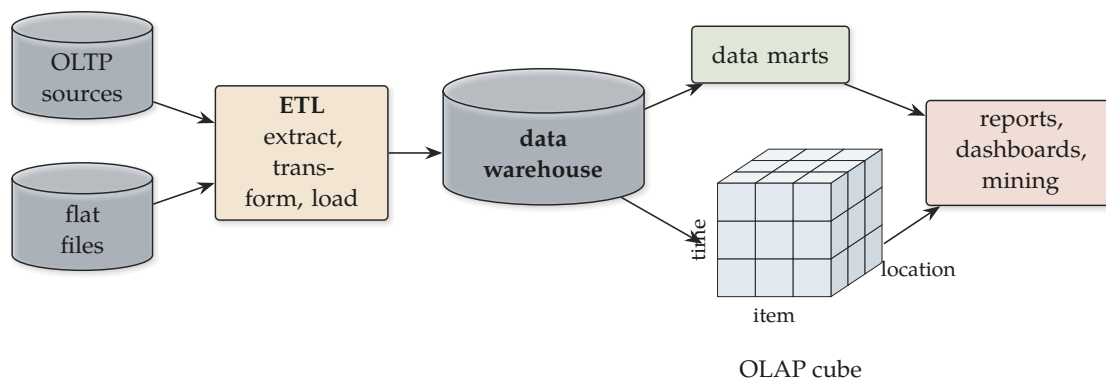


FIGURE 4.10. Data-warehouse architecture. The warehouse is subject-oriented, integrated, time-variant and non-volatile (Inmon); OLAP views it as a multidimensional cube that can be sliced, diced, rolled up and drilled down.

- Definition by **W.H. Inmon**: subject-oriented, integrated, time-variant, non-volatile. Kimball: dimensional modelling (bottom-up data marts).
- Schemas: **Star** (one fact table + denormalised dimensions), **Snowflake** (normalised dimensions), **Fact constellation** / **Galaxy** (multiple fact tables).
- OLAP operations: **roll-up** (aggregate up / drill-up), **drill-down**, **slice** (fix one dimension), **dice** (sub-cube on multiple), **pivot** (rotate).
- ROLAP (relational), MOLAP (multidimensional arrays), HOLAP (hybrid).
- Number of cuboids in n-dimensional cube without hierarchies: 2^n ; with L_i levels: $\prod(L_i + 1)$.
- OLTP (current, detailed, many short transactions, normalised) vs OLAP (historical, summarised, complex queries, denormalised).

4.10.1 Data mining (KDD process)

Selection → Pre-processing → Transformation → **Data mining** → Interpretation/Evaluation.

Association rules (Apriori):

Support of $A \Rightarrow B$	$\frac{\text{count}(A \cup B)}{N}$	
Confidence of $A \Rightarrow B$	$\frac{\text{support}(A \cup B)}{\text{support}(A)}$	
Lift	$\frac{\text{confidence}(A \Rightarrow B)}{\text{support}(B)}$	(> 1: positive correlation)
Apriori property	every subset of a frequent itemset is frequent	
FP-growth	mines an FP-tree without generating candidates	

- Classification (supervised): decision tree (ID3 – information gain, C4.5 – gain ratio, CART – Gini), Naive Bayes, k-NN, SVM.
- Clustering (unsupervised): k-means (partitioning), k-medoids (PAM), hierarchical (agglomerative/divisive), DBSCAN (density).
- Outlier detection, regression.

4.11 Big Data & NoSQL

When data become too large, too fast or too varied for a single relational server, systems distribute them across many machines and often relax strict consistency in exchange for availability.

- **5 Vs**: Volume, Velocity, Variety, Veracity, Value (3 Vs originally — Doug Laney).
- Types: structured, semi-structured (JSON, XML), unstructured (text, video).
- **Hadoop**: HDFS (NameNode = master/metadata, DataNode = storage; default block 128 MB, replication 3) + **MapReduce** (Map → Shuffle & Sort → Reduce) + **YARN** (ResourceManager, NodeManager). Ecosystem: Hive (SQL-like), Pig (Pig Latin), HBase (column store), Sqoop (RDBMS ↔ HDFS), Flume (logs), ZooKeeper (coordination), Oozie (workflow), Spark (in-memory, RDDs).

- **CAP theorem** (Brewer): a distributed system can guarantee only **2 of 3**: Consistency, Availability, Partition tolerance. (CP: HBase, MongoDB; AP: Cassandra, CouchDB, DynamoDB.)
- **BASE**: Basically Available, Soft state, Eventual consistency (vs ACID).

NoSQL type	Examples
Key-value	Redis, DynamoDB, Riak
Document	MongoDB, CouchDB
Column-family	Cassandra, HBase, Bigtable
Graph	Neo4j, JanusGraph

4.12 Deeper Dive — SQL on Real Tables, Minimal Cover, Full Normalisation & Schedules

4.12.1 SQL Worked on Sample Data

EMP

eid	name	dept	salary
1	Asha	CS	50000
2	Ravi	CS	40000
3	Meena	EE	45000
4	John	EE	30000
5	Priya	ME	60000
6	Kiran	NULL	35000

DEPT

dept	building
CS	B1
EE	B2
CE	B3

#	Query	Result
1	SELECT dept, COUNT(*), AVG(salary) FROM EMP GROUP BY dept HAVING COUNT(*) > 1	(CS, 2, 45000), (EE, 2, 37500)
2	SELECT name FROM EMP WHERE salary > (SELECT AVG(salary) FROM EMP)	avg = 43333.33 → Asha, Meena, Priya
3	SELECT COUNT(dept), COUNT(DISTINCT dept) FROM EMP	5, 3 (NULL ignored)
4	EMP JOIN DEPT ON EMP.dept = DEPT.dept	4 rows (Asha, Ravi, Meena, John)
5	EMP LEFT JOIN DEPT ...	6 rows (Priya & Kiran get NULL building)
6	EMP RIGHT JOIN DEPT ...	5 rows (CE appears with NULL employee)
7	EMP FULL OUTER JOIN DEPT ...	7 rows
8	Correlated: SELECT name FROM EMP e1 WHERE salary = (SELECT MAX(salary) FROM EMP e2 WHERE e2.dept = e1.dept)	Asha, Meena, Priya (Kiran excluded: NULL = NULL is UNKNOWN)
9	SELECT dept FROM DEPT WHERE dept NOT IN (SELECT dept FROM EMP)	Empty! The subquery contains NULL, so every NOT IN test is UNKNOWN
10	Same with NOT EXISTS (SELECT * FROM EMP WHERE EMP.dept = DEPT.dept)	CE ✓ — NOT EXISTS is NULL-safe

4.12.2 Minimal (Canonical) Cover — Worked

$F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

Step	Result
1. Split right-hand sides	$A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C$
2. Remove duplicates	$A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C$
3. Remove extraneous LHS attributes	in $AB \rightarrow C$, B is extraneous since $A^+ \ni C$; gives $A \rightarrow C$ (duplicate)
4. Remove redundant FDs	$A \rightarrow C$ follows from $A \rightarrow B$ and $B \rightarrow C$
Minimal cover	$F_c = \{A \rightarrow B, B \rightarrow C\}$

4.12.3 Normalisation — From 1NF to BCNF

$R(S, C, I, P, G)$ — Student, Course, Instructor, instructor Phone, Grade. $F = \{SC \rightarrow G, C \rightarrow I, I \rightarrow P\}$. S and C never appear on the right $\rightarrow$ **candidate key** = SC .

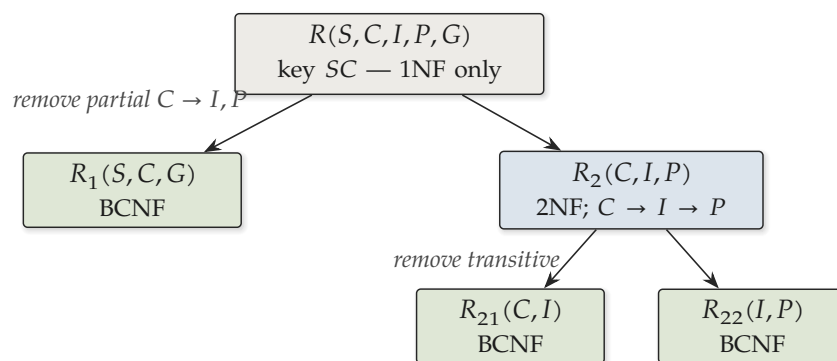


FIGURE 4.11. Decomposing $R(S, C, I, P, G)$ with $SC \rightarrow G, C \rightarrow I, I \rightarrow P$. The final schemas are lossless and preserve every dependency.

Final schema: $(S, C, G), (C, I), (I, P)$ — every determinant is a key $\rightarrow$ BCNF; decomposition is lossless (common attributes C and I are keys of a side) and preserves all three FDs.

4.12.4 Recoverability Examples ($W = \text{write}, R = \text{read}, C = \text{commit}$)

Schedule	Classification
W1(A) R2(A) C2 C1	Not recoverable — T2 commits after reading T1's uncommitted data, before T1 commits
W1(A) R2(A) C1 C2	Recoverable, but not cascadeless (abort of T1 forces abort of T2)
W1(A) C1 R2(A) C2	Cascadeless (reads only committed data)
W1(A) W2(A) C1 C2	Cascadeless but not strict (T2 overwrote uncommitted A)

4.12.5 Precedence Graphs

Schedule S: R1(A) W1(A) R2(A) W2(A) R1(B) W1(B) R2(B) W2(B)

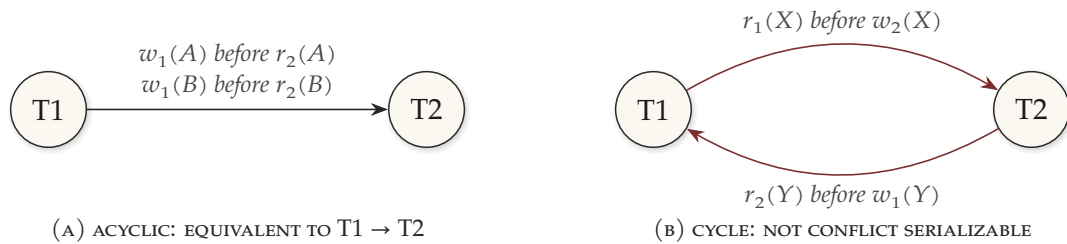


FIGURE 4.12. Precedence graphs for the two schedules S and S' below.

Acyclic $\rightarrow$ conflict serializable, equivalent to serial order **T1 $\rightarrow$ T2**.

Schedule S': R1(X) R2(Y) W2(X) W1(Y)

Cycle $\rightarrow$ **not** conflict serializable.

4.12.6 Two-Phase Locking — Lock Point

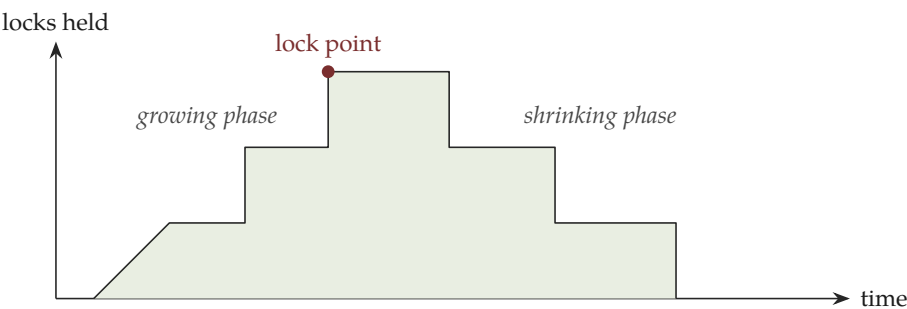


FIGURE 4.13. Two-phase locking: a transaction acquires all its locks before releasing any. Serializability order follows the order of the lock points.

4.12.7 Extendible Hashing (sketch)

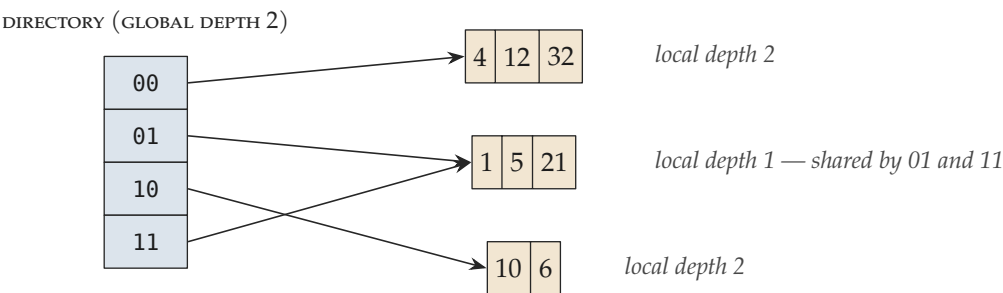


FIGURE 4.14. Extendible hashing. An overflowing bucket whose local depth equals the global depth forces the directory to double; otherwise only the bucket splits.

4.12.8 Data Cube Lattice ($3 \text{ dimensions} \rightarrow 2^3 = 8 \text{ cuboids}$)

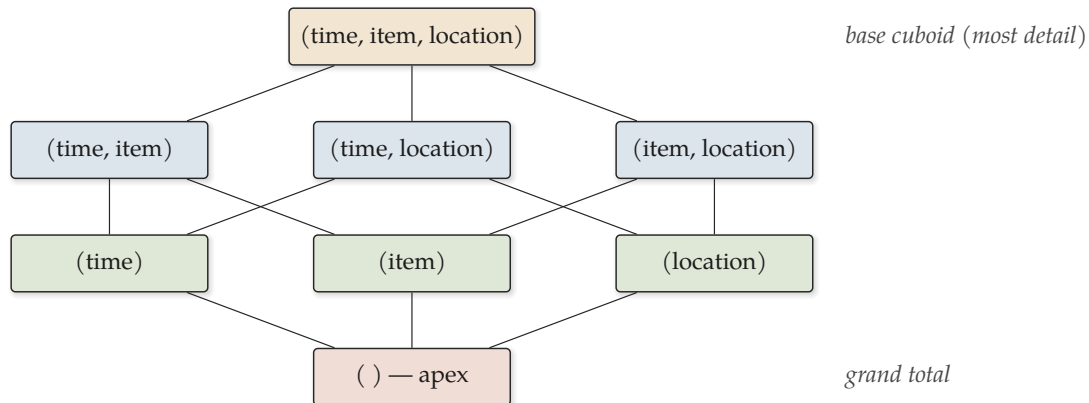


FIGURE 4.15. The lattice of $2^3 = 8$ cuboids for three dimensions. Rolling up moves down the lattice; drilling down moves up.

4.13 Previous Year Questions (PYQ pattern)

Architecture & ER

1. Ability to change the conceptual schema without changing application programs: **logical data independence**
2. The overall design of the database is called: **schema**
3. A weak entity set is represented by: **double rectangle**
4. Minimum number of tables for E1 (1)–R–(N) E2 with no attributes on R: **2**
5. Minimum tables for M:N relationship between two entities: **3**
6. Which is a DCL command? **GRANT**
7. TRUNCATE is a: **DDL command**

Keys

8. R(A,B,C,D) with only candidate key A — number of super keys: $2^3 = 8$
9. R(A,B,C,D) with candidate keys A and B: $8 + 8 - 4 = 12$
10. Which key cannot be NULL? **Primary key**

Relational algebra / SQL

11. R has 10 tuples, S has 5; $R \times S$ has: **50**
12. Which RA operation answers "for all" queries? **Division**
13. Which is not a fundamental RA operation? (a) σ (b) π (c) $\cap$ (d) — **Ans: (c)**
14. SELECT COUNT(*) vs COUNT(salary) on a table with 10 rows and 2 NULL salaries: **10 and 8**
15. HAVING is applied: **after GROUP BY on groups**
16. salary > ALL (SELECT salary FROM emp WHERE dept = 5) means: **greater than the maximum salary of dept 5**
17. Which set operator retains duplicates? **UNION ALL**
18. A trigger follows which model? **Event–Condition–Action**
19. Best protection against SQL injection: **parameterised/prepared statements**

Normalisation

- 20. $R(A,B,C,D)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$. Candidate key? **A**; Normal form? **2NF** (transitive dependencies exist)
- 21. $R(A,B,C,D)$, $F = \{AB \rightarrow C, C \rightarrow D\}$. Key **AB**; $C \rightarrow D$ is transitive $\rightarrow$ **2NF**, not 3NF
- 22. $R(A,B,C)$, $F = \{AB \rightarrow C, C \rightarrow A\}$. Keys **AB, CB**. Highest NF: **3NF** ($C \rightarrow A$, A is prime)
- 23. Every relation with two attributes is in: **BCNF**
- 24. Decomposition of $R(A,B,C)$ with $A \rightarrow B$ into $(A,B), (A,C)$ is: **lossless** (common $A \rightarrow AB$)
- 25. Which NF deals with multivalued dependency? **4NF**; join dependency? **5NF**
- 26. BCNF decomposition always guarantees: **lossless join** (not dependency preservation)
- 27. Closure of $\{A\}$ under $\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow E\}$: **ABCDE**

Transactions

- 28. Which ACID property is ensured by the concurrency control manager? **Isolation**
- 29. A schedule is conflict serializable iff its precedence graph is: **acyclic**
- 30. 2PL ensures: **conflict serializability** (not freedom from deadlock)
- 31. Strict 2PL avoids: **cascading rollbacks**
- 32. In wait-die, if an older transaction requests a lock held by a younger: **older waits**
- 33. In wound-wait, if an older transaction requests: **younger is aborted (wounded)**
- 34. Thomas' write rule modifies: **timestamp ordering protocol**
- 35. Number of serial schedules for 4 transactions: **24**
- 36. Reading uncommitted data is: **dirty read**
- 37. Write-ahead logging ensures: **log record on stable storage before data item**
- 38. After a crash, a transaction that has <start> and <commit> in log is: **redone**

Indexing

- 39. B+ tree leaves are linked to support: **range queries / sequential access**
- 40. Max number of primary indexes on a file: **1**
- 41. Order of B+ tree internal node: block 512 B, key 10 B, pointer 6 B: $p \cdot 6 + (p-1) \cdot 10 \leq 512 \rightarrow 16p \leq 522 \rightarrow p = 32$
- 42. Secondary index is usually: **dense**

Warehousing / mining / big data

- 43. Data warehouse is NOT: (a) subject-oriented (b) volatile (c) time-variant (d) integrated — **Ans: (b)**
- 44. OLAP operation that rotates the axes: **pivot**
- 45. Star schema has: **one fact table, denormalised dimension tables**
- 46. Support of {bread, butter} in 5 of 20 transactions: **25%**; if bread in 10 $\rightarrow$ confidence(bread $\rightarrow$ butter) = **50%**
- 47. Apriori uses the property: **all subsets of a frequent itemset are frequent**
- 48. CAP theorem: choose **2 of Consistency, Availability, Partition tolerance**
- 49. MongoDB is a: **document store**; Cassandra: **column-family**; Neo4j: **graph**
- 50. In Hadoop, metadata is stored by: **NameNode**
- 51. K-means is a: **partitioning clustering** algorithm

More practice questions

52. A NOT IN subquery whose result contains NULL returns: **no rows**
53. COUNT(DISTINCT dept) on values {CS, CS, EE, NULL}: **2**
54. Minimal cover of $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$: **$\{A \rightarrow B, B \rightarrow C\}$**
55. $R(A, B, C, D)$, $F = \{A \rightarrow B, C \rightarrow D\}$. Candidate key: **AC**; highest normal form: **1NF** (partial dependencies)
56. A schedule in which transactions read only committed data is: **cascadeless**
57. Schedule $R1(X) R2(Y) W2(X) W1(Y)$ is: **not conflict serializable**
58. In 2PL, the equivalent serial order follows the: **lock points**
59. In extendible hashing, the directory doubles when an overflowing bucket's local depth: **equals the global depth**
60. Number of cuboids for 4 dimensions without hierarchies: **16**
61. Apex cuboid represents: **the grand total (all dimensions aggregated)**
62. Full outer join of R (5 tuples) and S (3 tuples) with 2 matching pairs (one-to-one): $2 + 3 + 1 = 6$ **tuples**

POINTS TO REMEMBER

- 3NF: X superkey OR A prime · BCNF: X superkey
- Lossless: $(R1 \cap R2) \rightarrow R1$ or $R2$ · 3NF = lossless + dep. preserving
- Conflict serializable $\Leftrightarrow$ acyclic precedence graph · 2PL $\rightarrow$ serializable but deadlock possible
- Wait-die: old waits, young dies · Wound-wait: old wounds, young waits
- CAP: pick 2 · HDFS NameNode/DataNode · MapReduce Map $\rightarrow$ Shuffle $\rightarrow$ Reduce

System Software & Operating Systems

— . . . —

Expected questions: 10–12 · Target: 10+ · Type: scheduling/paging/disk numericals + Galvin concepts — highly scoring

SYLLABUS COVERAGE

- ☐ System software: assembly language, assemblers, compilers vs interpreters, loading, linking, relocation, macros, debuggers
 - ☐ OS basics: structure, services, system calls, boot
 - ☐ Process management: process states, PCB, IPC, synchronisation, critical section, Peterson, semaphores, classical problems
 - ☐ Threads: multithreading models, libraries, issues
 - ☐ CPU scheduling: criteria, FCFS, SJF, SRTF, priority, RR, multilevel queues, multiprocessor & real-time scheduling
 - ☐ Deadlocks: conditions, RAG, prevention, avoidance (Banker's), detection, recovery
 - ☐ Memory management: contiguous allocation, paging, segmentation, TLB, demand paging, page replacement, thrashing
 - ☐ Storage: disk structure, disk scheduling, RAID
 - ☐ File systems & I/O: access methods, directories, allocation methods, free-space management, I/O subsystem
 - ☐ Security & protection: access matrix, threats, authentication
 - ☐ Virtual machines; Linux & Windows internals; distributed systems
-

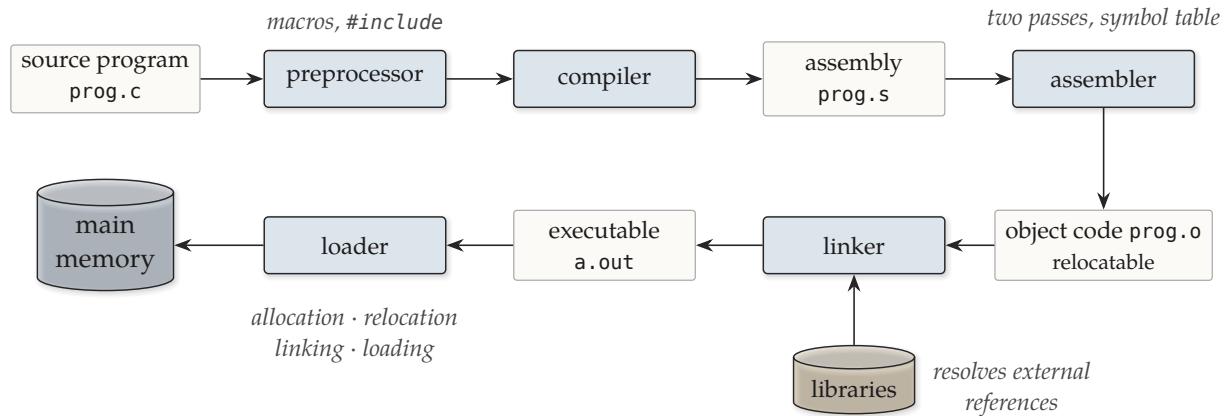


FIGURE 5.1. From source text to a running program. Translators (shaded) turn one file form into the next; the loader finally places the executable in memory.

5.1 System Software

System software is the layer of programs that makes a bare machine usable: translators that turn the programs we write into machine code, and the operating system that shares the hardware among them. The examination asks mostly about what each translator does and when it runs.

Tool	Job
Assembler	Assembly → machine code. Two-pass : pass 1 builds symbol table (location counter), pass 2 generates code. One-pass needs back-patching for forward references
Compiler	Whole program → target code at once; faster execution
Interpreter	Statement by statement; slower, easier debugging
Linker	Resolves external references, combines object modules
Loader	Places program in memory; functions: Allocation, Linking, Relocation, Loading
Macro processor	Expands macros (text substitution, no call overhead); macro vs subroutine
Debugger	Breakpoints, single-step, watch variables

- Loaders: **compile-and-go**, **absolute** (programmer does relocation), **relocating** (BSS loader), **direct-linking** loader, **dynamic loading** (load routine when called), **dynamic linking** (link at run time — DLL, .so).
- **Relocation**: adjusting address-sensitive instructions when program is loaded at a different address; relocation bits/table.
- Assembler data structures: **Symbol table (ST)**, **Literal table (LT)**, **Opcode table (MOT/OPTAB)**, **Pseudo-op table (POT)**, Pool table.
- Assembler directives (pseudo-ops): START, END, ORIGIN, EQU, LTORG, DC, DS.

5.2 OS Basics

An operating system has two jobs. As a **resource manager** it shares the CPU, memory and devices among competing programs; as an **extended machine** it hides awkward hardware behind clean abstractions such as processes, files and sockets. Programs ask for these services through system calls.

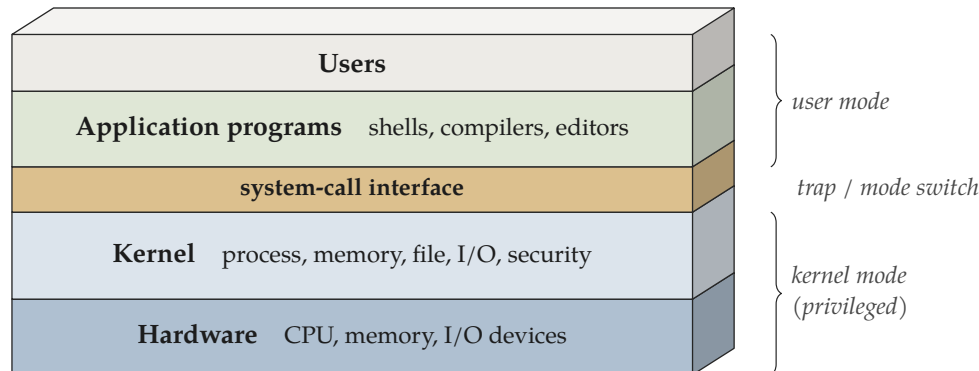


FIGURE 5.2. Layered view of a computer system. Programs reach the hardware only through the system-call interface, which switches the CPU from user mode to kernel mode.

Structure	Example
Simple / monolithic	MS-DOS, early UNIX, Linux (monolithic + loadable modules)
Layered	THE OS (Dijkstra)
Microkernel	Mach, QNX, Minix — minimal kernel, services in user space via message passing
Modular	Solaris, Linux LKMs
Hybrid	Windows NT, macOS (XNU)
Exokernel	Research — exposes hardware

- **Dual mode:** user mode / kernel mode (mode bit). System call → trap → kernel mode.
- System call categories: process control (fork, exec, exit, wait), file management (open, read), device management (ioctl), information maintenance (getpid), communication (pipe, shmget), protection (chmod).
- Parameter passing to system calls: registers, block/table in memory, stack.
- Boot: power on → BIOS/UEFI firmware (POST) → bootstrap loader (MBR / GRUB) → kernel loaded → init/systemd.
- OS types: batch, multiprogramming (maximise CPU utilisation), time-sharing (multitasking, response time), real-time (hard/soft), distributed, network, embedded.

fork() puzzle: n consecutive `fork()` calls create $2^n - 1$ child processes (2^n total).

```
fork(); fork(); fork();    // 8 processes total, 7 children
if (fork() && fork()) fork(); // count carefully: 4 processes total (parent + 3 children)
```

5.3 Process Management

A program is a passive file on disk; a **process** is that program in execution, together with its program counter, registers, stack and open files. The operating system records all of this in a **process control block** (PCB) and moves the process between states as it runs, waits and finishes.

5.3.1 Process States

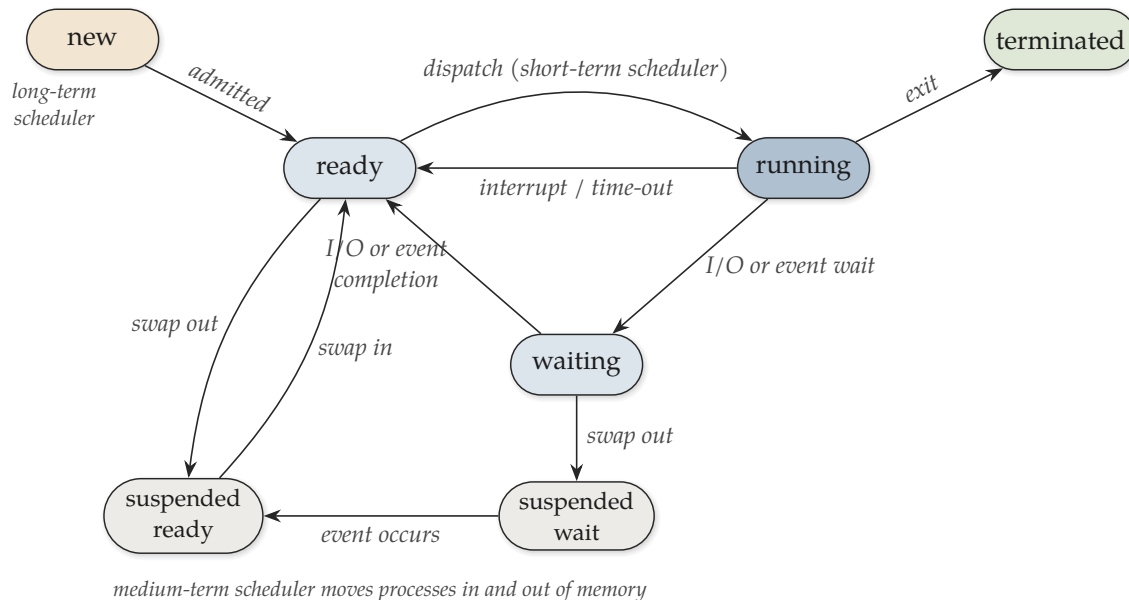


FIGURE 5.3. The process life cycle. The lower row exists only on systems that swap whole processes out of main memory.

- **PCB** (Process Control Block): PID, state, PC, registers, scheduling info, memory info (page tables), accounting, I/O status.
- Schedulers: **long-term** (job; controls degree of multiprogramming), **short-term** (CPU; very frequent), **medium-term** (swapping).
- **Context switch** time is pure overhead. **Dispatcher latency**.
- **Zombie**: terminated but parent hasn't called `wait()`. **Orphan**: parent terminated (adopted by `init`).

5.3.2 IPC

- **Shared memory** (fast, needs synchronisation) vs **message passing** (send/receive; direct/indirect via mailboxes; blocking = synchronous, non-blocking = asynchronous).
- Pipes (ordinary – parent-child, unidirectional; named pipes/FIFOs), sockets (IP + port), RPC (stubs, marshalling), signals.

5.3.3 Critical Section Problem

When processes share data, the parts of their code that touch it must not overlap in time. Any solution must give **mutual exclusion** (one process inside at a time), **progress** (no needless blocking when the section is free) and **bounded waiting** (no process waits forever). Requirements: **Mutual exclusion, Progress, Bounded waiting**.

Peterson's solution (2 processes):

```
// shared: bool flag[2]; int turn;
do {
    flag[i] = true;
    turn = j;                // politely give turn to other
    while (flag[j] && turn == j); // busy wait
    /* critical section */
    flag[i] = false;
    /* remainder section */
} while (true);
```

Satisfies all three requirements (on sequentially consistent memory).

Hardware: **TestAndSet, CompareAndSwap** (atomic). Simple TAS lock gives ME + progress but not bounded waiting.

5.3.4 Semaphores

A semaphore is an integer that can be changed only by two atomic operations, wait (P) and signal (V). Initialised to 1 it acts as a lock; initialised to 0 it forces one event to happen after another; initialised to n it counts free slots.

```
wait(S)  { while (S <= 0); S--; }    // P(), down()
signal(S) { S++; }                  // V(), up()
```

- **Binary semaphore** (mutex, 0/1) vs **counting semaphore**.
- Blocking implementation uses a waiting queue — negative value = number of waiting processes.
- Counting semaphore initial value 10, then 6 P and 4 V $\rightarrow 10 - 6 + 4 = 8$.
- Misuse $\rightarrow$ deadlock (wrong order of waits) or starvation. **Priority inversion** $\rightarrow$ solved by priority inheritance.
- **Monitors**: high-level construct; condition variables with wait()/signal().
- **Spinlock** = busy-waiting lock (good for short waits on multiprocessors).

5.3.5 Classical problems

Problem	Semaphores
Bounded buffer (producer–consumer)	<code>mutex = 1, empty = n, full = 0</code>
Readers–writers	<code>rw_mutex = 1, mutex = 1, read_count = 0</code> (first readers–writers: writers may starve)
Dining philosophers (5)	<code>chopstick[5] = 1</code> ; deadlock if all pick left — fixes: at most 4 at table, asymmetric pick, pick both atomically
Sleeping barber	<code>customers, barbers, mutex</code>

Producer	Consumer
<code>wait(empty);</code>	<code>wait(full);</code>
<code>wait(mutex);</code>	<code>wait(mutex);</code>
<i>add item to buffer</i>	<i>remove item from buffer</i>
<code>signal(mutex);</code>	<code>signal(mutex);</code>
<code>signal(full);</code>	<code>signal(empty);</code>

Initial values: `mutex = 1, empty = n, full = 0`. Swapping the two `wait` calls in the producer can cause **deadlock**: it may hold `mutex` while waiting on a full buffer, so the consumer can never enter to empty it.

5.4 Threads

- Thread = lightweight process: own **PC, registers, stack**; shares code, data, files with peers.
- **User-level threads** (fast, kernel unaware; one blocks → all block in many-to-one) vs **kernel-level threads**.
- Models: **Many-to-one** (Green threads), **One-to-one** (Linux, Windows), **Many-to-many**, two-level.
- Libraries: POSIX Pthreads, Windows threads, Java threads. Implicit threading: thread pools, OpenMP, GCD, fork-join.
- Issues: `fork()/exec()` semantics, signal handling, thread cancellation (asynchronous vs deferred), thread-local storage, scheduler activations (LWP, upcalls).
- **Amdahl's law** for multicore: $\text{speedup} \leq 1 / (S + (1-S)/N)$.

5.5 CPU Scheduling

Whenever the CPU falls idle, the short-term scheduler picks the next ready process. Different policies favour different goals (low average waiting time, quick response, fairness), and numerical questions ask you to draw the Gantt chart and read the times off it.

Turnaround time $TAT = Completion - Arrival$

Waiting time $WT = TAT - Burst$

Response time $RT = \text{first run} - Arrival$

Throughput $\frac{\text{number of processes}}{\text{total time}}$

Algorithm	Preemptive?	Notes
FCFS	No	Convoy effect
SJF	No	Optimal average waiting time (non-preemptive)
SRTF	Yes (preemptive SJF)	Optimal average WT overall; starvation
Priority	Both	Starvation → aging
Round Robin	Yes	Time quantum q ; large $q \rightarrow$ FCFS; small $q \rightarrow$ high context switch overhead
Multilevel queue	—	Fixed queues (foreground RR, background FCFS)
Multilevel feedback queue	Yes	Processes move between queues; most general
HRRN	No	Response ratio $= (W + S)/S$; favours short jobs, avoids starvation

5.5.1 Worked Example

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	9
P4	3	5

SRTF Gantt chart:

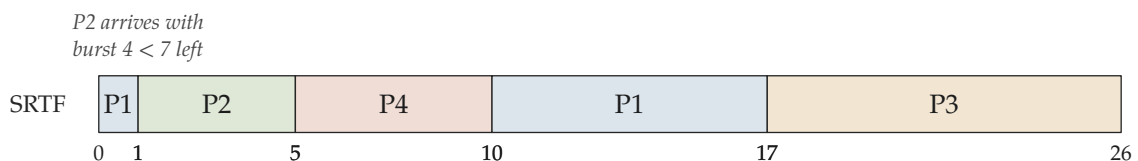


FIGURE 5.4. SRTF Gantt chart: at every arrival the process with the shortest *remaining* time runs.

P	CT	TAT	WT
P1	17	17	9
P2	5	4	0
P3	26	24	15
P4	10	7	2

Average WT = $26/4 = 6.5$.

Round Robin ($q = 4$) on same data:

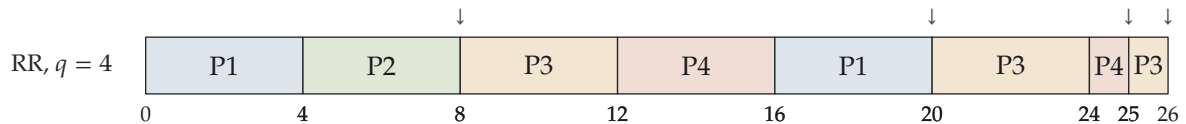


FIGURE 5.5. Round robin with quantum 4. Arrows mark the moments a process finishes (P2 at 8, P1 at 20, P4 at 25, P3 at 26).

CT: P1 = 20, P2 = 8, P3 = 26, P4 = 25 $\rightarrow$ WT: P1 = 12, P2 = 3, P3 = 15, P4 = 17 $\rightarrow$ average = **11.75**.

5.5.2 Multiprocessor & Real-time

- Asymmetric vs **symmetric multiprocessing (SMP)**; processor affinity (soft/hard); load balancing (push/pull migration).
- **Rate Monotonic** (static priority: shorter period $\rightarrow$ higher priority); schedulable if $U \leq n(2^{1/n} - 1)$ (≈ 0.69 for large n).
- **EDF** (Earliest Deadline First, dynamic): schedulable iff $U \leq 1$.
- Linux: **CFS** (Completely Fair Scheduler, red-black tree keyed by virtual runtime). Windows: 32-level priority-based preemptive.

5.6 Deadlocks

A deadlock is a set of processes each holding a resource and waiting for one held by another, so none can proceed. Systems either prevent one of the four necessary conditions, avoid unsafe states (Banker's algorithm), or detect deadlock and recover.

5.6.1 Four Necessary Conditions (Coffman)

Mutual exclusion · **Hold and wait** · **No preemption** · **Circular wait** — all four must hold.

Strategy	How
Prevention	Break one condition: e.g. request all at once (hold & wait), allow preemption, impose total order on resources (circular wait)
Avoidance	Banker's algorithm — grant only if state remains safe
Detection & recovery	Wait-for graph (single instance), detection algorithm; recover by killing processes / preemption
Ignorance	Ostrich algorithm (UNIX, Windows)

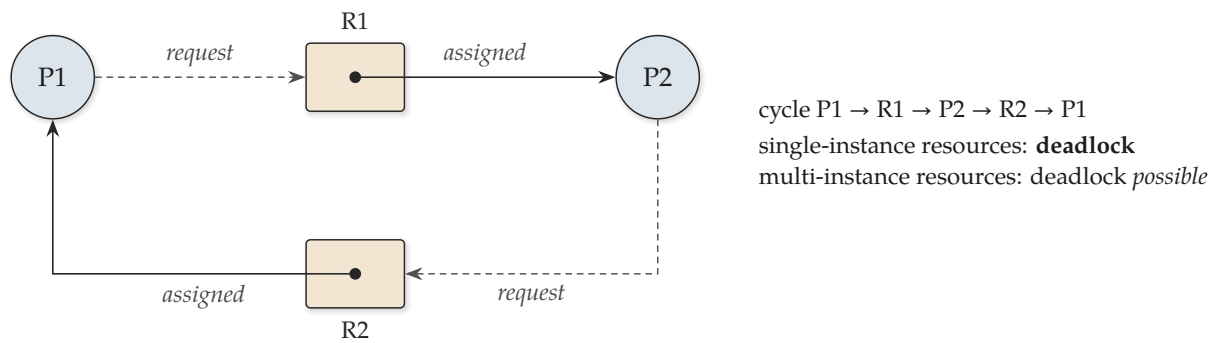


FIGURE 5.6. A resource-allocation graph. Request edges run from process to resource (dashed); assignment edges run from a resource instance (dot) to the process holding it.

5.6.2 Banker's Algorithm — Worked

5 processes, resources A(10), B(5), C(7).

P	Allocation (A B C)	Max	Need = Max – Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (10,5,7) – (7,2,5) = (3,3,2).

Work before	Choose	Need ≤ Work?	+ Allocation	Work after
(3, 3, 2)	P1	(1, 2, 2) ✓	(2, 0, 0)	(5, 3, 2)
(5, 3, 2)	P3	(0, 1, 1) ✓	(2, 1, 1)	(7, 4, 3)
(7, 4, 3)	P4	(4, 3, 1) ✓	(0, 0, 2)	(7, 4, 5)
(7, 4, 5)	P0	(7, 4, 3) ✓	(0, 1, 0)	(7, 5, 5)
(7, 5, 5)	P2	(6, 0, 0) ✓	(3, 0, 2)	(10, 5, 7)

Every process can finish, so the state is **safe**; the safe sequence is $\langle P1, P3, P4, P0, P2 \rangle$. A finished process returns its allocation, which is why Work grows row by row and ends at the total (10, 5, 7).

Deadlock-free condition formula: n processes each needing max k instances of a resource with R instances total → deadlock-free if $R \geq n(k - 1) + 1$. e.g. 3 processes each need 3 units → minimum $R = 3 \times 2 + 1 = 7$.

5.7 Memory Management

Programs are written as if they owned a large, private, contiguous memory. Memory management maps these **logical** addresses onto the **physical** memory actually available, and protects each process's region from the others.

5.7.1 *Contiguous Allocation*

- Fixed partitions → **internal fragmentation**. Variable partitions → **external fragmentation** (fix with compaction).
- Placement: **First fit**, **Best fit** (smallest adequate hole), **Worst fit** (largest), Next fit.
- **50-percent rule**: with first fit, N allocated blocks → 0.5N blocks lost to fragmentation.

5.7.2 Paging

Paging cuts logical memory into fixed-size **pages** and physical memory into frames of the same size, so any page can sit in any free frame. External fragmentation disappears; the price is a page table per process and an extra memory access per reference, which the TLB hides.

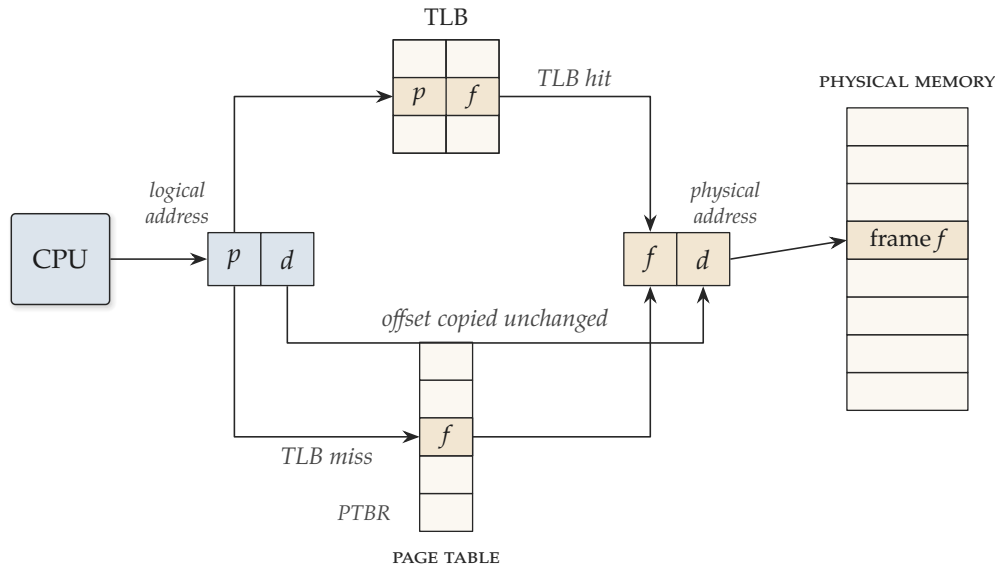


FIGURE 5.7. Paging hardware. The page number p is looked up first in the TLB; only on a miss is the page table in memory consulted. The offset d passes through untranslated.

Logical-address bits	$\log_2(\text{virtual address space})$
Offset bits	$d = \log_2(\text{page size})$
Number of pages	$2^{\text{LA bits} - d}$
Number of frames	physical memory size/page size
Page-table size	number of pages $\times$ PTE size
Effective access time with a TLB (hit ratio h , TLB time t , memory time m)	$\text{EAT} = h(t + m) + (1 - h)(t + 2m)$

Worked: 32-bit logical address, 4 KB pages, 4-byte PTE.

- Offset = 12 bits; pages = 2^{20} ; page table = $2^{20} \times 4 \text{ B} = 4 \text{ MB}$ per process (hence multilevel paging).
- Two-level (10 | 10 | 12): outer table 1024 entries $\times 4 \text{ B} = 4 \text{ KB} = \text{one page}$.
- Inverted page table: one entry per **frame** (size $\propto$ physical memory), searched by hashing.

5.7.3 Segmentation

- Logical address = (segment number, offset); segment table has **base & limit**; offset $\geq$ limit $\rightarrow$ trap.
- No internal fragmentation, but external fragmentation. Segmentation with paging (Intel x86).

5.7.4 Virtual Memory & Demand Paging

Virtual memory keeps only the pages a process is actually using in RAM and loads the others on demand when a **page fault** occurs. Programs can therefore be larger than physical memory, but each fault costs milliseconds, so the fault rate dominates performance.

$$\text{EAT} = (1 - p) \times ma + p \times \text{page-fault service time}$$

For example, with $ma = 200$ ns, a service time of 8 ms and $p = 0.001$:

$$\text{EAT} \approx 200 + 0.001 \times 8,000,000 \text{ ns} \approx 8.2 \mu\text{s}.$$

A fault rate of only one in a thousand slows memory down forty-fold; this is why page-fault rates must be kept tiny.

Page fault steps: trap $\rightarrow$ check valid $\rightarrow$ find free frame $\rightarrow$ schedule disk read $\rightarrow$ update table $\rightarrow$ restart instruction.

5.7.5 Page Replacement — Worked

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2, 3 frames.

FIFO

REFERENCE	7	0	1	2	0	3	0	4	2	3	0	3	2
FRAME 1	7	7	7	2	2	2	2	4	4	4	0	0	0
FRAME 2		0	0	0	0	3	3	3	2	2	2	2	2
FRAME 3			1	1	1	1	0	0	0	3	3	3	3
FAULT	●	●	●	●		●	●	●	●	●	●		

10 faults

FIGURE 5.8. FIFO replacement with three frames. Shaded cells show where a new page was loaded; each replaces the page that has been resident longest.

Optimal (replace page used farthest in future) $\rightarrow$ 7 faults. **LRU** (replace least recently used) $\rightarrow$ 9 faults.

- **Belady's anomaly**: more frames $\rightarrow$ more faults; occurs in **FIFO**, not in stack algorithms (LRU, Optimal). Classic string: 1 2 3 4 1 2 5 1 2 3 4 5 $\rightarrow$ FIFO 3 frames: 9 faults, 4 frames: 10 faults.
- Other algorithms: Second chance (clock), LFU, MFU, NRU (reference & modify bits).
- **Thrashing**: process spends more time paging than executing — when Σ working sets $>$ available frames; fix with **working-set model** or **page-fault frequency** control, reduce degree of multiprogramming.
- Frame allocation: equal, proportional; global vs local replacement.
- **Copy-on-write** for `fork()`. **Memory-mapped files**.

5.8 Disk & Storage

A magnetic disk spends most of each request moving the arm (seek) and waiting for the sector to rotate under the head (latency). Disk-scheduling algorithms reorder pending requests to cut total head movement.

5.8.1 Disk Scheduling — Worked

Queue: 98, 183, 37, 122, 14, 124, 65, 67; head at 53; cylinders 0–199.

Algorithm	Order	Total head movement
FCFS	53→98→183→37→122→14→124→65→67	640
SSTF	53→65→67→37→14→98→122→124→183	236
SCAN (towards 0)	53→37→14→0→65→67→98→122→124→183	236
C-SCAN (towards 199)	53→65→...→183→199→0→14→37	146 + 199 + 37 = 382 (counting return jump)
LOOK (towards 0)	53→37→14→65→...→183	39 + 169 = 208
C-LOOK (up)	53→65→67→98→122→124→183→14→37	130 + 169 + 23 = 322

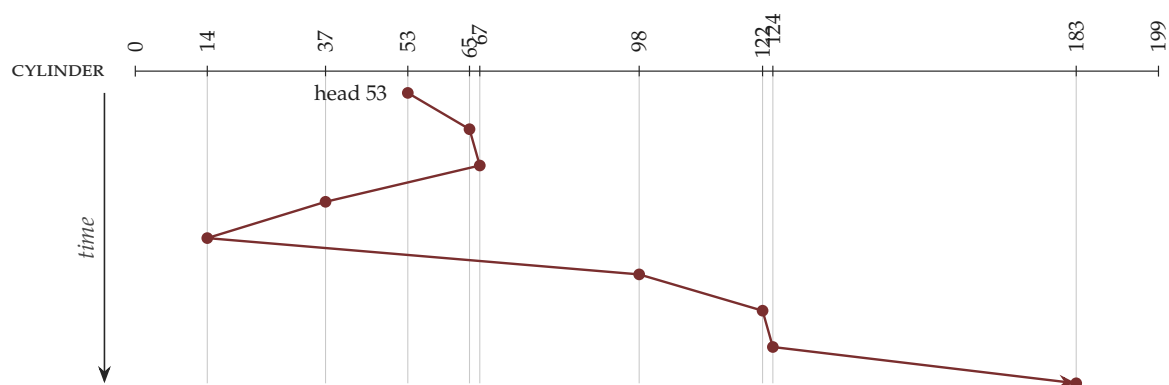


FIGURE 5.9. SSTF head path for the queue 98, 183, 37, 122, 14, 124, 65, 67 starting at cylinder 53: total movement $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236$ cylinders.

- SSTF may cause **starvation**. SCAN = elevator. C-SCAN gives more uniform wait time.

5.8.2 RAID

Level	Technique	Min disks	Fault tolerance
0	Striping (block)	2	None
1	Mirroring	2	1 disk
2	Bit striping + Hamming ECC	—	1
3	Byte striping + dedicated parity	3	1
4	Block striping + dedicated parity	3	1 (parity disk bottleneck)
5	Block striping + distributed parity	3	1
6	Distributed double parity (P+Q)	4	2
10 (1+0)	Mirror then stripe	4	1 per mirror pair

5.9 File Systems & I/O

A file system turns a disk's numbered blocks into named files and directories. Its central design choice is how a file's blocks are found: contiguously, by a chain of pointers, or through an index.

- Access methods: sequential, direct (relative), indexed.
- Directory structures: single-level → two-level → tree → **acyclic graph** (sharing via links) → general graph (cycles; needs garbage collection).
- Hard link (same inode, same FS, no dirs) vs soft/symbolic link (path, can cross FS, can dangle).

5.9.1 Allocation Methods

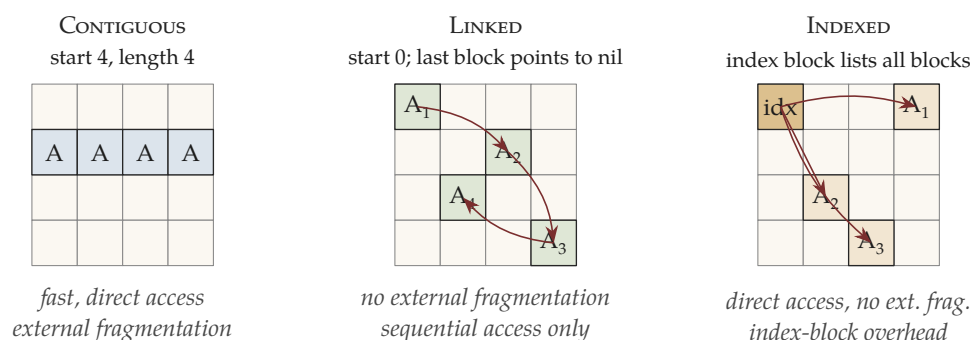


FIGURE 5.10. Three ways to lay a four-block file A on disk. FAT is a variant of linked allocation that keeps the chain in a table; the UNIX inode is a multilevel index.

UNIX inode max file size: 12 direct + 1 single + 1 double + 1 triple indirect. Block 4 KB, pointer 4 B → 1024 pointers per block:

For 4 KB blocks and 4-byte pointers, $N = 1024$:

$$(12 + 1024 + 1024^2 + 1024^3) \times 4 \text{ KB} \approx 1024^3 \times 4 \text{ KB} = 4 \text{ TB}.$$

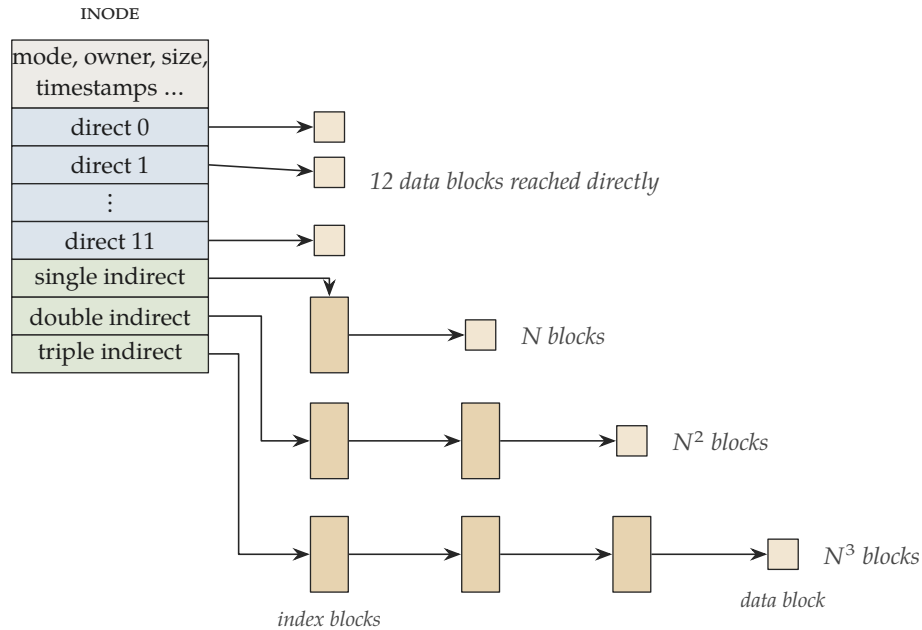


FIGURE 5.11. The UNIX inode. With N pointers per block ($N = \text{block size} / \text{pointer size}$), each extra level of indirection multiplies the reachable blocks by N .

- Free-space management: **bit vector** (1 bit per block), linked list, grouping, counting.
- Directory implementation: linear list, hash table.
- Mounting, file sharing, NFS. Journaling (log-structured) file systems for consistency.

I/O: polling, interrupts, DMA (see Unit 2). Kernel I/O subsystem: scheduling, **buffering** (single, double, circular), **caching**, **spooling** (printer), error handling. Device drivers. Block vs character devices.

5.10 Protection & Security

- **Access matrix**: rows = domains, columns = objects. Implementations: global table, **access control lists** (column-wise, per object), **capability lists** (row-wise, per domain), lock-key.
- Revocation: immediate/delayed, selective/general, partial/total.
- Principle of **least privilege**; need-to-know.
- Program threats: Trojan horse, trap door (back door), logic bomb, stack/buffer overflow, virus. System/network threats: worms, port scanning, DoS/DDoS.
- Authentication: passwords (salted hash), OTP, biometrics, multi-factor.
- Cryptography — see Unit 9.

5.11 Virtual Machines

- **Type 1 (bare-metal) hypervisor**: VMware ESXi, Xen, Hyper-V, KVM. **Type 2 (hosted)**: VirtualBox, VMware Workstation.
- Full virtualisation, para-virtualisation (guest OS modified — Xen), hardware-assisted (Intel VT-x, AMD-V), containers (OS-level: Docker — share host kernel), emulation (QEMU).
- JVM, .NET CLR = application/process VMs.

5.12 Linux & Windows

Feature	Linux	Windows
Kernel	Monolithic + loadable kernel modules	Hybrid (microkernel-inspired), HAL, executive
Process creation	<code>fork()</code> + <code>exec()</code> (clone for threads)	<code>CreateProcess()</code>
Scheduling	CFS (normal), real-time FIFO/RR	Priority-based preemptive, 32 levels
Memory	Buddy system + slab allocator, demand paging	Demand paging with clustering, working sets
File system	ext2/3/4 (inodes), VFS layer; <code>/proc</code>	NTFS (MFT, journaling, ACLs), FAT32, ReFS
IPC	pipes, signals, shared memory, message queues, sockets	ALPC, pipes, mailslots
Other	GPL; first released 1991 (Linus Torvalds)	Terminal services, fast user switching

Linux process states: Running, Interruptible sleep, Uninterruptible sleep, Stopped, Zombie.
Buddy system: memory split into power-of-2 blocks; request 70 KB from 1 MB → gets 128 KB block.

5.13 Distributed Systems

- Network OS (users aware of machines; remote login, file transfer) vs **Distributed OS** (transparent: data, computation, process migration).
- Topologies & communication: naming, routing, connection strategies, contention.
- Robustness: failure detection (heartbeat), reconfiguration, recovery.
- Design issues: transparency, fault tolerance, scalability.
- **Distributed File Systems:** NFS (stateless, v3), AFS (whole-file caching), GFS/HDFS. Naming transparency vs location independence; caching & consistency (write-through, delayed-write, callback).
- Clock synchronisation: **Lamport logical clocks** (happened-before →), vector clocks, Cristian's, Berkeley algorithms.
- Mutual exclusion: centralised, Ricart–Agrawala ($2(n-1)$ messages), token ring. Election: **Bully**, Ring.

5.14 Deeper Dive — More Scheduling, Semaphore Traces, Multilevel Paging & File Numericals

5.14.1 Priority & HRRN Scheduling — Worked

Process	Arrival	Burst	Priority (lower = higher)
P1	0	10	3
P2	0	1	1
P3	0	2	4
P4	0	1	5
P5	0	5	2

Non-preemptive priority:



FIGURE 5.12. Non-preemptive priority scheduling (lower number = higher priority): P2 (1), P5 (2), P1 (3), P3 (4), P4 (5).

Waiting times: P1 6, P2 0, P3 16, P4 18, P5 1 $\rightarrow$ average = $41/5 = 8.2$.

HRRN (non-preemptive; response ratio $RR = (W + S)/S$), processes A(0, 3), B(2, 6), C(4, 4), D(6, 5), E(8, 2):

Time	Waiting processes, response ratio $(W + S)/S$	Runs
0	only A has arrived	A, 0–3
3	only B has arrived	B, 3–9
9	C: $(5 + 4)/4 = 2.25$, D: $(3 + 5)/5 = 1.6$, E: $(1 + 2)/2 = 1.5$	C, 9–13
13	D: $(7 + 5)/5 = 2.4$, E: $(5 + 2)/2 = 3.5$	E, 13–15
15	only D is left	D, 15–20

A Gantt chart illustrating HRRN scheduling. The horizontal axis represents time from 0 to 20. The processes are scheduled as follows: A runs from 0 to 3; B runs from 3 to 9; C runs from 9 to 13; E runs from 13 to 15; and D runs from 15 to 20. The processes are color-coded: A is blue, B is green, C is yellow, E is pink, and D is light green.

Order **A B C E D**. A long-waiting job's ratio keeps rising, so HRRN cannot starve it.

5.14.2 Semaphore Tracing

```

semaphore S = 1, T = 0;
P1: wait(S); print("A"); signal(T);
P2: wait(T); print("B"); signal(S);

```

Whatever the scheduling, P2 blocks on T until P1 signals $\rightarrow$ output **"AB"** (and the pair can repeat as A B A B ... if looped). This is the standard **ordering** use of semaphores (initialise to 0 to force "happens-after").

Deadlock with wrong order:

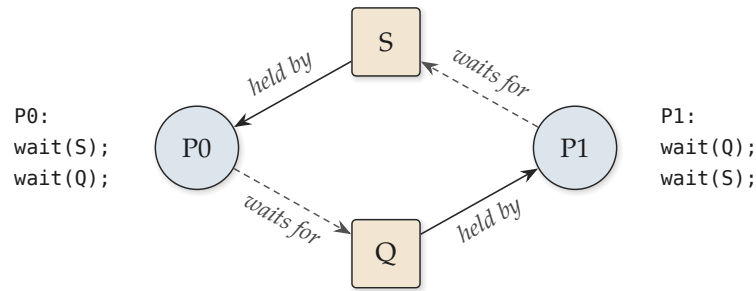


FIGURE 5.13. Acquiring two semaphores in opposite orders ($S = Q = 1$). If P0 takes S and P1 takes Q, each waits for the other forever. Acquiring in one global order prevents it.

5.14.3 Multilevel Paging Numericals

48-bit virtual address, 4 KB pages, 8-byte PTEs.

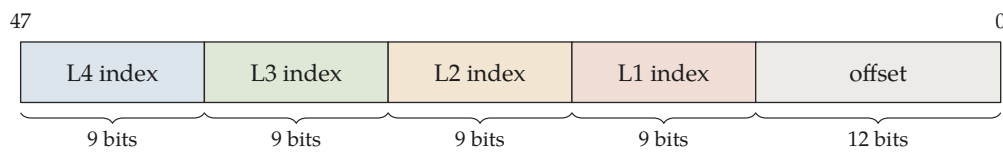


FIGURE 5.14. A 48-bit virtual address with 4 KB pages and 8-byte page-table entries: four 9-bit indexes and a 12-bit offset (the x86-64 layout).

$$\begin{aligned}
 \text{offset} &= \log_2 4096 = 12 \text{ bits} & \Rightarrow \text{page-number bits} &= 48 - 12 = 36 \\
 \text{entries per table page} &= 4096/8 = 512 = 2^9 & \Rightarrow \text{bits per level} &= 9 \\
 \text{levels} &= \lceil 36/9 \rceil = 4 & \text{accesses without TLB} &= 4 + 1 = 5
 \end{aligned}$$

Effective access time with TLB and 2-level paging: TLB 10 ns, memory 100 ns, hit ratio 90%:
 $EAT = 0.9 \times (10 + 100) + 0.1 \times (10 + 3 \times 100) = 99 + 31 = \mathbf{130 \text{ ns}}$.

5.14.4 Segmentation — Address Translation

Segment	Base	Limit
0	219	600
1	2300	14
2	90	100
3	1327	580

- (0, 430) $\rightarrow 430 < 600 \rightarrow$ physical **649**
- (1, 10) $\rightarrow$ **2310**
- (2, 500) $\rightarrow 500 \geq 100 \rightarrow$ **segmentation fault (trap)**
- (3, 400) $\rightarrow$ **1727**

5.14.5 Page Replacement with 4 Frames (Belady check)

Reference: 1 2 3 4 1 2 5 1 2 3 4 5

- FIFO, 3 frames → 9 faults; 4 frames → 10 faults (anomaly).
- LRU, 4 frames → 8 faults; Optimal, 4 frames → 6 faults.

5.14.6 File-System Numericals

Max file size with UNIX inode: block 1 KB, pointer 4 B → 256 pointers/block; 10 direct, 1 single, 1 double, 1 triple:

$$(10 + 256 + 256^2 + 256^3) \times 1 \text{ KB} = (10 + 256 + 65,536 + 16,777,216) \text{ KB} \approx 16 \text{ GB}$$

FAT size: 4 GB disk, 4 KB clusters, 32-bit entries → 2^{20} clusters × 4 B = 4 MB FAT.

Disk access: 7200 RPM, average seek 8 ms, transfer rate 100 MB/s, read 4 KB block: seek 8 + latency 4.17 + transfer 0.04 ≈ 12.2 ms.

5.14.7 Buddy System — Worked

1 MB block; requests A = 70 KB, B = 35 KB, C = 80 KB.

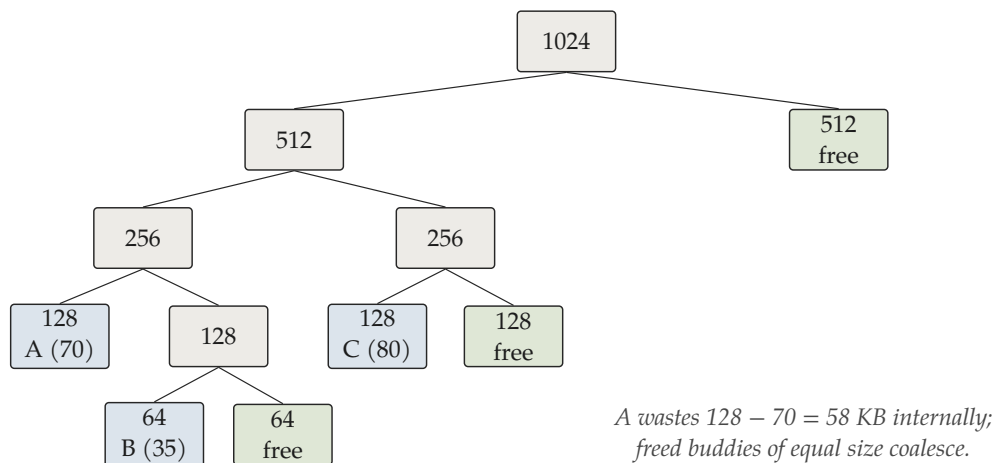


FIGURE 5.15. Buddy allocation in a 1 MB (1024 KB) block for requests A = 70 KB, B = 35 KB, C = 80 KB. Each request receives the smallest power-of-two block that fits.

5.14.8 Process Synchronisation Hardware

```
// Test-and-Set lock
bool lock = false;
do {
    while (test_and_set(&lock)); // atomically: old = lock; lock = true; return old
    /* critical section */
    lock = false;
} while (true);
```

Gives mutual exclusion and progress, but **not bounded waiting** (a process can be overtaken indefinitely); the bounded-waiting version uses a `waiting[]` array.

5.15 Previous Year Questions (PYQ pattern)

System software

1. A two-pass assembler's first pass: **builds the symbol table (assigns addresses)**
2. Forward reference problem in one-pass assembler is solved by: **back-patching**
3. Which loader function adjusts address-dependent locations? **Relocation**
4. Dynamic linking is performed at: **run time**
5. Macro expansion is done by: **macro processor / preprocessor** (before assembly/compilation)

Processes & threads

6. Number of child processes created by `for(i=0;i<n;i++) fork();`: $2^n - 1$
7. Which is NOT shared by threads of a process? **Stack (and registers)**
8. Degree of multiprogramming is controlled by: **long-term scheduler**
9. A process whose parent has not called `wait()`: **zombie**
10. Which multithreading model blocks the whole process on a blocking system call? **Many-to-one**

Synchronisation

11. Critical section solution requirements: **mutual exclusion, progress, bounded waiting**
12. Semaphore $S = 7$; 20 P and 15 V operations performed: $\text{final} = 7 - 20 + 15 = 2$
13. Producer-consumer with bounded buffer of size n : initial values $\text{mutex} = 1$, $\text{empty} = n$, $\text{full} = 0$
14. Priority inversion is solved by: **priority inheritance protocol**
15. Peterson's solution is for: **two processes**

Scheduling

16. Algorithm with minimum average waiting time: **SJF / SRTF**
17. Round robin with very large quantum becomes: **FCFS**
18. Starvation in priority scheduling is solved by: **aging**
19. Convoy effect occurs in: **FCFS**
20. Processes $P1(0,10)$, $P2(1,1)$, $P3(2,2)$ with SRTF: average waiting time = $P1: 3, P2: 0, P3: 0 \rightarrow 1$
21. Rate-monotonic schedulability bound for 2 tasks: $2(\sqrt{2} - 1) \approx 0.828$

Deadlock

22. Necessary conditions for deadlock: **mutual exclusion, hold & wait, no preemption, circular wait**
23. Banker's algorithm is for deadlock: **avoidance**
24. Ordering resources numerically prevents: **circular wait**
25. 4 processes each need 2 tape drives; minimum drives to ensure no deadlock: $4 \times 1 + 1 = 5$
26. Cycle in RAG with single-instance resources implies: **deadlock**

Memory

27. Belady's anomaly occurs in: **FIFO**

28. Page size 4 KB, logical address 32 bits → number of pages: 2^{20}
29. TLB 20 ns, memory 100 ns, hit 80%: $EAT = 0.8 \times 120 + 0.2 \times 220 = 140$ ns
30. Thrashing is caused by: **high degree of multiprogramming / insufficient frames**
31. Internal fragmentation occurs in: **paging (fixed-size)**; external in: **segmentation / variable partitions**
32. Best fit, memory holes 100K, 500K, 200K, 300K, 600K; request 212K → **300K hole**
33. LRU faults for 1,2,3,4,1,2,5,1,2,3,4,5 with 3 frames: **10**
34. Optimal page replacement is not implementable because: **future references are unknown**
35. Working set model is used to prevent: **thrashing**

Disk / file

36. Disk scheduling that may cause starvation: **SSTF**
37. Elevator algorithm: **SCAN**
38. RAID level with distributed parity: **RAID 5**
39. RAID with mirroring: **RAID 1**
40. Allocation method that supports direct access and has no external fragmentation: **Indexed**
41. FAT uses which allocation method? **Linked (table variant)**
42. Bit vector for 1 TB disk with 4 KB blocks: 2^{28} bits = **32 MB**

Security / VM / distributed

43. Access control lists correspond to: **columns of the access matrix**
44. Capability list corresponds to: **rows (domains)**
45. A program that appears useful but performs malicious actions: **Trojan horse**
46. VMware ESXi is a: **Type 1 hypervisor**
47. Lamport clocks capture: **happened-before ordering (logical time)**
48. Linux kernel is: **monolithic with loadable modules**
49. Windows file system with MFT: **NTFS**

More practice questions

50. HRRN favours: **short jobs, while preventing starvation of long jobs (waiting time raises the ratio)**
51. Response ratio of a process with waiting time 9 and burst 3: **4**
52. Semaphore initialised to 0 is used for: **ordering / signalling between processes**
53. 32-bit virtual address, 4 KB pages, 4-byte PTEs, single-level: page table size = **4 MB**
54. Levels of paging for 48-bit VA, 4 KB pages, 8 B PTEs (page-sized tables): **4**
55. Segment (2, 500) with limit 100: **trap / segmentation fault**
56. LRU with 4 frames on 1 2 3 4 1 2 5 1 2 3 4 5: **8 faults**
57. Optimal with 4 frames on the same string: **6 faults**
58. Buddy system allocation for a 70 KB request: **128 KB block**
59. Test-and-set spin lock fails to guarantee: **bounded waiting**
60. Average rotational latency at 15,000 RPM: **2 ms**

POINTS TO REMEMBER

- n forks $\rightarrow 2^n$ processes · Threads share code/data/files, not stack/registers
- SRTF optimal WT · RR big q = FCFS · aging fixes starvation
- Deadlock-free: $R \geq n(k-1)+1$ · Banker = avoidance
- $EAT(TLB) = h(t+m) + (1-h)(t+2m)$ · Belady only FIFO
- SSTF starvation · SCAN elevator · RAID 5 distributed parity

Software Engineering

— . . . —

Expected questions: 8–10 · Target: 9+ · Type: mostly theory (Pressman) + metrics numericals (COCOMO, FP, cyclomatic) — most scoring unit

SYLLABUS COVERAGE

- ☐ Software process models: waterfall, prototype, evolutionary (incremental, spiral), concurrent, component-based, formal methods, AOSD, unified process, Agile (XP, Scrum, DSDM, Crystal, FDD, ASD, Kanban)
 - ☐ Software requirements: functional & non-functional, eliciting, analysis, requirements specification, SRS, validation, management
 - ☐ Software design: abstraction, architecture, patterns, modularity, cohesion & coupling, information hiding, functional independence, refinement, architectural & UI design
 - ☐ Software quality: McCall, ISO 9126, quality control & assurance, risk management, RMMM, software reliability
 - ☐ Estimation & scheduling: size (LOC, FP), COCOMO, project scheduling & staffing, timeline charts
 - ☐ Software testing: verification & validation, error/fault/defect/failure, unit/integration/system/acceptance, white-box & black-box, regression, performance, stress, alpha/beta
 - ☐ Configuration management; maintenance & re-engineering; reverse engineering
-

6.1 Software Process Models

A process model fixes the order in which a team specifies, designs, builds and tests software. Plan-driven models (waterfall, V) suit stable requirements; iterative models (prototype, spiral, agile) suit requirements that will change. Questions usually describe a project and ask which model fits.

6.1.1 Waterfall (Royce, 1970)

- Linear sequential; each phase completes before next. Good when **requirements are clear & stable**. No working software until late; high risk.
- **V-Model**: verification phases (left) paired with validation/testing phases (right): Requirements ↔ Acceptance test, System design ↔ System test, Architecture ↔ Integration test, Module design ↔ Unit test.

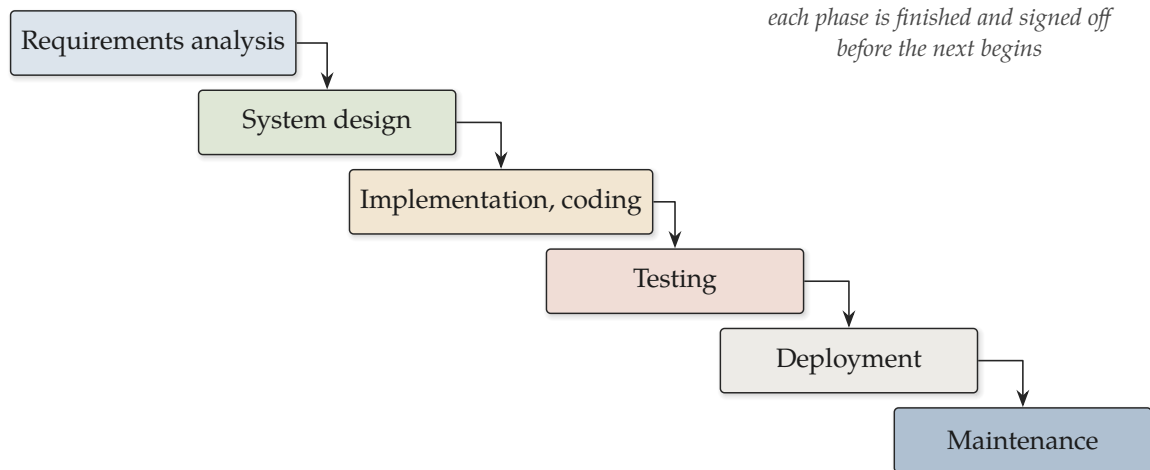


FIGURE 6.1. The waterfall model (Royce, 1970): a linear sequence of phases, cascading like a waterfall. Going back to an earlier phase is possible but expensive.

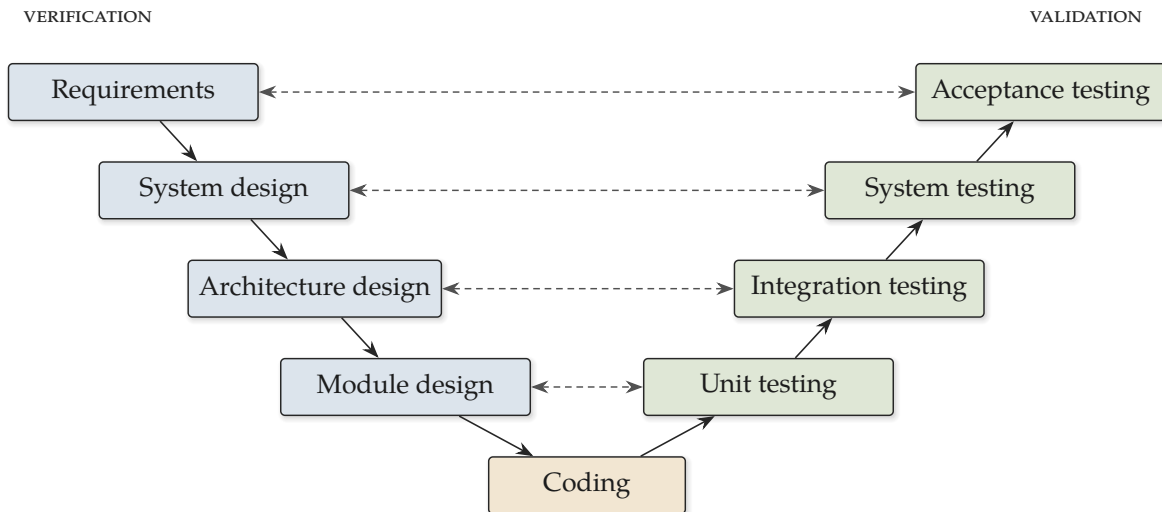


FIGURE 6.2. The V-model. Each design phase on the left defines the test phase that checks it on the right.

6.1.2 Prototype Model

Quick design → build prototype → customer evaluation → refine → (throwaway or evolutionary). Best when **requirements are unclear**.

6.1.3 Incremental & RAD

- **Incremental**: deliver in increments; first increment = core product.
- **RAD** (Rapid Application Development): 60–90 days, component reuse, multiple teams; needs modular & well-understood requirements.

6.1.4 Spiral Model (Barry Boehm, 1986) — risk-driven

- 4 quadrants: Planning (objectives) → **Risk analysis** → Engineering → Evaluation. Meta-model; suits large, high-risk projects.

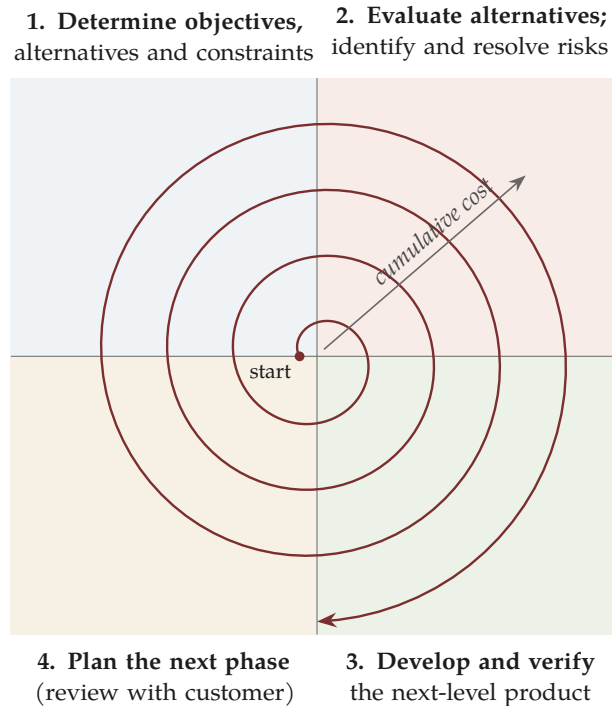


FIGURE 6.3. Boehm's spiral model. Each turn of the spiral is one phase passing through all four quadrants; the radius is cumulative cost. Risk analysis in quadrant 2 is what makes the model *risk-driven*.

6.1.5 Others

Model	Key idea
Concurrent	Activities exist simultaneously in different states (state chart)
Component-based (CBSE)	Assemble pre-built COTS components
Formal methods	Mathematical specification (Z, VDM); Cleanroom SE (statistical testing, defect prevention)
AOSD	Aspect-oriented: crosscutting concerns (logging, security) as aspects
Unified Process (RUP)	Use-case driven, architecture-centric, iterative & incremental. Phases: Inception → Elaboration → Construction → Transition (+ Production)

6.1.6 Agile

Agile methods deliver working software in short iterations and welcome changing requirements. The Agile Manifesto (2001) values individuals and interactions, working software, customer collaboration and responding to change over processes, documentation, contract negotiation and following a plan.

Agile Manifesto (2001) values:

- **Individuals & interactions** over processes & tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan (12 principles; e.g. deliver frequently, welcome changing requirements, sustainable pace, face-to-face communication.)

Method	Highlights
XP (Kent Beck)	Pair programming, test-first (TDD) , refactoring, continuous integration, small releases, user stories , spike solutions, CRC cards, collective ownership, 40-hour week
Scrum	Sprint (2–4 weeks, time-boxed), Product backlog , Sprint backlog, Daily Scrum (15 min stand-up), Sprint review, retrospective; roles: Product Owner , Scrum Master , Development Team ; burndown chart
DSDM	Time-boxing, MoSCoW prioritisation (Must, Should, Could, Won't); 80/20 principle
Crystal	Family (Clear, Orange...) by team size & criticality (Alistair Cockburn)
FDD	Feature Driven Development; features in ≤ 2 weeks
ASD	Adaptive SD: Speculation, Collaboration, Learning (Highsmith)
Kanban	Visual board, limit Work-In-Progress (WIP)
Lean	Eliminate waste

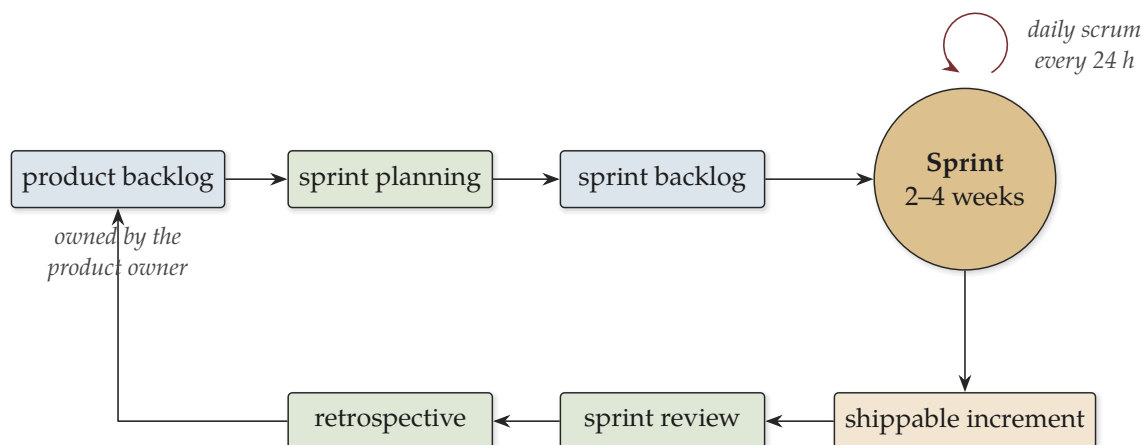


FIGURE 6.4. The Scrum cycle. Work is pulled from the product backlog into fixed-length sprints; every sprint ends with a working increment, a review with stakeholders and a retrospective on the process.

6.1.7 Model selection cheat-sheet

Situation	Model
Clear, fixed requirements	Waterfall
Unclear requirements	Prototyping
High risk, large	Spiral
Quick delivery, modular	RAD / Incremental
Changing requirements, small teams	Agile
Safety-critical	Formal methods / Cleanroom

6.2 Requirements Engineering

Requirements engineering finds out what the system must do (**functional** requirements) and how well it must do it (**non-functional** requirements: performance, security, usability), and records both in a Software Requirements Specification (SRS).



FIGURE 6.5. The seven requirements-engineering tasks (Pressman). Elicitation gathers needs; specification writes them down (the SRS); validation checks that they are complete, consistent and unambiguous.

- **Functional**: what the system does (services). **Non-functional**: quality constraints — performance, security, reliability, usability, portability, maintainability (FURPS+).
- Elicitation techniques: interviews, questionnaires, **JAD** (Joint Application Development), **QFD** (Quality Function Deployment: normal, expected, exciting requirements), brainstorming, use cases, observation, prototyping.
- **SRS** (IEEE 830): characteristics — correct, unambiguous, complete, consistent, ranked, **verifiable**, modifiable, traceable. SRS says **what**, not **how**.
- Requirement traceability matrix. Validation: reviews, prototyping, test-case generation.
- Analysis models: **DFD** (process), **ER** (data), **state diagrams** (behaviour), use case diagrams, data dictionary.

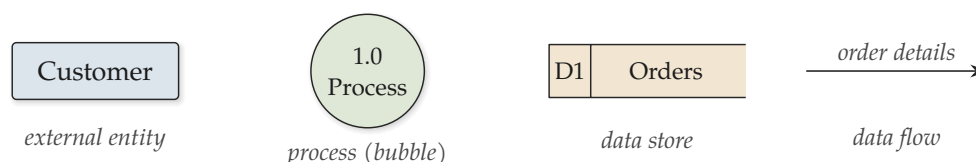


FIGURE 6.6. DFD notation (Yourdon/DeMarco). Every flow is named; data never flows directly between two stores or two external entities without passing through a process.

- A **level-0 DFD** is the *context diagram*: a single process and the external entities only.
- **Balancing**: the inputs and outputs of a child diagram must equal those of its parent process.

6.2.1 UML Diagrams

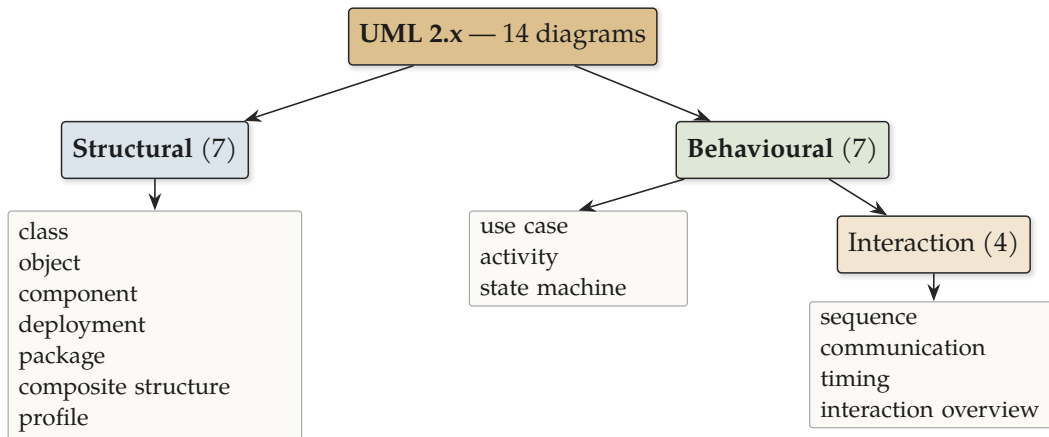


FIGURE 6.7. The fourteen UML 2.x diagrams: seven describe structure, seven describe behaviour (four of them interactions).

Relationships: association, aggregation ($\diamond$ hollow, “has-a”, weak), composition ($\blacklozenge$ filled, strong ownership), generalisation ($\triangle$ “is-a”), dependency (dashed arrow), realisation. Use case relations: «include» (mandatory), «extend» (optional).

6.3 Software Design

Design turns requirements into a structure of modules and interfaces. Its two classic measures are **cohesion** (how closely the parts of one module belong together) and **coupling** (how strongly modules depend on each other): good designs have high cohesion and low coupling.

6.3.1 Design Concepts

Abstraction (procedural, data), architecture, patterns, separation of concerns, **modularity**, **information hiding** (Parnas), **functional independence** (high cohesion + low coupling), stepwise refinement (Wirth), refactoring, aspects.

6.3.2 Cohesion (within a module) — best to worst

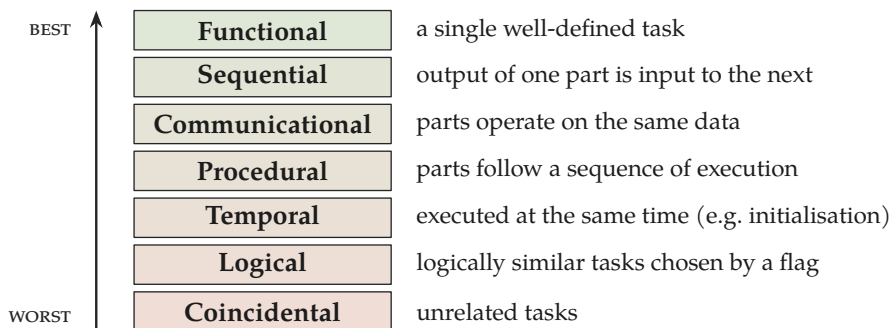


FIGURE 6.8. Cohesion, the strength of the bond *within* a module, from best (top) to worst. Aim for high cohesion.

Mnemonic (worst→best): “**C**oin **L**ogic **T**emp **P**roc **C**omm **S**eq **F**unc”.

6.3.3 Coupling (between modules) — best to worst

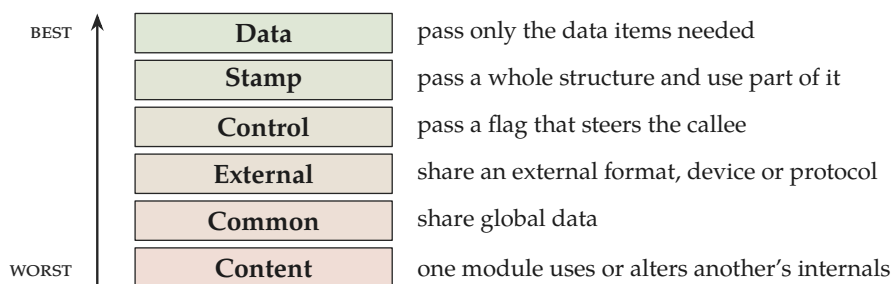


FIGURE 6.9. Coupling, the degree of dependence *between* modules, from best (top) to worst. Aim for low coupling: no coupling at all is better still.

Goal: high cohesion, low coupling.

6.3.4 Architectural Styles

Data-centred (repository/blackboard), data-flow (**pipe & filter**, batch sequential), call-and-return (main/subprogram, remote procedure), object-oriented, layered, client-server, **MVC**, event-driven, microservices, SOA.

6.3.5 Design Patterns (GoF – 23)

Category	Patterns
Creational (5)	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural (7)	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Behavioural (11)	Observer, Strategy, Command, Iterator, State, Template Method, Visitor, Mediator, Memento, Chain of Responsibility, Interpreter

6.3.6 UI Design

Golden rules (Mandel): place user in control, reduce memory load, make interface consistent. Usability heuristics (Nielsen).

6.4 Software Quality

Quality means conformance to requirements and fitness for use. Quality models such as McCall's and ISO 9126 break this into measurable factors; process standards such as ISO 9001 and CMMI judge the organisation that builds the software.

6.4.1 McCall's Quality Factors (1977) — 11 factors in 3 perspectives

ISO 9126 (6 characteristics): **Functionality, Reliability, Usability, Efficiency, Maintainability, Portability** (FRUEMP). Replaced by **ISO/IEC 25010** (8: + Security, Compatibility; "functional suitability", "performance efficiency").

- **Quality control (QC)**: inspections, reviews, tests — product-oriented, detect defects.

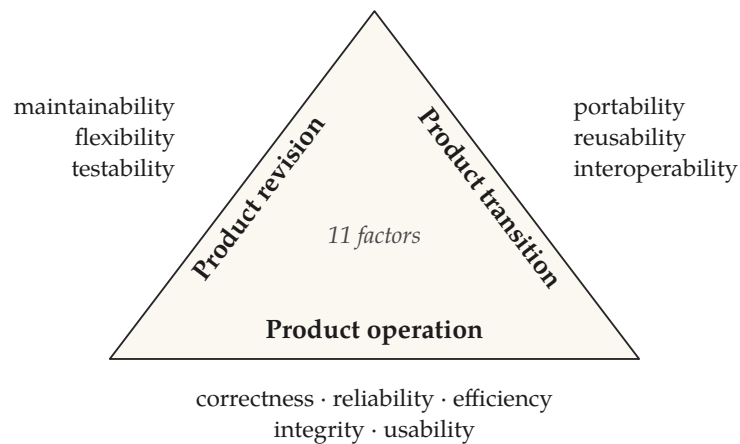


FIGURE 6.10. McCall's quality triangle (1977): eleven factors grouped by the three ways a product is used — operated, revised and moved to new environments.

- **Quality assurance (QA)**: auditing & reporting — process-oriented, prevent defects.
- **SQA activities**: FTR (Formal Technical Reviews — walkthrough, inspection: 3–5 people, ≤ 2 hours prep, ≤ 2 hours meeting), standards, audits.
- **Six Sigma**: 3.4 defects per million opportunities; DMAIC (Define, Measure, Analyse, Improve, Control).
- **CMMI levels**: 1 Initial → 2 Managed (Repeatable) → 3 Defined → 4 Quantitatively Managed → 5 Optimising.
- ISO 9001 — QMS standard applicable to software.

6.4.2 Reliability

Mean time between failures	$MTBF = MTTF + MTTR$
Availability	$A = \frac{MTTF}{MTTF + MTTR} \times 100\%$
Reliability with constant failure rate λ	$R(t) = e^{-\lambda t}$
Rate of occurrence of failure (ROCOF)	failures per unit time
Probability of failure on demand (POFOD)	$\Pr(\text{failure} \mid \text{demand})$

6.4.3 Risk Management

- **Reactive** (fire-fighting) vs **Proactive**.
- Risk categories: project, technical, business; known, predictable, unpredictable.
- **Risk exposure** $RE = P \times C$ (probability $\times$ cost/impact).
- Risk table → **RMMM**: Risk **Mitigation** (avoid), **Monitoring**, **Management** (contingency).

6.5 Estimation

Before a project starts, managers must estimate its size, effort and duration. Size is measured in lines of code or function points; empirical models such as COCOMO then turn size into effort and schedule.

6.5.1 LOC & Function Points

Function Point (Albrecht, 1979):

Function points	$FP = UFP \times VAF$
Value adjustment factor (14 general system characteristics, each rated 0–5)	$VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$
Range of VAF	$0.65 \leq VAF \leq 1.35$

Parameter	Simple	Average	Complex
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

Worked: EI = 10 (avg), EO = 8 (avg), EQ = 5 (avg), ILF = 4 (avg), EIF = 2 (avg); $\Sigma F_i = 42$.

$$UFP = 10 \times 4 + 8 \times 5 + 5 \times 4 + 4 \times 10 + 2 \times 7 = 40 + 40 + 20 + 40 + 14 = 154$$

$$VAF = 0.65 + 0.01 \times 42 = 1.07$$

$$FP = 154 \times 1.07 = \mathbf{164.78}$$

6.5.2 COCOMO (Boehm, 1981)

Effort (person-months)	$E = a (KLOC)^b$
Development time (months)	$D = c E^d$
Average staff	$Staff = E/D$

Mode	a	b	c	d	Description
Organic	2.4	1.05	2.5	0.38	Small team, familiar, < 50 KLOC
Semi-detached	3.0	1.12	2.5	0.35	Medium, mixed experience
Embedded	3.6	1.20	2.5	0.32	Tight constraints, hardware

Worked: Organic, 32 KLOC.

$$E = 2.4 \times 32^{1.05} \approx 2.4 \times 38.06 \approx 91 \text{ PM}$$

$$D = 2.5 \times 91^{0.38} \approx 2.5 \times 5.55 \approx 13.9 \text{ months}$$

$$\text{Staff} \approx 91/13.9 \approx 6.5 \text{ persons}$$

- **Intermediate COCOMO:** $E = a \cdot \text{KLOC}^b \times \text{EAF}$ (product of **15 cost drivers** in 4 groups: product, hardware/computer, personnel, project). Coefficients a: 3.2, 3.0, 2.8.
- **Detailed COCOMO:** phase-sensitive effort multipliers.
- **COCOMO II:** Application composition (object points), Early design, Post-architecture models; scale factors.
- **Putnam/SLIM model:** Rayleigh curve; $L = Ck \cdot K^{(1/3)} \cdot td^{(4/3)}$.
- **Brooks' law:** "Adding manpower to a late project makes it later." Communication paths among n people $= n(n-1)/2$.

6.5.3 Scheduling

- **WBS** (work breakdown structure), **Gantt / timeline charts**, **PERT/CPM** (critical path; see Unit 1).
- **Earned Value Analysis:** BCWS (planned value), BCWP (earned value), ACWP (actual cost).
 - $\text{SPI} = \text{BCWP}/\text{BCWS}$; $\text{CPI} = \text{BCWP}/\text{ACWP}$; $\text{SV} = \text{BCWP} - \text{BCWS}$; $\text{CV} = \text{BCWP} - \text{ACWP}$.
 $\text{SPI} < 1 \rightarrow$ behind schedule.
- 40-20-40 rule: 40% analysis & design, 20% coding, 40% testing.

6.6 Software Testing

Testing executes a program in order to find faults. It can show the presence of bugs but never their absence (Dijkstra), so test cases are chosen systematically, from the specification (black-box) or from the code (white-box).

6.6.1 Terminology

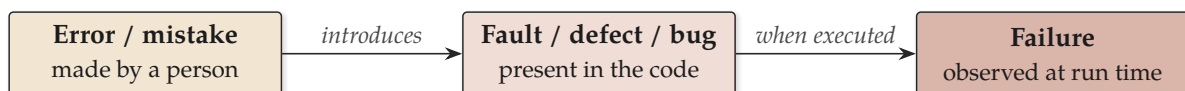


FIGURE 6.11. Error, fault and failure. Not every fault causes a failure: the faulty code must be executed with data that exposes it.

- **Verification:** "Are we building the product right?" (static — reviews, inspections).
- **Validation:** "Are we building the right product?" (dynamic — testing against user needs).
- Testing shows the **presence** of bugs, not their absence (Dijkstra).

6.6.2 Levels of Testing

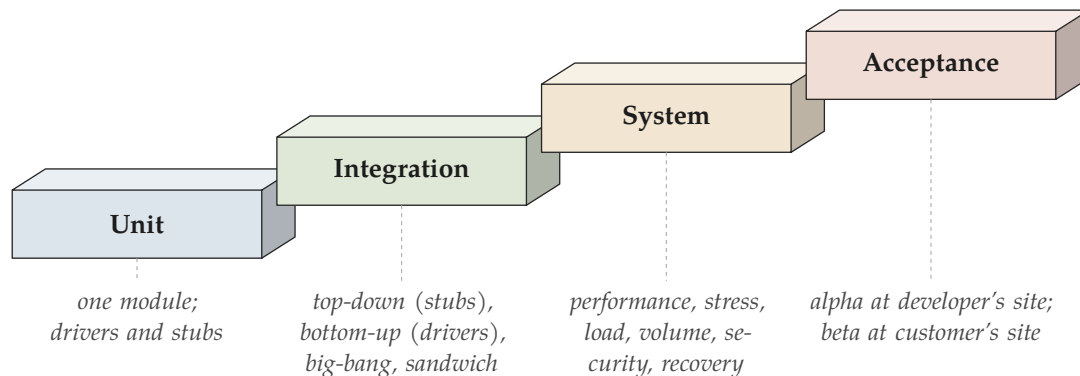


FIGURE 6.12. Levels of testing. Each level tests a larger assembly than the one before, ending with the customer's acceptance.

- **Stub** = dummy called module (top-down). **Driver** = dummy calling module (bottom-up).
- **Regression testing**: re-run tests after changes. **Smoke testing**: build verification. **Sanity**: narrow check after fix.
- **Alpha**: by customers at developer's site (controlled). **Beta**: by end users at their site (uncontrolled).
- Stress (beyond limits), load (expected load), volume (large data), performance, recovery, security testing.

6.6.3 Black-box vs White-box

Black-box (functional, behavioural)	White-box (structural, glass-box)
Equivalence partitioning	Statement coverage
Boundary value analysis	Branch/decision coverage
Cause-effect graphing / decision tables	Condition, MC/DC coverage
State transition testing	Basis path testing (cyclomatic complexity)
Orthogonal array testing	Data-flow testing, loop testing
Error guessing	Mutation testing

- **Boundary value analysis** for range $[1, 100]$: test 0, 1, 2, 99, 100, 101 (robust) or 1, 2, 50, 99, 100 (normal: $4n + 1$ for n variables).
- **Equivalence classes** for $[1, 100]$: one valid (1–100), two invalid (< 1 , > 100).
- Coverage strength: Statement $<$ Branch $<$ Condition/Decision $<$ MC/DC $<$ Multiple condition $<$ Path.

6.6.4 Cyclomatic Complexity (McCabe, 1976)

From the flow graph (E edges, N nodes, P connected components, usually 1)	$V(G) = E - N + 2P$
From decisions	$V(G) = \text{predicate nodes} + 1$
From the drawing (regions of the planar graph, outer region included)	$V(G) = \text{number of regions}$
Meaning	independent paths = minimum basis-path test cases

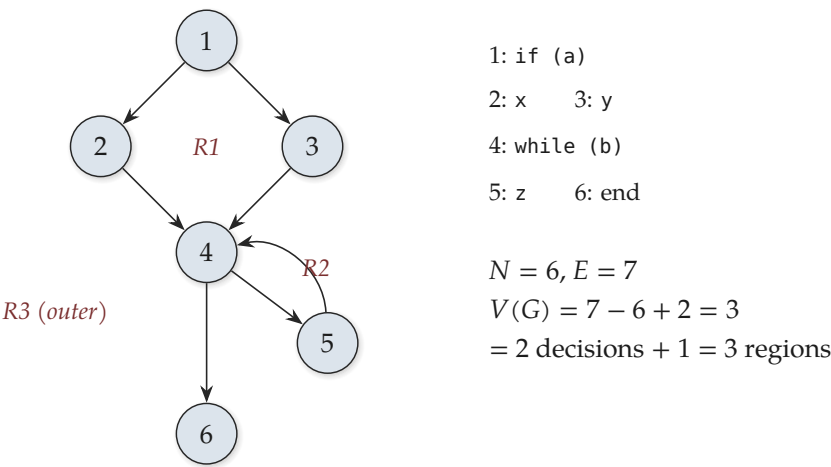


FIGURE 6.13. Flow graph for if (a) {x} else {y}; while (b) {z}. All three formulas give $V(G) = 3$, so three test cases cover a basis set of paths.

$V(G) \leq 10$ recommended.

6.7 Software Configuration Management

- **SCM** activities: identification, version control, change control, configuration auditing, status reporting.
- **Baseline**: formally reviewed & agreed work product; changes only via formal change control (Change Control Board / Authority).
- SCIs (Software Configuration Items). Tools: Git, SVN, CVS.

6.8 Maintenance & Re-engineering

Most of the cost of software is spent after delivery. Maintenance fixes faults, adapts the program to new environments, adds features and improves its structure; when a system becomes too hard to change it is re-engineered.

Maintenance type	Share (approx.)	Purpose
Corrective	~20%	Fix bugs
Adaptive	~25%	Adapt to new environment (OS, hardware)
Perfective	~50% (largest)	New features, performance
Preventive	~5%	Improve future maintainability (refactoring)

- **Lehman's laws** of software evolution (continuing change, increasing complexity...).
- **Re-engineering**: inventory analysis → document restructuring → **reverse engineering** → code restructuring → data restructuring → **forward engineering**.
- **Reverse engineering**: extracting design/specification from code (abstraction level ↑). Forward engineering: design → code.
- **Restructuring**: change form without changing functionality.
- Software maintenance cost is typically **60–80%** of total lifecycle cost.

6.8.1 Other Metrics

Halstead vocabulary and length (n_1, n_2 distinct operators and operands; N_1, N_2 totals)	$n = n_1 + n_2, \quad N = N_1 + N_2$
---	--------------------------------------

Halstead volume	$V = N \log_2 n$
-----------------	------------------

Halstead difficulty and effort	$D = \frac{n_1}{2} \cdot \frac{N_2}{n_2}, \quad E = D \times V$
--------------------------------	---

Defect density	defects/KLOC
----------------	--------------

Defect removal efficiency (E errors found before delivery, D defects after)	$DRE = \frac{E}{E + D}$
--	-------------------------

CK object-oriented metrics	WMC, DIT, NOC, CBO, RFC, LCOM
----------------------------	-------------------------------

6.9 Deeper Dive — Modelling Examples, Test Design, Estimation & Reliability Models

6.9.1 Level-1 DFD — Library System

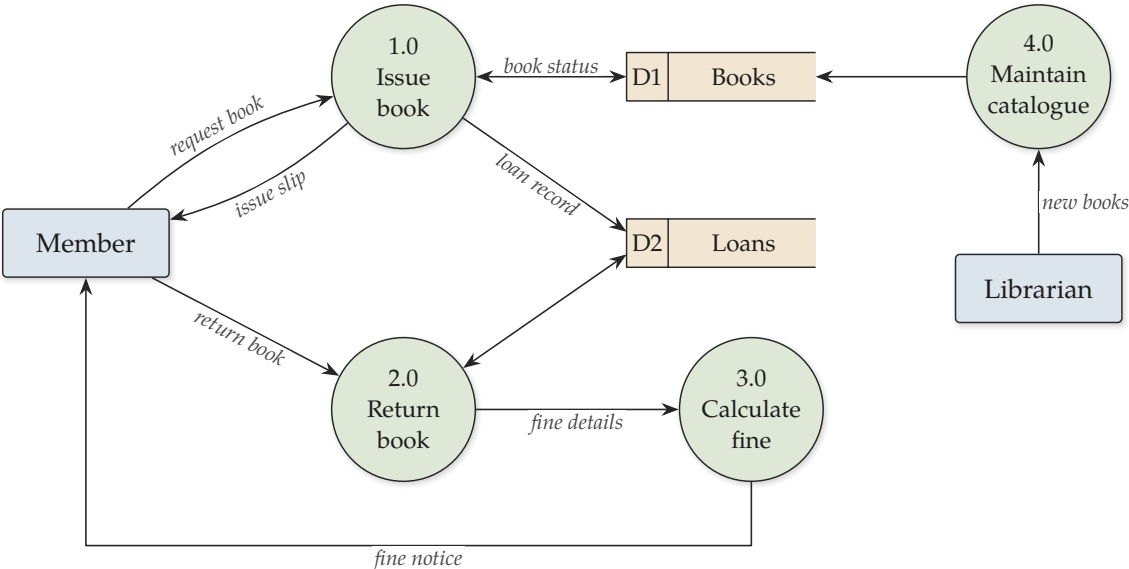


FIGURE 6.14. Level-1 DFD for a library system: four processes, two data stores (open rectangles) and two external entities.

Rules: every process has at least one input and one output; data stores connect only through processes; external entities never connect directly to data stores.

6.9.2 UML Class Diagram

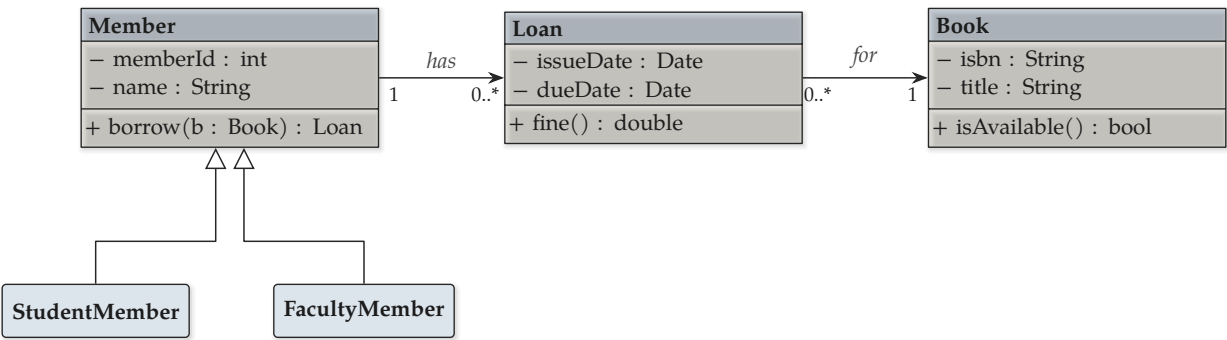


FIGURE 6.15. UML class diagram. Each class box lists name, attributes and operations (– private, + public); hollow triangles show inheritance; numbers at line ends give multiplicity.

Visibility: + public, - private, # protected, ~ package. Multiplicity: 1, 0..1, 0.., 1...

6.9.3 UML Sequence Diagram — ATM Withdrawal

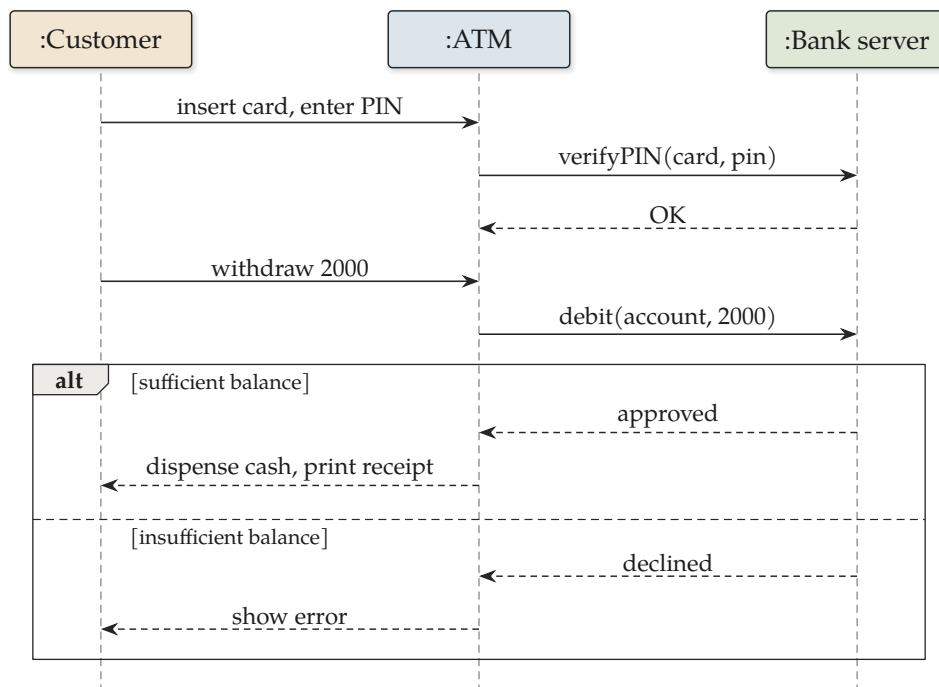


FIGURE 6.16. UML sequence diagram for an ATM withdrawal. Time runs downwards; solid arrows are calls, dashed arrows are replies; the alt fragment shows two mutually exclusive outcomes.

6.9.4 Black-box Test Design — Worked

Function: `eligible(age, grade)` where $\text{age} \in [18, 60]$ and $\text{grade} \in \{A, B, C\}$.

Technique	Test values
Equivalence classes (age)	valid 18–60 (e.g. 35); invalid < 18 (e.g. 10); invalid > 60 (e.g. 70)
Equivalence classes (grade)	valid {A, B, C}; invalid (e.g. D)
BVA (age, normal)	18, 19, 39, 59, 60
Robust BVA (age)	17, 18, 19, 39, 59, 60, 61
Worst-case BVA (2 variables)	$5^2 = 25$ combinations; robust worst-case $7^2 = 49$

Decision table (login):

Conditions / Actions	R1	R2	R3	R4
Valid user ID?	T	T	F	F
Valid password?	T	F	T	F
Grant access	✓			
Show "wrong password"		✓		
Show "unknown user"			✓	✓

6.9.5 Intermediate COCOMO — Worked

Semi-detached project, 50 KLOC, EAF = 1.2 (product of the 15 cost-driver multipliers).

Intermediate coefficients a : organic 3.2, semi-detached 3.0, embedded 2.8 (b as in basic COCOMO).

$$E = 3.0 \times 50^{1.12} \times 1.2 \approx 3.0 \times 80 \times 1.2 \approx 288 \text{ person-months}$$

$$D = 2.5 \times 288^{0.35} \approx 2.5 \times 7.26 \approx 18 \text{ months}$$

$$\text{Staff} \approx 288/18 = 16 \text{ persons}$$

Cost drivers include RELY (reliability), CPLX (complexity), ACAP (analyst capability), PCAP, TOOL, SCED (schedule constraint), etc.

6.9.6 Software Reliability Models

Model	Idea
Jelinski–Moranda (1972)	N initial faults; each fix reduces failure rate by a constant φ : $\lambda_i = \varphi(N - i + 1)$
Musa basic execution time	$\lambda(\mu) = \lambda_0 (1 - \mu/\nu_0)$: failure intensity falls linearly with failures experienced
Musa–Okumoto logarithmic	Failure intensity decreases exponentially with failures experienced
Goel–Okumoto (NHPP)	Expected failures $m(t) = a(1 - e^{(-bt)})$
Littlewood–Verrall	Bayesian; failure rates random

Worked (Musa basic): $\lambda_0 = 10$ failures/CPU-hr, $\nu_0 = 100$ total failures. After 50 failures: $\lambda = 10(1 - 50/100) = 5$ failures/CPU-hr.

6.9.7 Risk Table — Example

Risk	Category	Probability	Impact (1–4)	Exposure (₹)	RMMM response
Key developer leaves	Project	0.3	2	$0.3 \times 4 \text{ L} = 1.2 \text{ L}$	Cross-training, documentation
Requirements change late	Business	0.6	2	$0.6 \times 2 \text{ L} = 1.2 \text{ L}$	Agile iterations, change control
New tool unreliable	Technical	0.2	3	$0.2 \times 1 \text{ L} = 0.2 \text{ L}$	Prototype tool early

Risks are sorted by probability $\times$ impact; a **cut-off line** decides which get full RMMM plans.

6.9.8 CMMI Process Areas by Level (examples)

Level	Focus	Example process areas
2 Managed	Basic project management	Requirements management, project planning, configuration management, measurement & analysis, QA
3 Defined	Organisation-wide standard process	Requirements development, technical solution, verification, validation, risk management
4 Quantitatively managed	Statistical control	Organisational process performance, quantitative project management
5 Optimising	Continuous improvement	Causal analysis & resolution, organisational performance management

6.10 Previous Year Questions (PYQ pattern)

Process models

1. Which model is most suitable when requirements are not clear? **Prototyping**
2. The spiral model was proposed by: **Barry Boehm**; its key feature: **risk analysis**
3. In the spiral model, the radial dimension represents: **cumulative cost**
4. RUP phases in order: **Inception, Elaboration, Construction, Transition**
5. Which is NOT an agile method? (a) XP (b) Scrum (c) Waterfall (d) DSDM — **Ans: (c)**
6. Pair programming is a practice of: **Extreme Programming**
7. In Scrum, the time-boxed iteration is called: **Sprint**; daily meeting lasts: **15 minutes**
8. MoSCoW prioritisation is associated with: **DSDM**
9. V-model emphasises: **verification and validation at each phase**

Requirements & design

10. A level-0 DFD is also called: **context diagram**
11. "The system shall respond within 2 seconds" is a: **non-functional requirement**
12. SRS should specify: **what the system should do, not how**
13. Best type of cohesion: **functional**; worst: **coincidental**
14. Best coupling: **data coupling**; worst: **content coupling**
15. Modules sharing global data exhibit: **common coupling**
16. Passing a flag to control another module's logic: **control coupling**
17. Information hiding was proposed by: **David Parnas**
18. Which UML diagram shows the time-ordered interaction of objects? **Sequence diagram**
19. «include» vs «extend»: include is **mandatory**, extend is **optional/conditional**
20. Singleton pattern belongs to: **creational patterns**
21. Filled diamond in UML indicates: **composition**

Quality

22. Which is NOT a McCall product-operation factor? (a) correctness (b) reliability (c) portability (d) usability — **Ans: (c)**
23. ISO 9126 quality characteristics count: **6**
24. CMMI level 5 is: **Optimising**
25. Availability with MTTF = 90 h and MTTR = 10 h: **90%**
26. Risk exposure with probability 0.3 and cost ₹50,000: **₹15,000**
27. RMMM stands for: **Risk Mitigation, Monitoring and Management**
28. QA is ____-oriented while QC is ____-oriented: **process; product**

Estimation

29. Value adjustment factor range: **0.65 to 1.35**
30. Number of general system characteristics in FP: **14**
31. Basic COCOMO organic effort constants: **a = 2.4, b = 1.05**
32. Intermediate COCOMO uses how many cost drivers? **15**
33. "Adding manpower to a late project makes it later" — **Brooks' law**
34. Number of communication paths among 6 team members: **15**
35. SPI = 0.8 indicates the project is: **behind schedule**

Testing

36. Cyclomatic complexity of a graph with 10 edges, 8 nodes: $10 - 8 + 2 = 4$
37. A program has 3 if and 1 while: $V(G) = 4 + 1 = 5$
38. Boundary value testing is a: **black-box technique**
39. Basis path testing is: **white-box**
40. Testing at developer's site by customer: **alpha testing**
41. Re-running tests after modification: **regression testing**
42. In top-down integration, dummy modules are: **stubs**
43. "Are we building the product right?" — **Verification**
44. Mutation testing evaluates: **the quality of test cases**
45. Number of test cases in BVA (normal) for 2 variables: $4n + 1 = 9$
46. DRE when 90 errors found before release and 10 after: **0.9**

Maintenance / SCM

47. Maintenance to accommodate a new OS: **adaptive**
48. Largest share of maintenance effort: **perfective**
49. Extracting design from source code: **reverse engineering**
50. A formally reviewed work product serving as basis for further development: **baseline**

More practice questions

51. In a DFD, a data store can connect directly to: **a process only**
52. Number of test cases in robust BVA for one variable: **7** ($6n + 1$ for n variables)
53. Worst-case BVA for 3 variables: $5^3 = 125$
54. Intermediate COCOMO coefficient a for embedded mode: **2.8**
55. In UML, # before an attribute means: **protected**

56. A hollow triangle arrowhead in a class diagram denotes: **generalisation (inheritance)**
57. In Musa's basic model, failure intensity decreases: **linearly with the number of failures experienced**
58. Goel–Okumoto is an example of: **NHPP (non-homogeneous Poisson process) model**
59. Risk management process area is introduced at CMMI level: **3**
60. A decision table with 3 Boolean conditions has at most: **8 rules**
61. Which UML diagram best shows the order of messages between objects? **Sequence diagram**

POINTS TO REMEMBER

- Unclear req → Prototype · Risk → Spiral · Changing req → Agile
- Cohesion: Functional best, Coincidental worst · Coupling: Data best, Content worst
- FP: $VAF = 0.65 + 0.01\Sigma F$ (0.65–1.35), 14 GSCs · COCOMO organic 2.4/1.05/2.5/0.38
- $V(G) = E - N + 2 = \text{decisions} + 1$ · BVA $4n+1$
- Alpha at dev site · Beta at customer site · Perfective maintenance largest

Data Structures & Algorithms

...

Expected questions: 10–12 · Target: 9+ · Type: complexity, recurrences, tree/heap/hash numericals, algorithm traces

SYLLABUS COVERAGE

- ☐ Data structures: arrays, sparse matrices, stacks, queues, priority queues, linked lists, trees, binary trees, threaded trees, BST, AVL, B/B+/B* trees, sets (union-find), graphs, sorting, searching, hashing
 - ☐ Performance analysis: time & space complexity, asymptotic notation, recurrence relations
 - ☐ Design techniques: divide & conquer, dynamic programming, greedy, backtracking, branch & bound
 - ☐ Lower bound theory: comparison trees, reductions
 - ☐ Graph algorithms: BFS, DFS, shortest paths, maximum flow, MST
 - ☐ Complexity theory: P, NP, NP-complete, NP-hard, reducibility
 - ☐ Selected topics: number-theoretic algorithms, polynomial arithmetic, FFT, string matching
 - ☐ Advanced: parallel algorithms (sorting, searching, merging), approximation & randomised algorithms
-

7.1 Asymptotic Analysis

Asymptotic analysis describes how an algorithm's running time grows with the input size n , ignoring constant factors and small inputs. It lets us compare algorithms independently of the machine they run on.

O — upper bound	$f(n) \leq c g(n) \quad \text{for all } n \geq n_0$
-------------------	---

Ω — lower bound	$f(n) \geq c g(n) \quad \text{for all } n \geq n_0$
------------------------	---

Θ — tight bound	$c_1 g(n) \leq f(n) \leq c_2 g(n)$
------------------------	------------------------------------

o — strict upper bound	$\lim_{n \rightarrow \infty} f(n)/g(n) = 0$
--------------------------	---

ω — strict lower bound	$\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$
-------------------------------	--

Growth order (memorise):

$$1 < \log \log n < \log n < (\log n)^k < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 3^n < n! < n^n$$

Useful facts: $\log(n!) = \Theta(n \log n)$; $n^{1/\log n} = \Theta(1)$; $2^{\log_2 n} = n$; $n^{\log n}$ is super-polynomial.

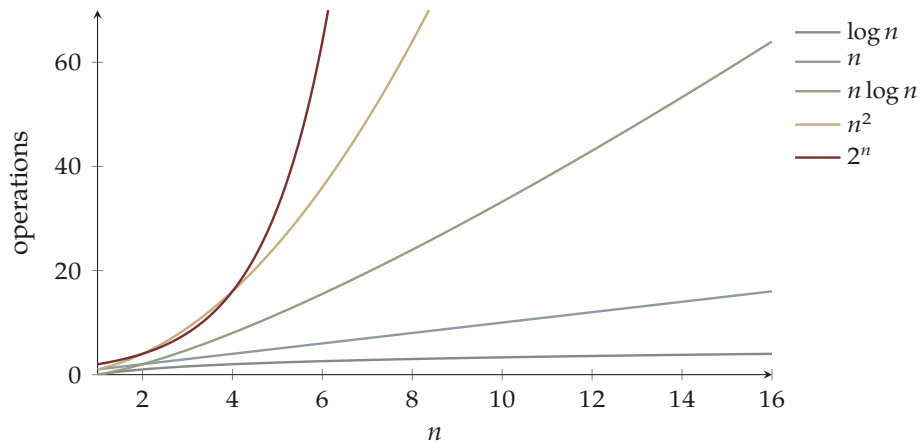


FIGURE 7.1. Growth of common running times. For small n the curves cross; asymptotic order describes only what happens as n grows large.

7.1.1 Recurrences — Master Theorem

$T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$. Compare $f(n)$ with $n^{\log_b a}$:

Case	Condition	Result
1	$f(n) = O(n^{\log_b a - \epsilon})$	$\Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \log^k n)$	$\Theta(n^{\log_b a} \log^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity	$\Theta(f(n))$

Recurrence	Solution	Algorithm
$T(n) = T(n/2) + 1$	$\Theta(\log n)$	Binary search
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	Merge sort
$T(n) = 2T(n/2) + 1$	$\Theta(n)$	Tree traversal, max-min
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Quick sort worst, selection
$T(n) = T(n-1) + 1$	$\Theta(n)$	Linear recursion
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	Tower of Hanoi
$T(n) = 7T(n/2) + n^2$	$\Theta(n^{2.81})$	Strassen
$T(n) = 8T(n/2) + n^2$	$\Theta(n^3)$	Naive D&C matrix multiply
$T(n) = 3T(n/2) + n$	$\Theta(n^{1.585})$	Karatsuba
$T(n) = T(\sqrt{n}) + 1$	$\Theta(\log \log n)$	—
$T(n) = 2T(\sqrt{n}) + \log n$	$\Theta(\log n \cdot \log \log n)$	—
$T(n) = T(n/3) + T(2n/3) + n$	$\Theta(n \log n)$	Recursion tree

7.2 Linear Data Structures

Linear structures arrange items in a sequence. They differ in where items may be inserted and removed: anywhere (array, list), at one end only (stack) or at opposite ends (queue).

7.2.1 Arrays

- Row-major address: $A[i][j] = B + w \cdot [(i - L_1) \cdot N + (j - L_2)]$ where N = number of columns.
- Column-major: $B + w \cdot [(j - L_2) \cdot M + (i - L_1)]$, M = rows.
- Lower triangular $n \times n$ stored row-wise: index of $(i, j) = i(i-1)/2 + j$ (1-based). Total elements $= n(n+1)/2$.
- **Sparse matrix**: triplet (row, col, value) representation; or linked lists.

7.2.2 Stack (LIFO)

- Applications: expression conversion/evaluation, recursion, backtracking, parenthesis matching, undo.
- **Infix $\rightarrow$ Postfix** example: $A + B * C - D / E \rightarrow A B C * + D E / -$.
- Postfix evaluation $6 \ 2 \ 3 \ + \ - \ 3 \ 8 \ 2 \ / \ + \ * \rightarrow 6 - 5 = 1; 8/2 = 4; 3 + 4 = 7; 1 \times 7 = 7$.
- Number of stack permutations of n elements = Catalan number C_n .

7.2.3 Queue (FIFO)

- Circular queue: full when $(rear + 1) \% n == front$ (one slot wasted). Deque, priority queue.
- Queue using two stacks: amortised $O(1)$.

7.2.4 Linked Lists

Singly, doubly, circular. Insert at head $O(1)$; search $O(n)$. Reverse a list: three pointers. Detect cycle: Floyd's tortoise-hare. Middle: slow/fast pointers.

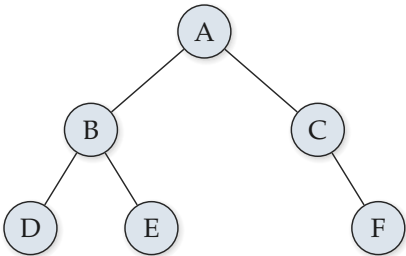
7.3 Trees

A tree stores items hierarchically: one root, and every other node has exactly one parent. Trees give logarithmic search when balanced, and they underlie heaps, indexes and syntax trees.

7.3.1 Binary Tree Facts

Maximum nodes at level i (root at level 0)	2^i
Maximum nodes in a tree of height h (root height 0)	$2^{h+1} - 1$
Minimum height with n nodes	$\lceil \log_2(n + 1) \rceil - 1$
Leaves versus nodes with two children	$n_0 = n_2 + 1$
Null pointers in an n -node binary tree	$n + 1$
Unlabelled binary trees with n nodes (Catalan number)	$C_n = \frac{1}{n + 1} \binom{2n}{n}$
Labelled binary trees with n nodes	$n! \cdot C_n$
Full k -ary tree with i internal nodes	$n = ki + 1, \quad L = (k - 1)i + 1$

7.3.2 Traversals



Order	Sequence
Preorder (NLR)	A B D E C F
Inorder (LNR)	D B E A C F
Postorder (LRN)	D E B F C A
Level order	A B C D E F

FIGURE 7.2. Tree traversals. Preorder visits a node before its subtrees, inorder between them and postorder after them; level order reads the tree row by row.

- Unique tree reconstruction needs **inorder + (preorder or postorder or level-order)**. Preorder + postorder is unique only for full binary trees.
- **Threaded binary tree**: null right pointers point to inorder successor, null left to predecessor → traversal without stack.

7.3.3 Binary Search Tree

- Search/insert/delete: $O(h)$; $h = O(\log n)$ average, $O(n)$ worst (skewed).
- Delete node with 2 children: replace with **inorder successor** (min of right subtree) or predecessor.
- Inorder traversal of BST = **sorted order**.

7.3.4 AVL Tree

Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$.

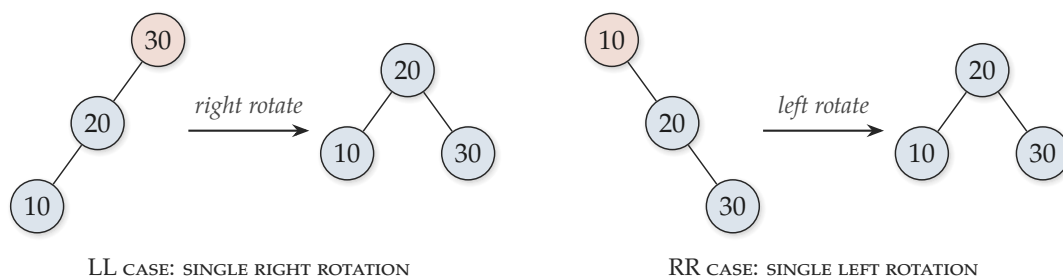


FIGURE 7.3. AVL single rotations; the unbalanced node is shaded. The double rotations are combinations: LR = left-rotate the child, then right-rotate the node; RL = right-rotate the child, then left-rotate the node.

- Min nodes in AVL of height h : $N(h) = N(h-1) + N(h-2) + 1$, $N(0)=1$, $N(1)=2 \rightarrow 1, 2, 4, 7, 12, 20, 33 \dots$
- Max height $\approx 1.44 \log_2 n$. All operations $O(\log n)$.
- **Red-Black tree**: root black, no two consecutive reds, same black-height on every path; height $\leq 2 \log_2(n+1)$.

7.3.5 B-Trees (multi-way, disk-based)

- Order m : each node $\leq m$ children, $\leq m-1$ keys; non-root $\geq \lceil m/2 \rceil$ children; all leaves same level.
- Insertion splits a full node and pushes the median up; tree grows at root.
- **B+ tree**: all data at leaves, leaves linked. **B* tree**: nodes at least $2/3$ full (redistribution before split).
- Details and order calculations: see **DBMS unit**.

7.3.6 Heaps

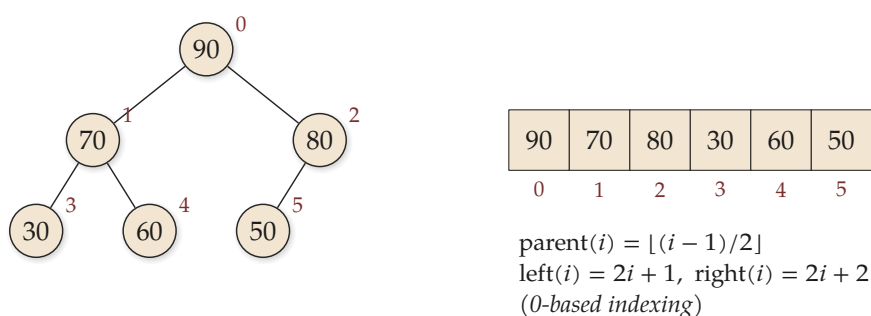


FIGURE 7.4. A max-heap and the array that stores it level by level. Every parent is at least as large as its children, so the maximum sits at index 0.

- Insert $O(\log n)$; delete-max $O(\log n)$; **build-heap** $O(n)$; find-max $O(1)$; search $O(n)$.
- In a max-heap, the minimum is among the leaves ($\lceil n/2 \rceil$ candidates).
- **Binomial heap** (union $O(\log n)$), **Fibonacci heap** (decrease-key $O(1)$ amortised $\rightarrow$ Dijkstra $O(E + V \log V)$).

7.3.7 Disjoint Sets (Union-Find)

Union by rank + path compression → nearly $O(1)$ amortised: $O(\alpha(n))$ (inverse Ackermann).
Used in Kruskal.

7.4 Hashing

Hashing stores a key at an address computed from the key itself, so search, insert and delete take $O(1)$ time on average. The design questions are the hash function and what to do when two keys collide.

Division method (m prime, not close to a power of 2)	$h(k) = k \bmod m$
Multiplication method (Knuth: $A \approx 0.618$)	$h(k) = \lfloor m (kA \bmod 1) \rfloor$
Load factor	$\alpha = n/m$

Collision resolution	Probe sequence	Issues
Chaining	Linked list per slot	Expected search $1 + \alpha$
Linear probing	$h(k) + i$	Primary clustering
Quadratic probing	$h(k) + c_1i + c_2i^2$	Secondary clustering
Double hashing	$h_1(k) + i \cdot h_2(k)$	Best of open addressing

Open addressing expected probes: unsuccessful $\leq 1/(1 - \alpha)$; successful $\leq (1/\alpha) \ln(1/(1-\alpha))$.

Worked (linear probing, $m = 10, h(k) = k \bmod 10$): insert 12, 18, 13, 2, 3, 23, 5, 15

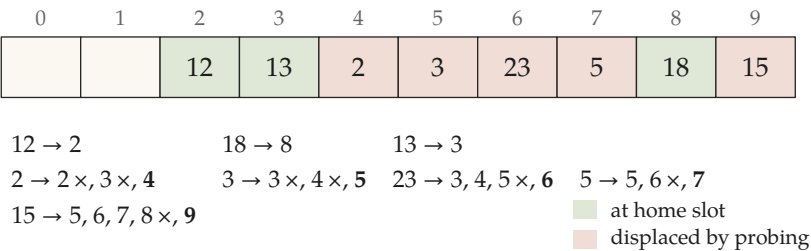


FIGURE 7.5. Linear probing, $m = 10, h(k) = k \bmod 10$, inserting 12, 18, 13, 2, 3, 23, 5, 15. Keys pile up in the run 2–9: this is *primary clustering*.

7.5 Sorting

Sorting questions test three things: time complexity in the best, average and worst cases; whether the sort is stable and in place; and the state of the array after one pass.

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble	n (with flag)	n^2	n^2	1	✓	✓
Selection	n^2	n^2	n^2	1	✗	✓
Insertion	n	n^2	n^2	1	✓	✓
Merge	$n \log n$	$n \log n$	$n \log n$	n	✓	✗
Quick	$n \log n$	$n \log n$	n^2	$\log n$	✗	✓
Heap	$n \log n$	$n \log n$	$n \log n$	1	✗	✓
Shell	$n \log n$	$\sim n^{1.3}$	n^2 (gap-dependent)	1	✗	✓
Counting	$n + k$	$n + k$	$n + k$	k	✓	✗
Radix	$d(n + k)$	$d(n + k)$	$d(n + k)$	$n + k$	✓	✗
Bucket	$n + k$	$n + k$	n^2	n	✓	✗

- Selection sort makes the **minimum number of swaps** ($n - 1$).
- Insertion sort is best for **nearly sorted** data; number of comparisons $\approx$ number of inversions.
- Quick sort worst case on sorted input with first/last element as pivot. Randomised pivot $\rightarrow$ expected $O(n \log n)$.
- **Comparison-sort lower bound:** $\Omega(n \log n)$ (decision tree with $n!$ leaves has height $\geq \log_2 n!$).
- Merging k sorted lists of total n : $O(n \log k)$ with min-heap.

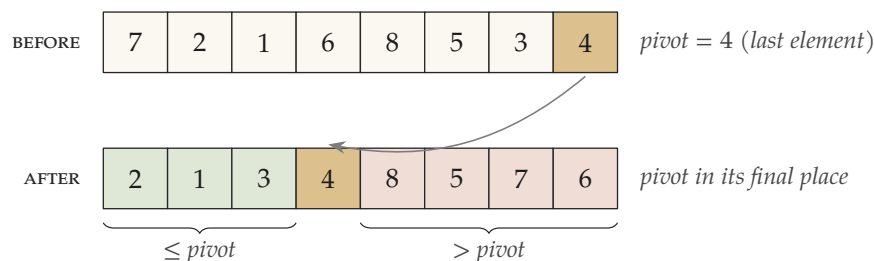


FIGURE 7.6. Lomuto partition. One pass places the pivot in its final position, with smaller keys to its left and larger keys to its right; quicksort then recurses on the two sides.

7.6 Searching

- Linear $O(n)$; **Binary** $O(\log n)$ on sorted array — max comparisons $\lceil \log_2 n \rceil + 1$.
- Interpolation search $O(\log \log n)$ average on uniform data.
- Selection (k -th smallest): Quickselect $O(n)$ average; **median of medians** $O(n)$ **worst**.
- Min & max together: $3n/2 - 2$ comparisons. Second largest: $n + \lceil \log_2 n \rceil - 2$.

7.7 Algorithm Design Techniques

Most algorithms in the syllabus follow one of five strategies. Recognising the strategy tells you the recurrence and therefore the running time.

Divide & conquer merge sort, quicksort, binary search, Strassen, closest pair, Karatsuba	Greedy Kruskal, Prim, Dijkstra, Huffman, fractional knapsack, activity selection, job sequencing	Dynamic programming 0/1 knapsack, LCS, matrix chain, Floyd–Warshall, Bellman–Ford, OBST, TSP (Held–Karp)
Backtracking N-queens, sum of subsets, graph colouring, Hamiltonian cycle, Sudoku	Branch & bound 0/1 knapsack, TSP, job assignment; prunes with bounds, BFS / least-cost search	

FIGURE 7.7. The five algorithm-design paradigms and the standard problems the examination attaches to each.

Greedy	Dynamic programming
Locally optimal choice, never reconsidered	Solves overlapping subproblems, stores results
Greedy-choice property + optimal substructure	Optimal substructure + overlapping subproblems
Faster, not always optimal	Always optimal (if formulated correctly)
Fractional knapsack ✓ ; 0/1 knapsack ✗	0/1 knapsack ✓ $O(nW)$ — pseudo-polynomial

7.7.1 Key DP Problems

LCS of $X = \text{ABCB DAB}$, $Y = \text{BDCABA} \rightarrow$ length 4 (e.g. BCBA). Time $O(mn)$.

$$\text{LCS}[i][j] = \begin{cases} \text{LCS}[i - 1][j - 1] + 1 & \text{if } x_i = y_j \\ \max(\text{LCS}[i - 1][j], \text{LCS}[i][j - 1]) & \text{otherwise} \end{cases}$$

Matrix chain multiplication: dims $10 \times 30, 30 \times 5, 5 \times 60$.

$$\begin{aligned} (AB)C &= 10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 1500 + 3000 = \mathbf{4500} \quad (\text{optimal}) \\ A(BC) &= 30 \cdot 5 \cdot 60 + 10 \cdot 30 \cdot 60 = 9000 + 18000 = 27000 \\ m[i][j] &= \min_{i \leq k < j} (m[i][k] + m[k + 1][j] + p_{i-1} p_k p_j) \quad O(n^3) \end{aligned}$$

The number of ways to parenthesise a product of n matrices is the Catalan number C_{n-1} .

0/1 Knapsack: $K[i][w] = \max(K[i-1][w], v_i + K[i-1][w - w_i])$.

7.7.2 Greedy Problems

- **Huffman coding**: repeatedly merge two least-frequent nodes. $O(n \log n)$. Frequencies a:5, b:9, c:12, d:13, e:16, f:45 → codes f:0, c:100, d:101, a:1100, b:1101, e:111. Average bits = 2.24.
- **Activity selection**: sort by finish time, pick compatible.
- **Job sequencing with deadlines**: sort by profit, place in latest free slot.
- **Fractional knapsack**: sort by value/weight.
- **Optimal merge pattern**: like Huffman.

7.7.3 Backtracking

- N-Queens: 4-Queens has **2** solutions; 8-Queens has **92**.
- State-space tree; prune when constraint violated (bounding function).
- Graph m-colouring, sum of subsets, Hamiltonian cycles.

7.7.4 Branch & Bound

- Uses BFS / **least-cost (LC) search** with bounds; FIFO B&B, LIFO B&B, LC B&B. Used for optimisation (TSP, 0/1 knapsack, assignment).

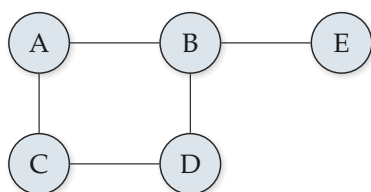
7.8 Graph Algorithms

A graph $G = (V, E)$ models any network of relationships. The standard algorithms traverse it, find shortest paths, build minimum spanning trees and compute maximum flows.

7.8.1 Representations

Adjacency matrix: $O(V^2)$ space, $O(1)$ edge check. Adjacency list: $O(V + E)$.

7.8.2 BFS & DFS



	Order from A	Uses
BFS (queue)	A B C E D	shortest paths in unweighted graphs
DFS (stack)	A B E D C	topological sort, SCCs, cycle detection

FIGURE 7.8. BFS explores level by level; DFS goes as deep as possible before backtracking. Both take $O(V + E)$ with adjacency lists and $O(V^2)$ with a matrix. (The exact order depends on the order in which neighbours are listed.)

- DFS edge types: tree, back (→ cycle), forward, cross. Undirected DFS has only tree and back edges.
- **Topological sort** (DAG only): DFS finishing order reversed, or Kahn's algorithm (in-degree 0).
- **Strongly connected components**: Kosaraju (2 DFS), Tarjan (1 DFS). $O(V + E)$.
- Articulation points & bridges: DFS with low values.

7.8.3 Shortest Paths

Algorithm	Type	Negative edges?	Complexity
BFS	Single-source, unweighted	—	$O(V + E)$
Dijkstra	Single-source	No	$O((V + E) \log V)$ binary heap; $O(V^2)$ array; $O(E + V \log V)$ Fibonacci
Bellman–Ford	Single-source	Yes; detects negative cycles	$O(VE)$
DAG shortest path	Single-source	Yes	$O(V + E)$
Floyd–Warshall	All-pairs (DP)	Yes (no neg. cycles)	$O(V^3)$
Johnson	All-pairs, sparse	Yes	$O(V^2 \log V + VE)$

Dijkstra worked (directed edges: $A \rightarrow B$ 4, $A \rightarrow C$ 1, $C \rightarrow B$ 2, $C \rightarrow D$ 5, $B \rightarrow D$ 1; source A)

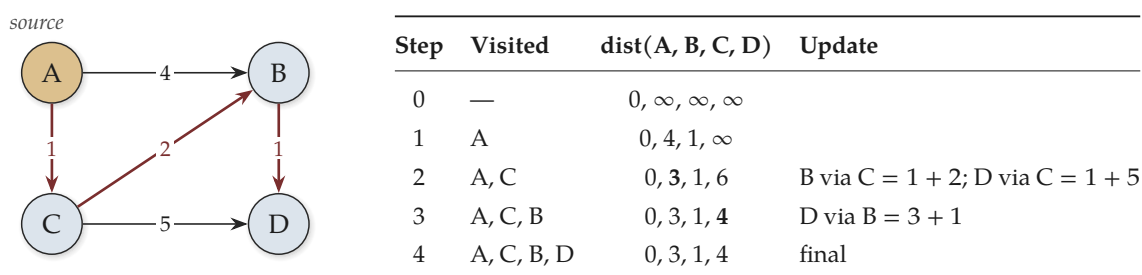


FIGURE 7.9. Dijkstra's algorithm from A. Each step fixes the closest unvisited vertex and relaxes its out-edges; the shortest-path tree is drawn in red.

7.8.4 Minimum Spanning Tree

Kruskal	Prim
Sort edges, add smallest that doesn't form a cycle (union-find)	Grow tree from a vertex, add cheapest edge leaving tree
$O(E \log E)$	$O(E \log V)$ heap; $O(V^2)$ matrix
Better for sparse graphs	Better for dense graphs
Forest during execution	Always a single tree

- MST is unique if all edge weights are distinct.
- **Cut property:** lightest edge across any cut belongs to some MST. **Cycle property:** heaviest edge in a cycle is not in any MST.
- Number of spanning trees of K_n : n^{n-2} .

7.8.5 Maximum Flow

- **Ford–Fulkerson** (augmenting paths): $O(E \cdot f^*)$. **Edmonds–Karp** (BFS augmenting paths): $O(VE^2)$.
- **Max-flow min-cut theorem**: value of max flow = capacity of min s–t cut.
- Bipartite matching reduces to max flow.

7.9 Lower Bound Theory

- **Comparison tree (decision tree)**: sorting needs $\geq \lceil \log_2 n! \rceil = \Omega(n \log n)$ comparisons; searching sorted array $\Omega(\log n)$.
- Merging two sorted lists of size n : $2n - 1$ comparisons (worst). Finding max: $n - 1$ (tournament argument).
- **Reductions**: if A reduces to B and A has lower bound L , B has lower bound L (e.g. sorting $\leq$ convex hull $\rightarrow$ convex hull $\Omega(n \log n)$).

7.10 Complexity Classes

Complexity classes group problems, not algorithms, by the resources the best possible algorithm needs. The open question $P = NP$ asks whether every problem whose solution can be checked quickly can also be solved quickly.

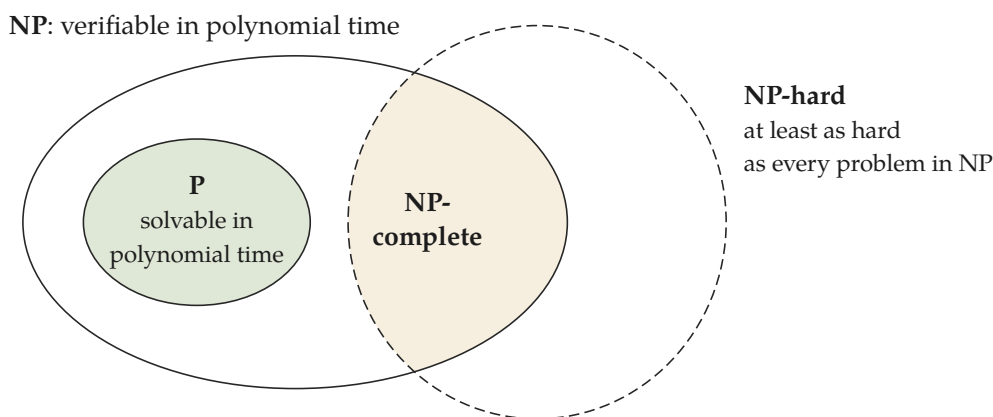


FIGURE 7.10. Complexity classes, assuming $P \neq NP$. NP-complete = $NP \cap NP\text{-hard}$. If any NP-complete problem were solvable in polynomial time, then $P = NP$.

- $P \subseteq NP$ ($P = NP?$ is open). NP-Complete = $NP \cap NP\text{-Hard}$.
- To prove X is NP-complete: (1) $X \in NP$; (2) reduce a known NP-complete problem **to** X ($Y \leq_p X$).
- **Cook–Levin theorem**: SAT is the first NP-complete problem.
- NP-complete: SAT, 3-SAT, CLIQUE, Vertex Cover, Independent Set, Hamiltonian cycle, TSP (decision), Subset sum, 0/1 Knapsack (decision), Graph colouring ($k \geq 3$), Set cover, Partition.
- In P : 2-SAT, shortest path, MST, 2-colouring (bipartite test), Euler circuit, sorting, matching, linear programming, primality (AKS 2002).
- **NP-Hard but not in NP**: Halting problem; optimisation TSP.

- Reductions chain: SAT $\rightarrow$ 3-SAT $\rightarrow$ CLIQUE $\rightarrow$ Vertex Cover $\rightarrow$ Hamiltonian cycle $\rightarrow$ TSP.

7.11 Selected Topics

Topic	Key result
GCD (Euclid)	$\gcd(a, b) = \gcd(b, a \bmod b)$; $O(\log \min(a, b))$; extended Euclid gives x, y with $ax + by = \gcd$
Modular exponentiation	Repeated squaring $O(\log e)$
RSA	$n = pq$, $\varphi = (p-1)(q-1)$, $ed \equiv 1 \bmod \varphi$
Primality	Miller–Rabin (randomised), AKS (deterministic polynomial)
Polynomial multiplication	Naive $O(n^2)$; FFT $O(n \log n)$ (evaluate at roots of unity, pointwise multiply, inverse FFT)
Naive string match	$O((n - m + 1)m)$
Rabin–Karp	Rolling hash; average $O(n + m)$, worst $O(nm)$
KMP	Prefix (failure) function; $O(n + m)$
Boyer–Moore	Bad-character & good-suffix; sub-linear in practice
Finite automaton matcher	$O(n)$ matching after $O(m)$

KMP prefix function for pattern “**ababaca**”: $\pi = [0, 0, 1, 2, 3, 0, 1]$.

7.12 Advanced Algorithms

- **Parallel algorithms** (PRAM models: EREW, CREW, CRCW). Parallel prefix sum $O(\log n)$ with n processors. Odd-even transposition sort $O(n)$ with n processors; bitonic sort $O(\log^2 n)$. Parallel merge $O(\log n)$. Brent's theorem.
- **Approximation algorithms**: Vertex cover — 2-approximation (pick both ends of uncovered edge); Metric TSP — 2-approx (MST doubling), **1.5-approx (Christofides)**; Set cover — $\ln n$ approx (greedy). Approximation ratio $\rho(n)$.
- **Randomised: Las Vegas** (always correct, random time — randomised quicksort) vs **Monte Carlo** (fixed time, may be wrong — Miller–Rabin, Karger's min-cut).

7.13 Deeper Dive — Traces You Must Be Able to Do by Hand

7.13.1 Recursion Tree for $T(n) = 2T(n/2) + n$

For $T(n) = T(n - 1) + n$: unrolling gives $n + (n - 1) + \dots + 1 = n(n + 1)/2 = \Theta(n^2)$.

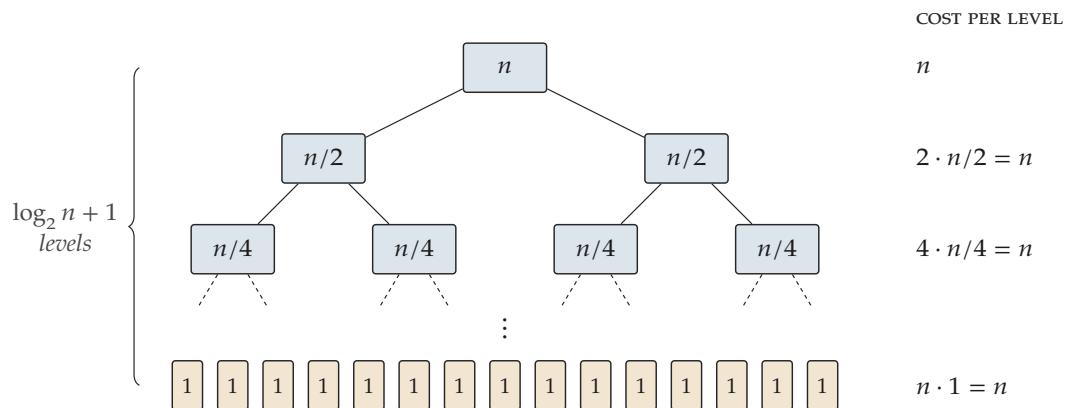


FIGURE 7.11. Recursion tree for $T(n) = 2T(n/2) + n$. Every level costs n and there are $\log_2 n + 1$ levels, so $T(n) = n(\log_2 n + 1) = \Theta(n \log n)$.

7.13.2 AVL Insertion Trace: 10, 20, 30, 40, 50, 25

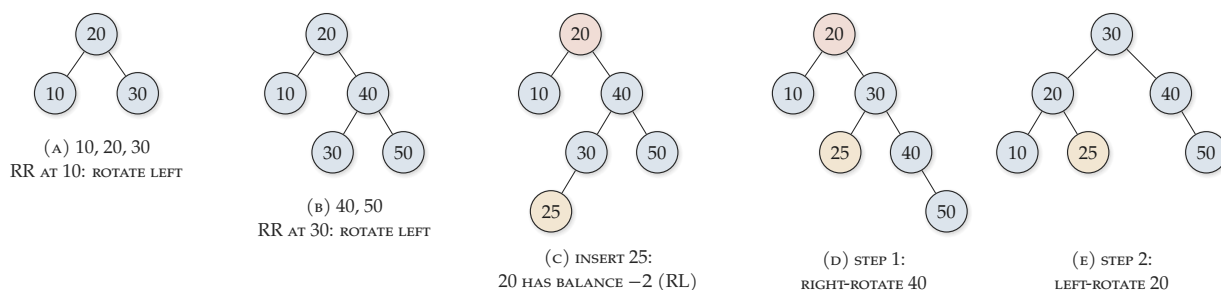


FIGURE 7.12. AVL insertion of 10, 20, 30, 40, 50, 25. The unbalanced node is shaded red, the newly inserted key yellow. The final RL case needs a double rotation.

7.13.3 Heap Sort Trace: [4, 10, 3, 5, 1]

Step	Array	Note
Build heap	$[4, 10, 3, 5, 1] \rightarrow [10, 4, 3, 5, 1] \rightarrow [10, 5, 3, 4, 1]$	heapify index 1, then 0
Swap 10 $\leftrightarrow$ 1, heapify	$[5, 4, 3, 1 10]$	
Swap 5 $\leftrightarrow$ 1, heapify	$[4, 1, 3 5, 10]$	
Swap 4 $\leftrightarrow$ 3, heapify	$[3, 1 4, 5, 10]$	
Swap 3 $\leftrightarrow$ 1	$[1 3, 4, 5, 10]$	sorted

7.13.4 Kruskal vs Prim on the Same Graph

Kruskal (sorted edges)	Decision	Prim (start A)	Edge added
A-C 1	take	{A}	A-C 1
B-C 2	take	{A, C}	B-C 2
D-E 3	take	{A, B, C}	B-D 5
A-B 4	skip (cycle)	{A, B, C, D}	D-E 3
B-D 5	take $\rightarrow$ done		

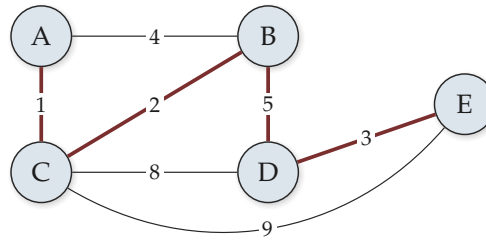


FIGURE 7.13. The minimum spanning tree (thick red) has weight $1 + 2 + 3 + 5 = 11$. Kruskal and Prim add the same edges, in different orders.

Both give MST weight $1 + 2 + 3 + 5 = 11$.

7.13.5 Floyd–Warshall Trace

Directed graph: $1 \rightarrow 2$ (4), $1 \rightarrow 3$ (11), $2 \rightarrow 1$ (6), $2 \rightarrow 3$ (2), $3 \rightarrow 1$ (3).

$$D^0 = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix} \quad D^1 = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad D^2 = \begin{pmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad D^3 = \begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

D^k allows vertices $1..k$ as intermediates: $d_{ij} \leftarrow \min(d_{ij}, d_{ik} + d_{kj})$. Changed entries are in red: $3 \rightarrow 1 \rightarrow 2 = 7$, then $1 \rightarrow 2 \rightarrow 3 = 6$, then $2 \rightarrow 3 \rightarrow 1 = 5$. D^3 is final.

Updates: $D^1[3][2] = 3 + 4 = 7$; $D^2[1][3] = 4 + 2 = 6$; $D^3[2][1] = 2 + 3 = 5$.

7.13.6 0/1 Knapsack DP Table

Capacity $W = 5$; items (weight, value): $(2, 3)$, $(3, 4)$, $(4, 5)$, $(5, 6)$.

$i \setminus w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (2, 3)	0	0	3	3	3	3
2 (3, 4)	0	0	3	4	4	7
3 (4, 5)	0	0	3	4	5	7
4 (5, 6)	0	0	3	4	5	7

Answer 7 (items 1 and 2). Traceback: $K[4][5] = K[3][5] = K[2][5] \neq K[1][5] \rightarrow$ item 2 taken; remaining capacity 2 $\rightarrow$ item 1 taken.

7.13.7 Topological Sort (Kahn's algorithm)

In-degrees: A 0, B 0, C 2, D 1, E 2. Queue A, B $\rightarrow$ remove A (C: 1) $\rightarrow$ remove B (C: 0, D: 0) $\rightarrow$ C $\rightarrow$ D (E: 0) $\rightarrow$ E. One valid order: **A, B, C, D, E** (B, A, D, C, E is also valid — topological orders are not unique).

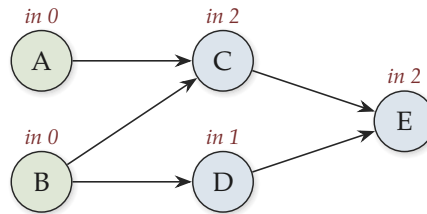


FIGURE 7.14. A DAG for Kahn's algorithm. Vertices with in-degree 0 (green) start the queue; removing a vertex lowers the in-degree of its successors.

7.13.8 Double Hashing

$m = 11$, $h_1(k) = k \bmod 11$, $h_2(k) = 7 - (k \bmod 7)$, probe i : $(h_1 + i \cdot h_2) \bmod 11$. Insert 22, 33 (22 already at 0): $22 \rightarrow 0$. $33 \rightarrow h_1 = 0$ (occupied); $h_2 = 7 - 5 = 2 \rightarrow (0 + 2) \bmod 11 = 2$. h_2 must never be 0 and should be co-prime with m (choose m prime).

7.14 Previous Year Questions (PYQ pattern)

Complexity & recurrences

1. Solution of $T(n) = 2T(n/2) + n \log n$: $\Theta(n \log^2 n)$
2. $T(n) = 4T(n/2) + n$: $\Theta(n^2)$
3. $T(n) = T(n/2) + n$: $\Theta(n)$
4. $T(n) = 3T(n/4) + n \log n$: $\Theta(n \log n)$ (case 3)
5. Which is the slowest-growing? (a) $n \log n$ (b) $n^{1.01}$ (c) $n/\log n$ (d) $\sqrt{n} \log n$ — (d)
6. Time complexity of the loop `for(i=1; i<n; i*=2)`: $O(\log n)$
7. Nested `for(i=1; i<=n; i++) for(j=1; j<=n; j+=i)`: $O(n \log n)$ (harmonic series)

Arrays, stacks, queues

8. $A[1..10][1..15]$, row-major, base 100, 4 bytes/element. Address of $A[5][7]$: $100 + 4[(4)(15) + 6] = 364$
9. Postfix of $(A + B) * (C - D)$: **AB+CD-***
10. Prefix of $A + B * C$: **+A*BC**
11. Minimum stacks to implement a queue: **2**
12. Circular queue of size n holds at most: **$n - 1$** elements (one-slot-empty convention)

Trees

13. A binary tree with 20 leaves has how many nodes of degree 2? **19**
14. Maximum nodes in a binary tree of height 5 (root at height 0): $2^6 - 1 = 63$
15. Number of distinct binary trees with 4 nodes: **14**
16. Inorder: D B E A F C; Preorder: A B D E C F $\rightarrow$ Postorder: **D E B F C A**
17. Minimum nodes in an AVL tree of height 4: **12**
18. In a complete ternary tree with 10 internal nodes, leaves = $2 \times 10 + 1 = 21$
19. Inorder traversal of BST gives: **sorted order**
20. Worst-case height of BST with n nodes: **$n - 1$**

Heaps & hashing

21. Time to build a heap of n elements: **$O(n)$**
22. Array 89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100 — after inserting 100 into max-heap, root = **100**
23. Linear probing suffers from: **primary clustering**
24. With chaining and load factor α , expected unsuccessful search: **$\Theta(1 + \alpha)$**
25. Keys 43, 36, 92, 87, 11, 4, 71, 13, 14 hashed with $h(k) = k \bmod 11$ using chaining. Keys in slot 3: **36 and 14** (43→10, 36→3, 92→4, 87→10, 11→0, 4→4, 71→5, 13→2, 14→3)

Sorting & searching

26. Which sorting algorithm has worst case $O(n \log n)$ and is in-place? **Heap sort**
27. Best algorithm for nearly sorted data: **Insertion sort**
28. Quick sort worst case occurs when: **array is already sorted (pivot = first/last)**
29. Which sorts are stable? **Merge, insertion, bubble, counting, radix**
30. Lower bound for comparison sorting: **$\Omega(n \log n)$**
31. Maximum comparisons in binary search on 1000 elements: $\lfloor \log_2 1000 \rfloor + 1 =$ **10**
32. Min and max of n elements need at least: **$\lceil 3n/2 \rceil - 2$ comparisons**

Design techniques

33. Matrix chain 10×20 , 20×30 , 30×40 : min multiplications = $(AB)C = 6000 + 12000 =$ **18000**
34. LCS of "ABCD" and "ACBD": **3**
35. 0/1 knapsack using DP: **$O(nW)$** (pseudo-polynomial)
36. Huffman coding is an example of: **greedy**
37. Floyd–Warshall uses: **dynamic programming**
38. N-Queens problem is solved by: **backtracking**
39. Which needs both optimal substructure & overlapping subproblems? **Dynamic programming**

Graphs

40. Dijkstra fails with: **negative edge weights**
41. Bellman-Ford time complexity: **$O(VE)$**
42. Kruskal's time complexity: **$O(E \log E)$**
43. BFS is used to find: **shortest path in unweighted graph**
44. Topological sort is possible only for: **DAG**
45. Number of edges in the MST of a connected graph with n vertices: **$n - 1$**
46. Back edge in DFS of a directed graph indicates: **a cycle**
47. Max-flow equals: **min-cut capacity**

Complexity theory

48. First problem proved NP-complete: **SAT (Cook–Levin)**
49. Which is in P? (a) 3-SAT (b) 2-SAT (c) Clique (d) Hamiltonian cycle — **(b)**
50. If $A \leq_p B$ and $B \in P$ then: **$A \in P$**
51. If an NP-complete problem is solved in polynomial time: **$P = NP$**
52. Halting problem is: **undecidable, NP-hard but not in NP**

Selected topics

53. KMP string matching time: $O(n + m)$
 54. FFT multiplies two polynomials of degree n in: $O(n \log n)$
 55. Randomised quicksort is a: **Las Vegas algorithm**
 56. Christofides algorithm gives approximation ratio: **1.5** (metric TSP)

More practice questions

57. After inserting 10, 20, 30 into an empty AVL tree, the root is: **20**
 58. Number of rotations needed for the RL case: **2**
 59. Build max-heap from [4, 10, 3, 5, 1]; the resulting array: [**10, 5, 3, 4, 1**]
 60. MST weight for the graph in §13.4: **11**
 61. Floyd–Warshall final distance from 2 to 1 in §13.5: **5**
 62. 0/1 knapsack $W = 5$ with items (2, 3), (3, 4), (4, 5), (5, 6): **7**
 63. Which is NOT a valid topological order of §13.7? (a) A B C D E (b) B A D C E (c) A C B D E (d) B D A C E — **Ans: (c)** (C before B violates $B \rightarrow C$)
 64. $T(n) = T(n - 1) + n$ solves to: **$\Theta(n^2)$**
 65. In double hashing, $h_2(k)$ must never evaluate to: **0**
 66. Kruskal's algorithm uses which data structure to detect cycles? **Disjoint-set (union–find)**

POINTS TO REMEMBER

- Master theorem: compare $f(n)$ with $n^{\log_b a}$
- $n_0 = n_2 + 1$ · Catalan 1, 1, 2, 5, 14, 42 · AVL min nodes 1, 2, 4, 7, 12, 20
- Build-heap $O(n)$ · Comparison sort $\Omega(n \log n)$
- Dijkstra no negatives · Bellman–Ford $O(VE)$ · Floyd $O(V^3)$
- SAT first NPC · 2-SAT in P · KMP $O(n + m)$

Theory of Computation & Compilers

— . . . —

Expected questions: 10–12 · Target: 9+ · Type: closure/decidability facts, automata construction, parsing tables — the differentiator unit for 250+

SYLLABUS COVERAGE

- Formal languages, non-computational problems, diagonal argument, Russell's paradox
 - Regular languages: DFA, NFA, ϵ -NFA, equivalence, regular expressions, regular grammars, pumping lemma, closure & decision properties, Mealy & Moore machines, minimisation
 - Context-free languages: CFG, derivations, parse trees, ambiguity, CNF, GNF, PDA (DPDA/NPDA), pumping lemma, closure & decision properties, CYK
 - Turing machines: variants, universal TM, Church–Turing thesis, recursive & RE languages, CSL & LBA, unrestricted grammars, Chomsky hierarchy
 - Undecidability: halting problem, PCP, undecidable CFL problems, Rice's theorem, complexity classes
 - Syntax analysis: precedence, associativity, left recursion & factoring, recursive descent, LL(1), LR(0), SLR(1), LALR(1), CLR(1)
 - Semantic analysis: SDD, synthesized & inherited attributes, S- and L-attributed, dependency graphs, type checking
 - Run-time environment: storage organisation, activation records, parameter passing, symbol table
 - Intermediate code: three-address code, quadruples, triples, DAG, translation of expressions, Booleans, control flow
 - Code generation & optimisation: basic blocks, flow graphs, data-flow analysis, local/global/loop/peephole optimisation, register allocation, instruction scheduling
-

8.1 Part A — Theory of Computation

8.2 Basics

- Alphabet Σ (finite set of symbols), string, language $L \subseteq \Sigma^*$. $|\Sigma^*|$ is countably infinite; the set of all languages 2^{Σ^*} is **uncountable**.
- **Diagonal argument (Cantor)**: proves uncountability of reals / power sets; used to show existence of non-RE languages (L_d = diagonal language).
- Since TMs are countable but languages are uncountable, **most languages are not even recursively enumerable**.
- **Russell's paradox**: the set of all sets that do not contain themselves — self-reference leads to contradiction (basis of diagonalisation).

8.3 Chomsky Hierarchy

Noam Chomsky (1956) classified grammars by the form their rules may take. Each type of grammar corresponds exactly to a type of machine, and the questions in this unit nearly always come down to placing a language in the right class.

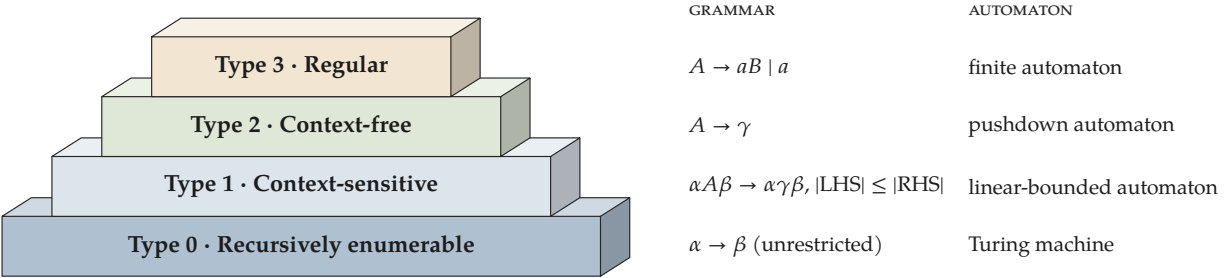


FIGURE 8.1. The Chomsky hierarchy. Each class contains the one above it; the restriction on grammar rules and the power of the matching machine grow together.

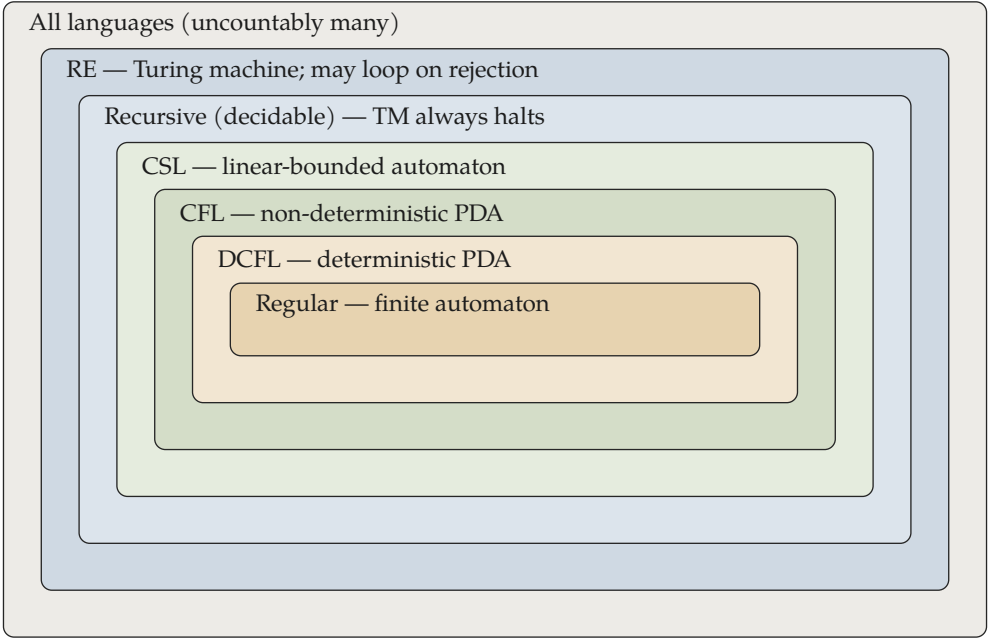


FIGURE 8.2. Language classes, each properly contained in the next. Regular, DCFL, CSL and recursive languages are closed under complement; CFL and RE are not.

8.4 Finite Automata

A finite automaton reads its input once, left to right, remembering only which of finitely many states it is in. That fixed memory is exactly what limits it to the regular languages.

8.4.1 DFA vs NFA

- DFA: $\delta: Q \times \Sigma \rightarrow Q$. NFA: $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$.
- **DFA $\equiv$ NFA $\equiv$ ϵ -NFA $\equiv$ Regular expressions $\equiv$ Regular grammars** in power.
- Subset construction: n -state NFA $\rightarrow$ DFA with up to 2^n states.

Example: DFA for binary strings ending in "01".

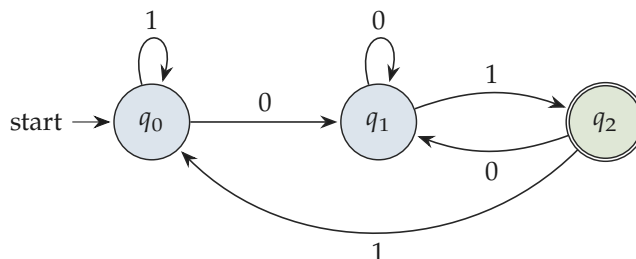


FIGURE 8.3. DFA for binary strings ending in 01. State q_1 means "last symbol was 0", q_2 means "last two symbols were 01".

Example: DFA for binary numbers divisible by 3 (state = remainder so far; reading bit b takes remainder i to $(2i + b) \bmod 3$).

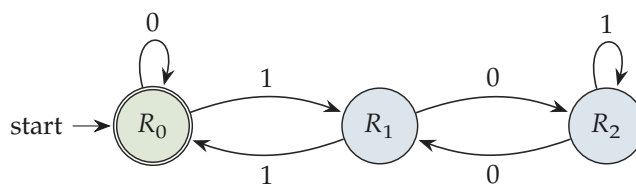


FIGURE 8.4. DFA for binary numbers divisible by 3. State R_i means "value so far $\equiv i \pmod{3}$ "; reading bit b moves from R_i to $R_{(2i+b) \bmod 3}$.

8.4.2 Counting minimum DFA states (common shortcuts)

Language over $\{a, b\}$	Min DFA states
Strings of length exactly n	$n + 2$
Length $\geq n$	$n + 1$
Length $\leq n$	$n + 2$
Strings where length $\equiv 0 \pmod{k}$	k
Strings ending with a given string of length n	$n + 1$
Strings containing a given substring of length n	$n + 1$
Strings starting with a given string of length n	$n + 2$
Number of a 's $\equiv 0 \pmod{m}$ AND b 's $\equiv 0 \pmod{n}$	$m \cdot n$
Binary numbers divisible by n (n odd)	n
n -th symbol from the end is ' a ' (DFA)	2^n (NFA: $n + 1$)

8.4.3 Minimisation (Myhill–Nerode / partition method)

1. Remove unreachable states.
 2. Partition into {final}, {non-final}.
 3. Repeatedly split groups whose members go to different groups on some symbol.
 4. Merge equivalent states.
- Myhill–Nerode:** L is regular $\Leftrightarrow$ the number of equivalence classes of $\equiv_L$ is finite; that number = states of minimal DFA.

8.4.4 Mealy & Moore

Moore	Mealy
Output depends on state only	Output depends on state + input
Output length = input length + 1	Output length = input length
More states usually	Fewer states
- Mealy $\rightarrow$ Moore may need up to	Q

8.5 Regular Expressions

- Operators: union (+), concatenation, Kleene star (*). Precedence: $*$ > concatenation > +.
- **Arden's theorem:** $R = Q + RP \Rightarrow R = QP^*$ (if P doesn't contain ϵ).
- Identities: $(a + b)^* = (ab)^* = (a^* + b^*)$; $(ab)a = a(ba)$; $\epsilon = \epsilon$; $\emptyset^* = \epsilon$; $R + R^* = R^*$; $(R^*)^* = R^*$.

Language	RE
All strings over {a,b}	$(a + b)^*$
Contains "aa"	$(a + b)aa(a + b)$
Even length	$((a + b)(a + b))^*$
Starts and ends with same symbol	$a(a+b)^*a + b(a+b)^*b + a + b$
No two consecutive a's	$(b + ab)^*(\epsilon + a)$
Exactly one 'a'	bab

8.5.1 Pumping Lemma (Regular)

If L is regular, $\exists p$ such that every $w \in L$ with $|w| \geq p$ can be split $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L$ for all $i \geq 0$. Used to prove **non-regularity** (it is a necessary, not sufficient condition). Non-regular: $a^n b^n$, ww^R , palindromes, $a^{(n^2)}$, a^p (p prime), balanced parentheses, $a^n b^m$ ($n < m$). Regular: $a^n b^m$ ($n, m \geq 0$), $(ab)^n$, $a^{(2n)}$, strings with equal number of "ab" and "ba" substrings, $a^n b^n$ for $n \leq 100$ (finite).

8.6 Context-Free Languages

Adding a stack to a finite automaton gives it unbounded memory of a restricted, last-in-first-out kind. That is enough to match nested structures such as brackets and $a^n b^n$, the backbone of every programming-language syntax.

8.6.1 Grammars & Ambiguity

- **Ambiguous** grammar: some string has ≥ 2 parse trees (leftmost derivations). $E \rightarrow E + E \mid E * E \mid \text{id}$ is ambiguous.
- **Inherently ambiguous language**: every grammar ambiguous, e.g. $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$.
- Checking ambiguity of a CFG is **undecidable**.
- Remove ambiguity by encoding precedence & associativity:

$$\begin{aligned} E &\rightarrow E + T \mid T && + \text{ left-associative, lower precedence} \\ T &\rightarrow T * F \mid F && * \text{ higher precedence} \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

8.6.2 Normal Forms

- **CNF**: $A \rightarrow BC \mid a$. A string of length n needs exactly $2n - 1$ derivation steps; parse tree height $\geq \lceil \log_2 n \rceil + 1$.
- **GNF**: $A \rightarrow a\alpha$ ($\alpha \in V^*$). Derivation of length- n string takes exactly n steps.
- Simplification order: remove ϵ -productions $\rightarrow$ unit productions $\rightarrow$ useless symbols.

8.6.3 PDA

- PDA = FA + stack. Acceptance by final state $\equiv$ by empty stack (for NPDA).
- **NPDA** $\not\supseteq$ **DPDA** (unlike FA). ww^R needs NPDA; wcw^R is DCFL.
- DCFLs are unambiguous; every LR(k) grammar generates a DCFL.

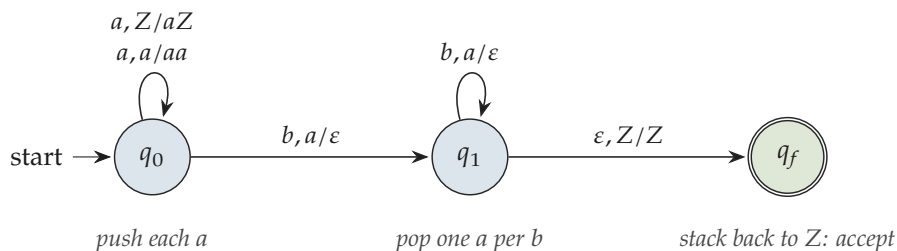


FIGURE 8.5. PDA for $\{a^n b^n \mid n \geq 1\}$. An edge label $x, Y/\gamma$ means: read x , pop Y , push γ .

8.6.4 Pumping Lemma (CFL)

$w = uvxyz$, $|vxy| \leq p$, $|vy| \geq 1$, $uv^i xy^i z \in L \forall i \geq 0$. Non-CFL: $a^n b^n c^n$, ww , $a^{(n^2)}$, a^p (prime), $a^n b^m c^n d^m$. CFL (non-regular): $a^n b^n$, ww^R , $a^n b^m c^m$, $a^n b^n c^m$, $a^i b^j c^k$ with $i = j$ or $j = k$.

8.6.5 CYK Algorithm

Membership for CNF grammar in $O(n^3 \cdot |G|)$; dynamic programming over substrings.

8.7 Closure & Decision Properties (MOST ASKED TABLE)

Operation	Regular	DCFL	CFL	CSL	Recursive	RE
Union	✓	✗	✓	✓	✓	✓
Intersection	✓	✗	✗	✓	✓	✓
Complement	✓	✓	✗	✓	✓	✗
Concatenation	✓	✗	✓	✓	✓	✓
Kleene star	✓	✗	✓	✓	✓	✓
Reversal	✓	✗	✓	✓	✓	✓
Homomorphism	✓	✗	✓	✗	✗	✓
Inverse homomorphism	✓	✓	✓	✓	✓	✓
Intersection with regular	✓	✓	✓	✓	✓	✓
Difference	✓	✗	✗	✓	✓	✗

Notes: $CFL \cap Regular = CFL$. $DCFL \cup Regular$, $DCFL \cap Regular = DCFL$. If L and L' are both RE $\Rightarrow L$ is recursive.

Decision problem	Regular	CFL	CSL	Recursive	RE
Membership	D	D	D	D	U (semi-decidable)
Emptiness	D	D	U	U	U
Finiteness	D	D	U	U	U
Equivalence	D	U (DCFL: D)	U	U	U
Ambiguity	—	U	—	—	—
$L = \Sigma^*$	D	U	U	U	U
Subset ($L_1 \subseteq L_2$)	D	U	U	U	U
Intersection empty?	D	U	U	U	U

D = decidable, U = undecidable.

8.8 Turing Machines

A Turing machine has an unbounded tape it can read, write and move along in both directions. It is the accepted mathematical model of what any computer can compute.

- $TM = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$; infinite tape, read/write head moves L/R.
- **Variants equivalent in power**: multi-tape, multi-track, two-way infinite tape, non-deterministic TM, multi-head, TM with stay option, 2-stack PDA, queue automaton, counter machine with 2 counters.
- **PDA with 2 stacks = TM**. FA with a queue = TM.
- **Universal TM**: simulates any TM given its encoding $\langle M, w \rangle$.
- **Church–Turing thesis**: every effectively computable function is computable by a TM (a hypothesis, not a theorem).
- **Recursive (decidable)**: TM halts on all inputs. **RE (semi-decidable)**: TM halts & accepts on members; may loop otherwise.
- **LBA** (tape bounded by input length) accepts CSLs. Membership for CSL is decidable; emptiness for CSL is undecidable.
- **Unrestricted grammar** $(\alpha \rightarrow \beta) \equiv TM$.

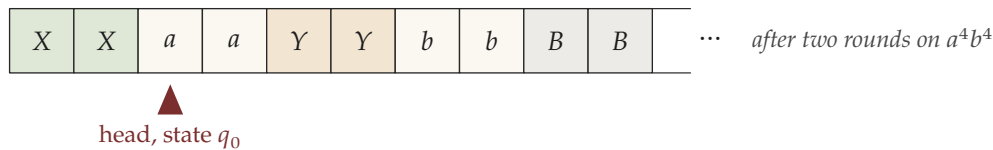


FIGURE 8.6. A Turing machine for $a^n b^n$ repeatedly marks the leftmost a as X , moves right to the leftmost b and marks it Y , then returns left. When no a remains it checks that no b remains, and accepts.

8.9 Undecidability

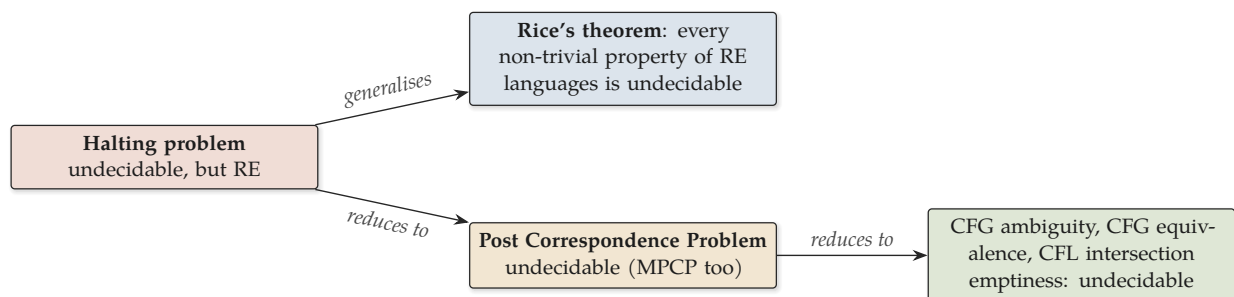


FIGURE 8.7. How the classic undecidability results follow from the halting problem by reduction.

- **Halting problem** $HALT_TM = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$: **RE but not recursive**; its complement is **not RE**.
- A_TM (acceptance) is RE, undecidable. Complement of A_TM is not RE.
- **Rice's theorem**: any non-trivial property of the **language** of a TM is undecidable (e.g. " $L(M)$ is empty", " $L(M)$ is regular", " $L(M)$ is finite").

- **PCP**: undecidable in general; decidable for single-letter alphabet. Bounded PCP is NP-complete.
- **E_TM** (emptiness) — not RE (its complement is RE). **EQ_TM** — neither RE nor co-RE.
- Decidable examples: whether a DFA accepts any string; whether a CFG generates the empty language; whether a TM makes more than k steps on input w (simulate k steps).
- Complexity: P, NP, PSPACE, EXPTIME; tractable (polynomial) vs intractable. See [Unit 7 §10](#).

8.10 Part B — Compiler Design

8.11 Phases of a Compiler

A compiler translates a program in one stage after another. The analysis phases (front end) break the source down and check it; the synthesis phases (back end) build the target program. All phases share a symbol table.

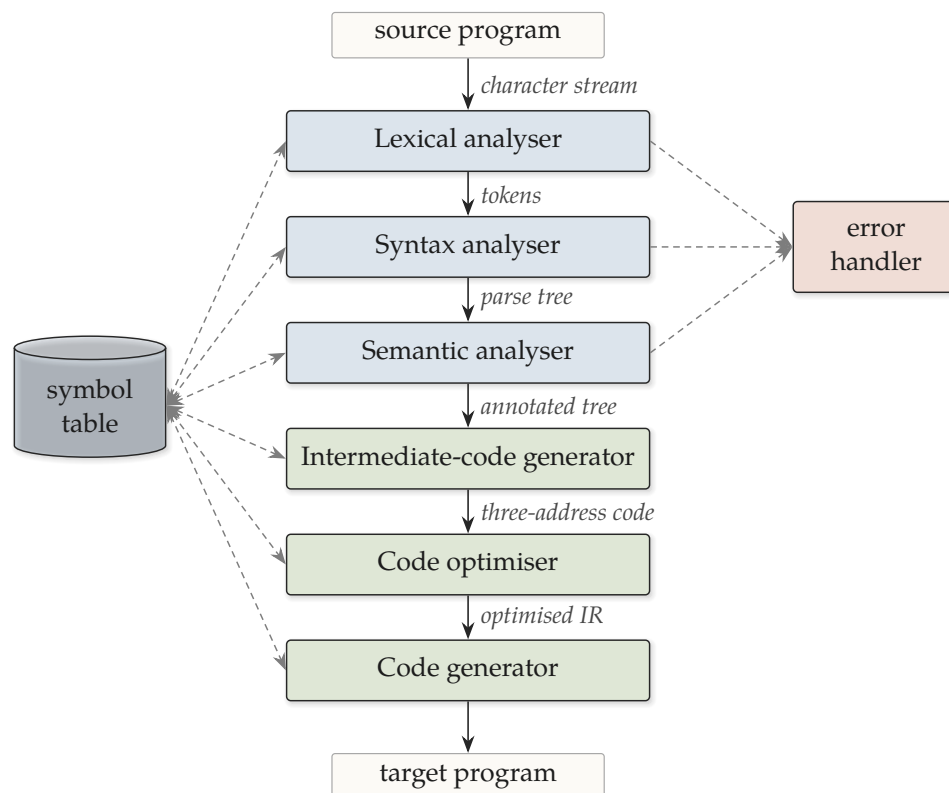


FIGURE 8.8. The phases of a compiler. The first three phases (blue) form the analysis front end and the last three (green) the synthesis back end. Every phase consults the symbol table; the analysis phases also report errors.

- **Front end** (analysis): lexical, syntax, semantic, ICG — language-dependent, machine-independent.
- **Back end** (synthesis): optimisation, code generation — machine-dependent.
- **Lexical analysis**: regular expressions → DFA (tools: **LEX/Flex**). Token, lexeme, pattern. Errors: illegal characters.
- **Count tokens**: `printf("i = %d", i);` → `printf(, "i = %d", ", ", i, ), ; = 7 tokens`.
- Parser generator: **YACC/Bison** (LALR(1)).

- Compiler passes: single-pass vs multi-pass. **Cross compiler**: runs on one machine, generates code for another. **Bootstrapping**: writing a compiler in its own language (T-diagrams).

8.12 Syntax Analysis (Parsing)

The parser checks that the token stream can be generated by the grammar and builds a parse tree. Top-down parsers build the tree from the root; bottom-up parsers build it from the leaves.

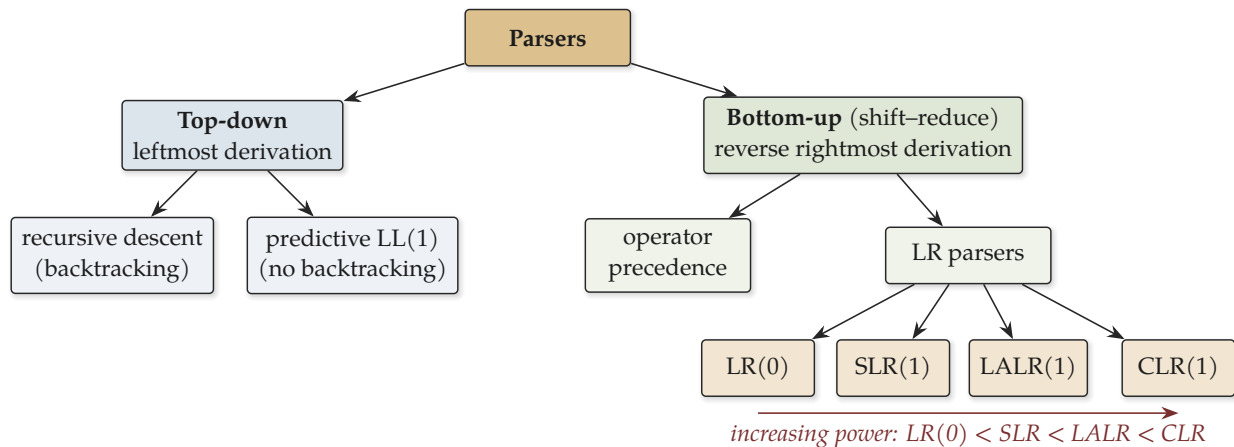


FIGURE 8.9. A family tree of parsers. Bottom-up LR parsers handle a strictly larger class of grammars than LL(1).

Power: $LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1)$; $LL(1) \subset LR(1)$. Number of states: $LR(0) = SLR = LALR \leq CLR$.

8.12.1 Grammar Transformations for Top-down

- **Left recursion removal:** $A \rightarrow A\alpha \mid \beta \Rightarrow A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$.
- **Left factoring:** $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \Rightarrow A \rightarrow \alpha A', A' \rightarrow \beta_1 \mid \beta_2$.

8.12.2 FIRST and FOLLOW

Grammar:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

NT	FIRST	FOLLOW
E	{ (, id }	{), \$ }
E'	{ +, ε }	{), \$ }
T	{ (, id }	{ +,), \$ }
T'	{ *, ε }	{ +,), \$ }
F	{ (, id }	{ *, +,), \$ }

LL(1) parsing table:

	id	+	*	(	)	\$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→FT'			T→FT'		
T'		T'→ ε	T'→*FT'		T'→ ε	T'→ ε
F	F→id			F→(E)		

LL(1) condition: for $A \rightarrow \alpha \mid \beta$: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$; if $\beta \Rightarrow^* \varepsilon$ then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$. A left-recursive or ambiguous grammar is **never** LL(1).

8.12.3 LR Parsing

Parser	Items	Reduce $A \rightarrow \alpha$ placed in
LR(0)	$[A \rightarrow \alpha \cdot \beta]$	all terminals
SLR(1)	LR(0) items	FOLLOW(A)
LALR(1)	LR(1) items with same core merged	lookaheads
CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$	lookahead a only

Conflicts: **shift-reduce (S/R)** and **reduce-reduce (R/R)**.

- Merging CLR states into LALR can introduce **R/R conflicts** but **never new S/R conflicts**.
- An unambiguous grammar can still have conflicts; every ambiguous grammar has conflicts in every LR table.

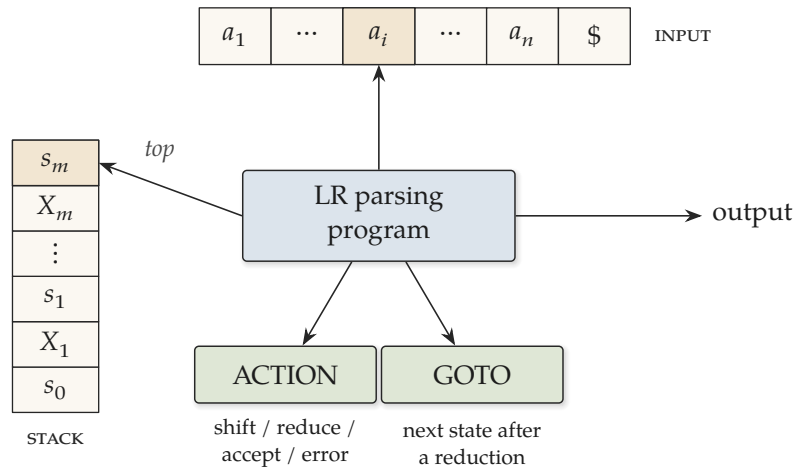


FIGURE 8.10. The LR parser model. The driver program is the same for every LR parser; only the ACTION and GOTO tables differ between LR(0), SLR, LALR and CLR.

Operator precedence parsing: for operator grammars (no ϵ , no two adjacent non-terminals); relations $<, \doteq, >$.

Handle: substring matching RHS whose reduction is a step of reverse rightmost derivation.

Viable prefixes are recognised by a DFA (LR(0) automaton).

8.13 Syntax-Directed Translation

Attribute	Computed from	Example
Synthesized	Children (bottom-up)	$E.val = E1.val + T.val$
Inherited	Parent and/or left siblings	type info in declarations: $L.in = T.type$

- **S-attributed SDD:** only synthesized attributes $\rightarrow$ evaluate in **bottom-up (postorder)**, natural for LR parsers.
- **L-attributed SDD:** inherited attributes depend only on parent & **left** siblings $\rightarrow$ evaluated in one **depth-first left-to-right** pass (LL parsers). **Every S-attributed is L-attributed.**
- **Dependency graph:** evaluation order = any **topological sort**; a cycle $\rightarrow$ no valid order.
- **Annotated parse tree:** shows attribute values.
- **Type checking:** static vs dynamic; type expressions; coercion; overloading resolution; strongly typed language.

Production	Semantic rule
$L \rightarrow E \text{ n}$	$print(E.val)$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

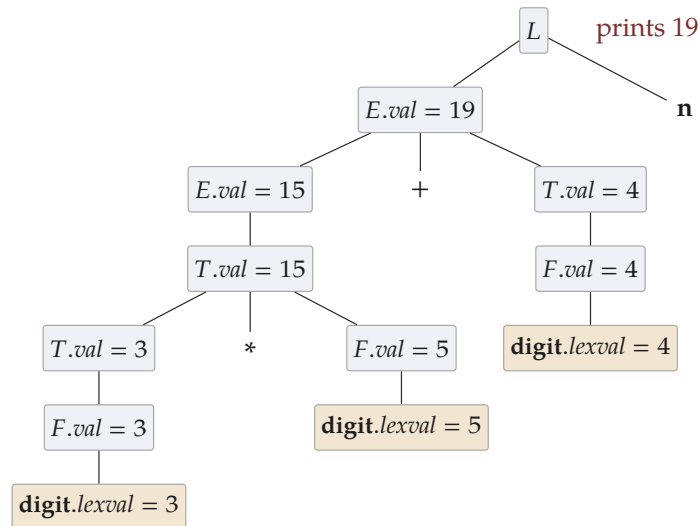


FIGURE 8.11. Annotated parse tree for $3 * 5 + 4 n$. Synthesised attributes flow upwards: each node's value is computed from its children's.

8.14 Run-Time Environment

At run time each procedure call needs storage for its parameters, locals and bookkeeping. The run-time environment decides where this storage lives and how a procedure finds non-local names.

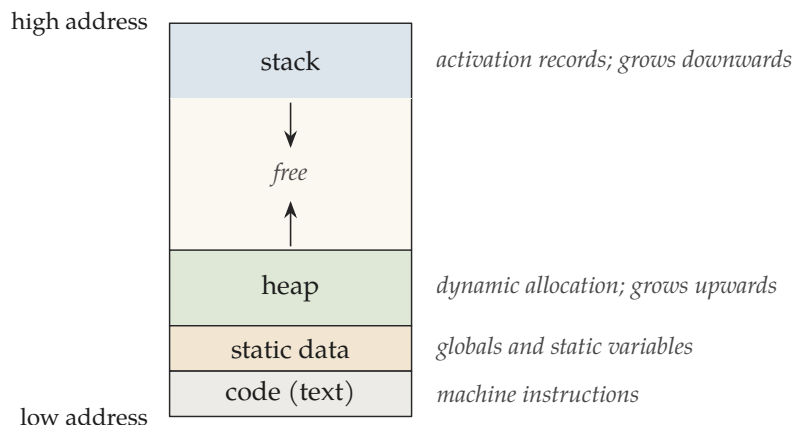


FIGURE 8.12. Run-time memory layout. Stack and heap grow towards each other into the free space between them.

Activation record (stack frame):

- **Activation tree:** each node = a procedure activation; the stack holds the path from root to the current node.
- Static allocation (FORTRAN — no recursion), stack allocation (C, recursion), heap allocation (closures, dynamic data).
- **Display:** array of pointers for fast non-local access in nested scopes.
- Parameter passing: see [Unit 3 §1.3](#).
- **Symbol table:** implemented with linear list, **hash table** (most common), BST; stores name, type, scope, offset. Scope handling via a stack of tables.

actual parameters	
returned value	
control link (dynamic)	<i>points to the caller's record</i>
access link (static)	<i>points to the record of the lexically enclosing procedure</i>
saved machine status	<i>return address, registers</i>
local data	
temporaries	

FIGURE 8.13. A typical activation record (stack frame), created on each procedure call and removed on return.

8.15 Intermediate Code

Expression: $a = b * -c + b * -c$

Three-address code:

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

Quadruples (op, arg1, arg2, result) · **Triples** (op, arg1, arg2 — results referred by position) · **Indirect triples** (list of pointers to triples, easy to reorder).

#	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

DAG for the same expression shares the common subexpression $b * -c$:

- Other IRs: syntax tree, postfix, **SSA (static single assignment)** — each variable assigned once, ϕ -functions at joins.
- Boolean expressions: numerical vs **short-circuit (jumping) code**; **backpatching** for one-pass generation of jumps (makelist, merge, backpatch).

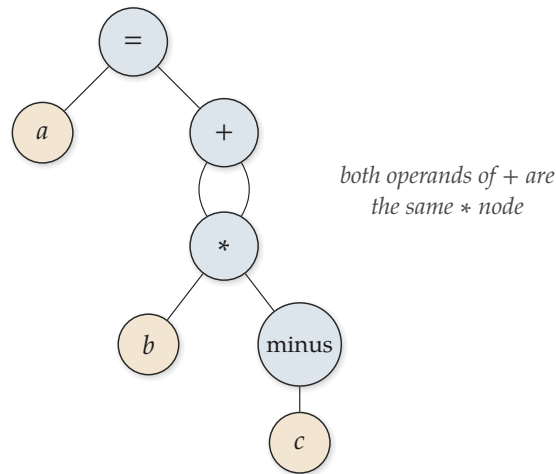


FIGURE 8.14. DAG for $a = b * -c + b * -c$. The common subexpression $b * -c$ is built once and shared.

8.16 Code Optimisation & Generation

Optimisation rewrites the intermediate code so that it runs faster or takes less space without changing what it computes. It works first inside basic blocks, then across the flow graph.

8.16.1 Basic Blocks & Flow Graphs

Leaders: (1) first statement; (2) target of any jump; (3) statement immediately after a jump. A basic block runs from a leader up to (not including) the next leader.

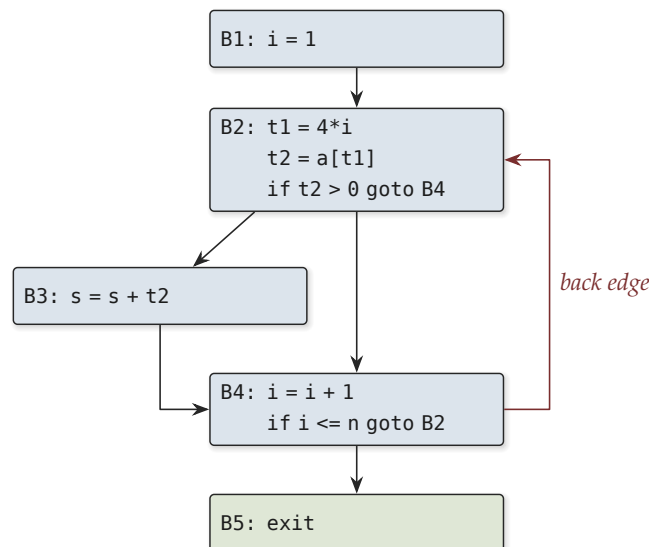


FIGURE 8.15. Basic blocks and the flow graph of a loop. The back edge $B4 \rightarrow B2$ marks a natural loop $\{B2, B3, B4\}$.

8.16.2 Optimisations

Level	Technique	Example
Local	Common subexpression elimination	$t1 = b*c; t2 = b*c \rightarrow \text{reuse } t1$
Local	Constant folding	$x = 2 * 3.14 \rightarrow x = 6.28$
Local	Constant/copy propagation	$x = y; z = x + 1 \rightarrow z = y + 1$
Local	Dead code elimination	Remove assignments never used
Loop	Code motion (loop-invariant)	Move $t = x*y$ out of the loop
Loop	Induction variable elimination	Replace i with pointer arithmetic
Loop	Strength reduction	$i*4 \rightarrow t = t + 4; x^2 \rightarrow x*x$
Loop	Loop unrolling, loop fusion (jamming), loop fission	
Global	Data-flow analysis based	
Peephole	Redundant load/store, unreachable code, flow-of-control (jump to jump), algebraic simplification, use of machine idioms (INC)	Window over target code

8.16.3 Data-Flow Analysis

Analysis	Direction	Meet	Use
Reaching definitions	Forward	$\cup$	Constant propagation
Live variables	Backward	$\cup$	Register allocation, dead code
Available expressions	Forward	$\cap$	Global CSE
Very busy expressions	Backward	$\cap$	Code hoisting

Reaching definitions (forward): transfer $OUT[B] = gen[B] \cup (IN[B] - kill[B])$

Reaching definitions: meet over predecessors $IN[B] = \bigcup_{P \in pred(B)} OUT[P]$

Live variables (backward): transfer $IN[B] = use[B] \cup (OUT[B] - def[B])$

Live variables: meet over successors $OUT[B] = \bigcup_{S \in succ(B)} IN[S]$

- **Loops** detected using **dominators** and back edges ($n \rightarrow d$ where $d \text{ dom } n$). Natural loop. Reducible flow graphs.

8.16.4 Code Generation

- Issues: instruction selection, **register allocation** (graph colouring — interference graph; spilling), evaluation order.
- **Sethi–Ullman** numbering gives the minimum registers for an expression tree.
- get reg, register & address descriptors. Next-use information.
- **Instruction scheduling**: reorder to avoid pipeline stalls (list scheduling); respects data dependencies.

8.17 Deeper Dive — Constructions & Parsing Traces

8.17.1 NFA $\rightarrow$ DFA (*Subset Construction*)

NFA for strings over $\{0, 1\}$ ending in "01": $q_0 \xrightarrow{0,1} q_0$, $q_0 \xrightarrow{0} q_1$, $q_1 \xrightarrow{1} q_2$ (final).

DFA state	on 0	on 1	Final?
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$	✓

Only 3 of the $2^3 = 8$ subsets are reachable — this is the same DFA drawn in §3.1.

8.17.2 DFA Minimisation (*partition refinement*)

State	on 0	on 1	Final?
$\rightarrow A$	B	C	
B	A	D	
C	E	F	✓
D	E	F	✓
E	E	F	✓
F	F	F	

Round	Partition	Reason
P_0	$\{C, D, E\} \{A, B, F\}$	final versus non-final
P_1	$\{C, D, E\} \{A, B\} \{F\}$	on 1, F goes to a non-final state while A, B go to final states
P_2	$\{C, D, E\} \{A, B\} \{F\}$	no group splits further: stable

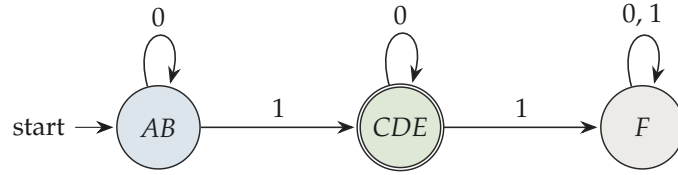


FIGURE 8.16. The minimal DFA has three states and accepts strings over $\{0, 1\}$ containing exactly one 1; F is a dead state.

8.17.3 CFG $\rightarrow$ CNF

$S \rightarrow aSb \mid ab$

1. Replace terminals in bodies of length ≥ 2 by new variables: $A \rightarrow a, B \rightarrow b$, giving $S \rightarrow ASB \mid AB$.
2. Break bodies longer than two symbols: $S \rightarrow AX$ with $X \rightarrow SB$.

CNF: $S \rightarrow AX \mid AB, \quad X \rightarrow SB, \quad A \rightarrow a, \quad B \rightarrow b$

8.17.4 Pumping-Lemma Proof Template ($L = \{ a^n b^n \mid n \geq 0 \}$ is not regular)

1. Assume L is regular with pumping length p .
2. Choose $w = a^p b^p \in L, |w| \geq p$.
3. Any split $w = xyz$ with $|xy| \leq p, |y| \geq 1$ forces $y = a^k (k \geq 1)$.
4. Pump $i = 2$: $xy^2z = a^{p+k} b^p \notin L$ — contradiction. Hence L is not regular.

8.17.5 LR(0) Automaton and Parse — Worked

Grammar: (0) $S' \rightarrow S$ (1) $S \rightarrow AA$ (2) $A \rightarrow aA$ (3) $A \rightarrow b$

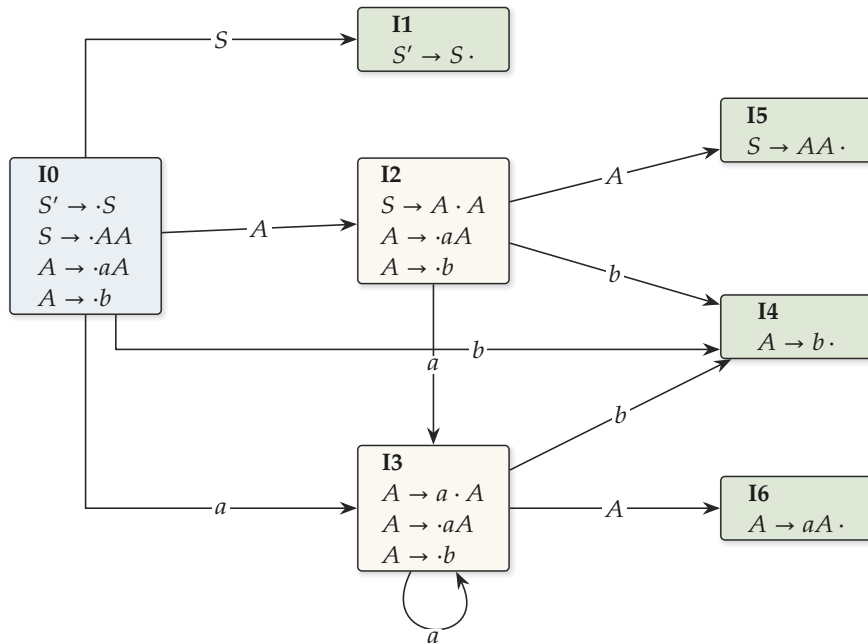


FIGURE 8.17. LR(0) automaton for $S \rightarrow AA, A \rightarrow aA \mid b$. Shaded states contain a complete item (a reduction); none also contains a shift, so the grammar is LR(0).

State	a	b	\$	A	S
0	s3	s4		2	1
1			acc		
2	s3	s4		5	
3	s3	s4		6	
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

No state contains both a shift and a complete item $\rightarrow$ the grammar is **LR(0)** (hence also SLR, LALR, CLR).

Parse of **abb\$**:

Stack	Input	Action
0	abb\$	shift 3
0 a 3	bb\$	shift 4
0 a 3 b 4	b\$	reduce $A \rightarrow b$, goto(3, A) = 6
0 a 3 A 6	b\$	reduce $A \rightarrow aA$, goto(0, A) = 2
0 A 2	b\$	shift 4
0 A 2 b 4	\$	reduce $A \rightarrow b$, goto(2, A) = 5
0 A 2 A 5	\$	reduce $S \rightarrow AA$, goto(0, S) = 1
0 S 1	\$	accept

A grammar that is LALR(1) but not SLR(1): $S \rightarrow L = R \mid R$, $L \rightarrow *R \mid \text{id}$, $R \rightarrow L$. In the state containing $S \rightarrow L \cdot = R$ and $R \rightarrow L \cdot$, $'=' \in \text{FOLLOW}(R)$, so SLR has a shift/reduce conflict on $'='$; LR(1) lookaheads resolve it.

8.17.6 Basic Blocks — Worked

8.17.7 DAG-Based Local Optimisation

- $d = a - d$ recomputes node n_2 (same operator, same operands, nothing changed in between) $\rightarrow$ replaced by the copy $d = b$.
- $c = b + c$ is **not** a repeat of $a = b + c$, because b was reassigned in between (its operand is n_2 , not b_0).

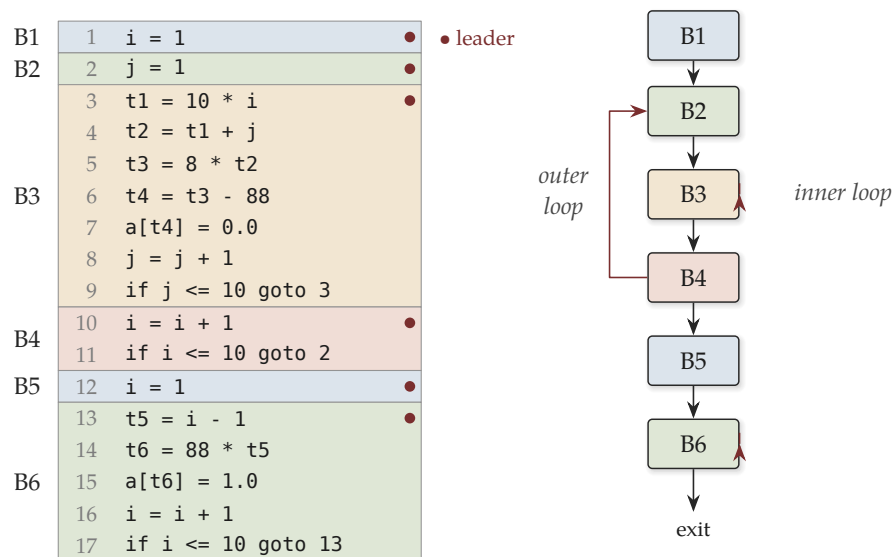


FIGURE 8.18. Partitioning code into basic blocks. Leaders are the first statement, every jump target (2, 3, 13) and every statement after a conditional jump (10, 12); each block runs from a leader to just before the next.

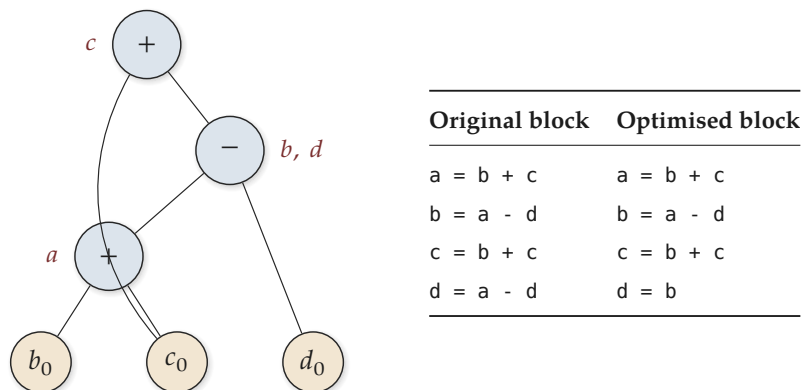


FIGURE 8.19. DAG of a basic block. When $d = a - d$ is processed, the node $-(n_1, d_0)$ already exists (it is labelled b), so d is simply attached to it: the last statement becomes the copy $d = b$.

8.18 Previous Year Questions (PYQ pattern)

Finite automata & regular languages

1. Minimum states in DFA accepting strings over $\{0,1\}$ where the number of 0s is divisible by 3 and number of 1s divisible by 2: **6**
2. Minimum DFA states for binary strings whose 3rd symbol from the right is 1: **8**
3. Minimum DFA states for strings over $\{a,b\}$ of length exactly 3: **5**
4. An NFA with n states can be converted to a DFA with at most: **2^n states**
5. Which is NOT regular? (a) ab (b) $a^n b^n$ (c) $(ab)^*$ (d) $a^{(2n)}$ — **(b)**
6. $(a + b)^*$ is equivalent to: **$(ab)^*$**
7. Regular languages are NOT closed under: (a) union (b) complement (c) infinite union (d) intersection — **(c)** (infinite union can give any language)
8. Moore vs Mealy: in Moore the output depends on: **present state only**
9. Arden's theorem solution of $R = Q + RP$: **$R = QP^*$**
10. Pumping lemma for regular languages is used to prove a language is: **not regular**

CFLs & PDAs

11. Which language is context-free but not regular? $a^n b^n$
12. Which is not context-free? $a^n b^n c^n$
13. CFLs are NOT closed under: **intersection and complement**
14. DCFLs are closed under: **complement** (not union/intersection)
15. In CNF, a string of length n is derived in: **$2n - 1$ steps**
16. Which is undecidable for CFGs? **ambiguity / equivalence / $L = \Sigma^*$**
17. Membership of a string in a CFL can be decided by: **CYK algorithm ($O(n^3)$)**
18. ww^R is accepted by: **NPDA, not DPDA**
19. $CFL \cap regular$ is: **context-free**
20. Grammar $S \rightarrow aSb \mid ab$ generates: **$a^n b^n, n \geq 1$**

TMs & decidability

21. A language accepted by an LBA is: **context-sensitive**
22. Halting problem is: **recursively enumerable but not recursive**
23. Complement of an RE but non-recursive language is: **not RE**
24. If L and its complement are both RE then L is: **recursive**
25. Rice's theorem applies to: **non-trivial properties of the languages of TMs**
26. PCP is: **undecidable**
27. Which is equivalent in power to a TM? **PDA with two stacks**
28. Which is decidable? (a) whether a TM halts on all inputs (b) whether a CFG is ambiguous (c) whether a DFA accepts an infinite language (d) PCP — **(c)**
29. Recursive languages are closed under: **complement** (RE are not)
30. Type-1 grammars are: **context-sensitive**

Compilers

31. Number of tokens in `int a = b + 10;; int, a, =, b, +, 10, ; = 7`
32. Lexical analysis is based on: **regular expressions / finite automata**
33. YACC generates: **LALR(1) parsers**
34. Which parser is most powerful? **Canonical LR(1)**
35. Number of states: SLR vs LALR for the same grammar: **equal**
36. Left-recursive grammar cannot be parsed by: **top-down (LL / recursive descent) parsers**
37. Merging CLR(1) states to LALR(1) may introduce: **reduce-reduce conflicts**
38. FOLLOW of the start symbol always contains: **\$**
39. A bottom-up parser produces: **reverse of the rightmost derivation**
40. S-attributed definitions use only: **synthesized attributes**
41. Inherited attributes from parent and left siblings: **L-attributed**
42. Access link in an activation record points to: **the AR of the lexically enclosing procedure**
43. Representation where results are referenced by position: **triples**
44. Replacing $x * 2$ by $x + x$ (or $x \ll 1$): **strength reduction**
45. Moving a loop-invariant computation outside the loop: **code motion**
46. Live variable analysis is a: **backward data-flow problem**
47. Peephole optimisation is applied to: **target (or intermediate) code over a small window**

48. Register allocation is commonly modelled as: **graph colouring**
49. A DAG for a basic block helps to detect: **common subexpressions**
50. Number of basic blocks in a flow graph — first find **leaders**: first statement, jump targets, statements after jumps

More practice questions

51. Subset construction for the "ends in 01" NFA yields how many reachable DFA states? **3**
52. Minimal DFA for "exactly one 1" over $\{0, 1\}$ (with dead state): **3 states**
53. CNF of $S \rightarrow aSb \mid ab$ needs how many non-terminals? **4** (S, X, A, B)
54. In the pumping-lemma proof for $a^n b^n$, pumping y changes: **only the number of a's**
55. Number of LR(0) item sets for $S \rightarrow AA, A \rightarrow aA \mid b$: **7**
56. The grammar $S \rightarrow L = R \mid R, L \rightarrow *R \mid id, R \rightarrow L$ is: **LALR(1) but not SLR(1)**
57. Number of basic blocks in the code of §15.6: **6**
58. In $a = b + c; b = a - d; c = b + c; d = a - d$, the redundant computation is: **$d = a - d$** (equals b)
59. A shift/reduce conflict in SLR arises when a terminal in $FOLLOW(A)$ also: **labels a shift from the same state**
60. The handle in a right-sentential form is reduced by: **a bottom-up (shift-reduce) parser**

POINTS TO REMEMBER

- Regular closed under everything basic · CFL ✗ $\cap$, ✗ complement · DCFL ✓ complement · RE ✗ complement
- CNF $2n-1$ steps · GNF n steps · CYK $O(n^3)$
- Halting: RE not recursive · $L \ \& \ \bar{L} \text{ RE} \Rightarrow$ recursive · Rice: non-trivial language properties undecidable
- $LR(0) \subset SLR \subset LALR \subset CLR$ · $SLR = LALR$ states · LALR merge $\rightarrow$ R/R only
- S-attributed $\subset$ L-attributed · Live variables backward · Reaching definitions forward

Data Communication & Computer Networks

Expected questions: 10–12 · Target: 10+ · Type: subnetting, sliding window, Shannon/Nyquist numericals + Forouzan concepts

SYLLABUS COVERAGE

- ☐ Data communication: components, data flow, topologies, protocols & standards, OSI & TCP/IP
 - ☐ Analog & digital signals: bandwidth, impairments, Nyquist & Shannon limits, performance
 - ☐ Digital transmission: line coding, block coding, scrambling, PCM, delta modulation
 - ☐ Analog transmission: ASK, FSK, PSK, QAM
 - ☐ Multiplexing (FDM, WDM, TDM), spread spectrum, transmission media
 - ☐ Switching: circuit, message, packet (datagram, virtual circuit)
 - ☐ Data link layer: framing, error detection & correction, flow control, Stop-and-Wait, Go-Back-N, Selective Repeat, HDLC, PPP
 - ☐ MAC: ALOHA, CSMA, CSMA/CD, CSMA/CA, polling, token passing, FDMA/TDMA/CDMA; Ethernet
 - ☐ Network layer: IPv4, subnetting, CIDR, NAT, IPv6, routing (DV, LS, path vector), RIP, OSPF, BGP, ARP, ICMP, DHCP
 - ☐ Transport layer: UDP, TCP, SCTP, congestion control, QoS
 - ☐ Application layer: DNS, HTTP, FTP, SMTP, TELNET, SNMP
 - ☐ Network security: cryptography, RSA, Diffie-Hellman, digital signatures, VPN, firewalls
 - ☐ Mobile technology: GSM, CDMA, GPRS, Mobile IP, MANETs, satellites, wireless LANs
 - ☐ Cloud computing & IoT
-

9.1 Network Models

Networking software is built in layers. Each layer offers a service to the layer above and uses the service of the layer below, so that, for example, a web browser never needs to know whether its data travels over Wi-Fi or fibre.

OSI (7 LAYERS)	TCP/IP	PDU	PROTOCOLS (DEVICES)
7 Application	Application	<i>message</i>	HTTP, FTP, SMTP, DNS, SNMP, DHCP (gateway)
6 Presentation			
5 Session			
4 Transport	Transport	<i>segment</i>	TCP, UDP, SCTP
3 Network	Internet	<i>packet</i>	IP, ICMP, ARP, IGMP (router)
2 Data link	Network access	<i>frame</i>	Ethernet, PPP, HDLC (switch, bridge)
1 Physical		<i>bits</i>	(hub, repeater)

FIGURE 9.1. The OSI reference model beside the TCP/IP suite. Mnemonic, top to bottom: “All People Seem To Need Data Processing”.

9.1.1 OSI vs TCP/IP

Layer	Key responsibilities	Address
Physical	Bit transmission, encoding, topology, data rate	—
Data link	Framing , physical addressing, flow & error control (hop-to-hop), MAC	MAC (48-bit)
Network	Logical addressing, routing (host-to-host / source-to-destination)	IP
Transport	Process-to-process (end-to-end) delivery, segmentation, flow, error, congestion control	Port (16-bit)
Session	Dialog control, synchronisation (checkpoints)	—
Presentation	Translation, encryption, compression	—
Application	User services	Specific (URL, email)

9.1.2 Topologies

Topology	Cables / links (n devices)	Notes
Mesh	$n(n-1)/2$ links; each device $n-1$ I/O ports	Robust, expensive
Star	n links	Central hub; hub failure kills network
Bus	1 backbone + drop lines	Cheap; backbone fault fails all
Ring	n links	Token passing; one break can disable
Tree / Hybrid	—	

Data flow: **simplex** (keyboard), **half-duplex** (walkie-talkie), **full-duplex** (telephone).

9.2 Signals & Data Rates

Every link has a limited bandwidth and some noise. Two classical results bound how fast data can be sent: Nyquist's for a noiseless channel and Shannon's for a noisy one.

Nyquist bit rate (noiseless channel, bandwidth B , L signal levels)	$\text{bit rate} = 2B \log_2 L$
Shannon capacity (noisy channel)	$C = B \log_2(1 + \text{SNR})$
Signal-to-noise ratio in decibels	$\text{SNR}_{\text{dB}} = 10 \log_{10} \text{SNR} \quad (30 \text{ dB} = 1000)$
Bandwidth–delay product (bits “in the pipe”)	bandwidth $\times$ propagation delay
Propagation and transmission delay	$T_p = \frac{\text{distance}}{\text{speed}}, \quad T_t = \frac{\text{frame size}}{\text{bandwidth}}$
Latency	$T_p + T_t + \text{queuing} + \text{processing}$
Baud (signal) rate, r bits per signal element	$S = N/r$

Worked: telephone line $B = 3000 \text{ Hz}$, $\text{SNR} = 3162$ ($\approx 35 \text{ dB}$): $C = 3000 \times \log_2(3163) \approx 3000 \times 11.62 = \mathbf{34.86 \text{ kbps}}$. Nyquist: $B = 3000 \text{ Hz}$, 4 levels $\rightarrow 2 \times 3000 \times 2 = \mathbf{12 \text{ kbps}}$.

Impairments: **attenuation** (loss of energy, $\text{dB} = 10 \log(P_2/P_1)$), **distortion** (different delays of components), **noise** (thermal, induced, crosstalk, impulse).

9.3 Digital & Analog Transmission

9.3.1 Line Coding

Line coding turns a bit sequence into a digital signal. Good codes keep the receiver's clock synchronised (by including transitions) and avoid a DC component.

Scheme	Category	Notes
NRZ-L, NRZ-I	Polar	No self-sync; baseline wandering
RZ	Polar	3 levels; 2 transitions per bit
Manchester , Differential Manchester	Biphase	Self-synchronising; bandwidth = $2 \times$ NRZ; Ethernet (10 Mbps) / Token ring
AMI, Pseudoternary	Bipolar	No DC component
2B1Q, 8B6T, 4D-PAM5	Multilevel	
MLT-3	Multitransition	100Base-TX

- **Block coding:** 4B/5B (Fast Ethernet), 8B/10B (Gigabit) — for synchronisation & error detection.

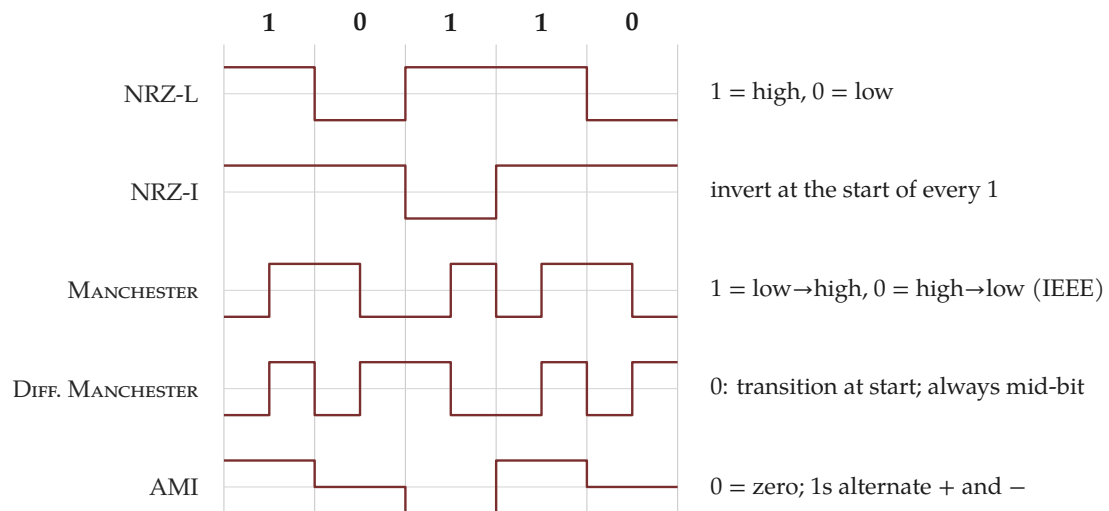


FIGURE 9.2. Line codes for the bits 10110. Manchester codes put a transition in the middle of every bit, so the receiver can recover the clock; AMI has no DC component.

- **Scrambling:** B8ZS (North America), HDB3 (Europe) — replace long zeros in AMI.
- **PCM:** Sampling → Quantisation → Encoding. **Nyquist sampling theorem:** $f_s \geq 2 \cdot f_{\text{max}}$. Bit rate = $f_s \times n$ bits. Voice: 8000 samples/s $\times$ 8 bits = **64 kbps**.
- **Delta modulation:** 1 bit per sample (up/down).
- Transmission modes: **parallel; serial** — asynchronous (start/stop bits, gaps), synchronous, isochronous.

9.3.2 Digital-to-Analog

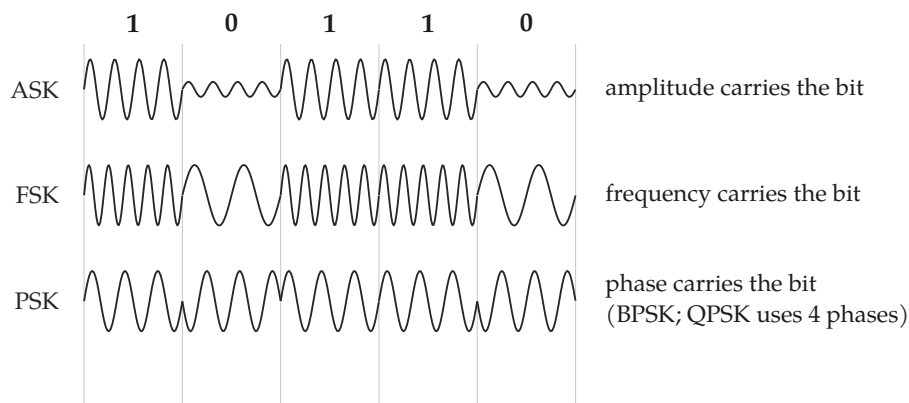


FIGURE 9.3. Digital-to-analog conversion of the bits 10110. QAM combines amplitude and phase changes to send several bits per signal element.

- QPSK: 2 bits per symbol; 16-QAM: 4 bits per symbol; 64-QAM: 6. Bits per baud = $\log_2(\text{constellation points})$.
- Analog-to-analog: AM, FM, PM. FM bandwidth (Carson) = $2(1 + \beta)B$.

9.4 Multiplexing & Media

Technique	Domain	Use
FDM	Frequency (analog)	Radio, TV, cable
WDM	Wavelength (optical)	Fibre (DWDM)
Synchronous TDM	Time slots fixed	T1 (24 channels, 1.544 Mbps), E1 (32 channels, 2.048 Mbps)
Statistical TDM	Slots on demand	Needs address in slot
Spread spectrum	FHSS, DSSS	Wireless, security

– $T1 = (24 \times 8 + 1 \text{ framing bit}) \times 8000 = \mathbf{1.544 \text{ Mbps}}$.

Transmission media:

	Medium	Key facts
Guided	Twisted pair	UTP Cat 5/6, RJ-45; twisting reduces crosstalk
	Coaxial cable	BNC connectors; cable TV
	Optical fibre	total internal reflection; single-mode vs multimode; highest bandwidth, immune to EMI
Unguided	Radio waves	3 kHz–1 GHz; omnidirectional
	Microwaves	1–300 GHz; unidirectional, line of sight
	Infrared	300 GHz–400 THz; short range, cannot penetrate walls

9.5 Switching

Circuit switching	Packet (datagram)	Virtual circuit	Message switching
Dedicated path, setup/teardown	No setup; each packet routed independently	Setup phase; packets follow same path	Store-and-forward whole message
Telephone	Internet (IP)	ATM, Frame Relay, X.25	Telegraph
In-order, constant delay	Out-of-order possible	In-order	Large buffers
Physical layer	Network layer	Data link (ATM) / network	Application

Delay for packet switching: message split into k packets over h hops: total $\approx (k + h - 1) \times$ transmission time per packet (+ propagation).

9.6 Data Link Layer

The data link layer moves frames between two directly connected nodes. It frames the bit stream, detects (and sometimes corrects) errors and keeps a fast sender from swamping a slow receiver.

9.6.1 Framing

Character count, **byte stuffing** (ESC/FLAG), **bit stuffing** (insert 0 after five consecutive 1s — HDLC flag 01111110), physical layer coding violations. Bit-stuffing example: data 011111111110 → 011111011111010.

9.6.2 Error Detection & Correction

- **Parity**: detects odd number of bit errors. 2-D parity detects all 1, 2, 3-bit errors and corrects single.
- **Checksum** (1's complement sum; used in IP, TCP, UDP).
- **CRC**: append (degree r) zeros, divide by generator (mod-2), remainder = CRC. Detects all burst errors of length $\leq r$; all odd errors if generator has factor $(x + 1)$.

CRC worked: data 1101011011, generator $x^4 + x + 1 = 10011 \rightarrow$ append 4 zeros, divide $\rightarrow$ remainder **1110**; transmitted **1101011011110**.

- **Hamming distance**: detect d errors needs $d_{\min} \geq d + 1$; correct t errors needs $d_{\min} \geq 2t + 1$.
- Hamming code (7,4): r parity bits with $2^r \geq m + r + 1$.

9.6.3 Flow & Error Control (Sliding Window)

Ratio of propagation to transmission time	$a = T_p/T_t$
Stop-and-wait efficiency	$\eta = \frac{1}{1 + 2a}$
Sliding window of size W	$\eta = \frac{W}{1 + 2a}$ if $W < 1 + 2a$, else 1
Window for full utilisation	$W \geq 1 + 2a$
Go-Back-N with n -bit sequence numbers	$W_s \leq 2^n - 1, \quad W_r = 1$
Selective Repeat with n -bit sequence numbers	$W_s = W_r \leq 2^{n-1}$
Throughput	$\eta \times \text{bandwidth}$

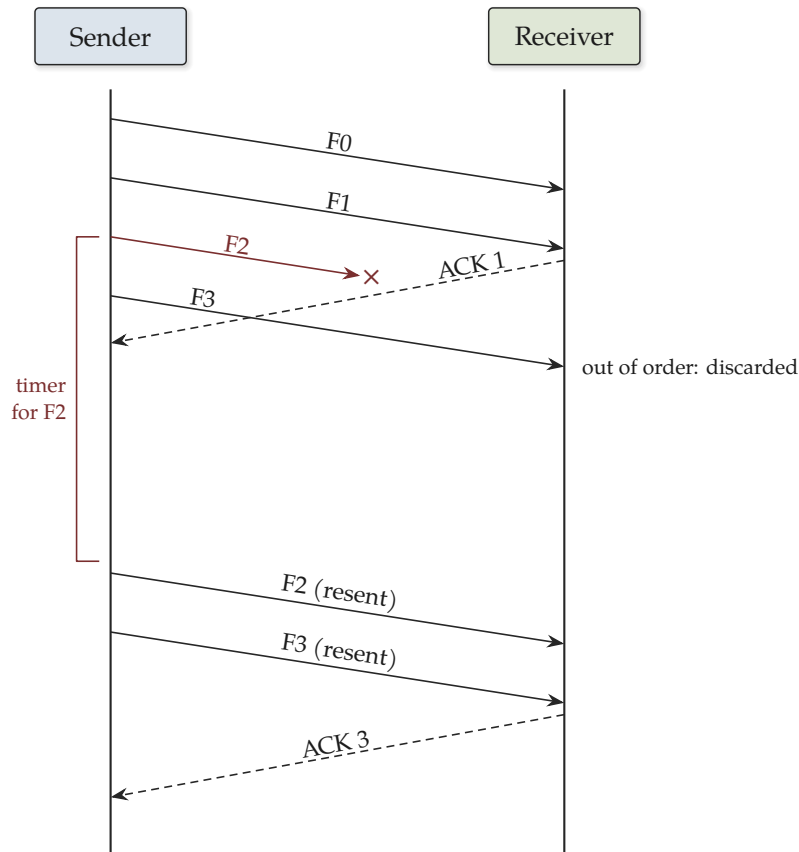


FIGURE 9.4. Go-Back-N with a window of 4 when frame 2 is lost. The receiver accepts only in-order frames, so after the timeout the sender resends F2 *and every frame after it*.

Protocol	Sender window	Receiver window	Retransmits	ACK
Stop-and-Wait ARQ	1	1	The one frame	Individual; seq numbers 0/1
Go-Back-N	$2^n - 1$	1	Lost frame and all after it	Cumulative
Selective Repeat	2^{n-1}	2^{n-1}	Only lost frame	Individual / NAK

Worked: 1 Mbps link, frame 1000 bits, $T_p = 20$ ms. $T_t = 1$ ms, $a = 20$. Stop-and-Wait $\eta = 1/41 \approx 2.4\%$. Window for 100% = $1 + 40 = 41 \rightarrow$ sequence bits = $\lceil \log_2 41 \rceil = 6$ (GBN; $2^6 - 1 = 63 \geq 41$).

9.6.4 HDLC & PPP

- **HDLC:** bit-oriented; frames I (information), S (supervisory), U (unnumbered); modes NRM, ABM, ARM; flag 01111110.
- **PPP:** byte-oriented, point-to-point (dial-up, DSL); LCP (link control), NCP, authentication **PAP** (plain-text) and **CHAP** (challenge-handshake, more secure). No flow control/sequence numbers by default.

9.7 Medium Access Control

When many stations share one channel, a medium-access protocol decides who may transmit when.

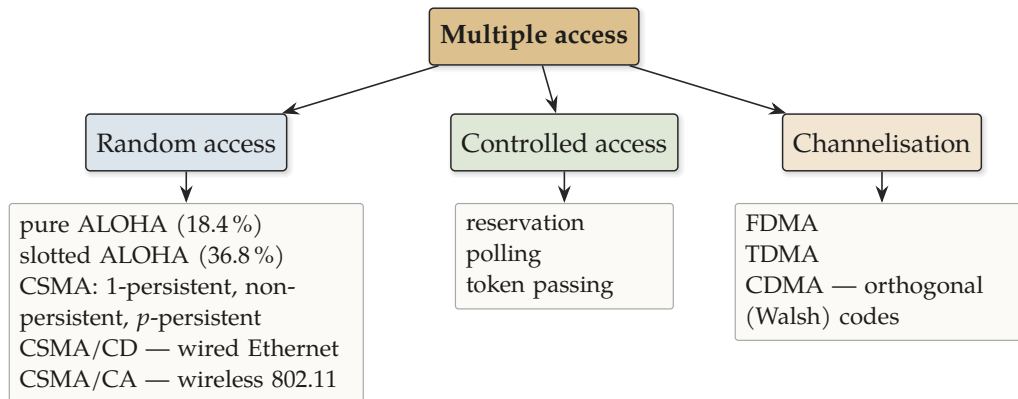


FIGURE 9.5. Medium-access protocols. Random access lets stations transmit at will and handle collisions; controlled access takes turns; channelisation divides the channel by frequency, time or code.

Pure ALOHA throughput (vulnerable time $2T_t$) $S = G e^{-2G}, \quad S_{\max} = \frac{1}{2e} \approx 0.184 \text{ at } G = 0.5$

Slotted ALOHA throughput (vulnerable time T_t) $S = G e^{-G}, \quad S_{\max} = \frac{1}{e} \approx 0.368 \text{ at } G = 1$

CSMA/CD minimum frame $T_t \geq 2T_p \Rightarrow L_{\min} = 2T_p B$

CSMA/CD efficiency $\eta = \frac{1}{1 + 6.44a}$

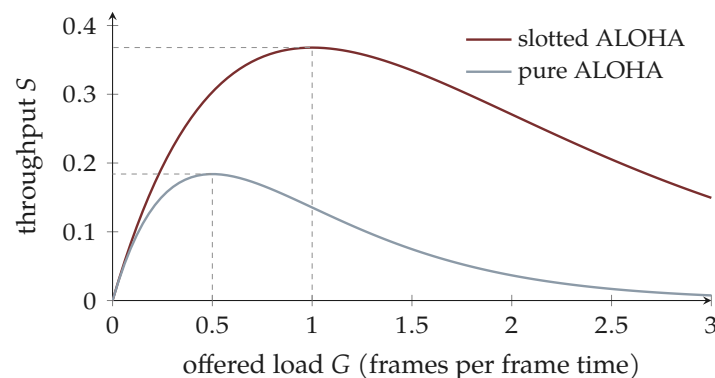


FIGURE 9.6. ALOHA throughput. Slotting halves the vulnerable period and so doubles the peak throughput.

Worked: 10 Mbps Ethernet, max RTT $51.2 \mu\text{s} \rightarrow L_{\min} = 10^7 \times 51.2 \times 10^{-6} = 512 \text{ bits} = 64 \text{ bytes}$.

- CSMA/CD: on collision send **jam signal**, use **binary exponential backoff** (wait random $k \times$ slot, $k \in [0, 2^n - 1]$; give up after 16 attempts).
- CSMA/CA (802.11): **IFS** (DIFS, SIFS), **contention window**, **RTS/CTS** (solves **hidden terminal**), NAV, ACK.

- Token ring (802.5), token bus (802.4).

9.7.1 Ethernet (IEEE 802.3)

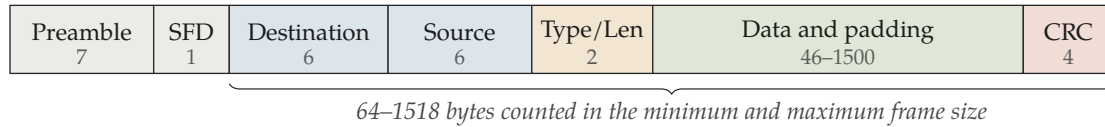


FIGURE 9.7. The IEEE 802.3 (Ethernet) frame; field sizes in bytes.

- MAC address: 48 bits (24-bit OUI + 24 bits). Broadcast FF:FF:FF:FF:FF:FF; multicast when the least-significant bit of the first byte is 1.

Standard	Speed	Media
10Base5 / 10Base2	10 Mbps	Thick / thin coax
10Base-T	10 Mbps	UTP
100Base-TX / FX	100 Mbps	UTP / fibre
1000Base-T	1 Gbps	UTP Cat5e
10GBase	10 Gbps	Fibre/Cat6a

Devices: **Repeater/hub** (L1, one collision & broadcast domain), **Bridge/switch** (L2, separates collision domains, one broadcast domain; learning, STP), **Router** (L3, separates broadcast domains), **Gateway** (all layers / protocol conversion). VLANs split broadcast domains on switches.

9.8 Network Layer

The network layer carries packets from source host to destination host across many networks. Its two jobs are addressing (IP) and routing (choosing the path).

9.8.1 IPv4 Addressing

Class	First bits	First octet	Default mask	Networks	Hosts/net
A	0	0–127	/8 255.0.0.0	2^7 (126 usable)	$2^{24} - 2$
B	10	128–191	/16	2^{14}	$2^{16} - 2$
C	110	192–223	/24	2^{21}	254
D	1110	224–239	—	Multicast	—
E	1111	240–255	—	Reserved	—

- Private ranges: **10.0.0.0/8**, **172.16.0.0/12**, **192.168.0.0/16**. Loopback: 127.0.0.0/8. APIPA 169.254/16.
- **Subnetting worked:** 192.168.10.0/26
- Usable hosts in $/n = 2^{(32-n)} - 2$ (except /31, /32).

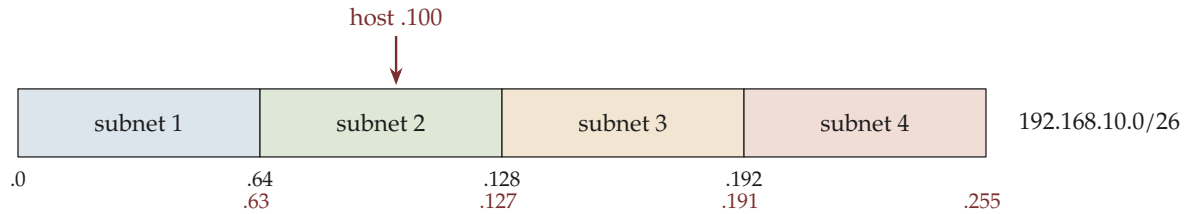


FIGURE 9.8. 192.168.10.0 with mask 255.255.255.192 (/26): four subnets of 64 addresses, 62 usable hosts each (the first address is the network, the last, in red, the broadcast). Host .100 lies in subnet 2: network .64, broadcast .127.

- **CIDR & supernetting:** 200.1.0.0/24 ... 200.1.3.0/24 aggregate to **200.1.0.0/22**. Longest prefix match in forwarding.
- **NAT:** maps private addresses to public; PAT uses ports.

9.8.2 IPv4 Header

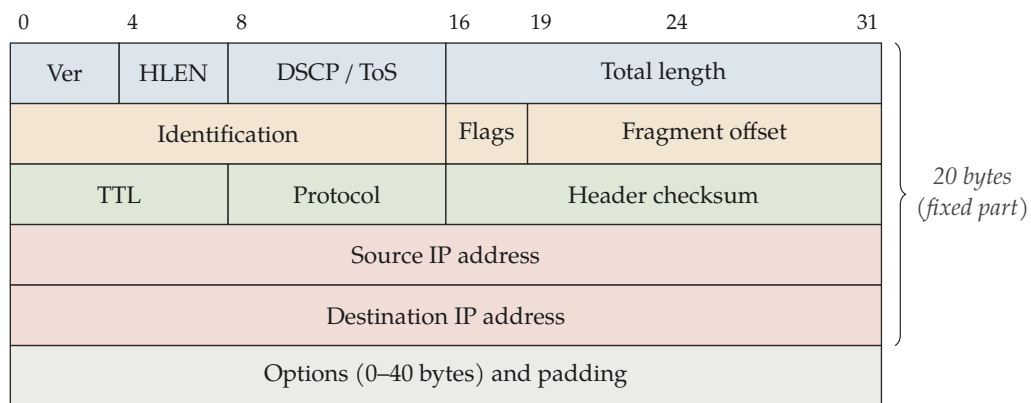


FIGURE 9.9. The IPv4 header, 32 bits per row. Header length = HLEN × 4 bytes (20–60); flags are reserved, DF and MF; fragment offset counts 8-byte units; protocol 1 = ICMP, 6 = TCP, 17 = UDP; maximum datagram 65 535 bytes.

Fragmentation worked: 4000-byte datagram (20 B header) over MTU 1500 → data per fragment = 1480 (multiple of 8): fragments carry 1480, 1480, 1020 bytes; offsets **0, 185, 370**; MF = 1, 1, 0.

9.8.3 IPv6

- 128-bit address (hex, colon notation; :: compresses zeros once), fixed **40-byte header**, no header checksum, **no fragmentation at routers** (source only), extension headers, flow label, built-in IPsec support, types: unicast, multicast, **anycast** (no broadcast).
- Transition: dual stack, tunnelling, header translation.

9.8.4 Routing

	Distance Vector	Link State	Path Vector
Algorithm	Bellman–Ford	Dijkstra	—
Knowledge	Neighbours' tables	Whole topology (LSPs flooded)	Paths (AS list)
Protocol	RIP	OSPF (also IS-IS)	BGP
Problem	Count-to-infinity (fix: split horizon, poison reverse, hold-down)	Heavy computation/flooding	Policy-based
Scope	Intra-AS (IGP)	Intra-AS (areas, backbone area 0)	Inter-AS (EGP)

- RIP: hop count metric, max **15** (16 = infinity), updates every **30 s**, runs over **UDP 520**.
- OSPF: cost metric, runs directly over **IP (protocol 89)**, hierarchical areas.
- BGP: runs over **TCP 179**; eBGP & iBGP.
- Multicast routing: DVMRP, MOSPF, PIM; **IGMP** for group membership.

9.8.5 Support Protocols

Protocol	Purpose
ARP	IP → MAC (broadcast request, unicast reply)
RARP / BOOTP / DHCP	Obtain IP address (DHCP: DORA — Discover, Offer, Request, Ack; UDP 67/68)
ICMP	Error reporting & query: echo (ping), destination unreachable, time exceeded (traceroute), redirect, source quench
IGMP	Multicast group membership

9.9 Transport Layer

The transport layer provides process-to-process delivery using port numbers. TCP adds reliability, ordering, flow control and congestion control on top of IP; UDP adds almost nothing and is therefore fast.

Feature	TCP	UDP	SCTP
Connection	Connection-oriented	Connectionless	Connection-oriented (association)
Reliability	Reliable, ordered	Unreliable	Reliable, multi-stream
Unit	Byte stream	Message	Message (chunks)
Header	20–60 B	8 B	12 B common header
Flow/congestion control	Yes	No	Yes
Use	HTTP, FTP, SMTP	DNS, SNMP, DHCP, VoIP, TFTP	Signalling; multihoming

9.9.1 TCP Connection Management

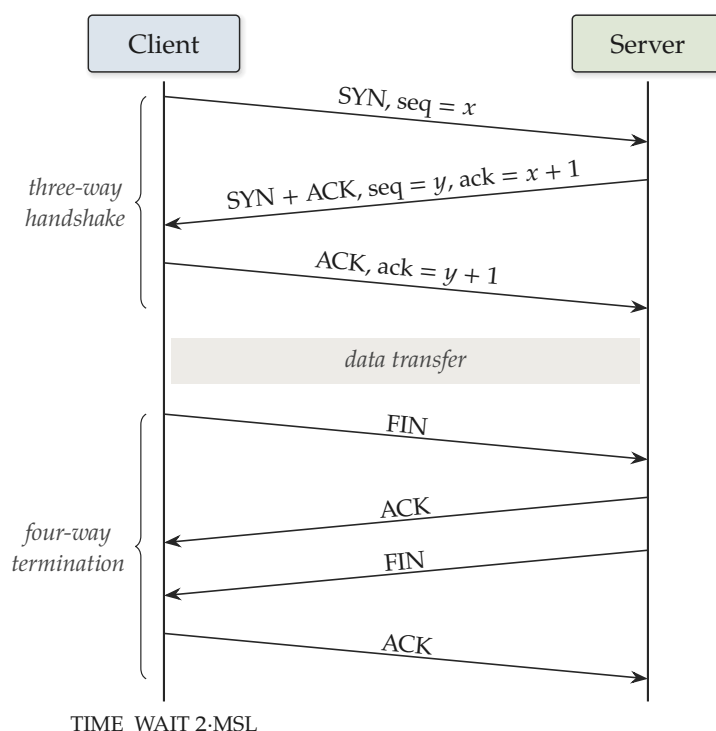


FIGURE 9.10. TCP connection establishment and release.

- SYN and FIN each **consume one sequence number**. Sequence numbers count bytes.
- TCP header flags: URG, ACK, PSH, RST, SYN, FIN. Window field 16 bits (window scaling option extends).
- **Wrap-around time** = $2^{32} / \text{bandwidth (bytes/s)}$.
- **Silly window syndrome**: Nagle's algorithm (sender), Clark's solution (receiver).
- Timers: retransmission (RTO via Jacobson/Karn), persistence, keep-alive, TIME-WAIT.
- RTT estimate: $\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstRTT} + \alpha \cdot \text{SampleRTT}$ ($\alpha = 1/8$).

9.9.2 TCP Congestion Control

Worked: ssthresh = 16, starting cwnd = 1 MSS: 1, 2, 4, 8, 16 (slow start), then 17, 18, 19 ... (congestion avoidance). If timeout at cwnd = 20: ssthresh = 10, cwnd = 1.

- Effective window = $\min(\text{cwnd}, \text{rwnd})$.
- **QoS**: leaky bucket (constant output rate), **token bucket** (allows bursts: $\text{max burst } S = C / (M - \rho)$), IntServ (RSVP), DiffServ.

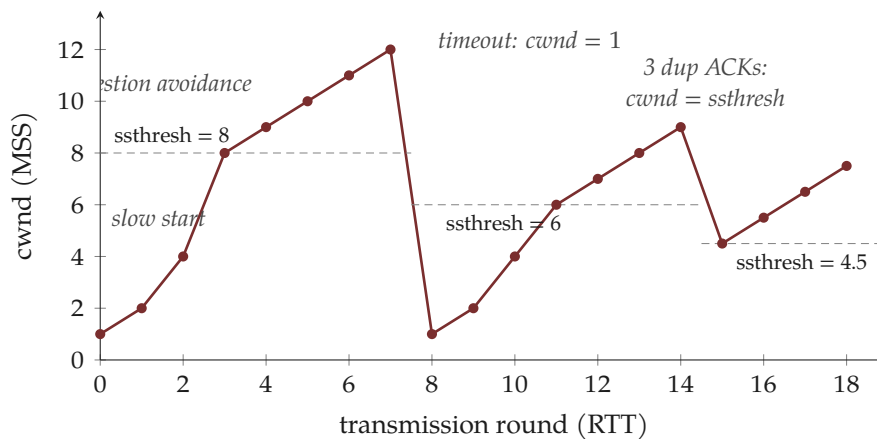


FIGURE 9.11. TCP congestion control. Slow start doubles cwnd each RTT up to ssthresh, then congestion avoidance adds one MSS per RTT. A timeout halves ssthresh and restarts from 1 (Tahoe); three duplicate ACKs halve cwnd and skip slow start (Reno fast recovery).

9.9.3 Well-known Ports

FTP	20 / 21	DNS	53	POP3	110	SNMP	161 / 162
SSH	22	DHCP	67 / 68	NTP	123	BGP	179
TELNET	23	TFTP	69	IMAP	143	HTTPS	443
SMTP	25	HTTP	80				

Port ranges: well-known 0–1023, registered 1024–49151, dynamic 49152–65535. (FTP: 20 data, 21 control.)

9.10 Application Layer

- **DNS**: hierarchical (root → TLD → authoritative); records **A** (IPv4), **AAAA** (IPv6), **MX** (mail), **CNAME** (alias), **NS**, **PTR** (reverse), **SOA**; recursive vs iterative resolution; UDP 53 (TCP for zone transfers). **DDNS** updates records dynamically.
- **HTTP**: stateless; non-persistent (HTTP/1.0; 2 RTT per object) vs persistent (HTTP/1.1, pipelining); cookies for state; methods GET, POST, PUT, DELETE, HEAD; status 1xx info, 2xx success (200 OK), 3xx redirect (301), 4xx client error (404), 5xx server error (500). HTTP/2 multiplexing, HTTP/3 over QUIC (UDP).
- **FTP**: two connections — control (21, persistent, out-of-band) & data (20, per file).
- **Email**: SMTP (push), POP3/IMAP (pull), MIME for attachments. See [Paper 1 ICT](#).
- **TELNET** (insecure remote login; NVT), **SSH** (secure).
- **SNMP**: manager-agent, **MIB**, **SMI**; GET, SET, TRAP; UDP 161/162.
- **WWW**: URL = protocol://host:port/path; web caching (proxy).
- **Bluetooth** (IEEE 802.15.1): piconet (1 primary + up to 7 active secondaries), scatternet; 2.4 GHz FHSS.

9.11 Network Security

Network security protects data in transit (confidentiality, integrity, authentication) and the network itself (availability). Cryptography supplies the first three; firewalls and intrusion-detection systems defend the network.

9.11.1 Goals & Attacks

CIA: **Confidentiality, Integrity, Availability** + authentication, non-repudiation. Passive attacks (eavesdropping, traffic analysis) vs active (masquerade, replay, modification, DoS). Malware: virus, worm, Trojan, ransomware, spyware, rootkit, botnet, logic bomb, keylogger.

9.11.2 Cryptography

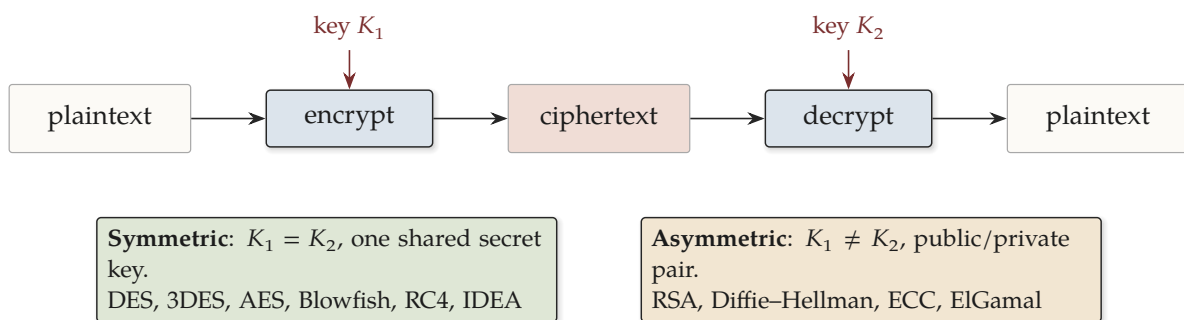


FIGURE 9.12. Encryption and decryption. Symmetric ciphers are fast but need a shared secret; public-key ciphers solve key distribution and enable digital signatures.

Algorithm	Type	Key facts
Caesar	Substitution (mono-alphabetic)	Shift by k ($k = 3$)
Vigenère	Poly-alphabetic	Keyword shifts
Playfair	Digraph substitution	5×5 matrix
Rail fence, columnar	Transposition	Rearranges letters
DES	Symmetric block (Feistel)	64-bit block, 56-bit key, 16 rounds
3DES	Symmetric	EDE, 112/168-bit key
AES (Rijndael)	Symmetric block (SPN)	128-bit block; keys 128/192/256 → 10/12/14 rounds
IDEA	Symmetric	64-bit block, 128-bit key
RC4	Stream cipher	WEP
RSA	Asymmetric	Factoring problem
Diffie–Hellman	Key exchange	Discrete log; vulnerable to man-in-the-middle
ECC	Asymmetric	Smaller keys for same security
MD5 / SHA-1 / SHA-256	Hash	128 / 160 / 256-bit digests

- Number of keys for n users: symmetric $\mathbf{n(n - 1)/2}$, asymmetric **2n**.
- **RSA worked:** $p = 3, q = 11 \rightarrow n = 33, \phi = 20$; choose $e = 3$ ($\gcd(3,20)=1$); $d = 7$ ($3 \times 7 = 21 \equiv 1 \pmod{20}$). Encrypt $M = 4$: $C = 4^3 \pmod{33} = 64 \pmod{33} = \mathbf{31}$. Decrypt: $31^7 \pmod{33} = \mathbf{4}$.

- **Diffie–Hellman worked:** $p = 23, g = 5, a = 6, b = 15 \rightarrow A = 5^6 \bmod 23 = 8, B = 5^{15} \bmod 23 = 19$; shared key $= 19^6 \bmod 23 = 8^{15} \bmod 23 = 2$.

9.11.3 Digital Signature & PKI

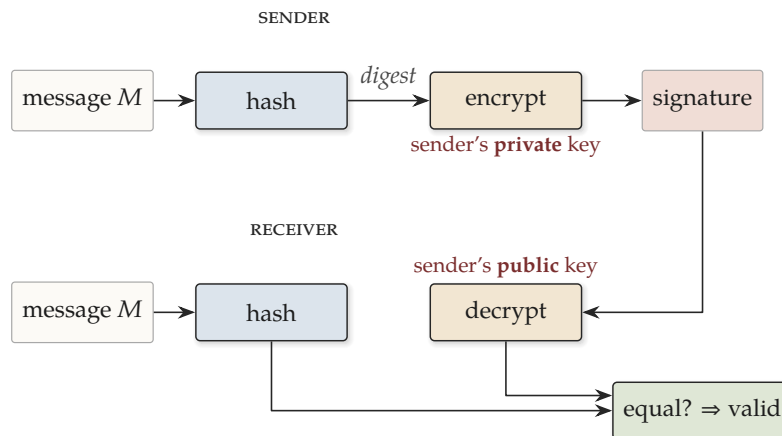


FIGURE 9.13. Digital signature. Signing uses the sender's private key, so anyone with the public key can verify it. For confidentiality the roles reverse: encrypt with the receiver's public key, decrypt with the receiver's private key.

Provides authentication, integrity, **non-repudiation**. Certificates (X.509) issued by **CA**; MAC/HMAC for message integrity with shared key.

- **Steganography:** hiding the existence of a message (e.g. in image LSBs), vs cryptography (hides meaning).

9.11.4 Security Protocols & Devices

- **IPsec** (network layer): AH (authentication), ESP (encryption); transport vs tunnel mode; IKE.
- **SSL/TLS** (transport layer): handshake, record protocol; HTTPS. **PGP** & S/MIME (email).
- **VPN:** secure tunnel over public network (IPsec, SSL VPN, PPTP, L2TP).
- **Firewalls:** packet filter (network/transport headers — stateless), stateful inspection, **proxy / application-level gateway**, circuit-level gateway. DMZ. **IDS/IPS** (signature vs anomaly).
- Kerberos (tickets, KDC: AS + TGS).

9.12 Mobile & Wireless

9.12.1 Cellular Concepts

- Frequency reuse; cluster size $N = i^2 + ij + j^2$ (1, 3, 4, 7, 12...); co-channel reuse distance $D = R\sqrt{3N}$.
- Handoff (hard — GSM, soft — CDMA). Generations: 1G (analog AMPS), 2G (GSM/CDMA digital voice, SMS), 2.5G (**GPRS**), 2.75G (EDGE), 3G (UMTS/WCDMA, CDMA2000), 4G (LTE, all-IP, OFDMA), 5G (mmWave, massive MIMO, URLLC, eMBB, mMTC).

9.12.2 GSM Architecture

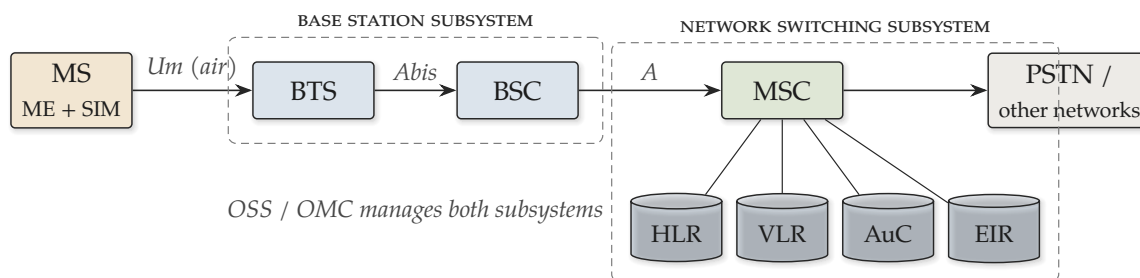


FIGURE 9.14. GSM architecture. The HLR holds permanent subscriber data, the VLR data for visitors currently in the area, the AuC authentication keys and the EIR the identities of handsets.

- **GSM**: TDMA + FDMA; 900/1800 MHz; 200 kHz carriers, 8 time slots; **HLR** (permanent subscriber data), **VLR** (temporary for visitors), **AuC** (authentication keys), **EIR** (IMEI database). IMSI in SIM.
- **GPRS**: packet-switched over GSM; SGSN & GGSN nodes. **SMS**: 160 characters, via SMSC, uses signalling channels.
- **CDMA (IS-95)**: spread spectrum, unique codes, soft handoff, RAKE receiver.

9.12.3 Mobile IP & Others

- **Mobile IP**: home agent, foreign agent, **care-of address (CoA)**, registration, **tunnelling**, triangle routing problem (route optimisation).
- Mobile computing middleware & gateways (WAP; WAP gateway translates between WAP & HTTP).
- **MANET**: infrastructure-less, self-configuring; routing — proactive (**DSDV**, **OLSR**), reactive (**AODV**, **DSR**), hybrid (**ZRP**).
- **Wireless LAN (IEEE 802.11)**: infrastructure (BSS, ESS via APs) vs ad hoc (IBSS); 802.11a/b/g/n/ac/ax (Wi-Fi 6). Security WEP → WPA → WPA2 (AES-CCMP) → WPA3.
- **Satellites**: GEO (35,786 km; ~270 ms one-way delay; 3 satellites cover earth), MEO (GPS ~20,200 km), LEO (500–2000 km; Starlink, Iridium). Uplink frequency > downlink.
- **Wireless geolocation**: GPS (trilateration, ≥ 4 satellites for 3-D fix), cell-ID, AoA, ToA, TDoA, RSSI. India: **NavIC (IRNSS, 7 satellites)**.

9.13 Cloud Computing & IoT

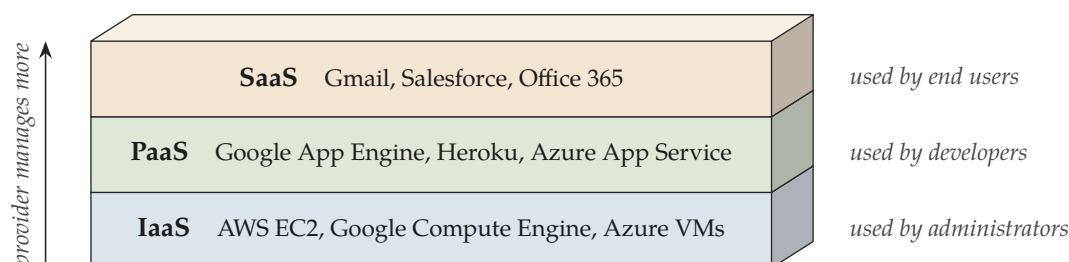


FIGURE 9.15. Cloud service models. Going up the stack the provider manages more and the customer less.

- **NIST essential characteristics (5)**: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.
- Deployment: **public, private, community, hybrid**.
- **Virtualisation**: hypervisors (see **OS unit**); virtual servers; live migration.
- Cloud storage (object: S3; block: EBS; file), database-as-a-service; multi-tenancy; resource management (load balancing, auto-scaling).
- **SLA** (Service Level Agreement): availability (e.g. 99.9% → ~8.76 h downtime/year), response time, penalties.
- **IoT**: things with sensors + connectivity. Architecture (3-layer): **Perception (sensing) → Network → Application** (5-layer adds processing/middleware & business). Protocols: **MQTT** (publish–subscribe over TCP), **CoAP** (REST over UDP), 6LoWPAN, Zigbee (802.15.4), BLE, LoRaWAN, NB-IoT, RFID/NFC. Edge/fog computing reduces latency.

9.14 Deeper Dive — VLSM, Routing Tables, TCP Numbers, Hamming & Ciphers

9.14.1 VLSM — Worked

Split **192.168.1.0/24** for LANs needing 100, 50, 25 and 10 hosts (allocate largest first):

LAN	Hosts needed	Prefix (usable)	Subnet	Range	Broadcast
A	100	/25 (126)	192.168.1.0	.1 – .126	.127
B	50	/26 (62)	192.168.1.128	.129 – .190	.191
C	25	/27 (30)	192.168.1.192	.193 – .222	.223
D	10	/28 (14)	192.168.1.224	.225 – .238	.239
Free	—	/28	192.168.1.240	—	—

9.14.2 Distance-Vector Update — Worked

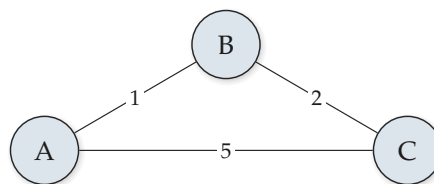


FIGURE 9.16. After B advertises “C at cost 2”, A replaces its direct route to C (cost 5) by the route via B (cost 3).

A's table	Initially	After receiving B's vector (B: A 1, B 0, C 2)
to A	0	0
to B	1 (direct)	1 (direct)
to C	5 (direct)	$\min(5, 1 + 2) = 3$ via B

Bellman–Ford equation: $D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$.

Count-to-infinity: suppose link B–C fails. B may then believe A's stale advertisement "C at cost 3" (which was itself via B) and set $C = 1 + 3 = 4$ via A; A then updates to $1 + 4 = 5$ via B, and the two keep raising each other's cost one step at a time. Here the climb stops once it exceeds A's direct link (cost 5), but with no alternative path it would continue until "infinity" (16 in RIP). **Split horizon** (never advertise a route back to the neighbour it was learned from) and **poison reverse** (advertise it as ∞) cure this two-node loop.

9.14.3 TCP Sequence and Acknowledgement Numbers

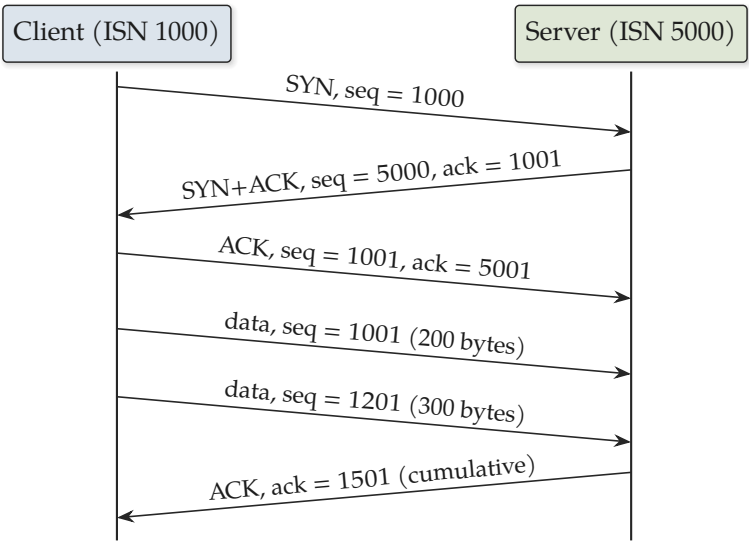


FIGURE 9.17. TCP sequence numbers count bytes. SYN and FIN each consume one number; the acknowledgement names the next byte expected.

Next expected byte = last seq + length; SYN and FIN consume one number each.

Go-Back-N count: window 4, frames 0–6, frame 2 lost once (no other losses). Assume frames 0–4 have been sent when frame 2's timer expires (ACKs for 0 and 1 slid the window to 2–5). Sent 0, 1, 2 (lost), 3, 4 → on timeout resend 2, 3, 4, then send 5, 6. Total transmissions = 5 + 3 + 2 = 10. Selective Repeat resends only frame 2: 7 + 1 = 8.

9.14.4 Hamming (7, 4) — Encode and Correct

Data 1011 → positions: 1 p1, 2 p2, 3 d1, 4 p4, 5 d2, 6 d3, 7 d4 (even parity).

POSITION	1	2	3	4	5	6	7	
ROLE	p_1	p_2	d_1	p_4	d_2	d_3	d_4	
CODEWORD	0	1	1	0	0	1	1	$c_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7 = 0$ $c_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7 = 1$ $c_4 = b_4 \oplus b_5 \oplus b_6 \oplus b_7 = 1$
RECEIVED	0	1	1	0	0	0	1	syndrome $c_4c_2c_1 = 110_2 = 6$

FIGURE 9.18. Hamming (7, 4) code for data 1011. Parity bits (ochre) sit at the power-of-two positions; p_1 covers positions 1, 3, 5, 7, p_2 covers 2, 3, 6, 7 and p_4 covers 4, 5, 6, 7. When bit 6 flips, the syndrome spells 6 in binary, so the receiver flips it back.

9.14.5 Delay Calculations

- Store-and-forward: a 1000-byte packet over 3 links of 1 Mbps (propagation ignored) $\rightarrow 3 \times 8 \text{ ms} = 24 \text{ ms}$.
- Message of 3 packets over the same path (pipelining): $(3 + 3 - 1) \times 8 = 40 \text{ ms}$.
- Circuit switching instead: setup time + message/bandwidth + propagation.

9.14.6 IPv6 Address Compression

Full	Compressed
2001:0db8:0000:0000:0000:ff00:0042:8329	2001:db8::ff00:42:8329
fe80:0000:0000:0000:0202:b3ff:fe1e:8329	fe80::202:b3ff:fe1e:8329
0000:0000:0000:0000:0000:0000:0000:0001	::1 (loopback)

Rules: drop leading zeros in each group; replace **one** longest run of all-zero groups with ::.

9.14.7 DNS Resolution

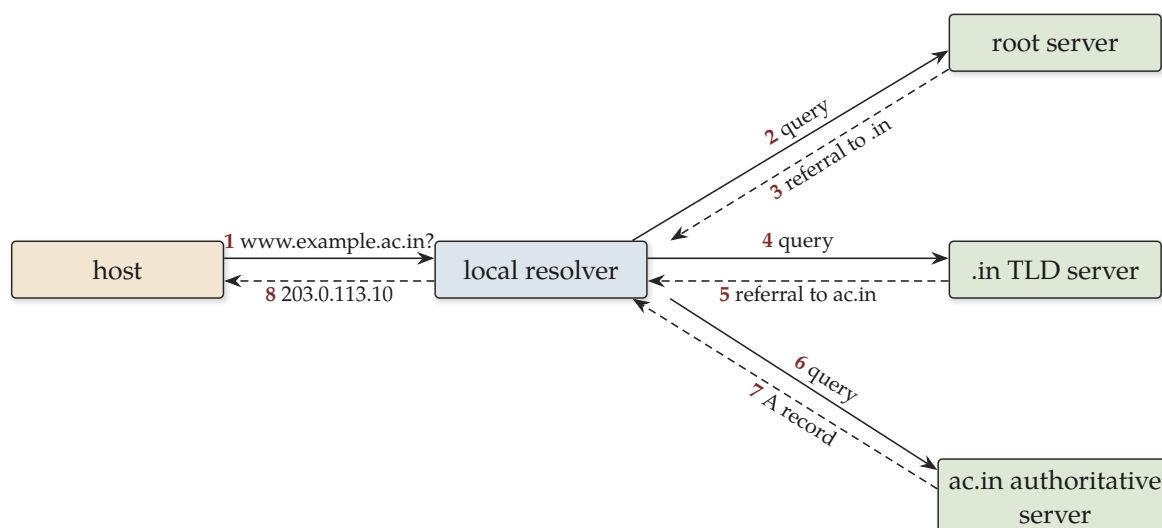


FIGURE 9.19. DNS resolution. The host asks its local resolver recursively (1, 8); the resolver queries the hierarchy iteratively (2–7) and caches the answer for its TTL.

9.14.8 Classical Ciphers — Worked

- **Caesar** ($k = 3$): NET $\rightarrow$ QHW.
- **Vigenère**, key LEMON: ATTACK $\rightarrow$ A+L = L, T+E = X, T+M = F, A+O = O, C+N = P, K+L = V $\rightarrow$ LXFOPV.
- **Rail fence** (2 rails): HELLOWORLD $\rightarrow$ rails HLOOL / ELWRD $\rightarrow$ HLOOLELWRD.
- **Columnar transposition** rearranges letters by a key order; **substitution** replaces letters. Product ciphers (DES, AES) combine both (confusion + diffusion — Shannon).

9.14.9 HTTP Exchange (*what a request looks like*)

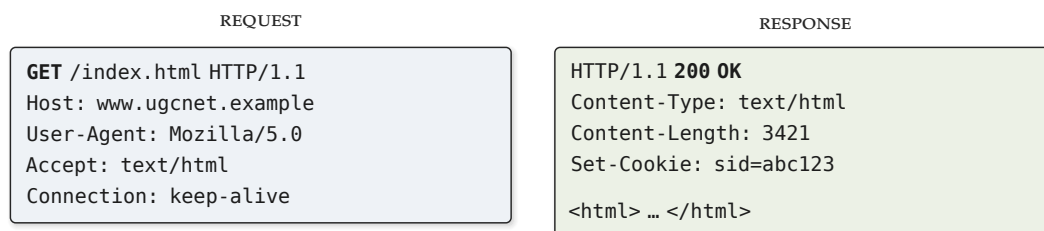


FIGURE 9.20. An HTTP request and its response: a start line, header lines, a blank line and (for the response) the body.

9.15 Previous Year Questions (PYQ pattern)

Models & topologies

1. Which OSI layer handles encryption & compression? **Presentation**
2. Process-to-process delivery is the job of: **Transport layer**
3. Dialog control & synchronisation: **Session layer**
4. Number of links in a fully connected mesh of 10 nodes: **45**
5. Router operates at: **Network layer**; switch at: **Data link layer**
6. Number of collision domains with a 24-port switch: **24**; broadcast domains: **1**

Physical layer

7. Channel B = 4 kHz, SNR = 63: $C = 4000 \times \log_2 64 =$ **24 kbps**
8. Noiseless 3 kHz channel with 8 levels: $2 \times 3000 \times 3 =$ **18 kbps**
9. Manchester encoding needs bandwidth: **twice that of NRZ** (it is self-clocking)
10. Sampling a 4 kHz voice signal per Nyquist: **8000 samples/s**
11. 16-QAM carries how many bits per symbol? **4**
12. T1 line data rate: **1.544 Mbps**
13. Medium immune to electromagnetic interference: **optical fibre**

Data link

14. Bit stuffing after five consecutive 1s inserts: **a 0**
15. CRC with generator of degree 4 appends how many bits? **4**
16. Min Hamming distance to correct 2-bit errors: **5**
17. Go-Back-N with 3-bit sequence numbers: max sender window **7**; Selective Repeat: **4**
18. Stop-and-wait, $T_t = 1$ ms, $T_p = 2$ ms: $\eta = 1/(1 + 4) =$ **20%**
19. Window size for full utilisation with $a = 10$: **21**
20. Which protocol uses cumulative ACKs and discards out-of-order frames? **Go-Back-N**
21. PPP authentication protocol using a challenge: **CHAP**

MAC

22. Max throughput of slotted ALOHA: **36.8%**; pure ALOHA: **18.4%**

23. CSMA/CD: 1 Gbps, 2 km cable, signal speed 2×10^8 m/s $\rightarrow T_p = 10 \mu\text{s} \rightarrow L_{\text{min}} = 2 \times 10 \mu\text{s} \times 10^9 = \mathbf{20,000 \text{ bits}}$
24. Hidden terminal problem is solved by: **RTS/CTS (CSMA/CA)**
25. Minimum Ethernet frame size: **64 bytes**
26. Binary exponential backoff is used in: **CSMA/CD Ethernet**

Network layer

27. Class of 191.10.20.1: **Class B**
28. Usable hosts in a /27 subnet: **30**
29. Network address of 172.16.45.14/20: $45 = 0010 \ 1101 \rightarrow /20$ keeps 0010 $\rightarrow \mathbf{172.16.32.0}$
30. Broadcast address of 10.1.1.130/25: **10.1.1.255**
31. Subnet mask for 500 hosts per subnet: **/23 (255.255.254.0)**
32. IPv4 header field preventing infinite looping: **TTL**
33. Fragment offset is measured in units of: **8 bytes**
34. IPv6 address size: **128 bits**; header: **40 bytes**
35. RIP uses: **distance vector, hop count (max 15)**
36. OSPF uses: **link state (Dijkstra)**
37. BGP is a: **path vector, inter-domain protocol over TCP 179**
38. Count-to-infinity occurs in: **distance vector routing**
39. Which protocol maps IP to MAC? **ARP**
40. ping and traceroute use: **ICMP**
41. DHCP message sequence: **Discover, Offer, Request, Acknowledge**

Transport & application

42. UDP header size: **8 bytes**
43. TCP 3-way handshake segments: **SYN, SYN-ACK, ACK**
44. After a timeout with cwnd = 32, new ssthresh: **16**, cwnd: **1**
45. Port of SMTP **25**, DNS **53**, HTTP **80**, HTTPS **443**, FTP control **21**
46. DNS record for mail server: **MX**; for IPv6: **AAAA**
47. FTP uses __ connections: **two (control + data)**; control channel is **out-of-band**
48. HTTP is: **stateless** (state maintained via cookies)
49. SNMP database of managed objects: **MIB**
50. Token bucket: capacity $C = 1 \text{ MB}$, token rate $\rho = 2 \text{ MBps}$, max rate $M = 10 \text{ MBps} \rightarrow \text{max burst time} = C / (M - \rho) = \mathbf{0.125 \text{ s}}$

Security

51. DES key length: **56 bits**; rounds: **16**
52. AES-128 rounds: **10**
53. RSA with $p = 5$, $q = 11$, $e = 3 \rightarrow \phi = 40$, $d = 27$
54. In digital signatures, the sender signs with: **own private key**
55. Number of keys for 100 users with symmetric crypto: **4950**; asymmetric: **200**
56. Diffie–Hellman is vulnerable to: **man-in-the-middle attack**
57. Hiding a message inside an image: **steganography**
58. IPsec operates at: **network layer**; SSL/TLS at: **transport layer**

59. A firewall that examines application-layer data: **proxy (application-level gateway)**

Mobile, cloud, IoT

60. GSM database holding permanent subscriber information: **HLR**; temporary visitor data: **VLR**

61. GPRS is a: **packet-switched 2.5G service**

62. In Mobile IP, the address used at the foreign network: **care-of address**

63. AODV is a: **reactive (on-demand) MANET routing protocol**

64. GEO satellite altitude: **$\approx 35,786$ km**

65. Google App Engine is an example of: **PaaS**; AWS EC2: **IaaS**

66. Rapid elasticity is a characteristic of: **cloud computing (NIST)**

67. MQTT follows: **publish-subscribe model**

68. Bluetooth piconet can have at most __ active secondaries: **7**

More practice questions

69. Using VLSM on 192.168.1.0/24, the prefix for a LAN with 50 hosts: **/26**

70. Broadcast address of 192.168.1.192/27: **192.168.1.223**

71. Split horizon is a remedy for: **count-to-infinity**

72. Client ISN 1000; after the handshake it sends 200 bytes. Sequence number of the next segment: **1201**

73. In the GBN scenario of §14.3 (frames 0–4 sent before the timeout), total transmissions: **10**; with Selective Repeat: **8**

74. Hamming (7, 4) codeword for data 1011 (even parity): **0110011**

75. Syndrome 101 in Hamming (7, 4) means the error is in bit: **5**

76. Compressed form of 2001:0db8:0000:0000:0000:0000:0001: **2001:db8::1**

77. :: may appear in an IPv6 address: **only once**

78. Vigenère encryption of "ATTACK" with key "LEMON": **LXFOPV**

79. Caesar cipher with key 3 encrypts "NET" as: **QHW**

80. Store-and-forward delay of a 1000-byte packet over 3 links of 1 Mbps: **24 ms**

POINTS TO REMEMBER

- Shannon $C = B \log_2(1 + \text{SNR})$ · Nyquist $2B \log_2 L$
- $\eta_{\text{SW}} = 1/(1 + 2a)$ · GBN $W \leq 2^n - 1$ · SR $W \leq 2^{n-1}$ · $L_{\text{min}} = 2 \cdot T_p \cdot B$
- ALOHA 18.4% / 36.8% · Ethernet min 64 B
- $/n \text{ hosts} = 2^{(32-n)} - 2$ · RIP DV/15 hops/UDP 520 · OSPF LS/IP 89 · BGP PV/TCP 179
- TCP: slow start exponential $\rightarrow$ AIMD; timeout $\rightarrow$ cwnd 1
- DES 56/16 · AES 10/12/14 · sign with private, encrypt with receiver's public

Artificial Intelligence

...

Expected questions: 8–10 · Target: 8+ · Type: search traces, alpha-beta, fuzzy operations, perceptron, concepts (Rich & Knight, Russell & Norvig)

SYLLABUS COVERAGE

- ☐ Approaches to AI: Turing test, rational agents, state-space representation, heuristic search, game playing, minimax, alpha-beta pruning
- ☐ Knowledge representation: logic, semantic networks, frames, rules, scripts, conceptual dependency, ontologies, expert systems, uncertainty
- ☐ Planning: components, linear & non-linear planning, goal stack, hierarchical planning, STRIPS, partial-order planning
- ☐ NLP: grammars, parsing techniques, semantic analysis, pragmatics
- ☐ Multi-agent systems: agents vs objects vs expert systems, MAS structure, semantic web, agent communication, ontologies, tools
- ☐ Fuzzy sets: membership functions, fuzzification/defuzzification, operations, linguistic variables, fuzzy relations, fuzzy inference & control
- ☐ Genetic algorithms: encoding, operators, fitness function, GA cycle
- ☐ Neural networks: supervised, unsupervised, reinforcement learning; perceptron, MLP, SOM, Hopfield

10.1 Approaches to AI

Artificial intelligence studies how to build systems that perceive, reason, learn and act. Definitions differ in whether they aim to think or to act, and whether the yardstick is human behaviour or ideal rationality.

	Human-like	Rational
Thinking	Cognitive modelling (GPS — Newell & Simon)	Laws of thought (logic)
Acting	Turing test (1950, "Computing Machinery and Intelligence")	Rational agent (Russell & Norvig — the modern approach)

- **Turing test:** interrogator communicates via text with a human and a machine; machine passes if indistinguishable. Needed capabilities: NLP, knowledge representation, automated reasoning, machine learning (+ vision & robotics for "total Turing test").
- **Chinese Room** (John Searle): argues passing the test $\neq$ understanding (strong vs weak AI).

– Term “AI” coined by **John McCarthy** at **Dartmouth Conference (1956)**.

10.1.1 Agents

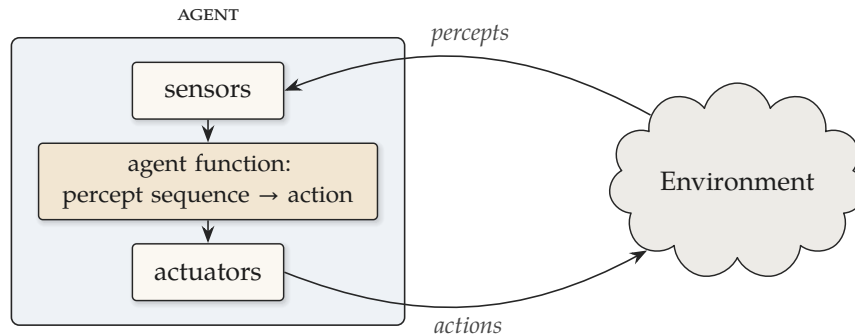


FIGURE 10.1. An agent perceives its environment through sensors and acts on it through actuators. A rational agent chooses the action expected to maximise its performance measure.

PEAS description: Performance measure, Environment, Actuators, Sensors. (Automated taxi: P = safety, speed, legality; E = roads, traffic; A = steering, brake; S = cameras, GPS, sonar.)

Agent type	Behaviour
Simple reflex	condition–action rules on current percept only
Model-based reflex	keeps internal state of the world
Goal-based	chooses actions to reach goals (search/planning)
Utility-based	maximises expected utility
Learning agent	performance element + learning element + critic + problem generator

Environment properties: fully/partially observable, deterministic/stochastic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent, known/unknown. (Chess with clock: fully observable, strategic, sequential, semi-dynamic, discrete, multi-agent.)

10.2 State Space Search

Many problems can be posed as finding a path through a space of states, from a start state to a goal state, using operators that turn one state into another. Search strategies differ in the order in which they explore states.

Problem = initial state, actions, transition model, **goal test**, path cost. Examples: 8-puzzle ($9!/2 = 181,440$ reachable states), water jug, missionaries & cannibals, 8-queens, Tower of Hanoi.

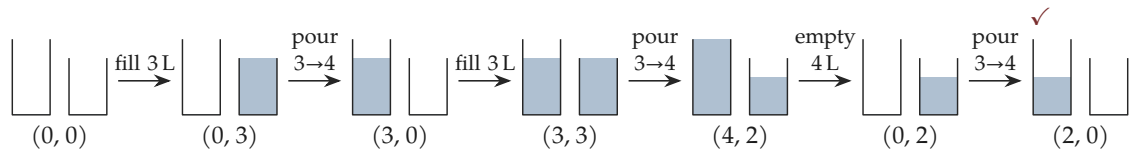


FIGURE 10.2. Water-jug problem (4 L and 3 L jugs, goal: 2 L in the 4 L jug). Each state is (4 L contents, 3 L contents); the search space is all reachable pairs.

10.2.1 Uninformed Search

b = branching factor, d = depth of shallowest goal, m = max depth, l = depth limit, C^* = optimal cost.

Strategy	Complete	Optimal	Time	Space
BFS	Yes (finite b)	Yes (unit cost)	$O(b^d)$	$O(b^d)$
Uniform-cost	Yes	Yes	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	same
DFS	No (infinite paths)	No	$O(b^m)$	$O(bm)$
Depth-limited	No	No	$O(b^l)$	$O(bl)$
Iterative deepening (IDDFS)	Yes	Yes (unit cost)	$O(b^d)$	$O(bd)$
Bidirectional	Yes	Yes	$O(b^{(d/2)})$	$O(b^{(d/2)})$

IDDFS combines BFS's completeness/optimality with DFS's memory — preferred uninformed method for large spaces.

10.2.2 Heuristic (Informed) Search

Greedy best-first: fast, but neither optimal nor complete

$$f(n) = h(n)$$

A*: optimal if h is admissible (tree search) or consistent (graph search)

$$f(n) = g(n) + h(n)$$

Admissible: never overestimates the true cost h^*

$$h(n) \leq h^*(n)$$

Consistent (monotone); consistent $\Rightarrow$ admissible

$$h(n) \leq c(n, n') + h(n')$$

Dominance: both admissible, h_2 expands fewer nodes

$$h_2(n) \geq h_1(n)$$

8-puzzle heuristics: h_1 = misplaced tiles, h_2 = **Manhattan distance** (h_2 dominates h_1).

A* worked — $h(S)=7$, $h(A)=6$, $h(B)=2$, $h(C)=1$, $h(G)=0$ (all admissible)

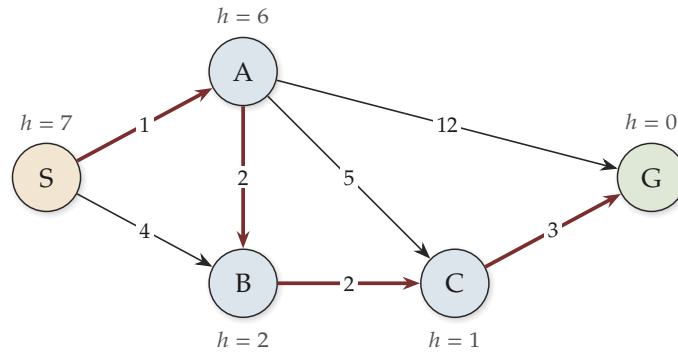


FIGURE 10.3. A* example. The optimal path S–A–B–C–G (cost 8) is drawn in red. Since $h(A) = 6 > c(A, B) + h(B) = 4$, the heuristic is admissible but not consistent.

Step	Expand (g, f)	Open list after expansion (g, f = g + h)
1	S (0, 7)	A (1, 7), B (4, 6)
2	B (4, 6)	A (1, 7), C (6, 7)
3	A (1, 7)	B (3, 5) — cheaper path found, B re-opened; C (6, 7); G (13, 13)
4	B (3, 5)	C (5, 6) ; G (13, 13)
5	C (5, 6)	G (8, 8)
6	G (8, 8)	Goal reached: S → A → B → C → G, cost 8 (optimal)

Note: h here is admissible but **not consistent** ($h(A) = 6 > c(A, B) + h(B) = 4$), which is why B had to be re-opened. With a consistent heuristic, a node is never re-expanded.

- **AO*** — for AND-OR graphs (problem decomposition).
- **IDA*** — iterative deepening with f-cost limits (memory $O(bd)$). **SMA*** — memory bounded.
- **Local search: Hill climbing** (steepest ascent) — problems: **local maxima, plateaus, ridges**; fixes: random restart, sideways moves. **Simulated annealing** — accepts worse moves with probability $e^{-(\Delta E/T)}$, T decreases. **Local beam search**. **Genetic algorithms** (§8).
- **Constraint satisfaction (CSP)**: variables, domains, constraints; backtracking + MRV (minimum remaining values), degree heuristic, LCV (least constraining value), forward checking, **arc consistency (AC-3)**.
- **Means-ends analysis (GPS)**: reduce the difference between current and goal state.

10.3 Game Playing

In two-player games the opponent also chooses moves. The game tree alternates MAX levels (our moves) and MIN levels (the opponent's), and the value of a position is worked out from the leaves upwards.

10.3.1 Minimax

MAX picks maximum, MIN picks minimum child value; complete & optimal against an optimal opponent. Time $O(b^m)$, space $O(bm)$.

10.3.2 Alpha-Beta Pruning

α = best value MAX can guarantee so far (lower bound); β = best value MIN can guarantee (upper bound). **Prune when $\alpha \geq \beta$.**

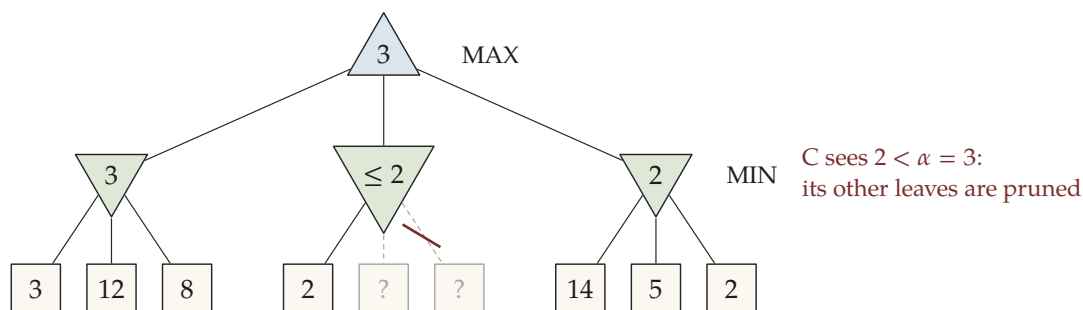


FIGURE 10.4. Alpha-beta pruning. MAX nodes point up, MIN nodes down. After B returns 3, MAX is guaranteed at least 3, so once C finds a 2 the rest of C need not be examined. The root value equals plain minimax: 3.

- Pruning does **not** change the final result.
- With perfect move ordering: time $O(b^{(m/2)})$ (effectively doubles the searchable depth); random ordering $\approx O(b^{(3m/4)})$.
- Stochastic games: **Expectiminimax** (chance nodes). Imperfect real-time decisions: cutoff test + evaluation function, quiescence search, horizon effect.

10.4 Knowledge Representation

An intelligent program needs knowledge in a form it can reason with: logic for precise inference, structured representations (semantic nets, frames, scripts) for organised everyday knowledge.

10.4.1 Logic

- **Propositional logic; First-order predicate logic (FOPL)** — see [Unit 1](#).
- Inference: **resolution** (refutation — negate goal, convert to **CNF/clausal form**, derive empty clause), **forward chaining** (data-driven; production systems, OPS5), **backward chaining** (goal-driven; Prolog, MYCIN).
- CNF conversion steps: eliminate $\leftrightarrow, \rightarrow$; move $\neg$ inward; standardise variables; **Skolemise** (remove $\exists$ using Skolem constants/functions); drop $\forall$; distribute $\vee$ over $\wedge$.

- **Unification**: find substitution making literals identical (most general unifier, MGU); occurs check.
- Horn clause: at most one positive literal — basis of Prolog.
- Monotonic vs **non-monotonic** reasoning (default logic, closed-world assumption, circumscription, truth-maintenance systems).

10.4.2 Structured Representations

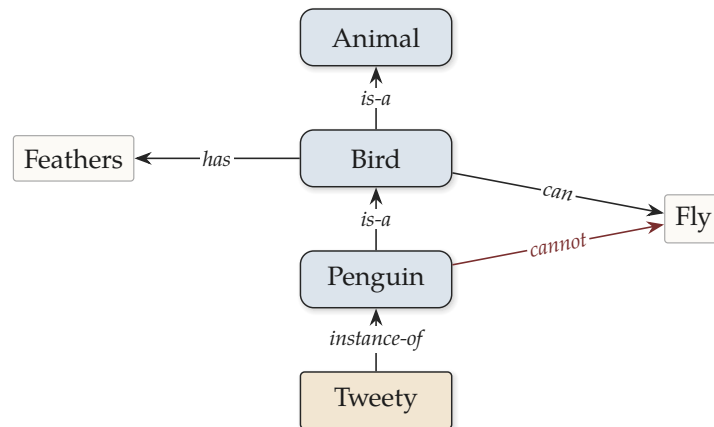


FIGURE 10.5. A semantic network. Properties are inherited down is-a links (Tweety has feathers); a more specific link (Penguin cannot fly) overrides an inherited default.

- **Semantic network** (Quillian): nodes = concepts, arcs = relations (is-a, has-a, instance); inheritance.
- **Frames** (Minsky, 1975): slots & fillers, default values, procedural attachments (if-needed, if-added demons).
- **Scripts** (Schank & Abelson): stereotyped event sequences — e.g. restaurant script: entry conditions, roles, props, scenes (entering, ordering, eating, exiting), results.
- **Conceptual Dependency** (Roger Schank): language-independent primitives — **ATRANS** (transfer possession: give), **PTRANS** (physical transfer: go), **MTRANS** (mental info: tell), **MBUILD** (build info: decide), **PROPEL**, **MOVE**, **GRASP**, **INGEST** (eat), **EXPEL**, **SPEAK**, **ATTEND** (11 primitive acts).
- **Production rules**: IF condition THEN action; conflict resolution (refractoriness, recency, specificity).
- **Ontologies**: formal specification of a shared conceptualisation (Gruber); OWL, RDF; upper ontologies.

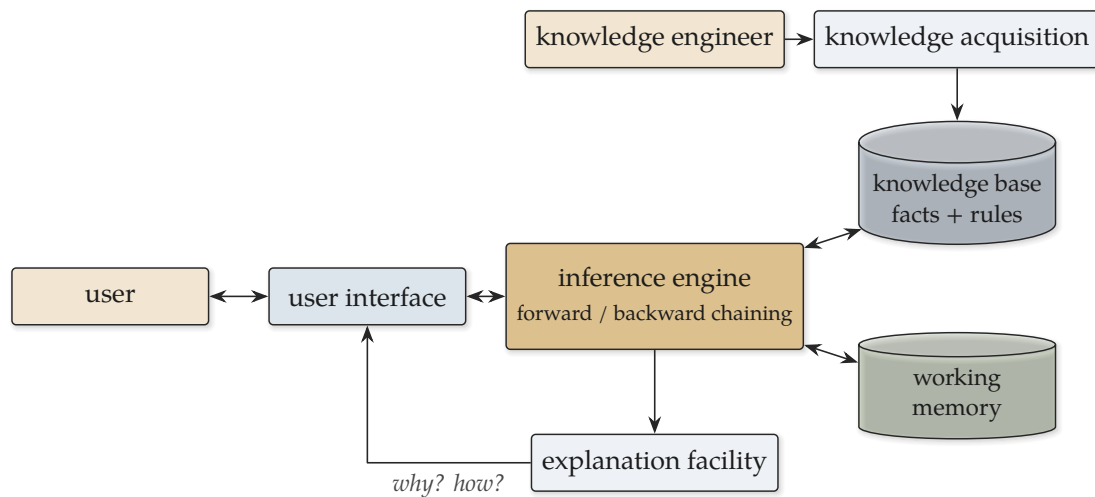


FIGURE 10.6. Architecture of an expert system. The knowledge base holds domain rules; the inference engine applies them to the facts in working memory and can explain its reasoning.

10.4.3 Expert Systems

Expert system	Domain
DENDRAL (first)	Chemical structure (mass spectrometry)
MYCIN	Blood infections — backward chaining, certainty factors
EMYCIN	Shell from MYCIN
PROSPECTOR	Mineral exploration
XCON / R1	DEC computer configuration — forward chaining
INTERNIST / CADUCEUS	Internal medicine

10.4.4 Uncertainty

Bayes' rule	$P(H E) = \frac{P(E H) P(H)}{P(E)}$
Certainty factor (MYCIN)	$CF = MB - MD \in [-1, 1]$
Combining two positive certainty factors	$CF = CF_1 + CF_2 (1 - CF_1)$
Dempster-Shafer: mass m over subsets	$\text{Bel}(A) \leq \text{Pl}(A)$
Bayesian network: a DAG with conditional probability tables	$P(x_1, \dots, x_n) = \prod_i P(x_i \text{parents}(x_i))$

Other: fuzzy logic (vagueness — §7), Markov models, HMMs.

10.5 Planning

Concept	Meaning
STRIPS (Fikes & Nilsson, 1971)	Operators with preconditions , add list , delete list ; closed-world assumption
Linear planning (goal stack)	Solve goals one at a time using a stack — can fail on interacting goals (Sussman anomaly)
Non-linear planning	Goals interleaved; constraint posting
Partial-order planning (POP)	Least-commitment; causal links, ordering constraints; resolves threats by promotion/demotion
Hierarchical planning (HTN)	Abstract actions decomposed into sub-tasks (ABSTRIPS)
Graphplan	Planning graph with mutex links
Forward (progression) vs backward (regression) state-space planning	

	PICKUP(x)	STACK(x, y)
PRE	ONTABLE(x) $\wedge$ CLEAR(x) $\wedge$ HANDEEMPTY	HOLDING(x) $\wedge$ CLEAR(y)
ADD	HOLDING(x)	ON(x, y), CLEAR(x), HANDEEMPTY
DEL	ONTABLE(x), CLEAR(x), HANDEEMPTY	HOLDING(x), CLEAR(y)

A STRIPS operator lists its preconditions and the facts it adds to and deletes from the state; every fact it does not mention is assumed unchanged (the STRIPS assumption).

Sussman anomaly: initial C on A, A and B on table; goal $\text{ON}(A, B) \wedge \text{ON}(B, C)$ — goal-stack (linear) planning cannot solve it without undoing a subgoal.

10.6 Natural Language Processing

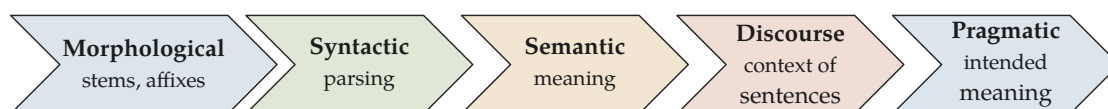


FIGURE 10.7. The stages of natural-language understanding, from the form of single words to the speaker's intention.

- Grammars: CFG, **Transformational grammar** (Chomsky), **ATN (Augmented Transition Network** — Woods), Case grammar (Fillmore), DCG, unification grammars, Systemic grammar.
- Parsing: top-down, bottom-up, **chart parsing** (Earley — $O(n^3)$), CYK, deterministic parsing.
- Ambiguity: lexical ("bank"), syntactic/structural ("I saw a man with a telescope"), semantic, anaphoric/referential ("he"), pragmatic.
- Semantic analysis: lexical semantics, word sense disambiguation, semantic grammar, case roles.
- **Pragmatics:** speech acts (Austin, Searle) — what the speaker intends; **discourse:** anaphora resolution.

- Modern NLP: n-grams, tokenisation, stemming (Porter) vs lemmatisation, POS tagging (HMM), NER, TF-IDF, word embeddings (word2vec), transformers/LLMs.
- Early systems: **ELIZA** (Weizenbaum, pattern matching therapist), SHRDLU (Winograd, blocks world), LUNAR, PARRY.

10.7 Multi-Agent Systems

	Object	Agent	Expert system
Autonomy	Methods invoked by others	Decides itself whether to act	Advises user
Behaviour	Passive	Proactive, reactive, social	Passive reasoning
Interaction	Method calls	Communication languages	User interface

- Agent properties: **autonomy, reactivity, pro-activeness, social ability** (Wooldridge & Jennings); BDI architecture (**B**eliefs, **D**esires, **I**ntentions).
- MAS structure: agents + environment + interactions (cooperation, coordination, negotiation — contract net protocol, auctions).
- **Agent communication languages**: **KQML** (performatives: ask, tell, achieve...), **FIPA-ACL**; content language **KIF**.
- Knowledge sharing via **ontologies**. **Semantic web** (Tim Berners-Lee): RDF (triples subject–predicate–object), RDFS, **OWL**, SPARQL; layered cake.
- Tools: **JADE** (Java, FIPA-compliant), **JACK**, Jason (AgentSpeak), Zeus, Aglets, NetLogo.

10.8 Fuzzy Sets & Logic (Zadeh, 1965)

In a classical set an element is either in or out. In a fuzzy set it belongs to a degree $\mu \in [0, 1]$, which lets systems reason with vague terms such as “hot” or “tall”.

Membership $\mu_A(x) \in [0, 1]$ (vs crisp $\{0, 1\}$).

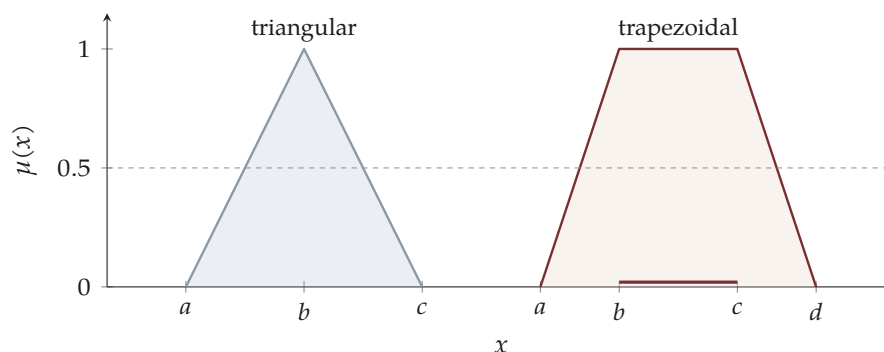


FIGURE 10.8. Membership functions. For the trapezoid, the *core* ($\mu = 1$) is $[b, c]$, the *support* ($\mu > 0$) is (a, d) and the *crossover* points are where $\mu = 0.5$ (dashed). A Gaussian has $\mu(x) = e^{-(x-c)^2/2\sigma^2}$.

10.8.1 Operations (standard Zadeh)

Union	$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
Intersection	$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
Complement	$\mu_{A'}(x) = 1 - \mu_A(x)$
Algebraic sum; algebraic product	$\mu_A + \mu_B - \mu_A \mu_B; \quad \mu_A \mu_B$
Bounded sum; bounded difference	$\min(1, \mu_A + \mu_B); \quad \max(0, \mu_A - \mu_B)$
Concentration ("very"); dilation ("somewhat")	$\mu^2; \quad \sqrt{\mu}$

Laws that FAIL in fuzzy sets: law of excluded middle ($A \cup A' \neq U$) and law of contradiction ($A \cap A' \neq \emptyset$). De Morgan, associativity, distributivity still hold.

Worked: $A = \{0.2/x_1, 0.7/x_2, 1/x_3\}$, $B = \{0.5/x_1, 0.3/x_2, 0.8/x_3\}$

$$\begin{array}{lll}
 A \cup B = \{0.5, 0.7, 1\} & A \cap B = \{0.2, 0.3, 0.8\} & A' = \{0.8, 0.3, 0\} \\
 A \cap A' = \{0.2, 0.3, 0\} \neq \emptyset & A_{0.5} = \{x_2, x_3\} \text{ (}\alpha\text{-cut)} & \text{height}(A) = 1 \text{ (normal)}
 \end{array}$$

- **Fuzzy relations:** max–min composition: $(R \circ S)(x, z) = \max_y \min(R(x, y), S(y, z))$. Max-product composition.
- **Linguistic variables:** e.g. Temperature $\in \{\text{cold, warm, hot}\}$; hedges: very, somewhat.
- **Fuzzy inference system:**

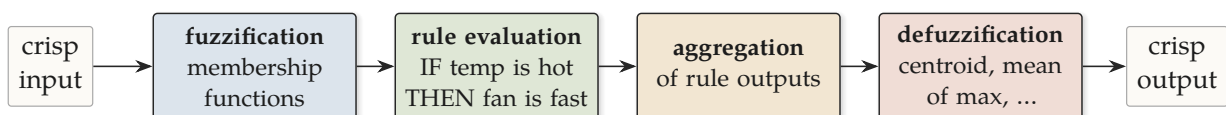


FIGURE 10.9. A fuzzy inference system (Mamdani). Crisp values are fuzzified, the rules fire to a degree, their outputs are combined, and a single crisp value is extracted.

- **Mamdani** (fuzzy output sets, centroid defuzzification — intuitive) vs **Sugeno / TSK** (output = linear function/constant, weighted average — efficient).
- Centroid: $z^* = \int \mu(z) \cdot z \, dz / \int \mu(z) \, dz$. Weighted average: $\sum \mu_i z_i / \sum \mu_i$.
- Applications: washing machines, ABS brakes, air-conditioners, cameras (fuzzy control).

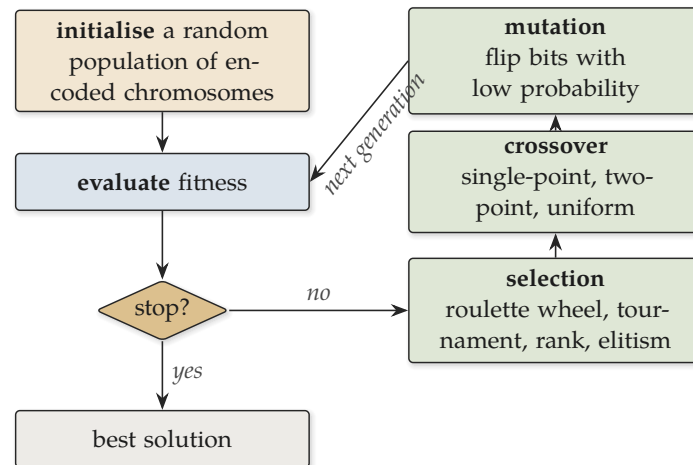


FIGURE 10.10. The genetic-algorithm cycle (Holland, 1975). Each generation keeps the fitter chromosomes, recombines them and mutates a few bits.

10.9 Genetic Algorithms (John Holland, 1975)

- **Encoding**: binary, permutation (TSP), value/real, tree (genetic programming).
- **Fitness function** guides selection. Roulette wheel probability $p_i = f_i / \sum f$.
- **Crossover** (exploitation of good genes) with probability ~ 0.6 – 0.9 ; **mutation** (exploration, maintains diversity) ~ 0.001 – 0.01 .
- Single-point crossover: $1011|010 \times 0110|111 \rightarrow 1011111, 0110010$.
- **Schema theorem** (Holland): short, low-order, above-average schemata grow exponentially (building block hypothesis). Order $o(H)$ = fixed positions; defining length $\delta(H)$ = distance between first & last fixed positions. Number of schemata in a string of length l : 3^l .
- Premature convergence; elitism keeps the best individuals.

10.10 Artificial Neural Networks

Artificial neural networks are built from simple units that compute a weighted sum of their inputs and pass it through an activation function. They learn by adjusting the weights from examples.

10.10.1 Learning Paradigms

Type	Data	Examples
Supervised	Labelled input–output pairs	Perceptron, back-propagation MLP, SVM, decision trees
Unsupervised	Unlabelled	SOM (Kohonen) , k-means, Hebbian, competitive learning, ART
Reinforcement	Rewards / penalties from environment	Q-learning, SARSA, TD learning; MDP (states, actions, rewards, policy)

Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.

10.10.2 Perceptron (Rosenblatt, 1958)

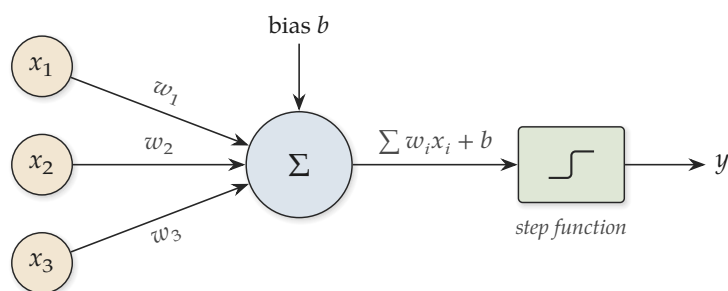


FIGURE 10.11. Rosenblatt's perceptron. Learning rule: $w_i \leftarrow w_i + \eta (t - y) x_i$, where t is the target and η the learning rate. A single perceptron can learn only linearly separable functions (not XOR).

- **McCulloch–Pitts neuron (1943)**: first model; binary threshold, fixed weights.
- Single-layer perceptron can learn only **linearly separable** functions — AND, OR, NAND ✓; **XOR ✗** (Minsky & Papert, 1969).
- AND with $w_1 = w_2 = 1$, threshold $\theta = 1.5$ (or bias -1.5); OR with $\theta = 0.5$.
- Perceptron convergence theorem: converges in finite steps if data is linearly separable.

10.10.3 Multilayer Perceptron & Backpropagation

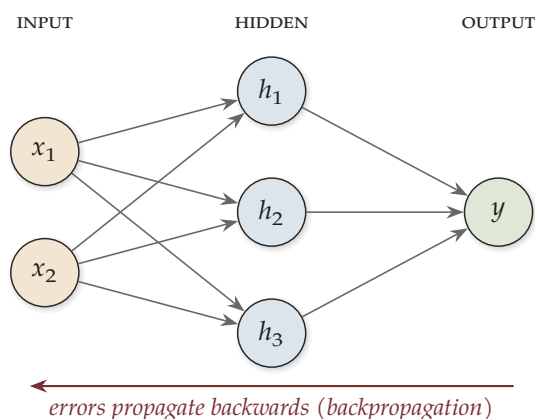


FIGURE 10.12. A multilayer perceptron. With a non-linear hidden layer it can represent XOR; training adjusts every weight by gradient descent on the output error.

- Input $\rightarrow$ hidden $\rightarrow$ output layers; non-linear activations: **sigmoid** $\sigma(x) = 1/(1 + e^{-x})$ with $\sigma' = \sigma(1 - \sigma)$, **tanh**, **ReLU** $\max(0, x)$, softmax (output).
- **Backpropagation** (Rumelhart, Hinton, Williams, 1986): forward pass $\rightarrow$ compute error $\rightarrow$ propagate gradients backwards (chain rule) $\rightarrow$ gradient descent: $w \leftarrow w - \eta \partial E / \partial w$. Problems: local minima, vanishing gradients (sigmoid); momentum, learning-rate tuning.
- MLP with one hidden layer is a **universal approximator**; can solve XOR.
- Overfitting $\rightarrow$ regularisation, dropout, early stopping.
- **Delta rule / Widrow–Hoff / LMS** (ADALINE): $\Delta w = \eta (t - y_{in})x$. MADALINE = multiple ADALINES.

10.10.4 Self-Organising Map (Kohonen)

- **Unsupervised, competitive** learning; maps high-dimensional input to a low-dimensional (2-D) grid preserving **topology**.
- Steps: find **Best Matching Unit (BMU)** (min Euclidean distance) → update BMU and neighbours: $w \leftarrow w + \eta \cdot h(t)(x - w)$; neighbourhood and η shrink over time.

10.10.5 Hopfield Network (1982)

- **Recurrent**, fully connected, **symmetric weights** ($w_{ij} = w_{ji}$), **no self-connections** ($w_{ii} = 0$); binary/bipolar states.
- **Associative (content-addressable) memory**: stores patterns via Hebbian rule $W = \sum p^T p - I$ (bipolar); retrieves from noisy input.
- Energy function $E = -\frac{1}{2} \sum \sum w_{ij} s_i s_j$ decreases monotonically → converges to stable state (local minimum).
- Capacity $\approx 0.138 N$ patterns for N neurons.
- Also: **Hebbian learning** ("neurons that fire together wire together") $\Delta w = \eta \cdot x \cdot y$; **BAM** (bidirectional associative memory — Kosko); **ART** (Grossberg – stability–plasticity); RBF networks; Boltzmann machine.

10.11.3 Forward vs Backward Chaining

Rules: R1: $A \wedge B \rightarrow C$; R2: $C \rightarrow D$; R3: $D \wedge E \rightarrow F$. Facts: A, B, E. Goal: F.

Forward chaining (data-driven)	Backward chaining (goal-driven)
A, B $\vdash$ C (R1)	Goal F $\leftarrow$ need D and E (R3)
C $\vdash$ D (R2)	E is a fact; D $\leftarrow$ need C (R2)
D, E $\vdash$ F (R3) ✓	C $\leftarrow$ need A and B (R1); both facts ✓

10.11.4 Bayes — Medical Test

Disease prevalence 1%, test sensitivity 99%, false-positive rate 5%.

$$P(D | +) = \frac{P(+ | D) P(D)}{P(+ | D) P(D) + P(+ | \neg D) P(\neg D)} = \frac{0.99 \times 0.01}{0.0099 + 0.05 \times 0.99} = \frac{0.0099}{0.0594} \approx 0.167$$

Only about 17 % of people who test positive actually have the disease: the disease is rare, so false positives from the large healthy group outnumber the true positives.

10.11.5 Fuzzy Max–Min Composition & Inference

$$R = \begin{pmatrix} 0.6 & 0.3 \\ 0.2 & 0.9 \end{pmatrix}, \quad S = \begin{pmatrix} 1.0 & 0.5 \\ 0.8 & 0.4 \end{pmatrix}, \quad T = R \circ S, \quad T_{ij} = \max_k \min(R_{ik}, S_{kj})$$

$$T_{11} = \max(\min(0.6, 1.0), \min(0.3, 0.8)) = \max(0.6, 0.3) = 0.6$$

$$T_{12} = \max(\min(0.6, 0.5), \min(0.3, 0.4)) = \max(0.5, 0.3) = 0.5$$

$$T_{21} = \max(\min(0.2, 1.0), \min(0.9, 0.8)) = \max(0.2, 0.8) = 0.8$$

$$T_{22} = \max(\min(0.2, 0.5), \min(0.9, 0.4)) = \max(0.2, 0.4) = 0.4$$

$$T = \begin{pmatrix} 0.6 & 0.5 \\ 0.8 & 0.4 \end{pmatrix}$$

Sugeno-style inference: temperature 30 °C $\rightarrow \mu_{\text{warm}} = 0.4, \mu_{\text{hot}} = 0.6$. Rules: warm $\rightarrow$ fan 40%, hot $\rightarrow$ fan 80%. Output = $(0.4 \times 40 + 0.6 \times 80) / (0.4 + 0.6) = 64\%$.

10.11.6 Genetic Algorithm — One Generation (maximise $f(x) = x^2$, 5-bit x)

String	x	$f(x)$	$p_i = f/\Sigma f$	Expected copies ($4p_i$)
01101	13	169	0.144	0.58
11000	24	576	0.492	1.97
01000	8	64	0.055	0.22
10011	19	361	0.309	1.23
$\Sigma = 1170$				

Crossover 0110|1 $\times$ 1100|0 at position 4 $\rightarrow$ **01100 (12)** and **11001 (25)** — the child 25 is fitter than any parent.

10.11.7 Perceptron Learning AND ($\eta = 1$, $w = (0, 0)$, $b = 0$, output 1 if $w \cdot x + b > 0$)

End of epoch	w_1	w_2	b	Errors in epoch
1	1	1	1	1
2	2	1	0	3
3	2	1	-1	3
4	2	2	-1	2
5	2	1	-2	1
6	2	1	-2	0 $\rightarrow$ converged

Check: (0,0) $\rightarrow -2 \rightarrow 0$; (0,1) $\rightarrow -1 \rightarrow 0$; (1,0) $\rightarrow 0 \rightarrow 0$; (1,1) $\rightarrow 1 \rightarrow 1$ ✓ = AND.

10.11.8 One Backpropagation Step (single sigmoid neuron)

$x = 1$, $w = 0.5$, $b = 0$, target $t = 1$, $\eta = 1$, $E = \frac{1}{2}(t - o)^2$.

$$\text{net} = 0.5, \quad o = \sigma(0.5) = 0.6225$$

$$\delta = (t - o) o (1 - o) = 0.3775 \times 0.6225 \times 0.3775 \approx 0.0887$$

$$\Delta w = \eta \delta x = 0.0887, \quad w_{\text{new}} = 0.5887$$

The weight grows, so the output moves towards the target.

10.11.9 Hopfield Storage & Recall

Store bipolar pattern $p = [1, -1, 1]$: $W = p p^T - I$.

$$W = \begin{pmatrix} 0 & -1 & 1 \\ -1 & 0 & -1 \\ 1 & -1 & 0 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad Wx = \begin{pmatrix} 0 \\ -2 \\ 0 \end{pmatrix} \xrightarrow{\text{sign, keep old value on 0}} \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}$$

The noisy input settles to the stored pattern $p = [1, -1, 1]$.

10.11.10 CSP — Map Colouring with Heuristics

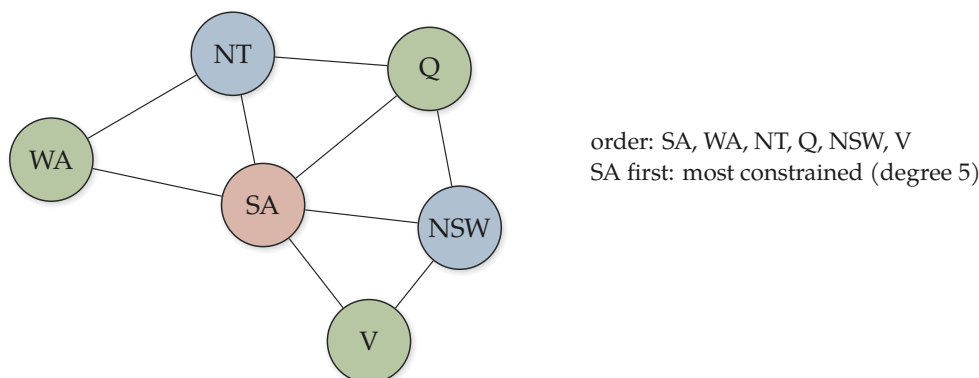


FIGURE 10.15. Map colouring of Australia as a CSP, solved with three colours and no backtracking using the MRV and degree heuristics with forward checking.

Three colours. **MRV/degree heuristic** picks SA first (degree 5). Assign SA = red $\rightarrow$ WA, NT, Q, NSW, V lose red; WA = green $\rightarrow$ NT = blue $\rightarrow$ Q = green $\rightarrow$ NSW = blue $\rightarrow$ V = green. Solution found without backtracking; **forward checking** removes inconsistent values early.

10.12 Previous Year Questions (PYQ pattern)

Foundations & agents

1. The term "Artificial Intelligence" was coined by: **John McCarthy (1956)**
2. Turing test evaluates: **whether a machine's behaviour is indistinguishable from a human's**
3. PEAS stands for: **Performance, Environment, Actuators, Sensors**
4. Agent that acts only on the current percept: **simple reflex agent**
5. Chess is: **fully observable, deterministic (strategic), sequential, discrete, multi-agent**

Search

6. Space complexity of BFS: **$O(b^d)$** ; of DFS: **$O(bm)$**
7. Search that combines benefits of BFS and DFS: **iterative deepening DFS**
8. A* is optimal if the heuristic is: **admissible (never overestimates)**
9. If $h(n) = 0$ for all n , A* reduces to: **uniform-cost search (Dijkstra)**
10. Greedy best-first search uses: **$f(n) = h(n)$**
11. Hill climbing can get stuck due to: **local maxima, plateau, ridges**
12. Simulated annealing accepts worse moves with probability: **$e^{-(\Delta E/T)}$**
13. Problem reduction (AND-OR graphs) uses: **AO***
14. Means-ends analysis was used in: **GPS (General Problem Solver)**
15. For 8-puzzle, Manhattan distance is: **admissible and dominates misplaced tiles**

Games

16. Alpha-beta pruning with perfect ordering reduces time to: **$O(b^{(m/2)})$**
17. Pruning occurs when: **$\alpha \geq \beta$**
18. Alpha-beta pruning changes the minimax value? **No**

19. Minimax value of tree with MIN nodes having leaves (3,12,8), (2,4,6), (14,5,2): $\max(3, 2, 2) = 3$

Knowledge representation

20. Frames were proposed by: **Marvin Minsky**
21. Scripts were proposed by: **Schank & Abelson**
22. "John gave Mary a book" in CD uses primitive: **ATRANS**
23. MYCIN used: **backward chaining with certainty factors**
24. First expert system: **DENDRAL**
25. Skolemisation removes: **existential quantifiers**
26. Prolog is based on: **Horn clauses with backward chaining (SLD resolution)**
27. Resolution proves a goal by: **refutation (deriving the empty clause from negated goal)**
28. Combining CF 0.6 and 0.4 (both positive): $0.6 + 0.4 \times 0.4 = 0.76$
29. Dempster-Shafer theory uses: **belief and plausibility**

Planning & NLP

30. STRIPS operators contain: **preconditions, add list, delete list**
31. Sussman anomaly illustrates a problem with: **linear (goal-stack) planning**
32. Partial-order planning follows: **least-commitment strategy**
33. ATN stands for: **Augmented Transition Network**
34. "I saw the man with the telescope" exhibits: **structural (syntactic) ambiguity**
35. Correct NLP pipeline: **morphological → syntactic → semantic → discourse → pragmatic**
36. ELIZA was developed by: **Joseph Weizenbaum**

MAS

37. BDI stands for: **Beliefs, Desires, Intentions**
38. KQML is used for: **agent communication**
39. JADE is: **a Java framework for FIPA-compliant multi-agent systems**
40. Semantic web ontology language: **OWL**

Fuzzy

41. Fuzzy logic was introduced by: **Lotfi Zadeh (1965)**
42. $\mu_A = 0.6$, $\mu_B = 0.3$: union **0.6**, intersection **0.3**, complement of A **0.4**
43. Which law does NOT hold in fuzzy sets? **Law of excluded middle / contradiction**
44. Defuzzification method computing centre of area: **centroid**
45. Sugeno fuzzy model output is: **a linear function or constant (crisp)**
46. Max-min composition of $R = [0.3 \ 0.8]$ and $S = [0.5; \ 0.9]$ (column): $\max(\min(0.3, 0.5), \min(0.8, 0.9)) = 0.8$
47. "Very" hedge applied to $\mu = 0.8$: **0.64**

GA

48. Genetic algorithms were introduced by: **John Holland**
49. Operator that maintains diversity: **mutation**

50. Roulette wheel: fitness values 10, 20, 30, 40 → probability of 3rd: **0.3**
 51. Number of schemata in binary string length 5: $3^5 = 243$

Neural networks

52. Single-layer perceptron cannot learn: **XOR**
 53. Perceptron learning rule: $\Delta \mathbf{w} = \eta(\mathbf{t} - \mathbf{y})\mathbf{x}$
 54. Backpropagation uses: **gradient descent with chain rule**
 55. Derivative of sigmoid: $\sigma(1 - \sigma)$
 56. Kohonen SOM is: **unsupervised, competitive learning preserving topology**
 57. Hopfield network weights are: **symmetric with zero diagonal**; used as **associative memory**
 58. Hopfield storage capacity: $\approx 0.138 N$
 59. Learning with reward/penalty: **reinforcement learning**
 60. Hebbian learning rule: $\Delta \mathbf{w} = \eta \cdot \mathbf{x} \cdot \mathbf{y}$

More practice questions

61. In the tree of §11.1, the number of leaves pruned by alpha-beta: **3**
 62. Resolution proves a goal by deriving: **the empty clause from the negated goal**
 63. Backward chaining starts from: **the goal**
 64. Prevalence 1%, sensitivity 99%, false-positive 5%: $P(\text{disease} | \text{positive}) \approx 0.17$
 65. Max-min composition entry $\max(\min(0.2, 1.0), \min(0.9, 0.8)) = 0.8$
 66. In a GA with fitness values 169, 576, 64, 361, the selection probability of the fittest: ≈ 0.49
 67. Single-point crossover of 01101 and 11000 after bit 4: **01100 and 11001**
 68. Perceptron weights $(w_1, w_2, b) = (2, 1, -2)$ implement: **AND**
 69. Derivative of the sigmoid at output 0.5: **0.25**
 70. Hopfield weight matrix for one stored pattern $\mathbf{p}$: $\mathbf{p} \mathbf{p}^T - \mathbf{I}$
 71. In map colouring, choosing the variable with the fewest legal values is the: **MRV heuristic**
 72. Degree heuristic chooses the variable: **involved in the most constraints with unassigned variables**

POINTS TO REMEMBER

- BFS $O(b^d)$ space · DFS $O(bm)$ · IDDFS best uninformed · A* optimal if admissible
- α - β : prune when $\alpha \geq \beta$ · best ordering $O(b^{(m/2)})$
- Minsky frames · Schank scripts & CD (ATRANS, PTRANS, MTRANS) · MYCIN backward + CF
- STRIPS: pre/add/delete · Sussman anomaly → linear planning fails
- Fuzzy: $\cup$ max, $\cap$ min, $' = 1 - \mu$; excluded middle fails
- Perceptron no XOR · SOM unsupervised · Hopfield symmetric, recurrent, $0.138N$

PART III

Revision

...

REVISION

Exam-Day Formula & Facts Sheet (UGC NET CS)

— . . . —

Read this the evening before and the morning of the exam. Every line here has appeared in past papers.

Paper 1

Topic	Must-remember
Levels of teaching	Memory–Herbart · Understanding–Morrison · Reflective–Hunt
Bloom (revised)	Remember → Understand → Apply → Analyse → Evaluate → Create
Evaluation	Formative (during) · Summative (end) · Diagnostic (difficulties) · Placement (before)
Research	Type I = reject true H_0 · Type II = accept false H_0 · NOIR scales
Plagiarism (UGC 2018)	$L0 \leq 10\%$ · $L1$ 10–40 · $L2$ 40–60 · $L3 > 60$
Communication	Lasswell 5Ws · Shannon–Weaver noise · Schramm field of experience · Berlo SMCR
Logic	$A(S)$ $E(S,P)$ $I(-)$ $O(P)$ distribution · contradictory $A-O$, $E-I$
Pramanas	Charvaka 1 · Buddhist/Vaisheshika 2 · Samkhya 3 · Nyaya 4 · Prabhakara 5 · Bhatta/Advaita 6
Hetvabhasa	Savyabhichara, Viruddha, Satpratipaksha, Asiddha, Badhita
Maths	$\text{Net \%} = a + b + ab/100$ · $CI - SI (2y) = P(R/100)^2$ · avg speed $2xy/(x+y)$
DI	$1\% = 3.6^\circ$ · Mode = 3 Median – 2 Mean
Environment	SDG 17/169 · EPA 1986 · NAPCC 8 missions · Kyoto 1997 · Paris 2015 · ISA Gurugram · Net zero 2070
Higher Ed	Nalanda–Kumaragupta I · Vikramashila–Dharmapala · Wood 1854 · Radhakrishnan 1948 · Kothari 1964–66 · NEP 2020 (GER 50% by 2035)

Discrete Mathematics

<i>Logic</i>	$p \rightarrow q \equiv \neg p \vee q \equiv \neg q \rightarrow \neg p$; Boolean functions of n variables: 2^{2^n}
<i>Relations on an n-set</i>	all 2^{n^2} ; reflexive 2^{n^2-n} ; symmetric $2^{n(n+1)/2}$; antisymmetric $2^n \cdot 3^{n(n-1)/2}$
<i>Counting</i>	onto functions $m \rightarrow n$: $\sum_k (-1)^k \binom{n}{k} (n-k)^m$; derangements $D_4 = 9, D_5 = 44$
<i>Sequences</i>	Bell 1, 1, 2, 5, 15, 52, 203; Catalan 1, 1, 2, 5, 14, 42, 132
<i>Graphs</i>	$\sum \deg = 2E$; $V - E + F = 2$; planar $E \leq 3V - 6$; Cayley n^{n-2} ; K_n has $n(n-1)/2$ edges
<i>Groups</i>	Lagrange: $ H $ divides $ G $; generators of $\mathbb{Z}_n$: $\varphi(n)$; $\mathbb{Z}_n$ is a field $\Leftrightarrow n$ prime
<i>Optimisation</i>	transportation BFS has $m + n - 1$ cells; PERT $t_e = (a + 4m + b)/6$, $\sigma^2 = ((b - a)/6)^2$

Computer Architecture

<i>Number formats</i>	2's complement range $-2^{n-1} \dots 2^{n-1} - 1$; IEEE single 1 8 23 (bias 127), double 1 11 52 (bias 1023)
<i>Hamming</i>	parity bits r : $2^r \geq m + r + 1$
<i>Pipeline</i>	$T = (k + n - 1) t$; speed-up $\rightarrow k$; Amdahl $S = 1 / ((1 - f) + f/s)$
<i>Cache</i>	hierarchical $T = h t_c + (1 - h)(t_c + t_m)$; simultaneous $T = h t_c + (1 - h) t_m$
<i>Address split</i>	direct: TAG LINE OFFSET; k -way: TAG SET OFFSET with sets = lines/ k
<i>Disk</i>	average rotational latency = $\frac{1}{2} \times 60/\text{RPM}$
<i>Circuits</i>	XOR from NAND: 4 gates; Johnson counter n flip-flops $\rightarrow 2n$ states; ring counter $\rightarrow n$ states

Programming & Graphics

<i>Bresenham</i>	$p_0 = 2\Delta y - \Delta x$; if $p_k < 0$: $p_k + 2\Delta y$, else $p_k + 2\Delta y - 2\Delta x$
<i>Midpoint circle</i>	$p_0 = 1 - r$; 8-way symmetry (ellipse 4-way)
<i>Frame buffer</i>	$W \times H \times \text{bpp}/8$ bytes
<i>Transformations</i>	rotation about P : $T(P) R(\theta) T(-P)$; Cohen-Sutherland codes TBRL, AND $\neq 0 \Rightarrow$ reject
<i>Curves</i>	Bézier: n control points $\rightarrow$ degree $n - 1$, global control; B-spline: local control
<i>Shading</i>	Gouraud interpolates intensities, Phong interpolates normals; Z-buffer works in image space
<i>C++</i>	operators that cannot be overloaded: <code>::</code> <code>.</code> <code>*</code> <code>?:</code> <code>sizeof</code>

DBMS

<i>Super keys</i>	one single-attribute candidate key among n : 2^{n-1} ; two single-attribute keys: $3 \cdot 2^{n-2}$
<i>Normal forms</i>	2NF: no partial dependency; 3NF: X superkey or A prime; BCNF: X superkey
<i>Decomposition</i>	lossless if $R_1 \cap R_2 \rightarrow R_1$ or R_2 ; 3NF is lossless and dependency-preserving, BCNF lossless only
<i>Transactions</i>	conflict serializable $\Leftrightarrow$ acyclic precedence graph; $n!$ serial schedules
<i>Deadlock schemes</i>	wait-die: old waits, young dies; wound-wait: old wounds, young waits
<i>B^+ tree order</i>	internal: $p P_b + (p - 1) K \leq B$; leaf: $q (K + P_r) + P_b \leq B$
<i>Mining</i>	support = $n(A \cup B)/N$; confidence = $\sup(A \cup B)/\sup(A)$; 2^n cuboids
<i>Big data</i>	CAP: choose 2 of 3; HDFS block 128 MB, replication 3

Operating Systems

<i>Processes</i>	n fork() calls $\rightarrow 2^n$ processes; TAT = CT - AT; WT = TAT - BT
<i>Deadlock</i>	deadlock-free if $R \geq n(k - 1) + 1$; semaphore final = initial - P + V
<i>Memory</i>	EAT (TLB) = $h(t + m) + (1 - h)(t + 2m)$; EAT (faults) = $(1 - p) ma + p \times \text{service}$
<i>Page table</i>	size = $(2^{LA}/\text{page size}) \times \text{PTE}$
<i>Real time</i>	RM bound $n(2^{1/n} - 1) \rightarrow 0.69$; EDF schedulable if $U \leq 1$
<i>Misc.</i>	Belady's anomaly: FIFO only; RAID 5 distributed parity (min 3 disks); RAID 6 two parities (min 4)
<i>Inode</i>	max file $\approx (12 + k + k^2 + k^3) \times \text{block}$, $k = \text{block} / \text{pointer size}$

Software Engineering

<i>Function points</i>	FP = UFP $\times (0.65 + 0.01 \sum F_i)$, 14 GSCs, VAF 0.65–1.35
<i>FP weights</i>	EI 3/4/6, EO 4/5/7, EQ 3/4/6, ILF 7/10/15, EIF 5/7/10
<i>COCOMO (a, b, c, d)</i>	organic 2.4, 1.05, 2.5, 0.38; semi-detached 3.0, 1.12, 2.5, 0.35; embedded 3.6, 1.20, 2.5, 0.32
<i>Cyclomatic</i>	$V(G) = E - N + 2P = \text{decisions} + 1 = \text{regions}$
<i>Other</i>	availability = $\text{MTTF}/(\text{MTTF} + \text{MTTR})$; risk exposure = $P \times C$; BVA $4n + 1$; paths $n(n - 1)/2$
<i>Design</i>	cohesion: functional (best) to coincidental (worst); coupling: data (best) to content (worst)

Algorithms

<i>Recurrences</i>	Master theorem: compare $f(n)$ with $n^{\log_b a}$
<i>Trees</i>	$n_0 = n_2 + 1$; max nodes at height h : $2^{h+1} - 1$; null links $n + 1$
<i>Balanced trees</i>	AVL minimum nodes 1, 2, 4, 7, 12, 20, 33; red-black height $\leq 2 \log(n + 1)$
<i>Sorting</i>	build-heap $O(n)$; comparison sort $\Omega(n \log n)$; min and max together $3n/2 - 2$
<i>Graphs</i>	Dijkstra $O((V + E) \log V)$, no negative edges; Bellman-Ford $O(VE)$; Floyd $O(V^3)$; Kruskal $O(E \log E)$
<i>DP and strings</i>	matrix chain $O(n^3)$; LCS $O(mn)$; 0/1 knapsack $O(nW)$; KMP $O(n + m)$
<i>Complexity</i>	SAT was the first NP-complete problem (Cook); 2-SAT $\in P$; halting is NP-hard but not in NP

TOC & Compilers

<i>Automata</i>	NFA with n states $\rightarrow$ DFA $\leq 2^n$; "nth symbol from the end" DFA needs 2^n states
<i>Grammars</i>	CNF derivation of length- n string: $2n - 1$ steps; GNF: n ; CYK $O(n^3)$
<i>Closure</i>	regular: all operations; CFL: not $\cap$, not complement; DCFL: complement $\checkmark$; RE: complement $\times$
<i>Decidability</i>	L and $\bar{L}$ both RE $\Rightarrow$ recursive; halting is RE, not recursive; Rice's theorem
<i>Undecidable for CFGs</i>	ambiguity, equivalence, $L = \Sigma^*$, intersection emptiness
<i>Parsers</i>	$LR(0) \subset SLR \subset LALR \subset CLR$; states $LR(0) = SLR = LALR \leq CLR$; LALR merging adds only R/R conflicts
<i>SDT and data flow</i>	S-attributed $\subset$ L-attributed; live variables backward; reaching definitions and available expressions forward

Networks

<i>Capacity</i>	Nyquist $2B \log_2 L$; Shannon $B \log_2(1 + \text{SNR})$; dB = $10 \log_{10}$
<i>Flow control</i>	$a = T_p/T_t$; $\eta_{\text{SW}} = 1/(1 + 2a)$; $\eta = W/(1 + 2a)$; GBN $W \leq 2^n - 1$; SR $W \leq 2^{n-1}$
<i>MAC</i>	pure ALOHA 18.4% ($G = 0.5$); slotted 36.8% ($G = 1$); CSMA/CD $L_{\min} = 2T_p B$
<i>Addressing and codes</i>	hosts in $/n$: $2^{32-n} - 2$; $d_{\min}$: detect $d + 1$, correct $2t + 1$
<i>Counting</i>	mesh links $n(n - 1)/2$; symmetric keys $n(n - 1)/2$; asymmetric keys $2n$
<i>Rates</i>	token bucket burst = $C/(M - \rho)$; T1 1.544 Mbps; E1 2.048 Mbps
<i>Routing</i>	RIP: DV, 15 hops, UDP 520; OSPF: LS, IP protocol 89; BGP: PV, TCP 179
<i>Ciphers</i>	DES 64-bit block, 56-bit key, 16 rounds; AES 128-bit block, keys 128/192/256 $\rightarrow$ 10/12/14 rounds
<i>Ports</i>	FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67/68, HTTP 80, POP3 110, IMAP 143, SNMP 161, HTTPS 443

AI

<i>Search</i>	BFS time and space $O(b^d)$; DFS space $O(bm)$; IDDFS time $O(b^d)$, space $O(bd)$
<i>A*</i>	$f = g + h$; admissible $\Rightarrow$ optimal; $h = 0 \Rightarrow$ uniform-cost search
<i>Games</i>	alpha-beta prunes when $\alpha \geq \beta$; best case $O(b^{m/2})$
<i>Fuzzy</i>	$\cup$ max, $\cap$ min, complement $1 - \mu$, “very” μ^2 , “somewhat” $\sqrt{\mu}$
<i>Neural</i>	perceptron $\Delta w = \eta(t - y)x$, cannot learn XOR; $\sigma' = \sigma(1 - \sigma)$
<i>Misc.</i>	Hopfield capacity $0.138N$; CF combine $CF_1 + CF_2(1 - CF_1)$; schemata 3^l

REVISION

120-Day Study Plan for 250+ (Paper 1 + Paper 2)

...

Assumes ~5–6 hours/day. Compress proportionally if you have less time (e.g. 90 days → shrink Phase 1).

Overview

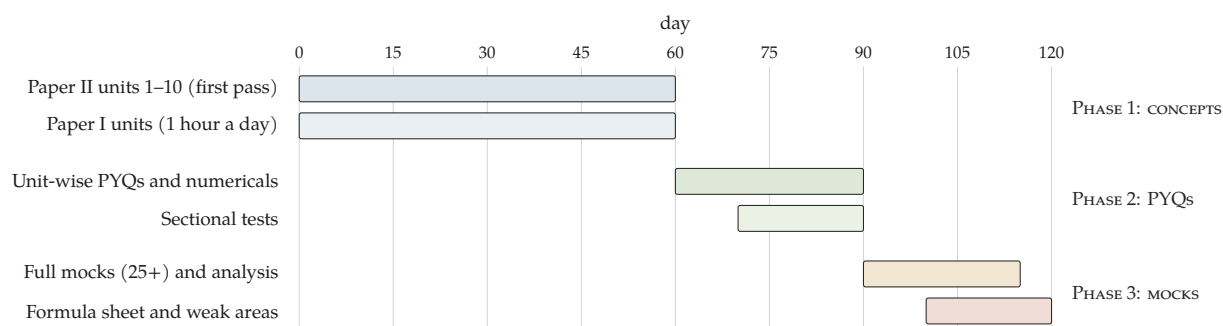


FIGURE 1. The 120-day plan in three phases. The phases overlap a little: practice starts before the first pass ends, and revision runs alongside the mocks.

Phase 1 — Concept Building (Day 1–60)

Daily split: 4 hrs Paper 2 + 1 hr Paper 1 + 30 min revision of previous day.

Days	Paper 2 unit (file)	Paper 1 unit (parallel)
1–6	Discrete Structures & Optimization	Teaching Aptitude
7–12	Computer System Architecture	Research Aptitude
13–18	DBMS	Communication
19–24	System Software & OS	Mathematical Reasoning (daily 10 Qs from here on)
25–30	Data Structures & Algorithms	Logical Reasoning (incl. Indian logic)
31–37	TOC & Compilers	Data Interpretation (daily 1 set from here on)
38–43	Computer Networks	ICT
44–48	Software Engineering	People, Development & Environment
49–53	Programming Languages & Graphics	Higher Education System
54–58	Artificial Intelligence	Comprehension (daily 1 passage from here on)
59–60	Buffer / catch-up	Buffer

For each unit: read the notes → redraw every diagram from memory → solve every PYQ in the file → mark wrong ones with ✗ → re-solve ✗ ones after 3 days.

Phase 2 — PYQs & Numerical Mastery (Day 61–90)

- Solve **official NTA papers** (last 5–6 cycles) **unit-wise** — attempt every question of a unit across years together; patterns become obvious.
- Daily numerical drill (30 min): rotate — cache/pipeline → scheduling/paging → subnetting/sliding window → recurrences → normalisation → COCOMO/FP/V(G) → DFA minimisation.
- Sectional tests: 25 questions per unit, 30 minutes, target $\geq 85\%$.
- Maintain an **error log**: question, why wrong (concept / calculation / misread), correct idea.

Phase 3 — Mocks & Final Revision (Day 91–120)

- **25+ full-length mocks** (150 Qs, 180 min). Analyse each for at least as long as it took.
- Days 91–105: 1 mock every alternate day; fix weak areas on off days.
- Days 106–115: 1 mock daily.
- Days 116–120: only revision — **Formula Sheet**, Quick Revision Boxes of each unit, error log. No new topics.

Score Tracker (fill in after every mock)

Mock	P1 /100	P2 /200	Total	Weakest unit	Action
1					
2					
3					
...					

Target trajectory: 180 → 210 (mock 5) → 235 (mock 12) → 250+ (mock 20 onwards, consistently).

Exam-Hall Strategy (3 hours, 150 questions, no negative marking)

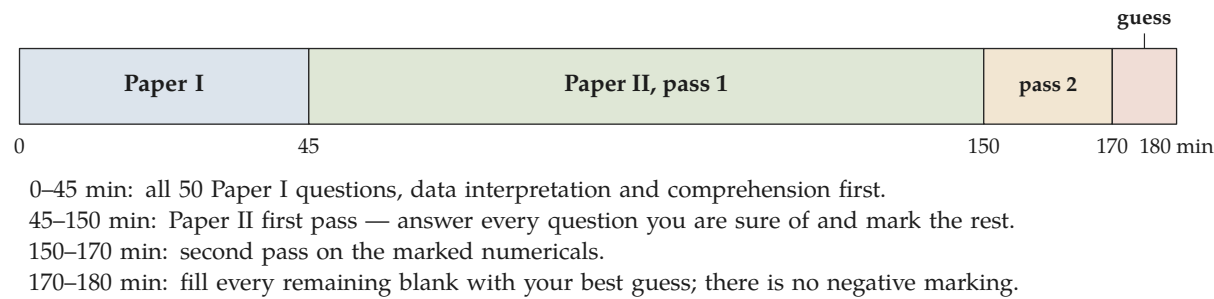


FIGURE 2. A time plan for the three-hour paper.

- 1. **Paper 1 in ≤ 45 minutes** — it carries 1/3 marks with easier questions; don't over-invest.
- 2. **Paper 2 two-pass method:** pass 1 answers what you know in < 40 s; mark anything needing long calculation.
- 3. **Statement/assertion questions:** evaluate each statement independently; eliminate options.
- 4. **Match-the-following:** fix the one pair you are surest of, then eliminate options.
- 5. **Never leave anything blank** — no negative marking. For pure guesses, eliminate first.
- 6. Watch for "NOT" / "incorrect" / "false" in question stems.

Recommended Standard Books (for depth where needed)

Unit	Book
Discrete	Kenneth Rosen — <i>Discrete Mathematics and Its Applications</i>
Optimization	Hamdy Taha — <i>Operations Research</i>
COA	M. Morris Mano — <i>Computer System Architecture</i>
C/C++	Kernighan & Ritchie; E. Balagurusamy
Graphics	Hearn & Baker — <i>Computer Graphics, C Version</i>
DBMS	Korth / Navathe
OS	Galvin (Silberschatz) — <i>Operating System Concepts</i>
SE	Roger Pressman — <i>Software Engineering</i>
Algorithms	Cormen (CLRS); Horowitz–Sahni
TOC	Peter Linz; Hopcroft & Ullman
Compilers	Aho, Lam, Sethi, Ullman (Dragon book)
Networks	Forouzan — <i>Data Communications and Networking</i> ; Tanenbaum
AI	Rich & Knight; Russell & Norvig; S.N. Sivanandam (fuzzy, GA, ANN)
Paper 1	Any standard Paper 1 guide + NTA official papers

Weekly Checklist

- ☐ 6 days study + 1 day revision & error-log review
- ☐ All diagrams of the week redrawn from memory
- ☐ At least 150 PYQ/practice questions solved
- ☐ One full or sectional mock analysed